Design (RTL / Netlist) → Constraints (SDC) → Parasitics (SPEF) → **STATIC TIMING ANALYSIS (STA)** → Timing Reports (Slack / Paths) → Violations (Setup / Hold) → Timing Sign-off (Tape-out)

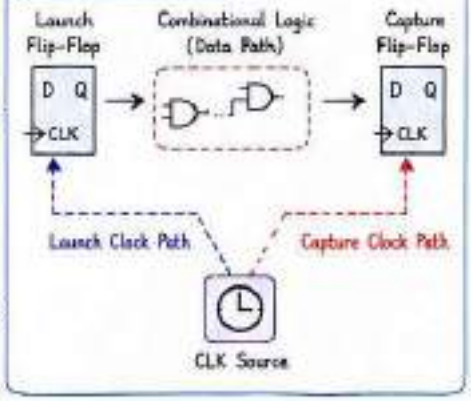
STATIC TIMING ANALYSIS (STA)

STA is a method to verify the timing correctness of a digital circuit without applying input vectors, by analyzing all possible timing paths.

1. WHAT STA CHECKS

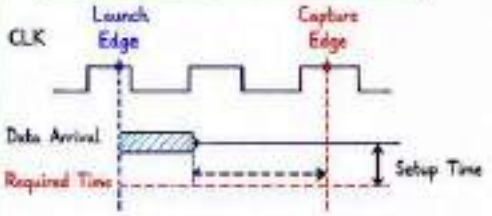
- ★ Setup Time (Data must arrive before capture edge)
- ★ Hold Time (Data must remain stable after capture edge)
- ★ Clock Relationships (Latency, Skew, Uncertainty)
- ★ Timing Exceptions (False Path, Multi-cycle, etc.)
- ★ Special Checks (Recovery, Removal, CDC, etc.)

2. TIMING PATH

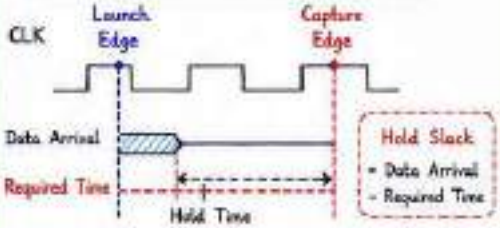


3. TIMING ANALYSIS

SETUP CHECK (Data Arrives Too Late)



HOLD CHECK (Data Arrives Too Early)



Slack > 0 → Timing Met | Slack < 0 → Timing Violation

4. KEY TIMING FACTORS

- **Clock Latency** — Source + Network Latency
- **Clock Skew** — Difference between Capture and Launch clock arrival
- **Clock Uncertainty** — Jitter, Variation, OCV, Skew uncertainty
- **Cell Delay** — Depends on PVT, Slew & Load
- **Net Delay** — Depends on RC, Wire length, Metal Layer
- **Slew (Transition)** — Affects cell delay
- **Load Capacitance** — Affects delay & slew
- **PVT Variation** — Process, Voltage, Temperature
- **OCV / AOCV / POCV** — On-Chip Variation models

Setup Slack = Required Time − Data Arrival

5. TIMING EXCEPTIONS

- **False Path** — Path that is functionally not required
- **Multi-Cycle Path** — Data allowed over multiple clock cycles
- **Half-Cycle Path** — Data required within half of the clock period
- **Asynchronous Path** — Between unrelated clock domains
- **Path Exclusion** — paths
- **Launch / Capture Edge Selection** — Control which edges are considered

6. STA FLOW OVERVIEW

1. Input Preparation	2. Timing Analysis	3. Report Generation	4. Debug & Optimization	5. Sign-off (STA)
• Netlist (Gate Level)	• Build Timing Graph	• Slack Summary	• Identify Root Cause	• MCMM Sign-off
• Timing Constraints (SDC)	• Propagate Arrival & Required Times	• Worst Paths	• Fix Setup / Hold	• PVT Corners
• Parasitics (SPEF)	• Check Setup / Hold	• Violations	• Optimize Design / Constraints	• OCV / AOCV / POCV
• Libraries (Lib)	• Apply Exceptions	• Path Details		• CPPR / CRPR
		• Constraints Check		• Final Timing Closure

Iterate until all timing requirements are met

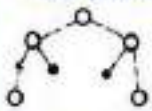
7. ANALYSIS METHODS

GBA (Graph Based Analysis)



- Fast
- Pessimism Possible
- Used for Early Stages

PBA (Path Based Analysis)



- More Accurate
- Less Pessimism
- Used for Sign-off

8. COMMON VIOLATION ROOT CAUSES

- Long data path (High logic / net delay)
- High fanout / High load capacitance
- Poor placement / Congested routing
- High RC (long wires / high resistance)
- Unfavorable clock skew
- High uncertainty / Variation
- Improper constraints / Exceptions

9. SETUP / HOLD FIXES

Setup Fixes (Improve Speed)	Hold Fixes (Add Delay)
✓ Increase drive strength	✓ Insert delay cells / Buffers
✓ Use faster cells / LVT cells	✓ Use smaller / Weaker cells
✓ Insert buffers / Upsize	✓ Increase wire length
✓ Reduce logic levels	✓ Use lower metal layers
✓ Reduce wire length	✓ Detour routing
✓ Use higher metal layers	✓ Increase clock-to-Q
✓ Improve placement	✓ Adjust clock skew (Push / Pull)
✓ Optimize clock skew (Useful Skew)	

10. SPECIAL CHECKS

- **Recovery / Removal** — For async SET-reset
- **Metastability / CDC** — Safe data transfer between clock domains
- **Maximum Transition (Slew)** — Avoid signal integrity issues
- **Maximum Capacitance** — Limit load on outputs
- **Maximum Fanout** — Limit number of loads

11. STA BENEFITS

Ensures Timing Correctness · Early Detection of Violations · No Test Vectors Required · Works for All PVT Corners · Improves Yield & Reliability · Enables Timing Sign-off

12. IMPORTANT REMINDERS

- ✓ Good Constraints → Accurate Results
- ✓ Check All Corners (MCMM)
- ✓ Verify Reports Carefully
- ✓ Fix Root Cause, Not Just Symptoms
- ✓ Re-run STA After Every Change

13. STA GOAL

Achieve TIMING CLOSURE with Positive Slack across all paths & all conditions for RELIABLE DESIGN SIGN-OFF.

Author : Konda Tharun Vivekananda

INDEX – STATIC TIMING ANALYSIS (STA)

S.No.	Main Heading / Concept	Page No.
1	CMOS Switching Waveform — Rising Transition; Falling Transition; Charging / Discharging Concept; Rise Time; Fall Time	1
2	Slew Rate — Transition Time; Load Capacitance & Delay; Propagation Delay; CMOS Delay Differences	2
3	Propagation Delay — Input Transition; Rise / Fall Propagation Delay; Average Propagation Delay; Clock-to-Q Delay	3
4	Cell Delay — Combinational & Sequential Cell Delay; Cell vs Net Delay; PVT, Load & Slew Dependency	4
5	Static Timing Analysis (STA) — Definition; Why STA is Required; Purpose; STA vs Functional Simulation	5
6	Advantages & Limitations of STA — Timing Verification Without Test Vectors; Best / Worst-Case Timing; Timing Signoff	6
7	Where STA Fits in VLSI Design — RTL; Synthesis; Floorplanning; Placement; Pre-CTS; CTS; Post-CTS; Routing; Post-Route STA	7
8	Post-Route STA — Signoff STA; Post-Route Parasitics; Estimated vs Actual RC Delay; Physical Timing Analysis	8
9	Inputs & Outputs of STA — Gate-Level Netlist; SDC Constraints; SPEF / Parasitics; Timing Reports; Slack; Violating Paths	9
10	Timing Path — Start Point; End Point; Launch Point; Capture Point; Launch / Capture Clock; Data Path	10
11	Timing Path Components — Capture Clock; Data Path; Clock Path; Combinational Logic; Sequential Elements	11
12	Timing Exceptions — Single-Cycle Path; Multi-Cycle Path; Half-Cycle Path; False Path	12
13	Timing Path Types — Synchronous Path; Asynchronous Path; Data Path; Clock Path	13
14	Setup Timing Analysis — Setup Time; Setup Violation; Launch / Capture Edge; Required Arrival Time	14
15	Setup Timing Analysis — Data Arrival Time; Setup Slack; Setup Timing Equation; Clock Period Effect	15
16	Setup Timing Analysis — Combinational Delay; Setup Time; Clock Skew; Clock Uncertainty	16
17	Hold Timing Analysis — Hold Time; Hold Violation; Data Arriving Too Early; Hold Slack	17
18	Hold Timing Analysis — Clock-to-Q; Minimum Data Path Delay; Clock Skew; Clock Uncertainty	18
19	Setup vs Hold — Complete Comparison; Setup Fixes; Hold Fixes; Setup / Hold Interaction	19
20	Slack, Arrival Time & Required Time — DAT; DRT; Slack Equation; Setup vs Hold	20
21	Timing Slack — Positive / Negative / Zero Slack; Setup Slack; Hold Slack; WNS; TNS; Critical Path	21
22	Clock Timing Fundamentals — Clock Period; Frequency; Rising / Falling Edge; Launch / Capture Edge; Clock Definition	22
23	Clock Latency — Source Latency; Network Latency; Total Latency; Timing Impact	23
24	Clock Insertion Delay — Clock Tree; Pre-CTS / Post-CTS; Source-to-Leaf Delay; Setup / Hold Impact	24
25	Clock Skew — Launch / Capture Clock Arrival; Skew Types; Local / Global Skew; Setup / Hold Impact	25

INDEX

S.No.	Main Heading / Concept	Page No.
26	Clock Skew & Uncertainty — Skew Calculation ; Setup/Hold Impact ; Clock Push ; Clock Pull	26
27	Useful Skew — Intentional Clock Skew ; Setup Improvement ; Hold Impact ; Clock Manipulation ; Timing Optimization	27
28	Clock Uncertainty — Setup/Hold Uncertainty ; Jitter ; Skew Variation ; OCV Contribution	28
29	Clock Uncertainty — Uncertainty Margin ; Pre-CTS/Post-CTS ; SDC Clock Uncertainty ; Setup/Hold Equations	29
30	Clock Jitter — Definition ; Ideal vs Actual Clock ; Early/Late Arrival ; Cycle-to-Cycle Jitter	30
31	Clock Jitter — PLL Jitter ; Oscillator Jitter ; Thermal Noise ; Power-Supply Noise	31
32	Clock Jitter Types — Absolute ; Cycle-to-Cycle ; Periodic ; Random ; Deterministic Jitter	32
33	Jitter Reduction — Buffers ; Power Filtering ; Shielding ; Interconnect Improvements	33
34	Slew / Transition — Rise/Fall Time ; Fast/Slow Slew ; Slew Impact on Cell Delay ; Slew Violations	34
35	Cell Delay — Gate Delay ; Input Slew ; Output Capacitance ; PVT Dependency ; Library Characterization	35
36	Net Delay / Interconnect Delay — Wire Delay ; Resistance ; Capacitance ; RC Delay ; Routing Impact	36
37	PVT Variation — Process ; Voltage ; Temperature ; PVT Corners ; Delay Impact	37
38	RC Corners — RC Extraction ; Resistance/Capacitance Effects ; Worst RC Corners ; Interconnect Impact	38
39	MCMM — Multi-Corner Multi-Mode ; Functional/Test Modes ; MCMM Flow ; Timing Closure	39
40	MCMM Importance — Timing Closure ; Setup/Hold Closure ; Yield ; Reliability ; Performance ; Signoff	40
41	OCV — On-Chip Variation ; Variation Sources ; Global/Local Variation ; Setup/Hold Impact	41
42	OCV Derating — Derating Concept ; Cell/Net Delay ; Early/Late Derating ; Setup/Hold Analysis	42
43	AOCV — Advanced OCV ; Path-Depth Dependency ; Short/Long Path Variation ; AOCV Setup/Hold	43
44	POCV — Parametric OCV ; Statistical Variation ; Mean/Sigma ; Cell Correlation ; Early/Late Derating	44
45	OCV / AOCV / POCV Comparison — Method Overview ; Path-Depth Dependency ; Table-Based Derating	45
46	CPPR / CRPR — Common Path Pessimism Removal ; Clock Path ; Setup/Hold Impact	46
47	Crosstalk Fundamentals — Coupling Capacitance ; Aggressor ; Victim ; Switching Interaction ; Crosstalk Delay	47
48	Crosstalk Noise / Glitch — Coupling Model ; Rise/Fall Glitch ; Overshoot/Undershoot ; Victim Response	48
49	Crosstalk Delay — Same/Opposite Direction Switching ; Positive/Negative Crosstalk Delay	49
50	Crosstalk Mitigation — Coupling Reduction ; NDR ; Buffer Insertion ; Shielding ; Multiple Vias	50

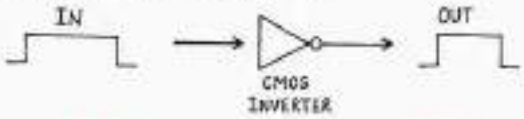
INDEX

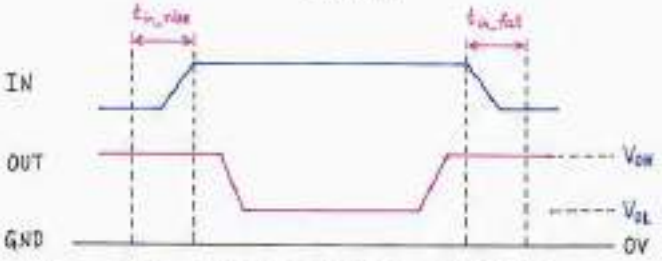
S.No.	Main Heading / Concept	Page No.
51	Timing Analysis Methods — GBA — Graph-Based Analysis ; Timing Graph ; Nodes / Edges ; Arrival Time ; Required Time ; Slack	51
52	Timing Analysis Methods — PBA — Path-Based Analysis ; PBA vs GBA ; Path Delay ; Reduced Pessimism ; Critical-Path Analysis ; Signoff Usage	52
53	Timing Violation Debugging — Setup Violation — Identify Violating Path ; Launch Point ; Capture Point ; Clock Path ; Clock Skew ; Uncertainty ; Cell Delay	53
54	Timing Violation Debugging — Hold Violation — Launch / Capture Clock ; Clock Skew ; PVT Corner ; RC ; OCV ; Critical-Path Analysis	54
55	Setup Fixes — Drive Strength ; Faster Cells ; LVT ; Buffer Insertion ; Logic Delay ; Wire Delay ; Metal Layer ; Placement ; Clock Skew Optimization	55
56	Hold Debugging — Critical Path ; Launch / Capture Clock ; Clock Skew ; Minimum Data Delay ; PVT ; RC ; OCV Derating ; Debug Flow	56
57	Hold Fixes — Delay Buffers ; Weaker Cells ; Wire Delay ; Detour Routing ; Clock-to-Q ; Clock Push / Pull	57
58	Special Timing Checks — Recovery & Removal Timing ; Violations ; Asynchronous Control ; Debugging	58
59	Metastability — Setup / Hold Relationship ; Unstable Flip-Flop State ; CDC ; Two-Stage Synchronizer ; Timing Margin	59
60	Special Timing Checks — CDC — Clock Domain Crossing ; Different Frequencies / Phases ; Synchronization ; FIFO ; CDC Constraints	60
61	Physical Effects on Timing — Placement ; Cell Proximity ; Wire Length ; Congestion ; Routing Layers ; Resistance ; Capacitance ; Overall Delay	61
62	Physical Effects on Timing — Buffering ; Via Resistance ; Parasitic Extraction ; Clock Tree Effects ; Post-Route Timing	62
63	Multi-Cut Vias / Redundant Vias — Multiple Via Cuts ; Resistance Reduction ; Reliability ; Current Handling ; Critical Nets	63
64	Timing Constraints — SDC Fundamentals — SDC ; Clock Definition ; Generated Clock ; Virtual Clock ; Clock Uncertainty	64
65	Timing Constraints — SDC Fundamentals — I/O Delays ; Timing Exceptions ; Max Transition ; Max Capacitance ; Max Fanout	65
66	Timing Constraints — Clock Constraints — Primary Clock ; Generated Clock ; Period ; Waveform ; Latency ; Uncertainty ; Clock Relationship ; Clock Source	66
67	Timing Exceptions — False Path ; Multi-Cycle Path ; Half-Cycle Behavior ; Asynchronous Path Considerations ; Path Exclusion ; Launch / Capture Edge Selection	67
68	Design Timing Constraints — Maximum Transition / Slew ; Maximum Capacitance ; Maximum Fanout ; Slew / Load / Fanout Violations	68
69	STA Report Analysis — Timing Report Reading ; Startpoint , Endpoint , Path Group ; Path Type ; Launch Clock , Capture Clock , Data Arrival / Required Time	69
70	STA Report Analysis — Slack ; Cell Delay ; Net Delay ; Slew ; Capacitance ; Fanout ; Clock Skew , Uncertainty ; Critical Path	70
71	Report Timing & STA Debugging — Timing Report ; Worst Setup / Hold Path ; Path Groups ; Startpoint / Endpoint Selection ; Violation Identification ; Debug Flow	71
72	Timing Closure — Fundamentals — Timing Closure ; Setup ; Hold ; Constraint ; Slew ; Capacitance ; Fanout ; Clock ; Noise ; Signoff Closure	72
73	Setup vs Hold Optimization Strategy — Setup Optimization ; Hold Optimization ; Delay / Drive / Buffer / Wire / Clock Strategies	73
74	ECO — Engineering Change Order — Definition ; Need ; Types ; ECO Flow ; Netlist Modification ; Buffer Insertion ; Cell Replacement ; Routing Modification ; Post-Implementation ECO	74
75	Signoff STA — Final Timing Verification — Why Signoff ; Final Verification ; Post-Route Parasitics ; PVT / MCMM ; Setup / Hold / Clock Signoff , OCV / AOCV / POCV ; CPPR / CRPR	75
76	Signoff STA — Key Considerations — Crosstalk ; Slew ; Capacitance ; Fanout ; Timing Exceptions ; Critical Paths ; ECO / Optimization ; Final Timing Closure ; Tapeout Readiness	76

STATIC TIMING ANALYSIS (STA)

1. CMOS SWITCHING WAVEFORM

CMOS inverter switching converts the input transition into an output transition.
The output waveform consists of a rising or falling transition depending on the input.



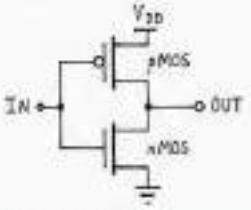


- When IN ↑, OUT ↓ (nMOS ON, pMOS OFF)
- When IN ↓, OUT ↑ (pMOS ON, nMOS OFF)

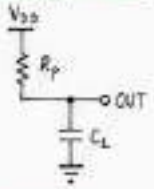
2. RISING TRANSITION (0 → 1)

Output rises from LOW (0) to HIGH (V_{DD}).

During rising:
- pMOS is ON → charges the load capacitance.
- nMOS is OFF → no discharge path.



RC Charging Model



Output voltage:

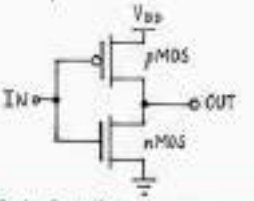
$$V_{out}(t) = V_{DD}\left(1 - e^{-\frac{t}{R_p C_L}}\right)$$

Key Points:
- Output increases exponentially.
- As time increases, V_{out} approaches V_{DD}.
- 63.2% of final value at $t = \tau$ (time constant).

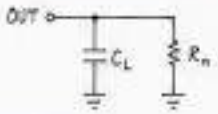
3. FALLING TRANSITION (1 → 0)

Output falls from HIGH (V_{DD}) to LOW (0).

During falling:
- nMOS is ON → discharges the load capacitance.
- pMOS is OFF → no charging path.



RC Discharging Model



Output voltage:

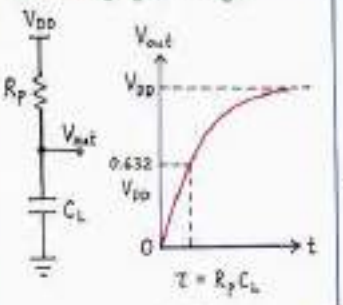
$$V_{out}(t) = V_{DD}\, e^{-\frac{t}{R_n C_L}}$$

Key Points:
- Output decreases exponentially.
- V_{out} approaches 0 as time increases.
- Drops to 36.8% of initial value at $t = \tau$.

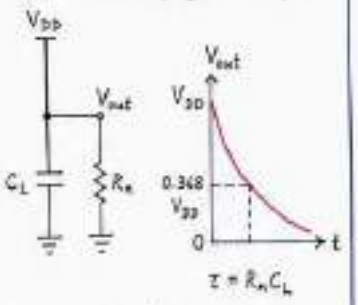
4. RC CHARGING / DISCHARGING CONCEPT

CMOS switching is dominated by charging and discharging of load capacitance through the effective resistance of the conducting device.

RC Charging (Rising)



RC Discharging (Falling)

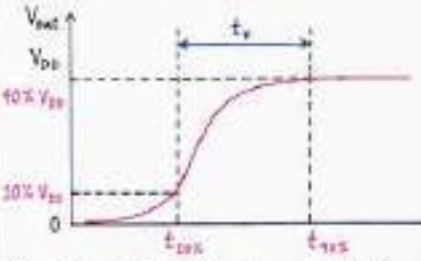


Time constant (τ) determines the speed of transition.

Larger R or C_L → Larger τ → Slower transition

5. RISE TIME (t_r)

Rise time is the time taken for output to rise from 10% to 90% of V_{DD}.



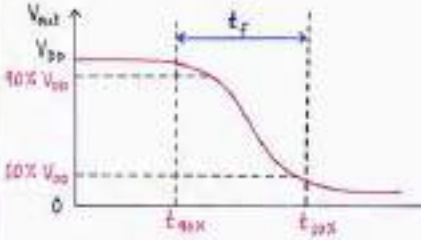
Approximation:

$$t_r \approx 2.2 R_p C_L$$

So, Rise time depends on pMOS resistance and load capacitance.

6. FALL TIME (t_f)

Fall time is the time taken for output to fall from 90% to 10% of V_{DD}.



Approximation:

$$t_f \approx 2.2 R_n C_L$$

So, Fall time depends on nMOS resistance and load capacitance.

SUMMARY TABLE

Parameter	Condition	Dominant Device	Model	Approx. Expression
Rising Transition	0 → 1	pMOS ON (Charging)	$R_p - C_L$	$t_r \approx 2.2 R_p C_L$
Falling Transition	1 → 0	nMOS ON (Discharging)	$R_n - C_L$	$t_f \approx 2.2 R_n C_L$
Rise Time	20% → 90%	—	—	$t_r \approx 2.2 R_p C_L$
Fall Time	90% → 10%	—	—	$t_f \approx 2.2 R_n C_L$

PRACTICAL OBSERVATIONS

- Smaller R_p/R_n or smaller C_L → faster transitions.
- Larger R_p/R_n or larger C_L → slower transitions.
- Slow transitions increase delay and affect timing slack.
- Accurate modeling of rise/fall times is essential for STA delay estimation.

VIVA QUESTIONS

Q1. What causes output transition in CMOS inverter?
Q2. Why is rise time usually different from fall time?
Q3. How do R_p, R_n and C_L affect rise/fall time?
Q4. Define rise time and fall time.
Q5. What is the time constant in RC charging?
Q6. Why is switching speed important in STA?

QUICK REVISION

- ✓ CMOS switching is based on RC charging / discharging.
- ✓ Rising: pMOS charges C_L through R_p.
- ✓ Falling: nMOS discharges C_L through R_n.
- ✓ Rise Time: 10% → 90% ($t_r \approx 2.2 R_p C_L$)
- ✓ Fall Time: 90% → 10% ($t_f \approx 2.2 R_n C_L$)
- ✓ Smaller τ → faster transition → smaller delay.
- ✓ Transition speed directly impacts STA timing.

STATIC TIMING ANALYSIS (STA)

1. SLEW RATE

Slew rate is the rate at which a signal voltage changes with time during a transition.
It indicates how "fast" the output is rising or falling.

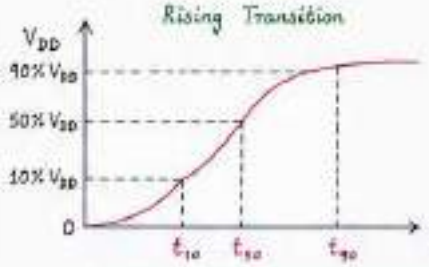
$$\text{Slew Rate (SR)} = \frac{\Delta V}{\Delta t} = \frac{V_{high} - V_{low}}{\Delta t}$$

Units : V/ns

Instantaneous Slew (at any instant t) :

$$SR_{inst} = \frac{dV}{dt}$$

Rising Transition



Slew rate is maximum near mid point (50%).

Higher slew rate → Faster transition
Lower slew rate → Slower transition

2. TRANSITION TIME

Transition time is the total time taken for output to change from one logic level to another between two reference points.
Commonly measured between 10% and 90% of V_{DD}.

For Rising Transition (0 → 1) :
t_r (10–90) = $t_{90} - t_{10}$

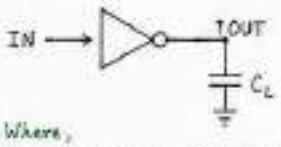
For Falling Transition (1 → 0) :
t_f (90–10) = $t_{10} - t_{90}$

Smaller transition time means faster edge.
Larger transition time means slower edge.

STA libraries characterize cell delays at different input transition times (slew).
Hence transition time strongly affects delay.

5. LOAD CAPACITANCE AND ITS EFFECT ON DELAY

Load capacitance (C_L) is the total capacitance seen at the output node (next gate input, interconnect, etc.).
Larger C_L requires more charge/discharge → longer delay.



Approximate Delay Relation

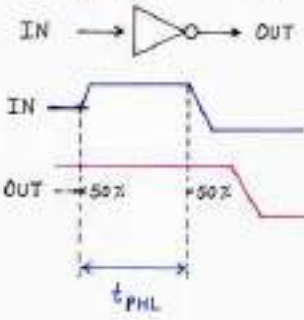
$$t_{delay} \propto R_{on} \times C_L$$

Where,
R_{on} → Effective On-Resistance of the conducting device
C_L → Load Capacitance at output

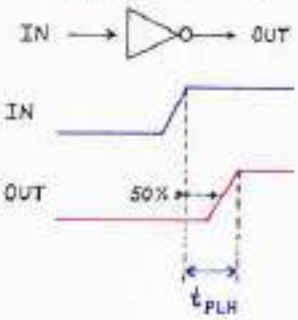
3. PROPAGATION DELAY

Propagation delay is the time taken for a change at input to produce a corresponding change at output.
Two types of propagation delay for an inverter :

t_{PHL} (High to Low)



t_{PLH} (Low to High)



Delays depend on driving device, load and input slew.

Output falling → Discharge → NMOS conducts → t_{PHL}
Output rising → Charge → PMOS conducts → t_{PLH}

4. WHY PMOS / NMOS HAVE DIFFERENT DELAYS

PMOS and NMOS have different drive strengths.

Parameter	NMOS	PMOS	Effect on Delay
Carrier Type	Electrons	Holes	Holes have lower mobility
Mobility (μ)	High	~2 to 3 × lower	Lower mobility → slower current
On-Resistance (R_{on})	Low	~2 to 3 × higher	Higher R_{on} in PMOS
Drive Strength	Stronger	Weaker	NMOS discharges faster
Typical Delay	Smaller	Larger	$t_{PHL} < t_{PLH}$

Hence : $t_{PLH} > t_{PHL}$ (in typical CMOS inverter)

6. DELAY VS LOAD CAPACITANCE

Propagation delay increases almost linearly with load capacitance for a given input slew.

Delay vs C_L



Reason :
- Larger C_L → More charge required during rise
- Larger C_L → More discharge required during fall
- Hence time increases.

NUMERICAL EXAMPLE

Given : V_{DD} = 1.0 V
$R_{on,n}$ = 2 kΩ, $R_{on,p}$ = 5 kΩ
C_L = 20 fF

Approx. t_{PHL} = 0.69 × $R_{on,n}$ × C_L
= 0.69 × 2k × 20f
= 27.6 ps

Approx. t_{PLH} = 0.69 × $R_{on,p}$ × C_L
= 0.69 × 5k × 20f
= 69 ps

Hence, $t_{PLH} > t_{PHL}$

PRACTICAL OBSERVATIONS

- Slower edge (large transition time) increases propagation delay.
- Library delays are characterized w.r.t. input slew and output load.
- PMOS is weaker → rise is slower than fall.
- Managing load capacitance is critical for achieving timing closure.

VIVA QUESTIONS

Q1. What is slew rate?
Q2. How is transition time measured?
Q3. Define propagation delay.
Q4. Why is t_{PLH} larger than t_{PHL}?
Q5. How does load capacitance affect propagation delay?
Q6. What is the relation between delay and C_L?

QUICK REVISION

- ✓ Slew rate = $\frac{dV}{dt}$
- ✓ Transition time = $t_{90} - t_{10}$ (rising), $t_{10} - t_{90}$ (falling)
- ✓ Delay : t_{PHL} (H→L), t_{PLH} (L→H)
- ✓ PMOS weaker → $t_{PLH} > t_{PHL}$
- ✓ Delay ∝ $R_{on} \times C_L$
- ✓ Larger C_L → Larger delay
- ✓ Smaller slew → Smaller delay

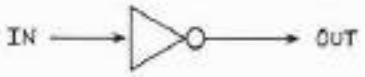
STATIC TIMING ANALYSIS (STA)

1. WHAT IS PROPAGATION DELAY ?

Propagation delay is the time taken for a change at the input of a logic gate (or cell) to be observed at the output. It represents the time required for the signal to propagate through the cell.

Two types :

- Rise propagation delay (t_{pLH})
- Fall propagation delay (t_{pHL})



Cell delay depends on :
- input transition
- output load
- supply, temp, process

2. INPUT TRANSITION vs OUTPUT TRANSITION

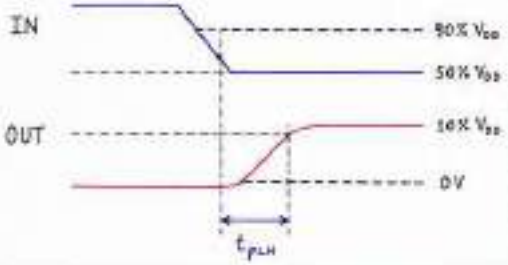
Parameter	Input Transition	Output Transition
Definition	Change of voltage at input from one logic level to the other.	Change of voltage at output caused by the input change after propagation delay.
Measured Between	10% ↔ 90% (rise) or 90% ↔ 10% (fall)	10% ↔ 90% (rise) or 90% ↔ 10% (fall)
Affects	Affects cell delay.	Represents output slew (speed of output edge).
Notation	t_{in} (rise / fall)	t_{out} or t_{slew} (rise / fall)
Range	Specified in STA libraries	Results from cell + load

Note :

A slower input transition (larger time) → Larger propagation delay.
A larger output transition (slower slew) also indicates higher load.

3. RISE PROPAGATION DELAY (t_{pLH})

Time taken for output to rise from LOW (10%) to HIGH (90%) in response to input falling (1 → 0) for an inverter.

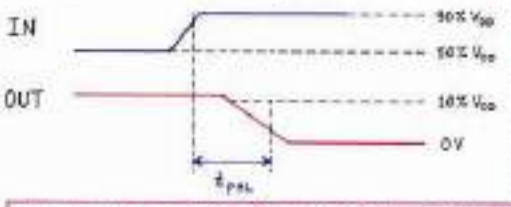


Measurement :
t_{pLH} = time between IN @ 50% (falling) to OUT @ 50% (rising)

Input falling → NMOS ON, PMOS OFF → Output rises

4. FALL PROPAGATION DELAY (t_{pHL})

Time taken for output to fall from HIGH (90%) to LOW (10%) in response to input rising (0 → 1) for an inverter.



Measurement :
t_{pHL} = time between IN @ 50% (rising) to OUT @ 50% (falling)

Input rising → PMOS ON, NMOS OFF → Output falls

5. AVERAGE PROPAGATION DELAY

Average propagation delay of a cell is the mean of rise and fall propagation delays.

$$t_{p(avg)} = \frac{t_{pLH} + t_{pHL}}{2}$$

Used to represent the overall delay of a cell when rise and fall are both possible.

Example :

If $t_{pLH} = 120$ ps and $t_{pHL} = 80$ ps

Then,

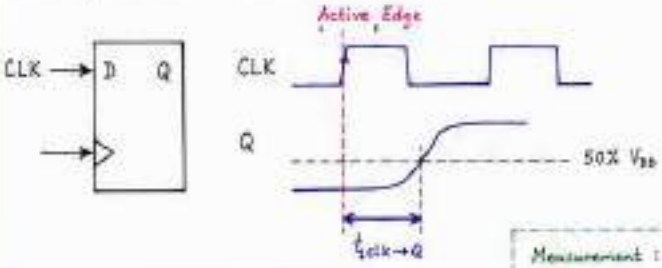
$$t_{p(avg)} = \frac{120 + 80}{2} = 100 \text{ ps}$$

Key Point :

In digital timing, both t_{pLH} and t_{pHL} are stored in STA libraries, and the appropriate one is used depending on the input transition direction.

6. CLOCK-TO-Q DELAY ($t_{clk \to Q}$)

Clock-to-Q delay is the time from the active clock edge at a flip-flop (register) to the corresponding output Q reaching 50% of its final value.



Measurement :
$t_{clk \to Q}$ = time from clock active edge to Q @ 50%

Important Notes
- Separate values for rising edge triggered and falling edge triggered flops.
- Depends on internal latch + clock path inside the flip-flop.
- Affected by PVT, load on Q output and input transition.
- Used as the first delay in timing path of launch flop.

COMPARISON TABLE

Parameter	t_{pLH}	t_{pHL}	$t_{p(avg)}$	$t_{clk \to Q}$
Meaning	Rise Propagation Delay	Fall Propagation Delay	Average Propagation Delay	Clock to Q Delay
Measured From	IN @ 50% (fall) to OUT @ 50% (rise)	IN @ 50% (rise) to OUT @ 50% (fall)	Mean of t_{pLH} and t_{pHL}	CLK active edge to Q @ 50%
Used For	Output rising cases	Output falling cases	General delay representation	Sequential path (launch flop)
Depends On	Input transition, Load, PVT	Input transition, Load, PVT	Both above delays	Flop design, PVT, load

PRACTICAL OBSERVATIONS

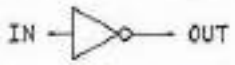
- Larger input transition time → larger propagation delay
- Larger output load → slower output transition → larger delay
- Rise and fall delays are usually not equal in CMOS due to PMOS / NMOS strength difference.
- Library delays are characterized with respect to input slew and output load.

VIVA QUESTIONS

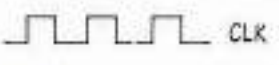
Q1. What is propagation delay ?
Q2. Define rise propagation delay.
Q3. Define fall propagation delay.
Q4. What is average propagation delay ?
Q5. What is clock-to-Q delay ?
Q6. What factors affect propagation delay ?
Q7. Why rise and fall delays are different ?

QUICK REVISION

- ✓ Propagation delay = time from input change to output change.
- ✓ Rise delay (t_{pLH}) : IN fall → OUT rise (10% to 90%).
- ✓ Fall delay (t_{pHL}) : IN rise → OUT fall (90% to 10%).
- ✓ Average delay = $(t_{pLH} + t_{pHL})/2$
- ✓ Clock-to-Q delay = CLK edge → Q @ 50%.
- ✓ Delays depend on input transition, load, PVT, cell design.



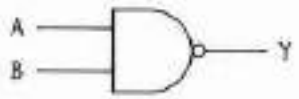
STATIC TIMING ANALYSIS (STA)



1. COMBINATIONAL CELL DELAY

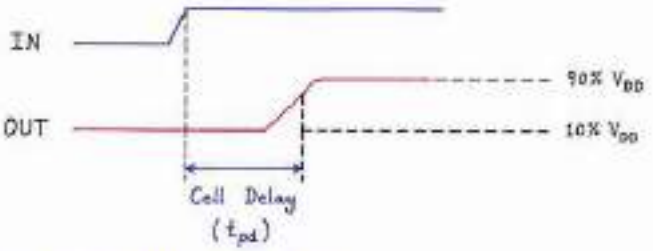
Combinational cell delay is the time taken for a change at input to propagate through a combinational logic cell and cause a change at the output.

Example : 2-input NAND gate



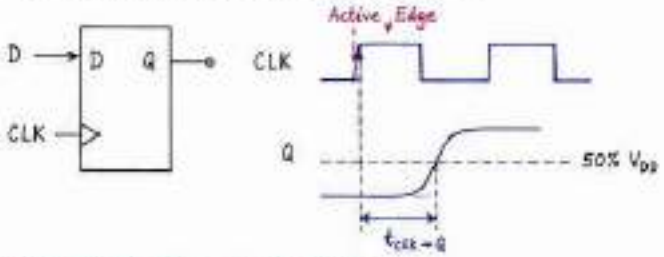
Delay depends on :
- input transition (slew)
- output load (C_L)
- supply, temp, process

Cell delay is specified in STA libraries for different input slews and output loads.



2. SEQUENTIAL CELL DELAY

Sequential cell delay is time associated with sequential elements (flip-flops, latches). Main delay of interest is from clock edge to output Q change (clock-to-Q delay).



Other related delays (in STA libs) :
- t_{rec} : Recovery time
- t_{rem} : Removal time

These are used in special timing checks (not main path delay).

3. CELL DELAY vs NET DELAY

Parameter	Cell Delay	Net Delay
Definition	Delay inside a logic cell (gate / FF)	Delay on interconnect wires between cells
Cause	Transistor switching inside the cell	RC charging / discharging of wires
Depends On	Input slew, output load, PVT, cell type	Wire length, R, C, Fanout, buffer, layer
Model	From timing libraries (.lib)	From parasitic extraction (SPEF / RC model)
Typical Size	ps to hundreds of ps	ps to ns (may be large)
Optimization	Use stronger cell, change logic	Buffer insertion, routing optimization

Key Point :
Total path delay = Sum of cell delays + Sum of net delays.
Both are required for accurate STA.

4. FACTORS AFFECTING CELL DELAY

Cell delay of a gate or flip-flop is not fixed. It varies with several factors.

① Input transition (slew)
② Output load capacitance
③ Supply voltage (V_{DD})
④ Temperature
⑤ Process variations
⑥ Cell strength / drive

In STA libs :
Delay = f ($slew_{in}$, C_L, V_{DD}, Temp, Process)
↓
Look-up in tables

5. PVT DEPENDENCY

Cell delay varies with Process, Voltage and Temperature.

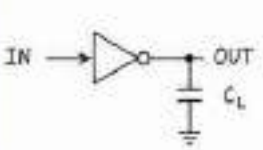
Slow conditions → larger delay
Fast conditions → smaller delay

Condition	Process	Voltage	Temperature	Effect on Delay
Slow Corner	SS (slow)	Low V_{DD}	High Temp	Maximum delay
Fast Corner	FF (fast)	High V_{DD}	Low Temp	Minimum delay

STA checks timing at required PVT corners.
Setup uses slow corner (max delay), Hold uses fast corner.

6. LOAD DEPENDENCY

As load capacitance (C_L) increases, more charge / discharge is required → delay increases.



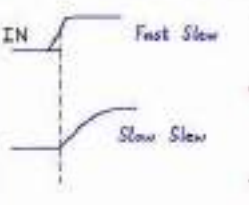


Approx. relation
$t_{pd} \propto C_L$

Reason :
More C_L → more RC time constant → slower transition → larger propagation delay.

7. SLEW DEPENDENCY

Larger input transition time (slew) → smaller drive current inside device → slower output transition → larger delay.



Approx. relation
$t_{pd} \propto Slew_{in}$

Reason :
$Slew_{in}$ ↑ → Drive current ↓ → Output charges / discharges slowly → Delay ↑.

NUMERICAL EXAMPLE

Given : 2-input NAND gate
Input slew = 40 ps
Load C_L = 20 fF
(From library) t_{pd} (rise) = 95 ps
t_{pd} (fall) = 70 ps
Average delay = (95 + 70) / 2 = 82.5 ps
If C_L = 40 fF → rise delay = 140 ps
(Delay increased due to larger load)

PRACTICAL OBSERVATIONS

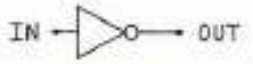
- Cell delay comes from library, depends on slew, load, PVT.
- Net delay comes from interconnect RC.
- Larger load → larger delay.
- Higher temperature → larger delay.
- Lower V_{DD} → larger delay.
- Accurate modeling of these factors is critical for STA accuracy.

VIVA QUESTIONS

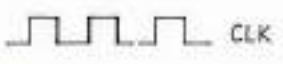
Q1. What is combinational cell delay?
Q2. What is sequential cell delay?
Q3. What is clock-to-Q delay?
Q4. What is the difference between cell delay and net delay?
Q5. List the factors affecting cell delay.
Q6. How does PVT affect cell delay?
Q7. How does load affect cell delay?
Q8. How does input slew affect delay?

QUICK REVISION

✓ Cell delay = delay inside logic cell.
✓ Sequential cell main delay = clock-to-Q.
✓ Cell delay ≠ Net delay.
✓ Delay depends on slew, load, PVT.
✓ Slow PVT → max delay.
✓ Larger load → larger delay.
✓ Larger input slew → larger delay.



STATIC TIMING ANALYSIS (STA)



1. DEFINITION OF STA

Static Timing Analysis (STA) is a method to verify whether all timing paths in a digital design meet the required timing constraints under specified operating conditions.

It analyzes circuit timing without applying any functional input vectors.

STA checks timing requirements using:
- Circuit structure (netlist)
- Timing libraries (delays)
- Timing constraints (SDC)
- Operating conditions (PVT, loads, slews)

2. WHY STA IS REQUIRED ?

- Modern designs have millions / billions of gates.
- Functional simulation requires huge input vectors and cannot guarantee all timing situations are covered.
- To ensure correct operation, every timing path must satisfy setup / hold and other timing constraints.
- STA provides exhaustive timing verification efficiently.

Key Need

Guarantee timing correctness for all possible input combinations across all paths, corners and modes.

3. PURPOSE OF STA

- Verify that timing paths meet specified constraints (setup, hold, recovery, removal, etc.).
- Identify critical paths and report slack.
- Ensure reliable operation across all conditions (PVT, voltage, load, temperature).
- Guide timing optimization and closure.
- Sign-off the design before tape-out.

Typical Checks Performed by STA

- Setup timing
- Hold timing
- Recovery / Removal
- Max transition (slew)
- Max capacitance
- Max fanout
- Min pulse width
- Clock gating checks

4. STA vs FUNCTIONAL SIMULATION

Parameter	Static Timing Analysis (STA)	Functional Simulation
Method	Analyzes timing without applying input vectors.	Applies input vectors and simulates logic behavior.
Input Vector	Not required	Required (testbench)
Coverage	100% of the timing paths (exhaustive)	Limited to applied vectors (not exhaustive)
Purpose	Verify timing correctness and report slack.	Verify functional correctness of logic.
Speed	Very fast (minutes to hours)	Slow for large designs (hours to days)
Memory Usage	Relatively low	High
Finds Timing Violations ?	Yes	No (unless additional timing simulation)
Depends on	Netlist, timing libs, constraints	Netlist + testbench vectors
Result	Slack, critical path, timing reports	Waveforms, logic outputs

Key Point :

Functional simulation proves "what the circuit does",
STA proves "whether it meets timing".

5. STA vs DYNAMIC TIMING ANALYSIS (DTA)

Parameter	STA (Static Timing Analysis)	DTA (Dynamic Timing Analysis)
Analysis Type	Static (analytical)	Dynamic (event driven)
Input Requirement	No input vectors	Requires input vectors
Nature	Pessimistic by nature	More accurate
Speed	Very fast	Slower
Coverage	100% of paths	Only exercised paths
Use	Sign-off timing verification	Validation, debug, small blocks
Complexity	Lower	Higher
Typical Tools	PrimeTime, Tempus, OpenSTA	VCS, Xcelium, Verilator (SDF annotated)
Result	Slack, path delay, reports	Waveforms with delays

Key Observation :

STA is used for sign-off timing closure.
DTA is used for timing validation on limited scenarios.

How STA Works (Conceptual View)



STA ensures all paths satisfy timing constraints for all operating conditions.

PRACTICAL OBSERVATIONS

- STA is the primary sign-off method used in industry.
- It is vectorless and provides 100% path coverage.
- STA can identify worst case delay even without applying any inputs.
- STA considers variations in PVT, load and slew through libraries.
- Good constraints (SDC) are essential for accurate STA.
- Negative slack means timing requirement is violated.

VIVA QUESTIONS

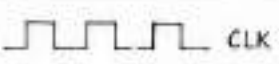
Q1. What is Static Timing Analysis (STA)?
Q2. Why is STA required in digital design flow?
Q3. What is the main purpose of STA ?
Q4. How is STA different from functional simulation?
Q5. What is the main advantage of STA over functional simulation ?
Q6. How is STA different from Dynamic Timing Analysis (DTA) ?

QUICK REVISION

- ✓ STA verifies timing without input vectors.
- ✓ It ensures all paths meet timing constraints.
- ✓ STA is fast, exhaustive and used for sign-off.
- ✓ Functional simulation checks logic, not timing.
- ✓ DTA uses vectors; STA is vectorless.
- ✓ STA reports slack for each timing path.



STATIC TIMING ANALYSIS (STA)



1. ADVANTAGES OF STA

STA provides several advantages over other verification methods.

- No test vectors required → analyzes all possible paths.
- Exhaustive coverage → 100% of timing paths are checked.
- Very fast → completes in minutes/hrs even for large designs.
- Early detection of timing issues before tape-out.
- Works at RTL, Gate-level (pre/post layout) and sign-off levels.
- Easy to integrate with design flow (uses libraries + SDC).
- Provides detailed reports → delays, slack, critical paths.
- Helps in timing optimization and closure.

> Key Advantage : STA verifies timing correctness without knowing how the circuit behaves functionally.

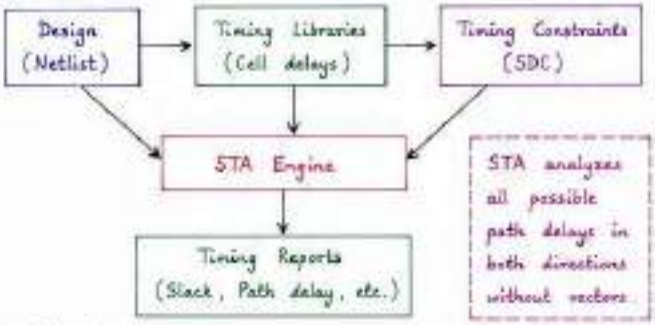
2. LIMITATIONS OF STA

- Does not detect functional errors (logic bugs).
- Results depend on quality of constraints (SDC).
- Pessimism can occur due to clock uncertainty, OCV etc.
- Cannot model all analog / dynamic behaviors.
- Assumes fixed delays from libraries → may not capture noise, IR drop, aging accurately.
- Requires accurate libraries and parasitic data.

> STA is not a replacement for functional verification, it only checks whether timing requirements are met.

3. TIMING VERIFICATION WITHOUT TEST VECTORS

STA builds a timing graph of the design and computes timing using library delays and constraints. It does not apply any input vectors.



STA analyzes all possible path delays in both directions without vectors.

How it works :

- STA propagates arrival / required times from startpoints to endpoints for all valid paths.
- It uses mathematical models of cell delays (rise/fall) and interconnect delays.
- It checks timing conditions for setup / hold at all capture points.

> Thus STA can verify timing for every possible input combination without applying any stimulus.

4. BEST-CASE AND WORST-CASE TIMING

Timing of a path can vary due to process, voltage, temperature, load and input transition. Hence STA evaluates best and worst cases.

Best-Case (Early) Timing

Data arrives earliest at the endpoint.

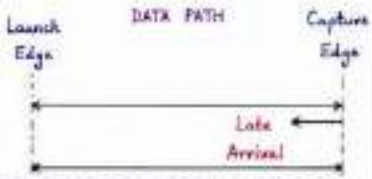
→ Used for HOLD check.



Worst-Case (Late) Timing

Data arrives latest at the endpoint.

→ Used for SETUP check.



> Key Point :
> Setup → check using worst-case (max delay).
> Hold → check using best-case (min delay).

5. TIMING CLOSURE CONCEPT

Timing closure is the process of ensuring that all timing paths meet the required constraints with acceptable slack.

Typical Objectives :

- ✓ All setup and hold slacks ≥ 0.
- ✓ No critical timing violations.
- ✓ Meet performance target.
- ✓ Achieve sign-off quality.

Steps involved :

1. Analyze timing (identify violations)
2. Find critical paths
3. Optimize design (cells, buffering, placement, routing, constraints)
4. Re-analyze
5. Iterate until closure achieved

> Goal : Positive slack for all paths under all required conditions.

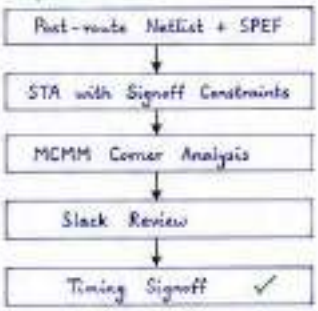
6. TIMING SIGNOFF CONCEPT

Timing signoff is the final verification step to declare the design meets timing requirements across all signoff scenarios.

Signoff Checks Include :

- Setup timing
- Hold timing
- All required PVT corners
- All RC corners (SPEF based)
- All modes (functional/scan etc.)
- All clocks and constraints
- Acceptable slack margins

Signoff Flow :



> Timing Signoff = Design is ready for Tape-out

COMPARISON TABLE

Feature	Best-Case Timing	Worst-Case Timing
Meaning	Earliest possible arrival	Latest possible arrival
Used For	Hold check	Setup check
Delay Used	Minimum delay path	Maximum delay path
Slack Type	Hold slack	Setup slack
Risk	Data changes too early	Data arrives too late

VIVA QUESTIONS

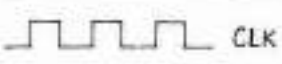
Q1. What are the main advantages of STA ?
Q2. What are the limitations of STA ?
Q3. How does STA verify timing without test vectors ?
Q4. Why do we check best-case timing for hold and worst-case timing for setup ?
Q5. What is timing closure ?
Q6. What are the major checks in timing signoff ?

QUICK REVISION

- ✓ STA verifies timing without input vectors.
- ✓ Best-case → used for Hold (early arrival).
- ✓ Worst-case → used for Setup (late arrival).
- ✓ STA is fast and exhaustive.
- ✓ Timing closure ensures all paths have non-negative slack.
- ✓ Timing signoff confirms the design meets all timing requirements across all conditions.



STATIC TIMING ANALYSIS (STA)



1. WHERE STA FITS IN THE VLSI DESIGN FLOW

STA is applied at multiple stages of the physical design flow to check and improve timing progressively.



STA is run repeatedly through these stages to detect violations early and fix them.

2. RTL STAGE

- STA at RTL is not based on gate delays.
- Designers specify timing intent using constraints (SDC).
- Used to check basic feasibility, clock definitions, I/O delays.
- Helps in identifying impossible or unrealistic constraints.

3. SYNTHESIS

- RTL is converted to gate-level netlist using logic synthesis.
- STA is run using cell delays from timing libraries.
- Provides estimated delays for all paths.
- Reports early setup/hold issues and guides optimization.

4. FLOORPLANNING

- Define die area, macro placement, power plan, I/O locations.
- Affects wirelength and congestion.
- Although no detailed timing, it impacts final delays.
- Good floorplan → shorter wires → better timing.

5. PLACEMENT

- Cells are placed at specific locations.
- STA is run with estimated interconnect delays.
- Gives more realistic delays than synthesis stage.
- Helps to fix violations by cell sizing, buffering, re-placement.

KEY POINTS:

- ✓ STA accuracy increases as we move from RTL → Signoff.
- ✓ Early STA helps early detection and reduces cost.
- ✓ Final signoff is based on post-route STA.

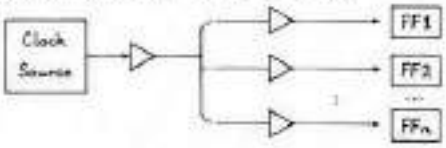
6. PRE-CTS TIMING

- Performed after placement and before clock tree synthesis.
- Clock network is ideal (zero skew).
- Data paths use estimated wire delays (RC estimation).
- Used to identify combinational timing issues and fix before CTS.



7. CTS (CLOCK TREE SYNTHESIS)

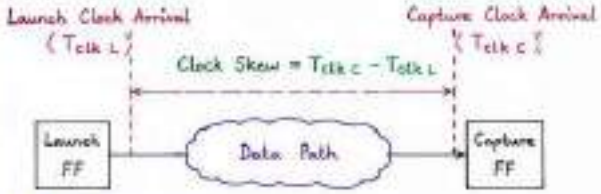
- Builds clock distribution network to all sequential elements.
- Ensures balanced insertion delay and low skew.
- Impacts both setup and hold timing.



Balanced clock tree → Low skew → Better timing

8. POST-CTS TIMING

- STA after clock tree is built.
- Includes real clock insertion delays and skew.
- More accurate than pre-CTS.
- Used to fix setup/hold violations due to clock skew.



9. ROUTING

- Final interconnect routing is performed.
- True wire lengths, layers and parasitics are extracted (SPEF).
- STA is run with extracted RC → most accurate timing.
- Fix remaining violations by buffering, resizing, re-routing.

FLOW SUMMARY TABLE

Stage	Timing Model	RC Data	Accuracy	Purpose
RTL Stage	No cell delays	None	Very Low	Timing intent check
Synthesis	Cell delays	None	Low	Initial timing check
Floorplanning	No detailed delays	None	Very Low	Plan for timing
Placement	Est. wire delays	Estimated RC	Moderate	Detect & fix violations
Pre-CTS Timing	Est. wire delays, Ideal clock	Estimated RC	Moderate	Before CTS check
CTS	Clock delays	Ideal clock tree model	Moderate	Build clock network
Post-CTS Timing	Clock + est. wires	Estimated RC	High	Fix skew related violations
Routing	Cell + actual wires	Extracted RC (SPEF)	Very High	Final accurate timing
Post-route STA (Signoff)	Cell + actual wires + clock	Extracted RC (SPEF)	Highest	Final signoff

PRACTICAL OBSERVATIONS

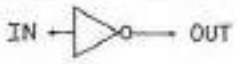
- Timing accuracy increases from RTL → Signoff.
- Early STA helps reduce iterations and design cost.
- Post-route STA is the most realistic and reliable.
- RC extraction is the key for accurate timing.
- Good physical design (placement, CTS, routing) is critical for timing closure.

VIVA QUESTIONS

Q1. At which stages is STA performed in the design flow?
Q2. What is the purpose of pre-CTS timing?
Q3. Why is post-route STA the most accurate?
Q4. What is the difference between estimated and extracted RC?
Q5. What is the role of CTS in timing?

QUICK REVISION

- ✓ STA is done at multiple stages of physical flow.
- ✓ Pre-CTS uses estimated delays, ideal clock.
- ✓ Post-CTS includes real clock delays and skew.
- ✓ Post-route STA uses extracted RC (SPEF).
- ✓ Signoff STA gives final and most accurate timing.



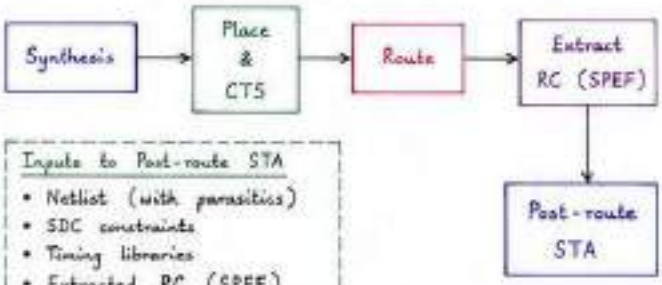
Static Timing Analysis (STA)



1. Post-Route STA

Post-route STA is performed after placement, CTS and routing are completed and interconnect parasitics are extracted.

- Uses extracted RC data (SPEF) for accurate timing.
- Provides realistic delays for cells and interconnect.
- Identifies real timing violations due to actual routing.
- Used to guide ECOs and timing closure.



Inputs to Post-route STA

- Netlist (with parasitics)
- SDC constraints
- Timing libraries
- Extracted RC (SPEF)

2. Signoff STA

Signoff STA is the final and complete timing verification performed before tape-out.

- Verifies timing across all required PVT corners and modes.
- Checks setup, hold, recovery/removal, max transition, etc.
- Uses accuracy during RC extraction and final constraints.
- Ensures the design is tape-out ready.

Typical Signoff Checks

- Setup timing
- Hold timing
- Recovery / Removal
- Max transition (slew)
- Max capacitance
- Max fanout
- Clock checks
- RC corners
- All PVT corners / modes

3. Why Timing Becomes More Accurate After Routing

Before routing, interconnect delays are only estimated. After routing, actual wires, layers and their parasitics are known.

- Real wire lengths are known.
- Actual resistance (R) and capacitance (C) of wires are extracted.
- Coupling and crosstalk effects are considered.
- Buffer insertion and clock tree delays are final.
- Hence timing numbers become much closer to silicon behavior.



4. Estimated Wire Delay vs Actual RC Delay

Feature	Estimated Wire Delay (Pre-Route)	Actual RC Delay (Post-Route)
Based On	Wirelength estimate	Actual routed wires
RC Data	Average R, C per unit length	Extracted R, C (SPEF)
Accuracy	Lower accuracy	High accuracy
Variation	Does not include coupling	Includes coupling & crosstalk
Typical Use	Early analysis	Final timing signoff
Impact	May hide violations	Real violations are seen

Key Point:

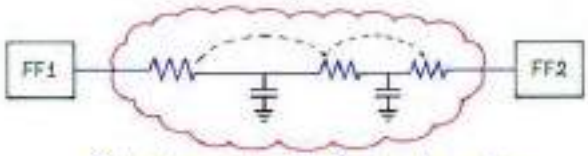
Estimated delay = $k \times$ length (k = avg. delay/unit length)

Actual RC delay = $f(R, C,$ topology, coupling, layer, etc.)

5. Physical Timing Analysis

Physical timing analysis (PTA) uses physical information of the design to compute timing.

- Includes placement, routing, extracted parasitics.
- Considers real interconnect effects.
- Gives most accurate arrival and required times.
- Basis for signoff timing closure.



Real interconnect with R, C and coupling

PTA Inputs

- Netlist with parasitics (SPEF)
- SDC constraints
- Timing libraries
- Physical information
- PVT conditions

Benefits

- Realistic timing numbers.
- Lower pessimism.
- Accurate critical path identification.
- Reliable timing closure and signoff.

6. Summary Comparison

Aspect	Pre-Route STA	Post-Route STA	Signoff STA
Stage	Before Routing	After Routing	Final (Tape-out)
RC Data	Estimated	Extracted (SPEF)	Signoff Extracted
Accuracy	Low / Moderate	High	Highest
Corner Coverage	Limited	Most Corners	All Required PVT/RC
Purpose	Early check, guide implementation	Find real violations, closure	Final verification & Tape-out
Used For	Planning	Optimization	Signoff

Practical Observations

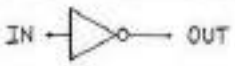
- Post-route STA usually shows more violations than pre-route STA.
- Wire RC extraction is the biggest reason for delay increase.
- Shielding, layer selection and routing topology affect timing.
- Good placement + CTS + routing → better timing.
- Signoff STA must pass all checks with required margins.

Viva Questions

Q1. What is post-route STA?
Q2. What is signoff STA?
Q3. Why does timing become more accurate after routing?
Q4. What is the difference between estimated wire delay and actual RC delay?
Q5. What is physical timing analysis?
Q6. What is the benefit of signoff STA?

Quick Revision

- ✓ Post-route STA → after routing with extracted RC.
- ✓ Signoff STA → final verification before tape-out.
- ✓ Timing more accurate after routing due to real RC extraction.
- ✓ Estimated delay uses average wire parameters.
- ✓ Physical timing analysis uses real interconnect + parasitics.
- ✓ Signoff ensures design meets all timing requirements.



Static Timing Analysis (STA)

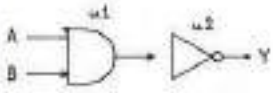


1. Inputs to STA

STA uses design, timing and physical information as inputs to build accurate timing model.

① Gate-Level Netlist

- Structural description of the design after synthesis.
- Contains cells, instances and their connections.
- Basis for building timing graph.



② SDC Constraints

- Defines clocks, input/output delays, uncertainties.
- Specifies false paths, multicycle paths, max transition, max capacitance, max fanout etc.
- Drives timing checks (setup, hold, recovery, removal).

Examples
set_clock
set_input_delay
set_output_delay
set_false_path
set_multicycle_path

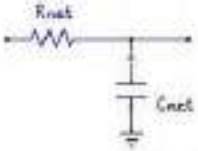
③ Timing Library (.lib)

- Characterizes timing of each cell.
- Provides delay, transition, constraint tables.
- Includes setup/hold, clk-to-q, recovery/removal, power, area etc.

Cell : INVX1
Cell : AOI21X1
Cell : DFFRX1
...

④ SPEF (Parasitic Information)

- Standard Parasitic Exchange Format.
- Contains extracted R, C for all nets after routing.
- Used for accurate interconnect delay calculation.



⑤ Parasitic Information (Extra Details)

- Coupling capacitance, crosstalk aggressors (if enabled).
- Layer, length, width information.
- Wire shielding / ground proximity.

Rwire
Cground
Ccoup
Length, Layer
...

Basic STA Flow

Read Inputs (Netlist, SDC, .lib, SPEF)
↓
Build Timing Graph
↓
Annotate Delays (Cell + Net Delays)
↓
Calculate Arrival Time (AT)
↓
Calculate Required Time (RT)
↓
Compute Slack (RT − AT)
↓
Generate Timing Reports
↓
Identify Violations / Critical Paths

During Analysis
1. For each startpoint → endpoint path
2. Compute AT & RT
3. Slack = RT − AT
4. If Slack < 0 → Violation
   else → Timing met

Key Formula
Slack = Required Time − Arrival Time
Slack > 0 → Pass
Slack = 0 → Critical
Slack < 0 → Fail

Practical Observation
- Quality of inputs directly impacts STA accuracy.
- Missing constraints or wrong libs → wrong timing results.
- SPEF is the most important input for post-route/signoff STA.

2. Outputs of STA

STA produces reports that help understand timing status and identify violations.

① Timing Reports

- Summary of timing for the design.
- Number of paths analyzed, violated, worst slack.
- Setup / Hold / Recovery / Removal reports.

Timing Summary Report

Check Type	Total Paths	Failed Paths	WNS	TNS
Setup	125430	12	-0.065 ns	-0.185 ns
Hold	125430	3	-0.032 ns	-0.090 ns
Recovery	5400	0	+0.120 ns	0.000 ns
...	...	...	...	...

② Violating Paths

- Lists paths that do not meet timing.
- Shows startpoint, endpoint, slack and path delay.

Path #	Startpoint	Endpoint	Type	Slack	Path Delay
1	regA/Q	regB/D	Setup	-0.065	1.965 ns
2	regC/Q	regD/D	Setup	-0.028	2.012 ns
3	regE/Q	regF/D	Hold	-0.032	0.032 ns
...	...	...	...	...	...

③ Slack Reports

- Provides slack distribution across all paths.
- Helps in understanding timing health.

Slack Distribution (Setup)

Slack Range (ns)	Number of Paths	% of Paths
Slack < 0	12	0.01 %
0 ≤ Slack < 0.1	854	0.68 %
0.1 ≤ Slack < 0.5	32560	25.92 %
0.5 ≤ Slack < 1.0	58300	46.73 %
Slack ≥ 1.0	35904	28.66 %

④ Path Delay Information

- Detailed breakdown of cell delays and net delays.
- Arrival time, required time, slack at each point.

Path Detail Example (Setup)

Point	Element	Cell/Net Delay (ns)	Arrival Time (ns)	Slack (ns)
Start	regA/Q	-	0.000	-
1	net1	0.120	0.120	-
2	u1 (NAND2)	0.350	0.470	-
3	net2	0.180	0.650	-
4	u2 (INV)	0.150	0.800	-
End	regB/D	-	0.800	0.065 (Setup)

Practical Observation
- Reports help designers identify, debug and fix timing issues.
- Slack reports show overall timing health.
- Path delay information is essential for root cause analysis.

Viva Questions

Q1. What are the main inputs to STA ?
Q2. Why is the timing library (.lib) required ?
Q3. What is the importance of SPEF in STA ?
Q4. What kinds of outputs are produced by STA ?
Q5. How do slack reports help in timing analysis ?
Q6. What is the purpose of violating paths report ?
Q7. What happens if SDC constraints are incorrect ?
Q8. What is the basic flow of STA ?

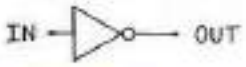
Quick Revision

- ✓ STA needs netlist, SDC, library, SPEF as inputs.
- ✓ Netlist builds the timing graph of the design.
- ✓ Library provides cell delays and timing arcs.
- ✓ SPEF provides real interconnect RC for accuracy.
- ✓ STA outputs timing summary, violations, slack and path details.
- ✓ Slack > 0 → pass, Slack = 0 → critical, Slack < 0 → fail.
- ✓ Good inputs → accurate STA results.

Key Takeaway

STA accuracy =
Good Constraints + Good Library
+ Accurate Parasitics
+ Correct Netlist

Better inputs → Better timing results
→ Better design closure



Static Timing Analysis (STA)



1. Definition of Timing Path

A timing path is a logical path through a circuit, along which a signal propagates from a point where it is launched to a point where it is captured, under the control of a clock.

> Key Idea :
> STA analyzes all possible timing paths to ensure that data arrives at the required time window for correct operation.

2. Start Point

- The beginning point of a timing path.
- Source from where data originates.
- Can be :
  1. Q output of a sequential element (FF / latch)
  2. Primary input port
  3. Output of a clock gating cell
  4. Output of a register with asynchronous control

> Example : Q of FF1 , Primary input A

3. End Point

- The terminating point of a timing path.
- Destination where data is captured or observed.
- Can be :
  1. D input of a sequential element
  2. Primary output port
  3. Input of asynchronous control pin
  4. Input of memory (address / data)

> Example : D of FF2 , Primary output Y

4. Launch Point

- The exact pin where data launches from the start point.
- For sequential elements → Q pin
- For primary input → port itself
- For combinational start → output pin of the cell

> Indicates the moment when data begins to propagate on the path.

5. Capture Point

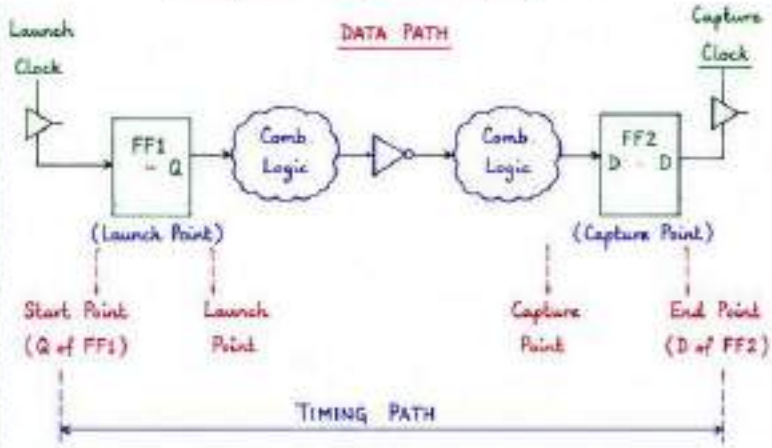
- The exact pin where data is captured at the end.
- For sequential elements → D pin
- For primary output → port itself
- For asynchronous input → specific control pin

> Data must arrive before the capture clock edge (Setup) or not violate hold window.

6. Launch Clock

- The clock that controls the launch point.
- Determines the reference edge from which data is launched.
- Defines the start time (t = 0) for timing calculation.
- Clock edge considered :
  → Rising edge or Falling edge (based on clock definition)
- Any latency, uncertainty and skew of this clock affects arrival / required time.

Conceptual Timing Path Diagram



Summary Table

Term	Description	Typical Location	Controlled By
Start Point	Beginning of timing path	Q of FF, PI, clock gating cell out	Launch Clock
Launch Point	Exact pin where data launches	Q pin of FF, output pin	Launch Clock
End Point	Termination of timing path	D of FF, PO, async pin	Capture Clock
Capture Point	Exact pin where data is captured	D pin of FF, async pin	Capture Clock
Launch Clock	Clock that launches data	Clock to start point	Clock network

Example – Timing Path Identification

- Path : FF1 → Logic → FF2
- Start Point : Q of FF1
- Launch Point : Q pin of FF1
- End Point : D of FF2
- Capture Point : D pin of FF2
- Launch Clock : Clock driving FF1 (Launch FF)
- Capture Clock : Clock driving FF2 (Capture FF)

> This path is checked for Setup and Hold timing.

Practical Observations

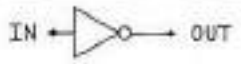
- Correct identification of start / end points is crucial for accurate STA.
- Wrong clock assignment or wrong pin selection leads to false violations.
- Launch / Capture concept is the foundation for all timing checks (Setup, Hold, etc.)
- Tools automatically detect all possible valid timing paths.

Viva Questions

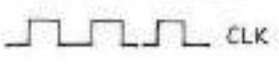
Q1. What is a timing path in STA ?
Q2. What is the difference between start point and launch point ?
Q3. What is the difference between end point and capture point ?
Q4. What is launch clock and why is it important ?
Q5. Give an example of a valid timing path.

Quick Revision

- ✓ Timing path = Start → Launch → Data Path → Capture → End
- ✓ Start Point = where path begins.
- ✓ End Point = where path ends.
- ✓ Launch Point = exact pin where data leaves start.
- ✓ Capture Point = exact pin where data is captured.
- ✓ Launch Clock = clock that triggers the start.

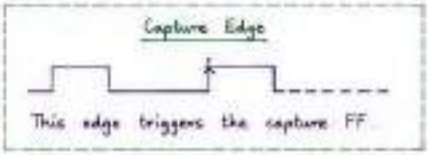


Static Timing Analysis (STA)



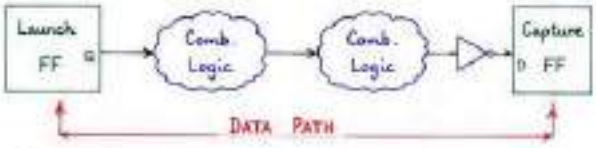
1. Capture Clock

- The clock that controls the capture (receiving) sequential element.
- Determines the time at which data must be stable.
- Defines the capture edge for setup / hold checks.
- Any delay, uncertainty or skew on this clock affects required time calculation.



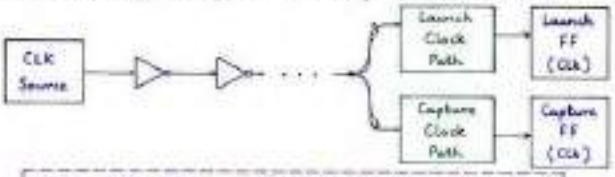
2. Data Path

- The path through which the data signal propagates from the launch point to the capture point.
- Consists of combinational logic, interconnect delays and any buffers / inverters.
- Determines the arrival time of data at the capture point.
- Total Data Path Delay = $\sum$ (cell delays + net delays)



3. Clock Path

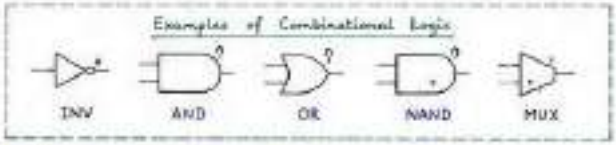
- The path taken by the clock signal from the clock source to the sequential element (either launch or capture).
- Consists of clock tree buffers / inverters and interconnect.
- Determines the clock arrival time at that sequential element.
- Affects timing through clock latency and clock skew.



Clock Path Delay = $\sum$ (buffer delays + net delays)

4. Combinational Logic

- Logic elements whose outputs depend only on present inputs.
- No memory element / clock storage.
- Includes gates (AND, OR, NAND, NOR, INV, MUX, etc.), buffers and inverters.
- Their propagation delays form a part of data path delay.

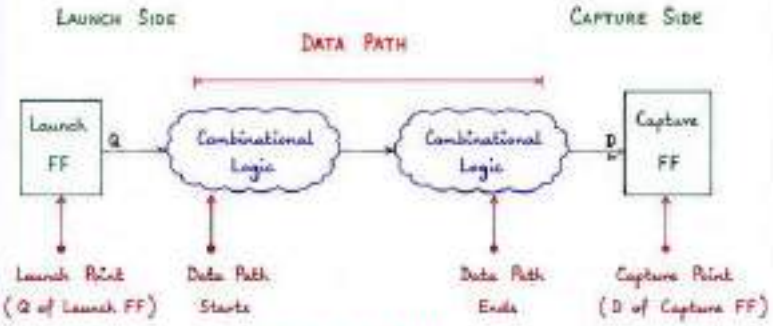


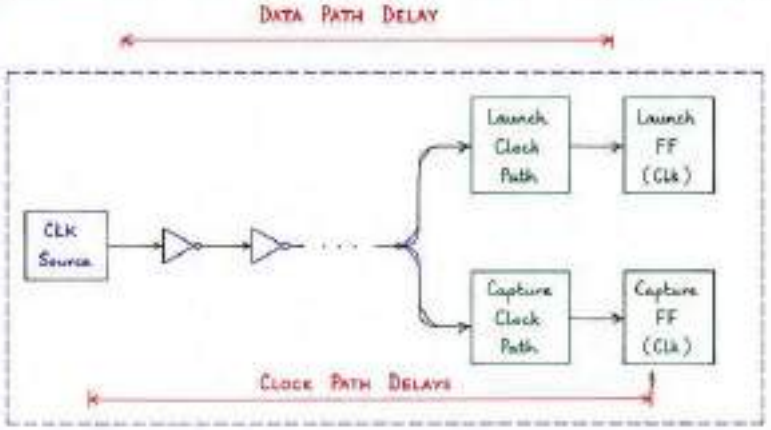
5. Sequential Elements

- Elements that store state and change output only on active clock edge.
- Examples : Flip-Flop (D, T, JK), Latch (level sensitive).
- Have timing specs like clk-to-q, setup, hold, recovery, removal.
- Form the start and end points of timing paths.



Conceptual Timing Path Diagram





Key Idea :

Data must travel from Launch Point to Capture Point within the time allowed by the Capture Clock.

Relation Between Data Path and Clock Path

- Data arrival time depends on data path delay + launch clock arrival.
- Required time depends on capture clock arrival.
- Clock skew = Capture Clock Arrival − Launch Clock Arrival.
- Skew < 0 (capture early) → harder for setup.
- Skew > 0 (capture late) → easier for setup.

Arrival Time (AT) = Launch Clock Arrival + Data Path Delay
Required Time (RT) = Capture Clock Arrival − Setup Time
Setup Slack = RT − AT

Summary Table

Term	Definition	Role in Timing Analysis	Affects
Capture Clock	Clock that triggers capture FF	Defines required time at capture point	Required time, Slack
Data Path	Path through which data travels	Determines arrival time of data	Arrival time
Clock Path	Path taken by clock signal	Determines clock arrival time	Skew, Slack
Combinational Logic	Logic without memory, depends on inputs	Contributes cell delay in data path	Arrival time
Sequential Elements	Elements with memory, clocked	Form start / end points of timing path	All timing checks

Practical Observations

- Incorrect definition of start / end points leads to wrong paths.
- Clock skew can make or break timing even with same data path delay.
- Combinational logic delay dominates critical path.
- Sequential elements define timing window boundaries.

Viva Questions

Q1. What is capture clock ?
Q2. What is the difference between launch point and capture point?
Q3. What is data path ? What does it include ?
Q4. What is clock path ? Why is it important ?
Q5. What is combinational logic ? Give examples.
Q6. What are sequential elements ? Give examples.
Q7. How does clock skew affect timing ?

Quick Revision

- ✓ Capture clock controls capture FF and required time.
- ✓ Data path carries data from launch point to capture point.
- ✓ Clock path carries clock to sequential elements.
- ✓ Combinational logic adds cell delay to data path.
- ✓ Sequential elements store state and set timing boundaries.
- ✓ All these together determine arrival time, required time and slack.

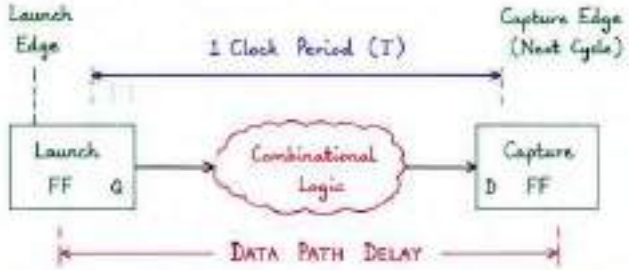
IN → OUT

Static Timing Analysis (STA)

CLK

1. Single-Cycle Path

- Data launched at one clock edge must be captured at the next active edge of the capture clock.
- This is the most common timing path.
- Available time = 1 clock period (T).

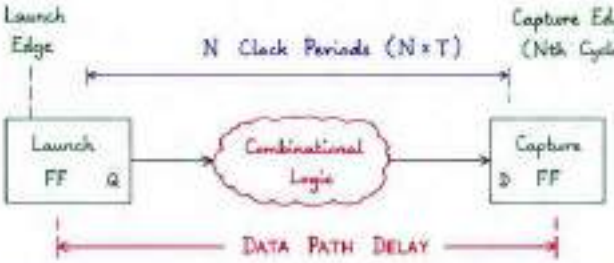


Required Time available = T − (capture clock latency + setup time)

Slack = Required Time − Arrival Time

2. Multi-Cycle Path

- Data is allowed to take more than one clock cycle to reach the destination.
- Used for paths with long delays or lower priority paths.
- Specified using multi-cycle path constraint (N cycles).
- Available time = N clock periods (N × T).



Required Time available = N × T − (capture clock latency + setup time)

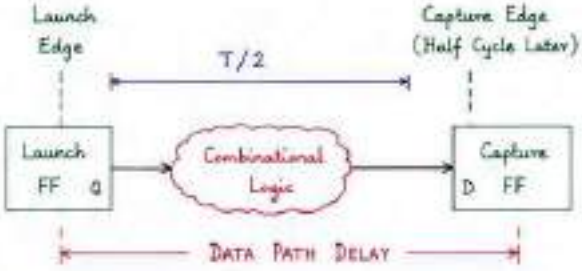
Slack = Required Time − Arrival Time

Example : If N = 3

- Data launched at cycle 0 can be captured at cycle 3.
- Available time = 3 × T.
- If delay exceeds 1×T but < 3×T → timing is acceptable.

3. Half-Cycle Path

- Data must be captured within half of the clock period.
- Used in time-borrowing structures or when capture edge occurs in the same cycle but half period later.
- Available time = T/2.



Required Time available = (T/2) − (capture clock latency + setup time)

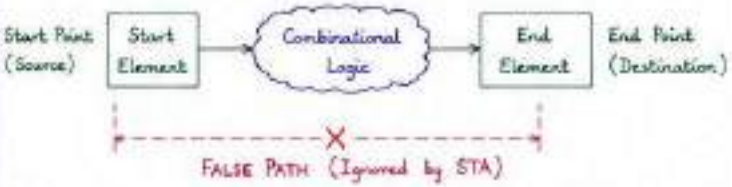
Slack = Required Time − Arrival Time

Example :

- Clock period T = 2 ns
- Half-cycle = 1 ns
- If path delay = 0.7 ns → slack = 1 − 0.7 = +0.3 ns ✓
- If path delay = 1.1 ns → slack = 1 − 1.1 = −0.1 ns ✗

4. False Path

- Paths that should not be analyzed for timing.
- No timing requirement exists between start and end points.
- Examples :
  1. Between different clock domains
  2. Asynchronous reset to data pin
  3. Test logic or scan configuration paths



No timing check is performed.

Path is excluded from timing analysis using false path constraint.

Comparison of Timing Path Types

Path Type	Capture Edge	Available Time	Purpose / Use	Constraint Command (SDC)
Single-Cycle Path	Next Active Edge	1 × T	Default timing paths	(Default)
Multi-Cycle Path	Nth Active Edge	N × T	Long paths / Lower priority paths	set_multicycle_path N
Half-Cycle Path	Half Period Later	T / 2	Time-borrowing / Early capture	set_multicycle_path 0.5
False Path	Not Applicable	Not Applicable	Ignore paths (no timing requirement)	set_false_path

Practical Observations

- Single-cycle paths dominate timing closure.
- Multi-cycle paths reduce unnecessary violations on non-critical long paths.
- Half-cycle paths are used carefully; not all tools support 0.5 cycle easily.
- False paths must be used correctly; overuse can hide real timing issues.

Viva Questions

Q1. What is a single-cycle path ?

Q2. When do we use multi-cycle path ?

Q3. What is half-cycle path ? Give an example.

Q4. What is a false path ? Give examples.

Q5. How does multi-cycle path affect required time ?

Q6. Which SDC command is used for false path ?

Quick Revision

- ✓ Single-cycle path → next active edge, time = T
- ✓ Multi-cycle path → Nth edge, time = N × T
- ✓ Half-cycle path → half period later, time = T/2
- ✓ False path → no timing check, ignored by STA
- ✓ Choose correct path type → accurate timing and efficient closure

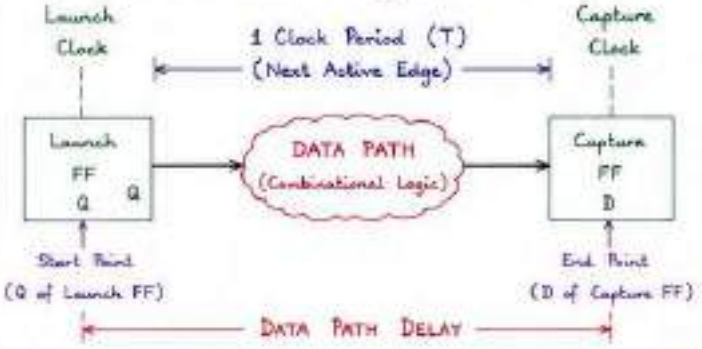
IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

1. SYNCHRONOUS PATH

- Start and end points are driven by the same clock domain.
- Launch edge of one FF and capture edge of another FF are related to the same clock.
- Data launched at one clock edge must be captured at the next valid active edge.
- Default and most common timing path.



Required Time (Setup) = T − (Capture Clock Latency + Setup Time)
Required Time (Hold) = Capture Clock Latency + Hold Time
Slack (Setup) = Required Time − Arrival Time
Slack (Hold) = Arrival Time − Required Time (Hold)

2. ASYNCHRONOUS PATH

- Start and end points are in different clock domains.
- Clocks are not phase related.
- No predictable capture edge relationship.
- Either ignore the path using false path or use special synchronizer / CDC structures.
- Common in CDC (clock domain crossing) paths.



Key Note : There is no guaranteed time window.
Path is either ignored using set_false_path or handled using synchronizer (e.g., 2-FF synchronizer).

3. DATA PATH

- The path through which the actual data signal propagates from launch point to capture point.
- Consists of combinational logic, interconnect (wires) and any buffers / inverters.
- Determines the data arrival time at the capture point.
- Total Data Path Delay = Σ (cell delays + net delays) along the path.

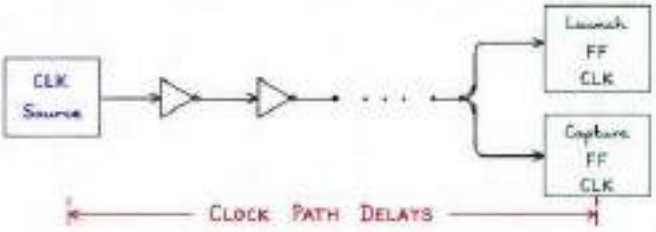


Components in Data Path

- Combinational cells (AND, OR, NAND, NOR, MUX, etc.)
- Buffers / Inverters
- Interconnect wires (with resistance & capacitance)
- Loads of next stage input ports

4. CLOCK PATH

- The path taken by the clock signal from the clock source to the sequential elements (FFs).
- Consists of clock tree buffers / inverters, interconnect wires and clock gating cells (if used).
- Determines the clock arrival time (latency) at each FF.
- Affects the required time and can introduce skew.



Components in Clock Path

- Clock tree buffers / inverters
- Interconnect wires
- Clock gating cells
- Clock pin capacitance of sequential elements

COMPARISON OF TIMING PATH TYPES

Path Type	Clock Relationship	Available Time	Used For	Typical Example	Handling in STA
Synchronous Path	Same Clock Domain	1 × T	Default timing checks (Setup & Hold)	FF to FF in same clock domain	Analyzed by default
Asynchronous Path	Different Clock Domains (No phase relation)	Not defined	CDC paths, crossings	Crossing from Clock A domain to Clock B domain	Ignore using set_false_path or use synchronizer
Data Path	N/A	N/A	Carries data from launch to capture	Combinational logic between two FFs	Delay is calculated and reported
Clock Path	From Clock Source to FFs	N/A	Delivers clock to sequential elements	Clock tree to each FF	Affects required time, skew and latency

PRACTICAL OBSERVATIONS

- Most performance is determined by synchronous paths.
- Asynchronous paths must be treated carefully to avoid false violations.
- Data path delays increase with logic depth and wire length.
- Clock path imbalance (skew) directly impacts timing margins.

VIVA QUESTIONS

Q1. What is a synchronous path?
Q2. What is an asynchronous path?
Q3. How is a data path different from a clock path?
Q4. What is the purpose of clock path analysis?
Q5. Why are asynchronous paths usually ignored?
Q6. Give an example of a CDC asynchronous path.

QUICK REVISION

✓ Synchronous path → same clock domain, 1 × T
✓ Asynchronous path → different clock domains, no fixed window.
✓ Data path → carries data between start and end.
✓ Clock path → delivers clock to sequential elements.
✓ Correct classification ensures accurate STA.

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

SETUP TIMING ANALYSIS

1. WHAT IS SETUP TIME ?

- Setup time (T_{setup}) is the minimum time that data must be stable BEFORE the capture (latching) clock edge so that the flip-flop captures correct data.
- If data violates this requirement (arrives too close to or after capture edge), FF may capture wrong value or become metastable.

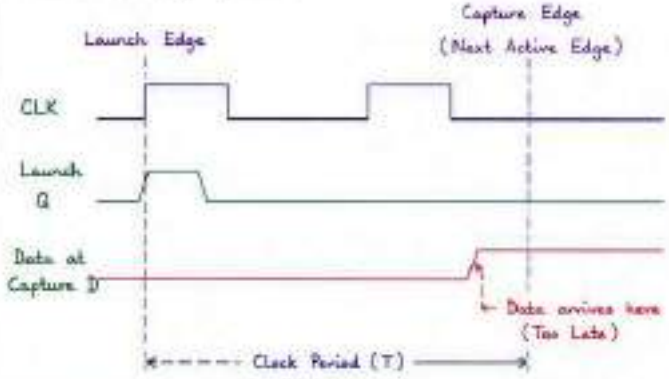
> Key Idea :
> Data must be ready and stable at the input of the capture FF at least T_{setup} time before the capture edge.

2. WHY SETUP VIOLATION OCCURS ?

- Occurs when data arrives at the capture FF too late.
- Causes :
  1. Longer than expected data path delay
  2. Increased logic / interconnect delay
  3. Clock skew in negative direction
  4. Reduced clock period
  5. High fan-out, loading or slow cells

3. DATA ARRIVING TOO LATE

- Data arrival time (AT) > Required arrival time (RT).
- Violation happens when data changes (or arrives) after the required window.
- Leads to setup failure.



> If Data changes after the required window, Setup Violation occurs.

4. LAUNCH EDGE

- The active clock edge that triggers the launching flip-flop (source) and makes data available at its Q output.
- Time reference for path delay calculation.
- Usually rising edge (↑), depending on clock definition.



5. CAPTURE EDGE

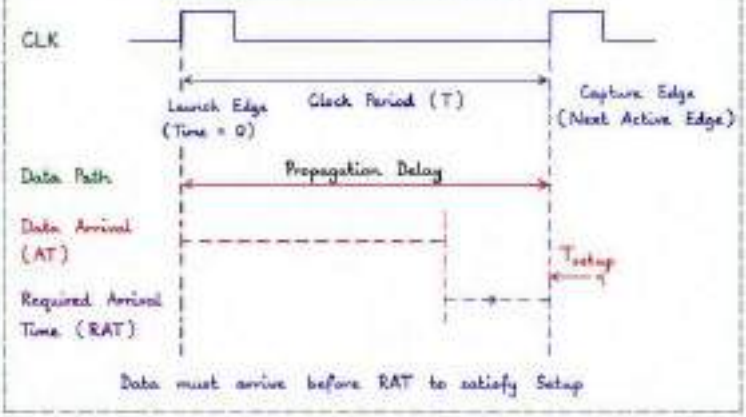
- The next active clock edge that triggers the capture flip-flop (destination) and requires data at its D input.
- Setup requirement must be met with respect to this edge.
- Defines the end of the timing window.

6. REQUIRED ARRIVAL TIME (RAT)

- It is the latest time by which data must arrive at the D input of capture FF to meet setup requirement.
- Calculated from capture edge going backwards by subtracting setup time and capture clock latency.

> Required Arrival Time (RAT)
> = Capture Edge Time − T_{setup} − $T_{clk_latency\ (capture)}$

SETUP TIMING CONCEPT DIAGRAM



- If AT ≤ RAT → Setup Requirement Met (No Violation)
- If AT > RAT → Setup Violation

KEY TIMING RELATIONSHIPS (SETUP)

Parameter	Description	Term	Equation / Relationship
T_{clk}	Clock Period	Data Arrival Time (AT)	AT = T_{launch}
T_{launch_clk}	Clock latency to launch FF	Required Arrival Time (RAT)	RAT = T_{clk} + T_{launch} − $T_{capture_clk}$ − T_{setup}
$T_{capture_clk}$	Clock latency to capture FF	Setup Slack	Slack = RAT − AT
T_{comb}	Combinational (data path) delay	Slack > 0	No setup violation
T_{setup}	Setup time of capture FF	Slack ≤ 0	Setup violation

NUMERICAL EXAMPLE

Given :
T_{clk} = 2.0 ns
T_{launch_clk} = 0.20 ns
$T_{capture}$ = 0.25 ns
T_{comb} = 1.35 ns
T_{setup} = 0.10 ns

AT = 0.20 + 1.35 = 1.55 ns
RAT = 2.0 + 0.20 − 0.25 − 0.10
= 1.85 ns
Slack = 1.85 − 1.55 = +0.30 ns ✓
(No Setup Violation)

PRACTICAL OBSERVATIONS

- Longer data path delay or negative clock skew reduces slack.
- Keep data arrival comfortably before required window.
- Setup violations cause wrong data capture or metastability.
- Tools report violating paths with negative slack.

VIVA QUESTIONS

Q1. What is setup time ?
Q2. What causes setup violation ?
Q3. What happens when data arrives too late ?
Q4. What is launch edge ?
Q5. What is capture edge ?
Q6. How is required arrival time calculated ?
Q7. What is setup slack and how is it interpreted ?

QUICK REVISION

- ✓ Setup time = data must be stable before capture edge.
- ✓ Data arriving too late → Setup violation.
- ✓ Launch edge = starts timing path.
- ✓ Capture edge = next active edge that requires data.
- ✓ Required arrival time = Capture edge − Setup time − capture clock latency.

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

SETUP TIMING ANALYSIS

1. DATA ARRIVAL TIME (AT)

- Data arrival time is the time at which data actually reaches the D input of the capture flip-flop.
- Calculated as the sum of delays from the launch point (Q of launch FF) through the data path.
- Includes clock-to-Q delay of launch FF and all interconnect, logic and net delays on the data path.



Equation (Arrival Time)

$AT = T_{clk,launch} + T_{clk\text{-}to\text{-}Q}\ (launch) + T_{data_path}$

2. SETUP SLACK

- Setup slack tells how much time margin exists for setup requirement.
- Positive slack → timing is safe.
- Zero slack → data arrives exactly at the required time.
- Negative slack → setup violation.

Equation (Setup Slack)

Setup Slack = Required Arrival Time (RAT) − Data Arrival Time (AT)

Slack Value	Interpretation	Status
> 0	Data arrives before required time	No violation (Safe)
= 0	Data arrives exactly at required time	Zero margin
< 0	Data arrives after required time	Setup violation

3. SETUP TIMING EQUATION

- Derived from setup requirement at capture flip-flop.

Required Arrival Time (RAT)

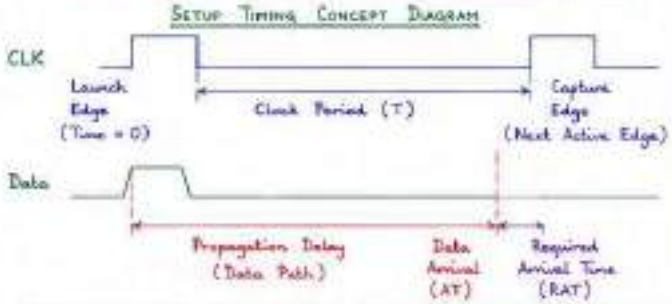
$RAT = \text{Capture Edge Time} - T_{setup} - T_{clk,latency(capture)}$

Setup Slack (Complete Form)

$\text{Setup Slack} = (\text{Capture Edge Time} - T_{setup} - T_{clk,latency(capture)}) - (T_{clk,latency(launch)} + T_{clk\text{-}to\text{-}Q} + T_{data_path})$

No Violation if : $AT \le RAT$ $(\text{Slack} \ge 0)$

SETUP TIMING CONCEPT DIAGRAM



Data must arrive before RAT to satisfy setup requirement.

4. EFFECT OF CLOCK PERIOD (T)

- Clock period determines the available time for data to travel from launch to capture.
- Larger clock period → more time → easier to meet setup.
- Smaller clock period → less time → high chance of violation.
- If T decreases, RAT decreases → slack reduces.

Effect of Clock Period on Setup

Clock Period (T)	Available Time (RAT Component)	Effect on Slack
Increase (T ↑)	RAT increases	Slack increases
Decrease (T ↓)	RAT decreases	Slack decreases

Example :

Assume $T_{setup} = 0.10$ ns, $T_{clk,lat(capture)} = 0.20$ ns

Case 1 : $T = 2.0$ ns → $RAT = 2.0 - 0.10 - 0.20 = 1.70$ ns

Case 2 : $T = 1.6$ ns → $RAT = 1.6 - 0.10 - 0.20 = 1.30$ ns

Hence, smaller T gives less time margin.

5. EFFECT OF CLOCK-TO-Q ($T_{clk\text{-}to\text{-}Q}$)

- Clock-to-Q delay of launch FF is the time from launch edge to data appearing at Q.
- If $T_{clk\text{-}to\text{-}Q}$ increases, data starts late → AT increases.
- Increased AT reduces slack and may cause setup violation.
- If $T_{clk\text{-}to\text{-}Q}$ decreases, data starts early → slack improves.

Effect of Clock-to-Q on Setup

Clock-to-Q ($T_{clk\text{-}to\text{-}Q}$)	Data Arrival (AT)	Effect on Slack
Increase	AT increases	Slack decreases
Decrease	AT decreases	Slack increases

Example : (Fixed path and clock)

Given : $RAT = 1.70$ ns, $T_{data_path} = 0.90$ ns, $T_{clk,lat(launch)} = 0.20$ ns

Case 1 : $T_{clk\text{-}to\text{-}Q} = 0.30$ ns → $AT = 0.20 + 0.30 + 0.90 = 1.40$ ns
Slack = 1.70 − 1.40 = +0.30 ns ✓

Case 2 : $T_{clk\text{-}to\text{-}Q} = 0.60$ ns → $AT = 0.20 + 0.60 + 0.90 = 1.70$ ns
Slack = 1.70 − 1.70 = 0.00 ns (Zero margin)

Case 3 : $T_{clk\text{-}to\text{-}Q} = 0.90$ ns → $AT = 0.20 + 0.90 + 0.90 = 2.00$ ns
Slack = 1.70 − 2.00 = −0.30 ns ✗ (Violation)

KEY RELATIONSHIPS SUMMARY

- Data must arrive at D input before RAT.
- $RAT = \text{Capture Edge Time} - T_{setup} - T_{clk,lat(capture)}$
- $AT = T_{clk,lat(launch)} + T_{clk\text{-}to\text{-}Q} + T_{data_path}$
- Slack = RAT − AT
- Slack ≥ 0 → No setup violation.
- Slack < 0 → Setup violation.

PRACTICAL OBSERVATIONS

- Shorter clock period and higher $T_{clk\text{-}to\text{-}Q}$ are the most common reasons for setup failures.
- Optimising logic, reducing interconnect, and using faster cells improve slack.
- Proper pipelining or architectural changes help meet target frequency.
- Always analyze worst-case path for setup.

VIVA QUESTIONS

Q1. What is data arrival time ?
Q2. What is setup slack and how is it interpreted ?
Q3. Write the setup timing equation.
Q4. How does clock period affect setup timing ?
Q5. How does clock-to-Q affect setup timing ?
Q6. What happens if data arrives after required time ?
Q7. What is the condition for no setup violation ?

QUICK REVISION

- ✓ Data must be stable at D input before capture edge.
- ✓ $AT = T_{clk,lat(launch)} + T_{clk\text{-}to\text{-}Q} + T_{data_path}$
- ✓ $RAT = \text{Capture Edge} - T_{setup} - T_{clk,lat(capture)}$
- ✓ Slack = RAT − AT
- ✓ Increase T → slack increases.
- ✓ Increase $T_{clk\text{-}to\text{-}Q}$ → slack decreases.



STATIC TIMING ANALYSIS (STA)

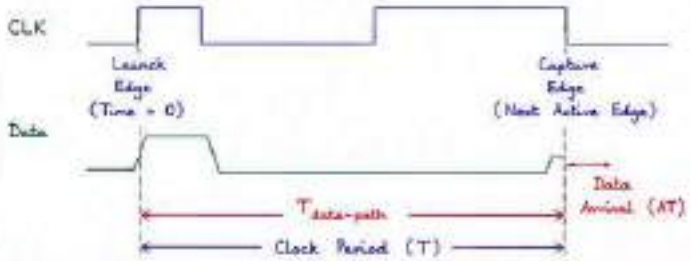


SETUP TIMING ANALYSIS

1. EFFECT OF COMBINATIONAL DELAY ($T_{data-path}$)

- Combinational delay is the total delay of logic between launch FF and capture FF.
- If combinational delay increases → data arrives later.
- Larger delay reduces slack and may cause setup violation.
- If delay decreases → data arrives earlier → slack increases.





Example : $T = 2.0$ ns, $T_{clk,lat(capture)} = 0.25$ ns, $T_{setup} = 0.30$ ns

Case 1 : $T_{data-path} = 1.30$ ns → AT = 1.30 ns → Slack = +0.35 ns ✓

Case 2 : $T_{data-path} = 1.70$ ns → AT = 1.70 ns → Slack = −0.05 ns ✗

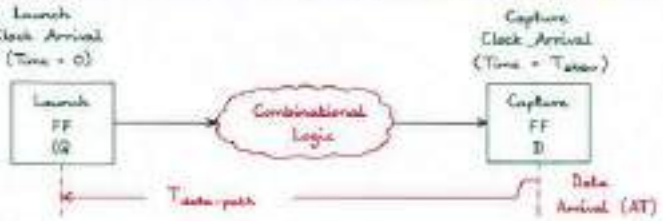
Observation : Increase in combinational delay pushes data arrival towards capture edge, reducing timing margin.

3. EFFECT OF SKEW (T_{skew})

- Skew = Capture Clock Arrival − Launch Clock Arrival
- Positive skew (capture clock late) → more time → slack increases.
- Negative skew (capture clock early) → less time → slack decreases.

RAT including skew

$RAT = (\text{Capture Edge Time} + T_{skew}) - T_{setup} - T_{clk,lat(capture)}$



Example : $T = 2.0$ ns, $T_{clk,lat(capture)} = 0.20$ ns, $T_{setup} = 0.10$ ns, $T_{data-path} = 1.90$ ns → AT = 1.90 ns

Case 1 (Positive Skew) :
RAT = (2.0 + 0.20) − 0.10 − 0.20 = 1.90 ns → Slack = +0.00 ns ✓

Case 2 (Negative Skew) :
RAT = (2.0 − 0.10) − 0.10 − 0.20 = 1.60 ns → Slack = +0.10 ns ✓

Case 3 (More Negative Skew) : $T_{skew} = -0.40$ ns
RAT = (2.0 − 0.40) − 0.10 − 0.20 = 1.30 ns → Slack = −0.50 ns ✗

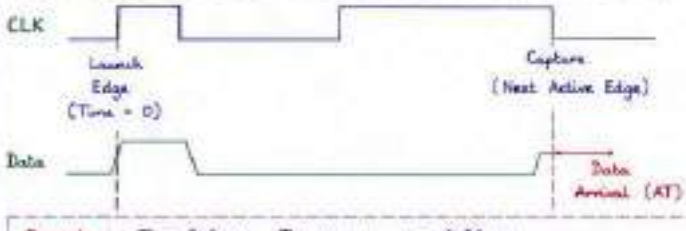
Observation : Negative skew is harmful for setup timing.

2. EFFECT OF SETUP TIME (T_{setup})

- Setup time is the minimum time data must be stable before capture edge.
- Larger setup time means data must arrive earlier.
- Increase in setup time reduces slack.
- Decrease in setup time increases slack.

Required Arrival Time (RAT)

$RAT = \text{Capture Edge Time} - T_{setup} - T_{clk,lat(capture)}$



Example : $T = 2.0$ ns, $T_{clk,lat(capture)} = 0.20$ ns

Case 1 : $T_{setup} = 0.10$ ns → RAT = 1.70 ns
AT = 1.55 ns → Slack = +0.15 ns ✓

Case 2 : $T_{setup} = 0.30$ ns → RAT = 1.50 ns
AT = 1.55 ns → Slack = −0.05 ns ✗

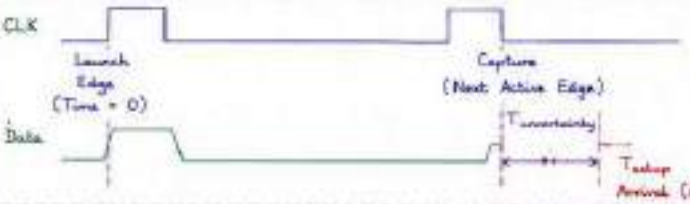
Observation : Higher setup time tightens the requirement.

4. EFFECT OF UNCERTAINTY ($T_{uncertainty}$)

- Uncertainty accounts for variations in clock, jitter, PVT, OCV, etc.
- It effectively reduces the available time.
- More uncertainty reduces slack.
- Less uncertainty increases slack.

RAT including uncertainty

$RAT = \text{Capture Edge Time} - T_{setup} - T_{clk,lat(capture)} - T_{uncertainty}$



Example : $T = 2.0$ ns, $T_{clk,lat(capture)} = 0.20$ ns, $T_{setup} = 0.10$ ns, $T_{data-path} = 1.90$ ns → AT = 1.90 ns

Case 1 : $T_{uncertainty} = 0.00$ ns
RAT = 1.70 ns → Slack = +0.30 ns ✓

Case 2 : $T_{uncertainty} = 0.10$ ns
RAT = 1.60 ns → Slack = +0.20 ns ✓

Case 3 : $T_{uncertainty} = 0.30$ ns
RAT = 1.40 ns → Slack = 0.00 ns (Zero margin)

Case 4 : $T_{uncertainty} = 0.40$ ns
RAT = 1.30 ns → Slack = −0.10 ns ✗

Observation : Uncertainty consumes available margin.

SUMMARY TABLE – EFFECTS ON SETUP SLACK

Factor	Increase in Factor	Effect on RAT (Required Time)	Effect on Slack	Reason
Combinational Delay ($T_{data-path}$)	Increases	RAT remains same, AT increases	Slack decreases	Data arrives late
Setup Time (T_{setup})	Increases	RAT decreases	Slack decreases	Need more time before capture
Skew (T_{skew})	More Negative (−ve)	RAT decreases	Slack decreases	Capture clock earlier
Uncertainty ($T_{uncertainty}$)	Increases	RAT decreases	Slack decreases	Less time guaranteed
Clock Period (T)	Increases	RAT increases	Slack increases	More time available
Clock-to-Q ($T_{clk-to-Q}$)	Increases	AT increases	Slack decreases	Data available late

PRACTICAL OBSERVATIONS

- Longer logic paths and negative skew are the major causes of setup violations.
- Increasing clock period and controlling uncertainty improve setup margin.
- Proper optimization of logic, clock tree and constraints is essential.

VIVA QUESTIONS

Q1. How does combinational delay affect setup slack?
Q2. What is the effect of increasing setup time?
Q3. What is clock skew and how does it affect setup?
Q4. How does uncertainty reduce timing margin?
Q5. Write the setup timing equation.
Q6. How does clock-to-Q affect setup timing?

QUICK REVISION

- ✓ Increase in combinational delay → slack decreases
- ✓ Increase in setup time → slack decreases
- ✓ Negative skew → slack decreases
- ✓ Increase in uncertainty → slack decreases
- ✓ More clock period → more margin
- ✓ Increase in clock-to-Q → slack decreases

IN → OUT

STATIC TIMING ANALYSIS (STA)

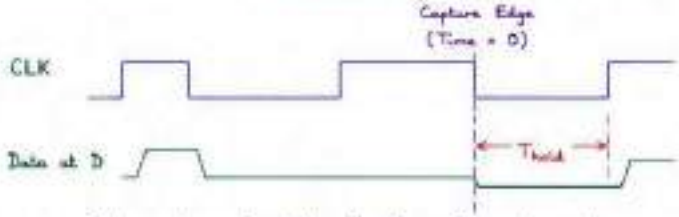
CLK

HOLD TIMING ANALYSIS

1. WHAT IS HOLD TIME ?

- Hold time (T_{hold}) is the minimum time that data must be stable AFTER the capture (latching) clock edge.
- Ensures that the capture flip-flop receives correct data and does not go metastable.
- Specified at the D input of the capture flip-flop.





Data must remain stable for T_{hold} after capture edge

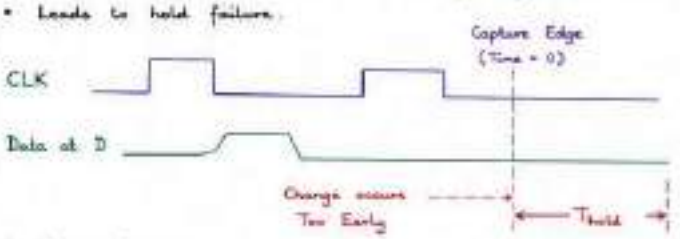
Key Idea :
After capture edge, data should not change for at least T_{hold}.
If it changes earlier → hold violation.

2. WHY HOLD VIOLATION OCCURS ?

- Occurs when data arrives at capture FF too early.
- Causes :
  1. Too small combinational delay (data path very short)
  2. Negative clock skew (capture clock early)
  3. Large launch clock latency or small capture clock latency
  4. Fast cells / strong drive
  5. Process, voltage, temperature variations

3. DATA ARRIVING TOO EARLY

- Data changes at D input before capture edge + T_{hold}.
- This means data does not remain stable for required hold time.
- Leads to hold failure.



4. HOLD SLACK

- Indicates how much margin exists for hold requirement.
- Positive slack → timing is safe.
- Zero slack → data changes exactly at required boundary.
- Negative slack → hold violation.

Hold Slack Formula

Hold Slack = Data Arrival Time (AT) − Required Time (RT)

Slack Value	Interpretation	Status
> 0	Data arrives after required time	No violation (Safe)
= 0	Data arrives exactly at required time	Zero margin
< 0	Data arrives before required time	Hold violation

5. REQUIRED TIME (RT)

- Earliest time by which data is allowed to arrive at capture FF to satisfy hold requirement.
- Calculated from capture edge going backwards by subtracting hold time and capture clock latency.

Equation (Required Time for Hold)

$$RT = \text{Capture Edge Time} - T_{hold} - T_{clk,lat(capture)}$$

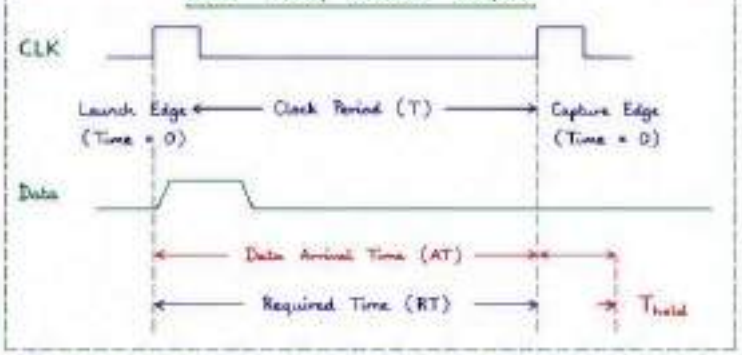
6. ARRIVAL TIME (AT)

- Actual time at which data reaches D input of capture FF.
- Calculated as sum of launch clock latency, clock-to-Q delay of launch FF and data path delay.

Equation (Arrival Time)

$$AT = T_{clk,lat(launch)} + T_{clk-to-Q\ (launch)} + T_{data-path}$$

HOLD TIMING CONCEPT DIAGRAM



AT ≥ RT → Hold requirement met (No violation)
AT < RT → Hold violation

HOLD TIMING NUMERICAL EXAMPLE

Given :

T_{clk} (Clock Period)	= 2.0 ns
$T_{clk,lat}$ (launch)	= 0.10 ns
$T_{clk-to-Q}$ (launch)	= 0.15 ns
$T_{data-path}$ (min delay)	= 0.25 ns
$T_{clk,lat}$ (capture)	= 0.20 ns
T_{hold}	= 0.10 ns

1) Arrival Time (AT) = 0.10 + 0.15 + 0.25 = 0.50 ns
2) Required Time (RT) = 0 (capture edge) − 0.10 − 0.20 = −0.30 ns
3) Hold Slack = AT − RT = 0.50 − (−0.30) = 0.80 ns ✓
(No Hold Violation)

HOLD TIMING RELATIONSHIP SUMMARY

- RT = Capture Edge Time − T_{hold} − $T_{clk,lat(capture)}$
- AT = $T_{clk,lat(launch)}$ + $T_{clk-to-Q\ (launch)}$ + $T_{data-path(min)}$
- Hold Slack = AT − RT
- Slack ≥ 0 → No hold violation
- Slack = 0 → Zero margin
- Slack < 0 → Hold violation

PRACTICAL OBSERVATIONS

- Short data path delay, negative skew or early capture clock can cause hold violations.
- Fixes include adding delay (buffers) in data path or adjusting clock network.
- Always check min delay corners for hold.
- Hold violations are more sensitive to PVT, OCV variations.

VIVA QUESTIONS

Q1. What is hold time?
Q2. Why does hold violation occur?
Q3. What happens when data arrives too early?
Q4. How is required time for hold calculated?
Q5. How is arrival time for hold calculated?
Q6. What is hold slack and how is it interpreted?

QUICK REVISION

- ✓ Hold time = minimum time data must be stable after capture edge.
- ✓ Early data arrival → hold violation.
- ✓ RT = Capture Edge − T_{hold} − $T_{clk,lat(capture)}$
- ✓ AT = $T_{clk,lat(launch)}$ + $T_{clk-to-Q}$ + $T_{data-path(min)}$
- ✓ Hold Slack = AT − RT
- ✓ Slack < 0 → Hold violation.

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

HOLD TIMING ANALYSIS

1. EFFECT OF CLOCK-TO-Q ($T_{clk\text{-}to\text{-}Q}$)

- Clock-to-Q is the delay from launching clock edge at source flip-flop to data appearing at its Q output.
- Larger $T_{clk\text{-}to\text{-}Q}$ → data appears later at start of data path.
- More time available to satisfy hold → improves hold slack.
- Smaller $T_{clk\text{-}to\text{-}Q}$ → data starts early → hold slack decreases.



Arrival Time (AT) including clock-to-Q

$AT = T_{clk\ (at\ launch)} + T_{clk\text{-}to\text{-}Q\ (launch)} + T_{data\text{-}path\ (min)}$

Effect of Clock-to-Q on Hold Slack

$T_{clk\text{-}to\text{-}Q}$ (Launch)	Arrival Time (AT)	Hold Slack (AT − RT)	Effect
Increase	AT increases	Slack increases	Safer
Decrease	AT decreases	Slack decreases	Risk of violation

2. MINIMUM DATA PATH DELAY ($T_{data\text{-}path\ (min)}$)

- For hold analysis, only minimum delay of data path is considered.
- Shorter path → data arrives earlier at capture FF.
- If minimum delay is too small, hold violation occurs.
- Buffers / intentional delay may be added to increase minimum delay.



Effect on Arrival Time

$AT \propto T_{data\text{-}path\ (min)}$

↓ Minimum delay → earlier AT → lower slack

3. EFFECT OF CLOCK SKEW (T_{skew})

- Skew = Capture clock arrival time − Launch clock arrival time.
- Positive skew (tap receive clock late) → effective required time moves right → easier to meet.
- Negative skew (capture clock early) → required time moves left → harder to meet hold.

Required Time (RT) with skew

$RT = (\text{Launch Edge Time} + T) + T_{skew} - T_{hold}$

Here, T = 0-fold cycle (same or previous edge)

$\Rightarrow RT = T_{skew} - T_{hold}$

Effect of Skew on Hold

Skew (T_{skew})	Required Time (RT)	Hold Slack	Effect
Positive (+)	RT increases	Slack increases	Safer
Zero	$RT = -T_{hold}$	Slack unchanged	Neutral
Negative (−)	RT decreases	Slack decreases	Violation risk

4. EFFECT OF UNCERTAINTY ($T_{uncertainty}$)

- Uncertainty accounts for clock jitter, PVT variations, OCV, measurement error etc.
- For hold, uncertainty is SUBTRACTED from required time (worst case).
- More uncertainty → required time reduces → slack reduces → higher chance of violation.

Required Time including uncertainty

$RT = T_{skew} - T_{hold} - T_{uncertainty}$

Effect of Uncertainty on Hold

$T_{uncertainty}$	Required Time (RT)	Hold Slack	Effect
Increase	RT decreases	Slack decreases	Risk ↑
Decrease	RT increases	Slack increases	Safer

HOLD TIMING CONCEPT DIAGRAM



AT ≥ RT → No hold violation

AT < RT → Hold violation

5. HOLD VIOLATION DIAGNOSIS

- Hold violation occurs when data changes (arrives) too early at capture FF, before hold window ends.
- Diagnose by checking: AT < RT (negative hold slack).
- Common indicators:
  1. Very small data path delay.
  2. Negative clock skew (capture clock early).
  3. Low clock-to-Q delay at launch.
  4. High uncertainty / variations.

Step-by-step Diagnosis

1. Check reported hold slack → negative?
2. Check arrival time (AT) and required time (RT).
3. Identify which factor is causing early arrival.
4. Apply fixes → add delay / buffers / adjust skew.

Hold Timing Numerical Example

Given: $T_{clk\text{-}to\text{-}Q} = 0.10$ ns, $T_{data\text{-}path\ (min)} = 0.25$ ns

$T_{skew} = -0.20$ ns, $T_{hold} = 0.10$ ns, $T_{uncertainty} = 0.05$ ns

1) $AT = 0.10 + 0.25 = 0.35$ ns

2) $RT = -0.20 - 0.10 - 0.05 = -0.35$ ns

3) Hold Slack = $AT - RT = 0.35 - (-0.35) = +0.70$ ns ✓

(No hold violation)

PRACTICAL OBSERVATIONS

- Hold violations are sensitive to small delays.
- Avoid using only fast cells on short paths.
- Add buffers / delay cells to fix violations.
- Negative skew is main reason for hold failures.
- Tools report worst-case hold slack.

VIVA QUESTIONS

Q1. How does clock-to-Q affect hold timing?

Q2. What happens if data path delay is too small?

Q3. How does positive skew affect hold slack?

Q4. How does uncertainty impact hold timing?

Q5. What is hold slack and how is it calculated?

Q6. What causes hold violation?

QUICK REVISION

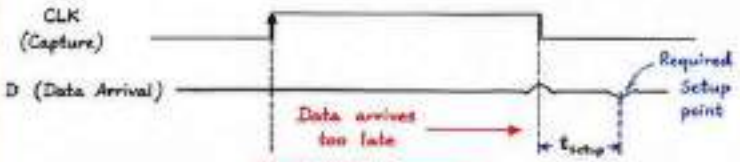
- ✓ More clock-to-Q → later arrival → higher slack.
- ✓ Shorter min data path → early arrival → violation.
- ✓ Positive skew → increases slack, Negative skew → reduces.
- ✓ More uncertainty → reduces slack.
- ✓ Attend report: AT, RT, Slack → fix if slack < 0.

STATIC TIME ANALYSIS

SETUP vs HOLD – Complete Comparison, Analysis, Fixes & Interactions

1. Data too late → Setup violation

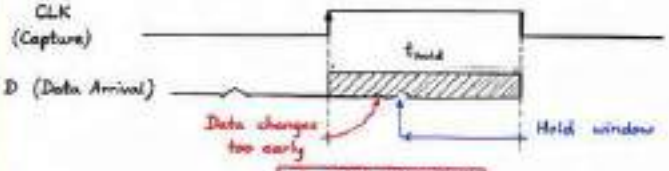
- Data arrives at the capture FF after the required setup point before the active clock edge.
- Capture FF may capture wrong / unstable data.



⇒ Setup Violation

2. Data too early → Hold violation

- Data arrives and changes at the capture FF before the hold time window after the active clock edge.
- Capture FF may go metastable or capture wrong data.



⇒ Hold Violation

3. Maximum-delay analysis (for Setup)

- Considers longest possible data path delay.
- Ensures data arrives early enough before the next active clock edge (satisfying setup time).



4. Minimum-delay analysis (for Hold)

- Considers shortest possible data path delay.
- Ensures data does not arrive / change before the hold window after the capture clock edge.



5. Setup worst-case corner

- Slow Process, High Temperature, Low Voltage (SLOW corner) → Increases cell delay.
- Maximum data path delay + Early capture clock.
- Most critical for Setup check.

Slow P + High T + Low V + Early Capture Clock

6. Hold worst-case corner

- Fast Process, Low Temperature, High Voltage (FAST corner) → Decreases cell delay.
- Minimum data path delay + Late capture clock.
- Most critical for Hold check.

Fast P + Low T + High V + Late Capture Clock

7. Setup fixes

① Reduce data path delay (buffer downsizing, logic restructuring)
② Increase capture clock latency (positive skew)
③ Reduce launch clock latency
④ Reduce clock period (if possible)
⑤ Pipeline / retime to break long paths
⑥ Use useful skew techniques

Effect: Gives more time for data to arrive.

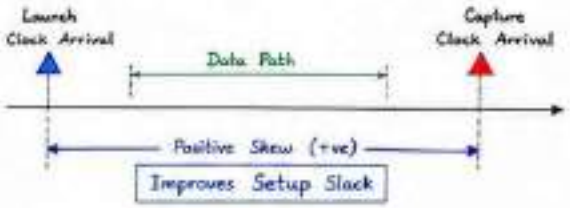
8. Hold fixes

① Increase data path delay (buffer upsizing, add delay cells)
② Reduce capture clock latency (negative skew)
③ Increase launch clock latency
④ Use hold buffers / delay chains
⑤ Physical placement adjustments to increase routing delay

Effect: Prevents data from arriving too early.

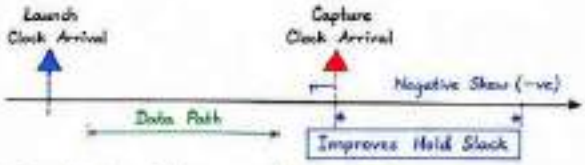
9. Setup effect of Positive Skew

- Capture clock arrives later than launch clock.
- Data has more time to arrive → Setup improves.



10. Hold effect of Negative Skew

- Capture clock arrives earlier than launch clock.
- Data has less chance to arrive early → Hold improves.



11. Why fixing Setup can hurt Hold

- Setup fix often uses positive skew (late capture clock).
- This reduces the hold window (data can arrive too early).

⇒ Setup improvement may cause hold violation.

12. Why fixing Hold can hurt Setup

- Hold fix often uses negative skew (early capture clock) or adds delay.
- This reduces time available for data to arrive.

⇒ Setup slack may reduce → Setup violation possible.

13. Setup vs Hold – Key Comparison

Aspect	Setup Check	Hold Check
Objective	Data must arrive early enough before the capture edge	Data must not change too early after capture edge
Analysis Type	Maximum-delay analysis	Minimum-delay analysis
Reference Edge	Next active capture edge	Same active capture edge
Critical Corner	Slow P, High T, Low V + Early Capture Clock	Fast P, Low T, High V + Late Capture Clock
Skew to Fix	Positive Skew (Late Capture)	Negative Skew (Early Capture)
Common Fix	Reduce data path delay or increase capture latency	Increase data path delay or reduce capture latency

14. Quick Numerical Example

(A) Setup Check (Max Delay)

Clock Period (Tclk) = 2.0 ns
Required Time = 2.0 ns
Data Arrival = 1.75 ns
Setup Time = 0.15 ns

Setup Slack = Required − Arrival
= 2.0 − (1.75 + 0.15)
= 0.10 ns ✓ (Pass)

(B) Hold Check (Min Delay)

Hold Time = 0.10 ns
Data Arrival = 0.05 ns

Hold Slack = Arrival − Hold
= 0.05 − 0.10
= −0.05 ns ✗ (Violation)

15. Practical Observation

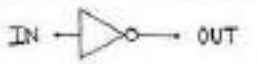
- Timing fixes must be balanced. Over-fixing one check can break the other.
- Engineers use useful skew, buffering, and optimization to satisfy both checks.
- Always validate in correct corners for Setup and Hold separately.

16. Viva Questions

Q1. What is the difference between Setup and Hold violations?
Q2. What is maximum-delay and minimum-delay analysis?
Q3. Which corners are worst for Setup and Hold?
Q4. How does positive skew help Setup?
Q5. How does negative skew help Hold?
Q6. Why does fixing Setup sometimes hurt Hold?
Q7. Why does fixing Hold sometimes hurt Setup?

17. QUICK REVISION

- Setup → Data too late
- Hold → Data too early
- Setup → Max delay analysis
- Hold → Min delay analysis
- Setup worst → Slow P, High T, Low V, Early Capture Clock
- Hold worst → Fast P, Low T, High V, Late Capture Clock
- Positive Skew → Setup better
- Negative Skew → Hold better
- Fixing one can hurt the other



Static Timing Analysis (STA)



Slack, Arrival & Required Time

Data Arrival Time (DAT) & Data Required Time (DRT) Relationship Between Them

1. Introduction

- In STA, every timing check compares two critical times at the capture flip-flop (D input).
- These are :
  1. Data Arrival Time (DAT)
  2. Data Required Time (DRT)

> DAT (Arrival) → Actual time when data reaches D input.
> DRT (Required) → Required time by which data must arrive to meet timing requirement.

2. Data Arrival Time (DAT)

- Time at which data actually arrives at the D input of capture FF.
- Calculated from launch point through the data path delays.
- Includes clock-to-Q of launch FF and all combinational delays (interconnect + logic).
- Represents the "actual" data timing.

Equation (Data Arrival Time)

$$DAT = T_{clk,lat(launch)} + T_{clk-to-Q(launch)} + T_{data-path(delay)}$$
$$= T_{launch} + T_{cq} + T_{pd(min\ or\ max)}$$

For Setup → Use MAX delay For Hold → Use MIN delay

3. Data Required Time (DRT)

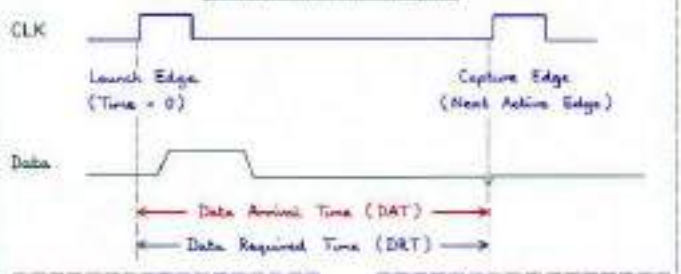
- Latest (for setup) or earliest (for hold) time by which data must arrive at D input to satisfy timing.
- Calculated backward from capture edge using clock period, clock latency and timing requirement (T_{setup} or T_{hold}).
- Represents the "allowed" data timing.

Equation (Data Required Time)

For Setup : $DRT = T_{clk,lat(capture)} + T_{period} - T_{setup}$
$= T_{cap} + T - T_{setup}$

For Hold : $DRT = T_{clk,lat(capture)} + T_{hold}$
$= T_{cap} + T_{hold}$

Timing Concept Diagram



AT = DAT (Arrival Time)
RT = DRT (Required Time)

Relationship

AT < RT → Timing Met
AT = RT → Zero Slack
AT > RT → Timing Violation

4. Relationship Between DAT & DRT

- Slack is the difference between required time and arrival time.
- Positive slack means data arrives early (safe).
- Negative slack means data arrives late (violation).

Slack Formula

$$Slack = DRT - DAT$$

Condition	Equation	Interpretation	Status
DAT < DRT	DRT − DAT > 0	Data arrives earlier than required	Positive Slack (Safe)
DAT = DRT	DRT − DAT = 0	Data arrives exactly at required time	Zero Slack (Margin = 0)
DAT > DRT	DRT − DAT < 0	Data arrives later than required	Negative Slack (Violation)

5. Example

Assume (Setup Check) :

T_{clk} (Clock Period)	= 2.00 ns
$T_{clk,lat}$ (launch)	= 0.10 ns
$T_{clk-to-Q}$ (launch)	= 0.15 ns
$T_{data-path}$ (max)	= 1.30 ns
$T_{clk,lat}$ (capture)	= 0.20 ns
T_{setup}	= 0.10 ns

Calculations

DAT = 0.10 + 0.15 + 1.30
= 1.55 ns

DRT = 0.20 + 2.00 − 0.10
= 2.10 ns

Slack = 2.10 − 1.55
= + 0.55 ns ✓

6. Numerical Comparison : Setup vs Hold

Aspect	Setup Check	Hold Check
Data considered	Max delay path	Min delay path
Required time	Latest allowed time (before next edge)	Earliest allowed time (after same edge)
DRT Calculation	$T_{cap} + T_{period} - T_{setup}$	$T_{cap} + T_{hold}$
Slack Formula	DRT − DAT (DAT using max delay)	DRT − DAT (DAT using min delay)
Violation when	DAT > DRT	DAT < DRT

7. Practical Interpretation

- STA computes both DAT and DRT for every endpoint.
- Slack decides whether the timing is safe or violating.
- Designers try to make DAT as early as possible (for setup) but not too early (for hold).
- Both high delay (late data) and very low delay (early data) can cause timing problems.

Practical Observations

- DAT increases with longer data path delay or clock-to-Q.
- DRT increases with longer clock latency at capture or longer clock period.
- Slack is the real indicator of timing quality.
- Always check both setup and hold for every critical path.

Viva Questions

Q1. What is Data Arrival Time (DAT)?
Q2. How is DAT calculated?
Q3. What is Data Required Time (DRT)?
Q4. How is DRT calculated for setup and hold?
Q5. What is the relationship between DAT and DRT?
Q6. What is slack and how is it interpreted?
Q7. What happens when DAT is greater than DRT?

Quick Revision

- ✓ DAT = Actual time data reaches D input
- ✓ DRT = Required time data must meet
- ✓ Slack = DRT − DAT
- ✓ DAT < DRT → Positive Slack (Safe)
- ✓ DAT = DRT → Zero Slack
- ✓ DAT > DRT → Negative Slack (Violation)

STATIC TIME ANALYSIS

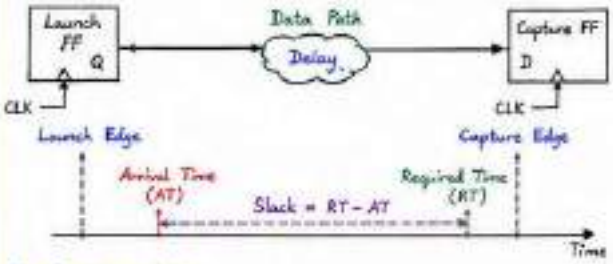
Timing Slack Analysis – Concepts, Types, Interpretation & Importance

1. What is slack?

- Slack is the difference between time available (Required Time) and time taken by data to arrive (Arrival Time).
- It tells us how much timing margin we have.

General Slack Formula

Slack = Required Time – Arrival Time



2. Positive slack

- When Required Time is greater than Arrival Time.
- Slack > 0 → Timing is met with margin.
- More margin → easier for variations, PVT, manufacturing.

Slack > 0 ⇒ Pass



3. Negative slack

- When Arrival Time is greater than Required Time.
- Slack < 0 → Timing violation occurs.
- Data arrives late → functional failure possible.

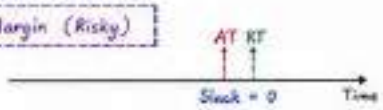
Slack < 0 ⇒ Violation



4. Zero slack

- When Arrival Time is exactly equal to Required Time.
- Slack = 0 → No margin at all.
- Any tiny variation can cause violation.

Slack = 0 ⇒ No Margin (Risky)



5. Setup slack

- Checks if data arrives early enough before capture clock edge.
- Formula: Setup Slack = Required Time (setup) – Arrival Time
- Positive → Pass | Negative → Setup Violation
- Required Time for setup is calculated w.r.t next active capture edge minus setup time.

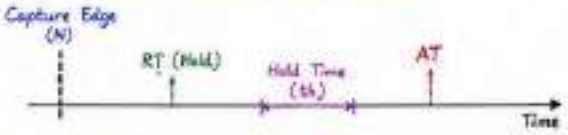
Setup Slack = RT(Setup) – AT



6. Hold slack

- Checks if data does not arrive too early after capture edge.
- Formula: Hold Slack = Arrival Time – Required Time (hold)
- Positive → Pass | Negative → Hold Violation
- Required Time for hold is calculated w.r.t same capture edge plus hold time.

Hold Slack = AT – RT(Hold)



7. Worst slack

- Among all paths of a particular check, the smallest slack.
- It is the path closest to failure.
- If worst slack < 0 → violation exists.

Worst Slack = Min (Slack of all paths for that check)

8. Total negative slack (TNS)

- Sum of negative slacks of all violating paths.
- Indicates overall severity of timing failure.
- More negative TNS → design is farther from closure.

TNS = Σ (Negative Slack of all violating paths) (Slack < 0 only)

9. Critical path

- The path having worst (minimum) slack.
- If worst slack < 0 → critical path is the failing path.
- If worst slack ≥ 0 → critical path is the one with minimum positive slack.
- Critical path decides maximum operating frequency.

10. Timing margin

- Extra time available beyond what is required.
- Equal to positive slack.
- Provides safety against variations (PVT, noise, etc.).

Timing Margin = Positive Slack

11. Required time vs Arrival time

Aspect	Required Time (RT)	Arrival Time (AT)
Definition	Time by which data must arrive at capture FF to meet timing.	Time when data actually arrives at capture FF from launch.
Source	Depends on clock period, timing constraints, setup / hold time.	Depends on data path delays from launch FF to capture FF.
Nature	A target / limit.	An actual measured value.
Relation with Slack	Slack = RT – AT (Setup) Slack = AT – RT (Hold)	Compared with RT to decide Pass / Fail.

If RT > AT → Positive Slack → Pass
If RT = AT → Zero Slack → No Margin
If RT < AT → Negative Slack → Violation

12. Slack Summary Table

Slack Type	Condition	Status / Interpretation	Meaning
Positive Slack	Slack > 0	Pass	Data arrives early, has margin.
Zero Slack	Slack = 0	Risky	No margin, any variation fails.
Negative Slack	Slack < 0	Violation	Data arrives late, failure.

13. Practical Observation

- Always aim for positive slack with good margin.
- Fixing negative slack on critical paths improves performance.
- Too much margin is also not good (wastes performance).
- Slack reports help prioritize which paths to fix first.
- Slack should be analyzed for both Setup and Hold.

14. Numerical Examples

(A) Setup Slack Example

Clock Period (Tclk) = 2.00 ns
Required Time (Setup) = 2.00 ns
Setup Time (Tsu) = 0.15 ns
Data Arrival Time = 1.75 ns

Setup Slack = RT – AT
= (2.00 – 0.15) – 1.75
= 0.10 ns (Positive → Pass)

(B) Hold Slack Example

Hold Time (th) = 0.10 ns
Required Time (Hold) = 0.10 ns
Data Arrival Time = 0.05 ns

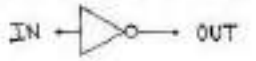
Hold Slack = AT – RT
= 0.05 – 0.10
= –0.05 ns (Negative → Violation)

15. Viva Questions

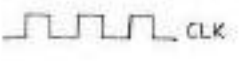
Q1. What is slack in STA?
Q2. What does positive slack indicate?
Q3. What does negative slack indicate?
Q4. What is the difference between setup slack and hold slack?
Q5. How is worst slack calculated?
Q6. Define total negative slack (TNS).
Q7. What is a critical path?
Q8. What is timing margin?
Q9. What is the relation between required time and arrival time?
Q10. How do slack values help in timing closure?

16. QUICK REVISION

- Slack = Required Time – Arrival Time
- Slack > 0 → Pass
- Slack = 0 → No Margin
- Slack < 0 → Violation
- Setup Slack = RT(Setup) – AT
- Hold Slack = AT – RT(Hold)
- Worst Slack = Min Slack
- TNS = Sum of negative slacks
- Critical Path = Path with worst slack
- Timing Margin = Positive Slack
- RT = Target | AT = Actual



STATIC TIMING ANALYSIS (STA)



CLOCK TIMING - CLOCK FUNDAMENTALS

1. CLOCK PERIOD (T)

- Clock period is the time for one complete cycle of clock
- It is the time between two consecutive identical edges (rising-to-rising or falling-to-falling)



Equation

T = Time between two consecutive rising edges

T = Time between two consecutive falling edges

2. CLOCK FREQUENCY (f)

- Clock frequency is the number of cycles per second
- It is the inverse of clock period

Equation

$f = 1/T$

Unit : Hz (or) GHz , MHz , kHz

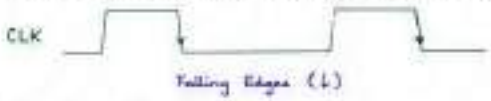
3. RISING EDGE

- Transition of clock from LOW (0) to HIGH (1)
- Denoted by ↑
- Used as reference for many timing checks



4. FALLING EDGE

- Transition of clock from HIGH (1) to LOW (0)
- Denoted by ↓
- Can also be used as reference in timing (based on design)



5. LAUNCH EDGE

- The clock edge at the launching flip-flop (source FF) that starts data on the path
- Usually the active edge of the launch FF
- Data becomes valid at launch FF output after clock-to-Q delay



6. CAPTURE EDGE

- The clock edge at the capturing flip-flop (destination FF) which latches data
- Data must arrive at D input before or after (hold) this edge depending on the check



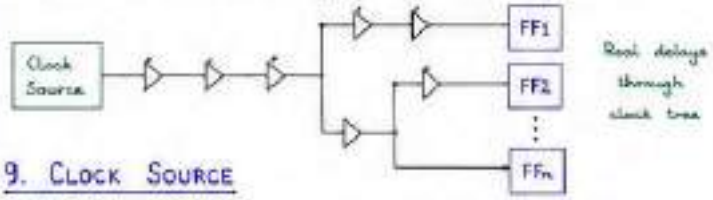
7. IDEAL CLOCK

- Clock defined at a single point and assumed to reach all points in the design with ZERO delay
- No clock tree delays , no skew
- Used in early design stages or for simple analysis



8. PROPAGATED CLOCK

- Clock travels through the actual clock tree (buffers , wires , inverters etc.)
- Each flop sees the clock after some delay
- Results in non-zero latency and skew across flops



9. CLOCK SOURCE

- The origin of the clock signal in the system
- Can be on-chip or off-chip
- It drives the clock network

Types of Clock Sources

- On-chip Clock Source
- Off-chip Clock Source
- PLL (Phase Locked Loop)
- External Oscillator

Source Type	Description
On-chip Clock Source	• Generated inside the chip using internal circuitry • Example : Ring oscillator , RC oscillator , on-chip clock generator • Less accurate than crystal but fully integrated
Off-chip Clock Source	• Generated outside the chip and provided to the chip through a pin • Example : Crystal oscillator on PCB
PLL (Phase Locked Loop)	• Circuit that locks the phase of output clock to the reference clock • Used for frequency synthesis / multiplication / division and jitter cleaning
External Oscillator	• Dedicated oscillator IC outside the chip • Provides stable , accurate reference clock to the system

10. CLOCK DEFINITION POINT

- The point in STA tool where the clock is considered to start (time = 0)
- Usually the output pin of the clock source (or virtual clock source in tool)
- All clock latencies are measured from this point

11. CLOCK ENDPOINT

- The points in the design where clock is used to trigger sequential elements
- Typically the clock pin (CLK) of flip-flops / latches
- STA computes latency to these endpoints from the clock definition point

PRACTICAL OBSERVATIONS

- Clock period sets the speed limit of the design
- Rising/falling edges determine when sequential elements operate
- Different clock sources affect accuracy and jitter
- Propagated clock introduces latency and skew
- Correct definition of clock source, definition point and endpoints is essential for accurate STA

VIVA QUESTIONS

Q1. What is clock period ?
Q2. How is clock frequency related to clock period ?
Q3. What is rising edge and falling edge ?
Q4. What is launch edge ?
Q5. What is capture edge ?
Q6. What is difference between ideal and propagated clock ?
Q7. What are different types of clock sources ?
Q8. What is clock definition point and clock endpoint ?

QUICK REVISION

- ✓ T = Time between two consecutive same edges
- ✓ f = 1/T
- ✓ Launch edge starts data at source FF
- ✓ Capture edge latches data at destination FF
- ✓ Ideal clock → Zero delay , No skew
- ✓ Propagated clock → Real delays , Skew exists
- ✓ Sources : On-chip , Off-chip , PLL , External Oscillator
- ✓ Checks depend on arrival vs required time at endpoints

STATIC TIME ANALYSIS

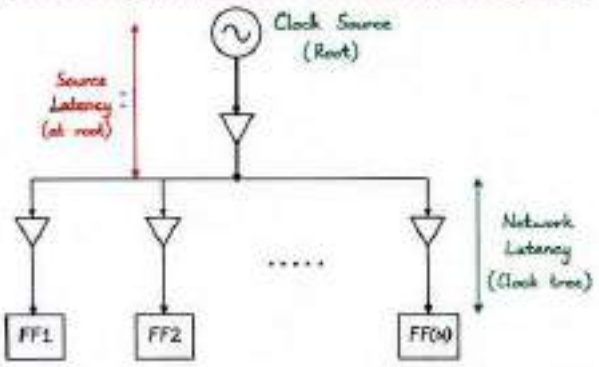
Clock Latency – Source latency, Network latency, Total Latency

1. What is Clock Latency?

- Clock latency is the time taken by the clock signal to reach a point (FF, pin, port) from the clock source.
- It is composed of source latency (at clock root) and network latency (through the clock tree).
- Latency affects the arrival time of clock edges and therefore the timing checks.

General Relationship

Total Clock Latency = Source Latency + Network Latency



Clock arrival time at each FF = Source Latency + Network latency of that path

2. Source Latency

- Time from the ideal clock reference (time zero) to the clock source output pin.
- Includes delays inside the clock source such as oscillator delay, PLL, clock buffer delay, clock gating cell delay, etc.
- Same for all endpoints driven by the same clock root.

Source Latency = Delay (Time zero → Clock source output)



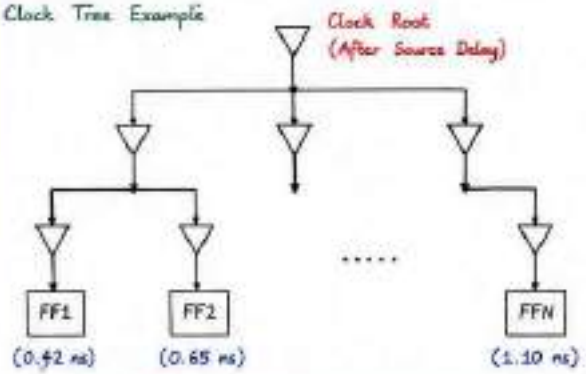
Example (Source Latency)
Clock source internal delay (PLL + Buffer) = 0.35 ns
Source Latency = 0.35 ns

3. Network Latency

- Time taken by clock to travel through the clock distribution network (clock tree) from the clock root to a specific endpoint.
- Depends on buffer delays, wire delays, fanout, load, routing.
- Different for each endpoint (FF, pin, port).

Network Latency (endpoint) = Delay (Root → Endpoint)

Clock Tree Example



Example (Network Latency)
Root → FF1 path delay = 0.42 ns
Root → FF2 path delay = 0.65 ns
Root → FFN path delay = 1.10 ns

4. Total Latency

- Sum of source latency and network latency to reach an endpoint.
- Determines clock arrival time at that endpoint.

Total Latency (endpoint) = Source Latency + Network Latency (endpoint)

Total Latency Example

Endpoint	Source Latency (ns)	Network Latency (ns)	Total Latency (ns)
FF1	0.35	0.42	0.35 + 0.42 = 0.77
FF2	0.35	0.65	0.35 + 0.65 = 1.00
FFN	0.35	1.10	0.35 + 1.10 = 1.45

Clock arrival time at each endpoint = Total Latency

Note:
- Higher latency → clock arrives late → less time for data path (setup risk).
- Lower latency → clock arrives early → more time for data path (hold risk).
- Latency differences between endpoints create clock skew.

5. Comparison of Source, Network and Total Latency

Aspect	Source Latency	Network Latency	Total Latency
Definition	Delay from time zero to clock source output.	Delay from clock root to a specific endpoint through clock tree.	Sum of source and network latency to reach endpoint.
Dependence	Same for all endpoints (from same root).	Different for each endpoint (varies with path).	Different for each endpoint.
Main Contributors	PLL delay, Oscillator delay, Clock buffer, Clock gating cell delay.	Buffer delays, wire delays, fanout, load, routing.	All source contributors + clock tree contributors.
Role in STA	Affects all paths equally.	Causes arrival time differences (clock skew).	Used for final clock arrival time in timing checks.

6. Numerical Example (Complete)

Given:
- Source Latency = 0.30 ns
- Network Latency to FF_A = 0.50 ns
- Network Latency to FF_B = 0.80 ns
- Network Latency to FF_C = 1.20 ns

Total Latency:
- FF_A = 0.30 + 0.50 = 0.80 ns
- FF_B = 0.30 + 0.80 = 1.10 ns
- FF_C = 0.30 + 1.20 = 1.50 ns

⇒ Clock arrives earliest at FF_A and latest at FF_C. → (Skew exists)

7. Practical Observation

- Use balanced clock tree to reduce latency variation.
- Excessive latency increases setup risk (late clock).
- Too little latency increases hold risk (early clock).
- Tools report latency values for each endpoint.
- Latency numbers change with PVT, load and routing.

8. Viva Questions

Q1. What is clock latency?
Q2. What is source latency?
Q3. What is network latency?
Q4. How is total latency calculated?
Q5. Is source latency same for all endpoints?
Q6. Does network latency vary across endpoints?
Q7. How does latency affect setup and hold?

9. QUICK REVISION

- Latency = time for clock to reach a point.
- Source Latency = time zero → clock source output.
- Network Latency = root → endpoint through tree.
- Total Latency = Source + Network.
- Total Latency decides clock arrival time.
- Latency variation creates skew.
- High latency → setup risk, Low latency → hold risk.

STATIC TIME ANALYSIS

Clock Insertion Delay – Definition, Types, Analysis and Impact

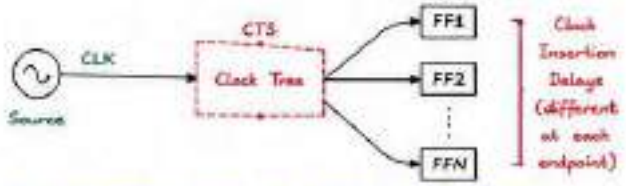
1. Clock Insertion Delay – Definition

- Clock insertion delay is the time taken by the clock signal to travel from the clock source to a specific sink (FF / pin / port).
- It is introduced by the clock distribution network (buffers, inverters, wires) during clock tree synthesis (CTS).
- Also called as Clock Tree Delay or Clock Propagation Delay.

> **Definition:**
> Clock Insertion Delay = Time for clock to reach a point from the clock source through the clock tree.

2. Insertion delay after CTS

- Before CTS : ideal clock (zero delay) is assumed.
- After CTS : actual clock tree is built and real delays are back-annotated to all endpoints.
- These annotated delays are called clock insertion delays.



After CTS, every endpoint has a different clock arrival time.

3. Physical vs Virtual Timing

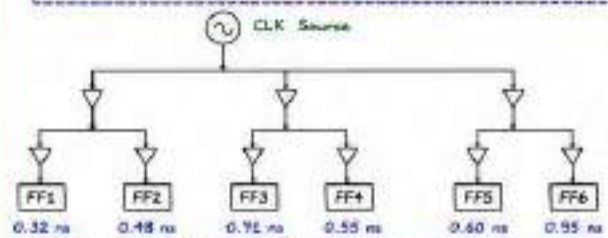
- Virtual Timing (pre-CTS) assumes zero clock network delay.
- Only data path delay is considered.
- Physical Timing (post-CTS) uses real clock insertion delays.
- Both data path and clock path delays are used.

Aspect	Virtual Timing (Pre-CTS)	Physical Timing (Post-CTS)
Clock model	Ideal clock (0 delay)	Real clock tree delays
Clock insertion delay	Ignored (0)	Considered (annotated)
Analysis type	Estimated timing	Accurate timing
Purpose	Planning / Early check	Sign-off timing
Result accuracy	Lower	Higher

4. Clock Tree Delay

- Total delay introduced by the entire clock distribution network from the source to a particular endpoint.
- Includes delays of all buffers, inverters, interconnects, and routing in the clock tree.
- Also called as Network latency.

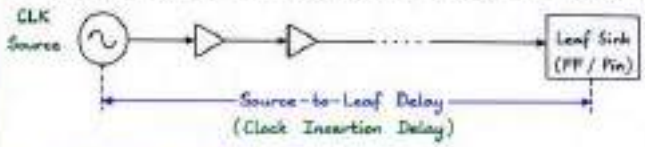
> Clock Tree Delay (endpoint) = Delay through the clock tree from source to the endpoint.



Different endpoints have different clock tree delays.

5. Source-to-Leaf Delay

- Delay from the clock source (insertion point) to a leaf node (sink) of the clock tree (FF / pin / port).
- This is the actual clock insertion delay seen by that endpoint.



6. Minimum Insertion Delay

- Smallest clock insertion delay among all endpoints in the design.
- Fastest clock arrival at any endpoint.
- Occurs at the nearest leaf from the clock source.

> Min Insertion Delay = Min (Clock Insertion Delay of all endpoints)

7. Maximum Insertion Delay

- Largest clock insertion delay among all endpoints in the design.
- Slowest clock arrival at any endpoint.
- Occurs at the farthest leaf from the clock source.

> Max Insertion Delay = Max (Clock Insertion Delay of all endpoints)

8. Relationship with Clock Latency

- Clock latency at an endpoint is composed of source latency and clock insertion delay (network latency).

> Clock Latency (endpoint) = Source Latency + Clock Insertion Delay
> = Source Latency + Clock Tree Delay

Where,
Source Latency → Delay from time zero to clock source output.
Clock Insertion Delay / Network Latency → Delay from source to the endpoint through the clock tree.

Example:

Endpoint	Source Latency (ns)	Clock Insertion Delay (ns)	Clock Latency (ns)
FF1	0.30	0.42	0.72
FF2	0.30	0.65	0.95
FF3	0.30	1.10	1.40

9. Impact on Setup / Hold

Setup Timing	Hold Timing
• Longer clock insertion delay at capture endpoint increases required time. • Helps setup (gives more time for data to arrive). • Smaller insertion delay reduces required time → hardens setup.	• Longer clock insertion delay at capture endpoint reduces slack for hold. • Can cause hold violations (data may arrive too early). • Smaller insertion delay helps hold.

Effect Summary

- Increase capture clock latency → Better for Setup, Worse for Hold.
- Increase launch clock latency → Worse for Setup, Better for Hold.
- Balance is needed to satisfy both checks.



10. Summary Table – Clock Insertion Delay Concepts

Concept	Meaning
Clock Insertion Delay	Time for clock to reach an endpoint from the source through clock tree.
After CTS	Real clock tree delays are back-annotated to endpoints.
Physical vs Virtual	Physical uses real delays (post-CTS), Virtual assumes zero delay.
Clock Tree Delay	Total delay through clock distribution network to the endpoint.
Source-to-Leaf Delay	Delay from clock source to a leaf (sink) endpoint.
Min Insertion Delay	Smallest clock delay among all endpoints.
Max Insertion Delay	Largest clock delay among all endpoints.
Relation with Latency	Clock Latency = Source Latency + Clock Insertion Delay.
Impact on Setup/Hold	More capture latency → good setup / bad hold. Opposite at launch.

11. Numerical Example

Given:

- Source Latency = 0.35 ns
- Clock insertion delays to endpoints:
  FF1 = 0.42 ns, FF2 = 0.70 ns, FF3 = 1.10 ns

Endpoint	Source Latency (ns)	Clock Insertion Delay (ns)	Clock Latency (ns)
FF1	0.35	0.42	0.77
FF2	0.35	0.70	1.05
FF3	0.35	1.10	1.45

Min Insertion Delay = 0.42 ns (FF1)

Max Insertion Delay = 1.10 ns (FF3)

12. Practical Observation

- Proper CTS builds balanced clock tree to reduce latency variation.
- Large insertion delay differences cause skew → impacts timing.
- Latency numbers depend on physical design (placement, routing).
- Always check min/max clock latency for timing closure.

13. Viva Questions

Q1. What is clock insertion delay?
Q2. What happens after CTS in terms of clock delays?
Q3. What is the difference between physical and virtual timing?
Q4. What is clock tree delay?
Q5. What is source-to-leaf delay?
Q6. How do you compute minimum and maximum insertion delay?
Q7. How is clock latency related to insertion delay?
Q8. How does clock insertion delay impact setup and hold?

14. QUICK REVISION

- Clock insertion delay = delay from source to endpoint.
- After CTS, real delays are back-annotated.
- Physical timing uses real clock tree delays.
- Clock tree delay = total network delay to endpoint.
- Min insertion delay = fastest clock arrival.
- Max insertion delay = slowest clock arrival.
- Clock Latency = Source Latency + Insertion Delay.
- More capture latency → good setup, bad hold.
- More launch latency → bad setup, good hold.

STATIC TIMING ANALYSIS (STA)

CLOCK SKEW & UNCERTAINTY → CLOCK SKEW

1. DEFINITION

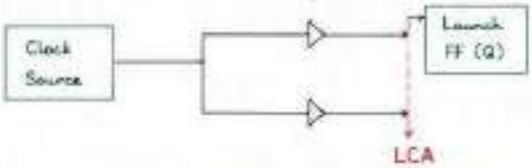
- Clock skew is the difference in arrival time of the same clock edge at two different sequential elements.
- It is the difference between capture clock arrival time and launch clock arrival time.
- Skew can be positive, negative or zero.

Equation

Skew = Capture Clock Arrival (CCA) − Launch Clock Arrival (LCA)

2. LAUNCH CLOCK ARRIVAL (LCA)

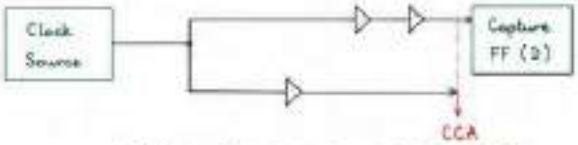
- Time at which the clock edge reaches the launch flip-flop (source FF) clock pin.
- Measured from the clock reference (clock definition point).
- Determines the start time of data launch.



(Arrival of clock edge at Launch FF)

3. CAPTURE CLOCK ARRIVAL (CCA)

- Time at which the clock edge reaches the capture flip-flop (destination FF) clock pin.
- Measured from the clock reference.
- Determines the end boundary for timing check.

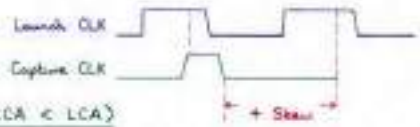


(Arrival of clock edge at Capture FF)

4. TYPES OF CLOCK SKEW

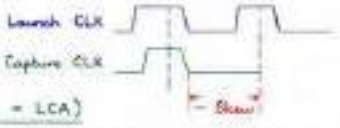
(i) Positive Skew (CCA > LCA)

- Capture clock arrives later than launch clock.
- Gives more time for data → helps setup timing.
- Hurts hold timing.



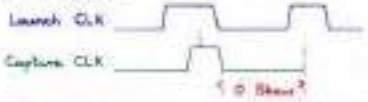
(ii) Negative Skew (CCA < LCA)

- Capture clock arrives earlier than launch clock.
- Reduces available time → hurts setup timing.
- Helps hold timing.



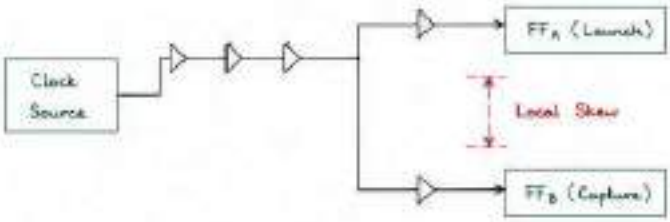
(iii) Zero Skew (CCA = LCA)

- Capture and launch clocks arrive at the same time.
- No skew between two points.



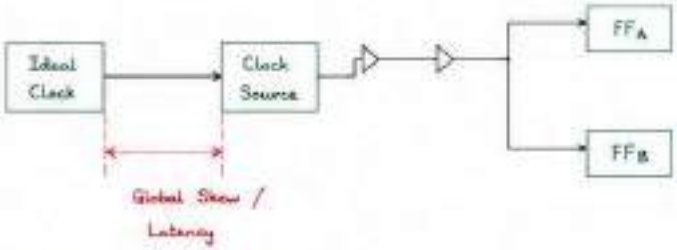
5. LOCAL SKEW

- Skew between two points on the same clock network.
- Example: between two flip-flops in the same clock domain.
- Affects the timing between those two sequential elements.



6. GLOBAL SKEW

- Skew between a clock point and the ideal/reference clock.
- Also called clock latency variation from ideal.
- Important for overall timing closure of the design.



7. RELATIONSHIP SUMMARY

Skew Type	Condition	Effect on Setup	Effect on Hold
Positive Skew	CCA > LCA	Improves (More time)	Degrades (Less margin)
Zero Skew	CCA = LCA	Normal	Normal
Negative Skew	CCA < LCA	Degrades (Less time)	Improves (More margin)

8. SKEW CALCULATION EXAMPLE

Given:

LCA = 1.20 ns

CCA = 1.65 ns (Case 1)

CCA = 0.90 ns (Case 2)

CCA = 1.20 ns (Case 3)

Calculations:

- Case 1 (Positive Skew):
  Skew = 1.65 − 1.20 = +0.45 ns (Positive)
- Case 2 (Negative Skew):
  Skew = 0.90 − 1.20 = −0.30 ns (Negative)
- Case 3 (Zero Skew):
  Skew = 1.20 − 1.20 = 0.00 ns (Zero)

9. PRACTICAL OBSERVATIONS

- Intentional positive skew is widely used to fix setup violations.
- Negative skew is sometimes used to fix hold violations.
- Excessive skew (either sign) can create timing problems in other paths.
- Clock tree design aims to control and balance skew.

VIVA QUESTIONS

Q1. What is clock skew?
Q2. How do you calculate clock skew?
Q3. What is the effect of positive clock skew on setup and hold?
Q4. What is the effect of negative clock skew on setup and hold?
Q5. What is local skew and global skew?
Q6. What is the difference between LCA and CCA?
Q7. When is skew zero?
Q8. How is skew used in real chip design?

QUICK REVISION

- ✓ Skew = CCA − LCA
- ✓ Positive skew (CCA > LCA) → helps setup, hurts hold
- ✓ Negative skew (CCA < LCA) → hurts setup, helps hold
- ✓ Zero skew (CCA = LCA) → no skew
- ✓ Local skew → between two points on same clock network
- ✓ Global skew → between clock point and ideal clock
- ✓ Skew is critical for timing closure and clock tree design

KEY FORMULAS

Clock Skew

Skew = CCA − LCA

Setup Slack

$Slack_{setup}$ = SRT − DAT

Hold Slack

$Slack_{hold}$ = DAT − HRT

= DAT − (CCA − T_{hold})

STATIC TIMING ANALYSIS (STA)

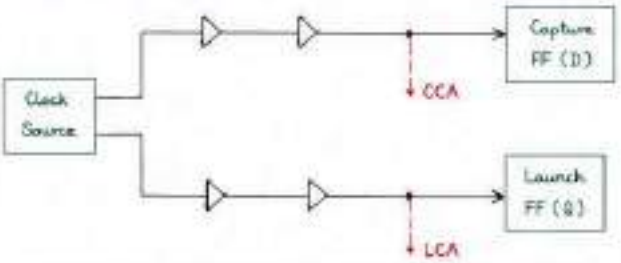
CLOCK SKEW & UNCERTAINTY → CLOCK SKEW

1. CLOCK SKEW CALCULATION

- Clock skew is the difference between capture clock arrival time and launch clock arrival time.

Equation

Clock Skew (Skew) = Capture Clock Arrival (CCA) − Launch Clock Arrival (LCA)

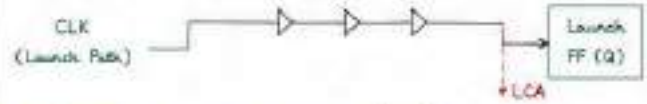


Interpretation

- ✓ Skew > 0 → CCA later than LCA (Positive skew)
- ✓ Skew = 0 → CCA equals LCA (Zero skew)
- ✓ Skew < 0 → CCA earlier than LCA (Negative skew)

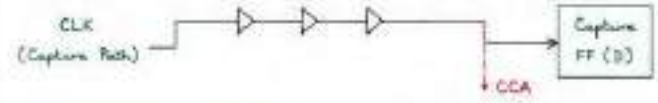
2. LAUNCH CLOCK ARRIVAL (LCA)

- Time at which the active clock edge reaches the launch flip-flop (source FF) clock pin.
- Measured from the clock reference / definition point.
- Determines the start time of data launch.



3. CAPTURE CLOCK ARRIVAL (CCA)

- Time at which the active clock edge reaches the capture flip-flop (destination FF) clock pin.
- Measured from the clock reference / definition point.
- Determines the capture boundary for timing check.



4. TYPES OF CLOCK SKEW

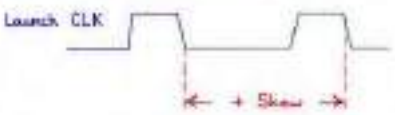
Skew Type	Condition	Meaning	Symbol
Positive Skew	CCA > LCA	Capture clock arrives later than launch	Skew > 0
Negative Skew	CCA < LCA	Capture clock arrives earlier than launch	Skew < 0
Zero Skew	CCA = LCA	Capture and launch arrive at same time	Skew = 0

Key Points:

Positive skew → more time for setup, less margin for hold.
Negative skew → less time for setup, more margin for hold.
Zero skew → neutral condition.

5. EFFECT OF SKEW ON SETUP TIMING

(i) Positive Skew (CCA > LCA)



- Capture edge delayed.
- More time for data to arrive.
- Setup slack improves.

(ii) Zero Skew (CCA = LCA)



- Capture and launch arrive same time.
- No skew effect.

(iii) Negative Skew (CCA < LCA)



- Capture edge early.
- Less time for data.
- Setup slack degrades.

Setup Required Time (RT_{setup})

RT_{setup} = Capture Clock Edge Time − T_{setup}
= Clock Period + (CCA − LCA) − T_{setup}
= Clock Period + Skew − T_{setup}

6. EFFECT OF SKEW ON HOLD TIMING

(i) Positive Skew (CCA > LCA)



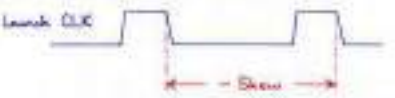
- Capture edge delayed.
- Data less likely to change early.
- Hold slack decreases (worse).

(ii) Zero Skew (CCA = LCA)



- Normal hold condition.

(iii) Negative Skew (CCA < LCA)



- Capture edge early.
- More margin for hold.
- Hold slack improves.

Hold Required Time (RT_{hold})

RT_{hold} = Launch Clock Edge Time + T_{hold}
= (LCA) + T_{hold}
= (CCA − Skew) + T_{hold}
= CCA + T_{hold} − Skew

8. CLOCK PUSH

- Increasing positive skewing.
- Delay capture clock more now.
- Data compared launch clocked.
- Used to gain setup slack time.



Push Capture CLK (delay more) ↓ Increase Setup Slack

9. CLOCK PULL

- Making skew more negative.
- Advance capture clock or delay launch clock.
- Used to fix hold violations.



Pull Capture CLK (advance) ↓ Increase Hold Slack

PRACTICAL OBSERVATIONS

- Skew exists due to different clock path delays.
- Skew can be global (chip-wide) or local (within domain).
- Excessive positive skew can create hold failures.
- Excessive negative skew can create setup failures.
- Useful skewing is a key technique in clock tree synthesis.

VIVA QUESTIONS

01. What is clock skew ?
02. How is clock skew calculated ?
03. What is positive skew effect on setup and hold ?
04. What is negative skew effect on setup and hold ?
05. What is useful skew ?
06. What is clock push and when is it used ?
07. What is clock pull and when is it used ?
08. Difference between local and global skew ?

QUICK REVISION

- ✓ Skew = CCA − LCA
- ✓ Skew > 0 → Positive → Good for Setup, Bad for Hold.
- ✓ Skew < 0 → Negative → Bad for Setup, Good for Hold.
- ✓ Skew = 0 → Zero → Neutral.
- ✓ Clock Push → Increase positive skew → Fix Setup.
- ✓ Clock Pull → Increase negative skew → Fix Hold.
- ✓ Local skew → Within same domain.
- ✓ Global skew → Between reference clock and clock point.

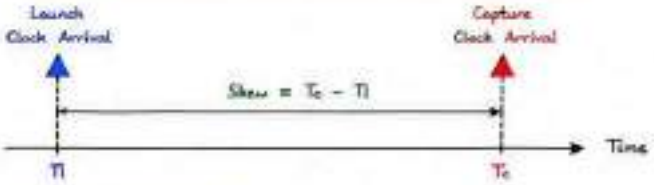
STATIC TIME ANALYSIS

Useful Skew – Intentional clock skew for Timing Optimization

1. Useful Skew – Intentional clock skew

- Deliberately creating a difference in arrival times of clock edges at launch and capture points.
- Skew = Capture clock arrival − Launch clock arrival
- Can be positive (capture later) or negative (capture earlier).

> Positive Skew : Capture later than launch (+)
> Negative Skew : Capture earlier than launch (−)



2. Setup improvement

- Positive skew (capture later) gives more time for data to arrive.
- It relaxes setup requirement.

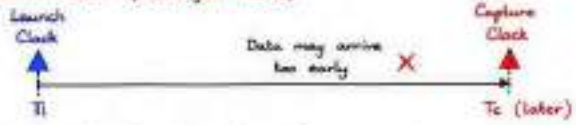


> Effect on Setup : Setup Slack ↑

3. Hold impact

- Positive skew reduces hold margin (data can arrive too early).
- Negative skew increases hold margin.

Positive Skew (Hold gets worse)



Negative Skew (Hold gets better)



4. Launch clock manipulation

- Change arrival time of clock at launch flip-flop.
- Increase launch latency → data appears later at launch.
- Decrease launch latency → data appears earlier at launch.

Methods:

- Add buffers / detours on launch clock path (increase latency).
- Use smaller buffers / shorter route (decrease latency).

> Increase launch latency → Positive skew → Better setup
> Decrease launch latency → Negative skew → Better hold

5. Capture clock manipulation

- Change arrival time of clock at capture flip-flop.
- Increase capture latency → capture edge moves later.
- Decrease capture latency → capture edge moves earlier.

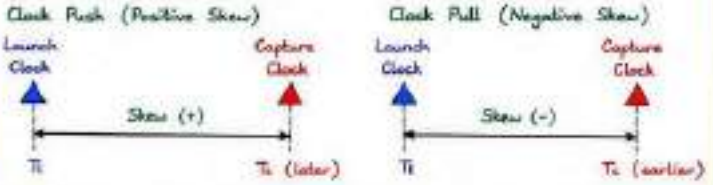
Methods:

- Add buffers / detours on capture clock path.
- Use shorter path / smaller buffers.

> Increase capture latency → Positive skew → Better setup
> Decrease capture latency → Negative skew → Better hold

6. Clock Push and Clock Pull

- Clock Push : Increase capture latency (push capture edge later) or decrease launch latency. ⇒ Positive skew is created.
- Clock Pull : Decrease capture latency (pull capture edge earlier) or increase launch latency. ⇒ Negative skew is created.

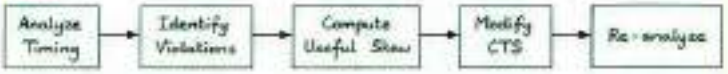


Term	Action	Skew	Setup	Hold
Clock Push	Push capture later or launch earlier	Positive (+)	Improves	Worsens
Clock Pull	Pull capture earlier or launch later	Negative (−)	Worsens	Improves

7. Timing Optimization

- Useful skew is applied to meet timing with minimal changes.
- Goal: Close setup violations while not creating hold violations.
- Use optimization tools to find optimal skew targets for endpoints.

Flow:



8. CTS Relationship

- Useful skew is achieved through the Clock Tree Synthesis (CTS).
- CTS controls buffer sizes, locations and routing to adjust clock arrival times.
- Skew targets are given to CTS to meet timing goals.
- Post-CTS skew must be within acceptable limits.

Skew in CTS:

$$\text{Skew}_{target_min} \le \text{Actual Skew} \le \text{Skew}_{target_max}$$

9. Critical-path Optimization

- Apply positive skew on critical setup paths to provide more time.
- Apply negative skew on critical hold paths to prevent early data.
- Focus skew on worst paths rather than all paths.
- Improves overall timing closure and performance.

Example:



Increase capture latency or decrease launch latency ⇒ Positive skew ⇒ Setup improves

Decrease capture latency or increase launch latency ⇒ Negative skew ⇒ Hold improves

10. Relationship Summary Table

Skew Type	How Achieved	Hold Slack
Positive Skew	Increase capture or decrease latency	Reduces slack
Negative Skew	Decrease capture Clock latency	No change (Better slack)

11. Key Equations & Relationships

- Clock Skew (S) = $T_c - T_l$
  (T_c = capture arrival, T_l = launch arrival)
- Setup Slack = $RT_{setup} - AT_{data}$
  (Positive skew ⇒ RT_{setup} effectively increases)
- Hold Slack = $AT_{data} - RT_{hold}$
  (Positive skew ⇒ AT_{data} effectively earlier)
- Useful Skew must balance Setup & Hold.

12. Practical Observation

- Too much positive skew can cause hold failures.
- Too much negative skew can cause setup failures.
- Skew budget must be carefully controlled.
- CTS with useful skew gives better timing closure and performance.

13. Viva Questions

Q1. What is useful skew?
Q2. How does positive skew improve setup?
Q3. What is the impact of positive skew on hold?
Q4. What is clock push and clock pull?
Q5. How is launch clock manipulation done?
Q6. How is capture clock manipulation done?
Q7. How does CTS help in useful skew?
Q8. How is useful skew used for critical-path optimization?

14. QUICK REVISION

- Useful skew = intentional difference in clock arrivals.
- Positive skew → better setup, worse hold.
- Negative skew → better hold, worse setup.
- Launch manipulation: change launch clock arrival.
- Capture manipulation: change capture clock arrival.
- Clock Push → push capture later (positive skew).
- Clock Pull → pull capture earlier (negative skew).
- Achieved through CTS by buffer sizing & routing.
- Optimize skew mainly on critical paths.
- Balance skew for timing closure.

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

CLOCK SKEW & UNCERTAINTY → CLOCK UNCERTAINTY

1. DEFINITION

- Clock uncertainty is the time margin added to account for variations and imprecisions in the clocking environment.
- It reduces the available time for setup and increases the constraint for hold to ensure robust timing.

Total Clock Uncertainty (U)

U = Setup Uncertainty (U_{setup}) for setup checks

U = Hold Uncertainty (U_{hold}) for hold checks

2. WHY UNCERTAINTY IS REQUIRED?

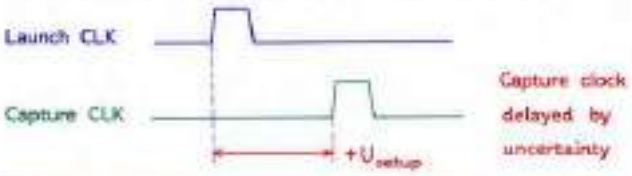
- Real clocks are not ideal.
- Variations in clock arrival due to jitter, skew variations, PVT, OCV, etc.
- To avoid false timing closure and ensure silicon reliability.
- To cover worst-case possibilities.

Main Sources

Jitter + Skew Variation + OCV (On-chip Variation) + Other modeling uncertainties

3. SETUP UNCERTAINTY (U_{setup})

- Worst-case variation that can delay the capture clock or advance the launch clock.
- It reduces the available time for the data to arrive.



Effect:

Required Time = Captured Edge Time − T_{setup} − U_{setup}

⟹ Required time decreases.

4. HOLD UNCERTAINTY (U_{hold})

- Worst-case variation that can advance the capture clock or delay the launch clock.
- It increases the minimum time for requirement for data.



Effect:

Required Time = Launch Edge Time + $T_{clk\to q(min)}$ + T_{hold} + U_{hold}

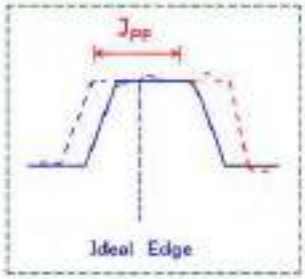
⟹ Required time increases.

5. JITTER CONTRIBUTION

- Cycle-to-cycle random clock edge variation.
- Causes uncertainty in both setup and hold.
- If T_{pp} = peak-to-peak jitter,

$$U_{jitter\ (setup)} = +\frac{T_{pp}}{2}$$

$$U_{jitter\ (hold)} = +\frac{T_{pp}}{2}$$

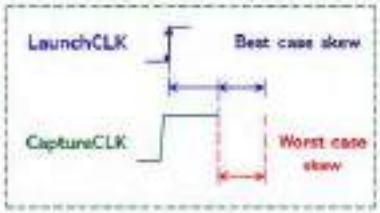


6. SKEW CONTRIBUTION (VARIATION)

- Skew changes with PVT, voltage, temperature, aging.
- Variation in skew must be added as uncertainty.
- Let σ_{skew} = variation in skew,

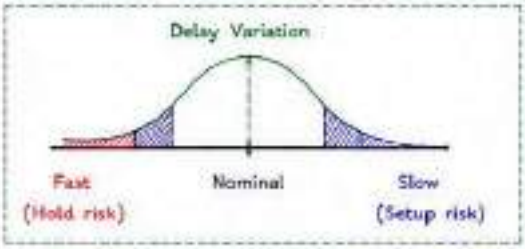
$$U_{skew\ (setup)} = +\sigma_{skew}$$

$$U_{skew\ (hold)} = +\sigma_{skew}$$



7. OCV CONTRIBUTION

- On-chip variations (process, voltage, temperature) change delays and clock arrival times.
- OCV denotes cause uncertainty in both setup and hold.
- Let $U_{ocv\ (setup)}$ and $U_{ocv\ (hold)}$ be OCV contributions.



8. TOTAL UNCERTAINTY CALCULATION

- Total uncertainty is the sum (or root-sum-square) of all individual sources.

Setup Uncertainty	Hold Uncertainty
$U_{setup} = U_{jitter\ (setup)} + U_{skew\ (setup)} + U_{ocv\ (setup)} + U_{other\ (setup)}$	$U_{hold} = U_{jitter\ (hold)} + U_{skew\ (hold)} + U_{ocv\ (hold)} + U_{other\ (hold)}$
or (RMS)	or (RMS)
$U_{setup} = \sqrt{U_j^2 + U_{sk}^2 + U_{ocv}^2 + U_{other}^2}$	$U_{hold} = \sqrt{U_j^2 + U_{sk}^2 + U_{ocv}^2 + U_{other}^2}$

9. SUMMARY TABLE

Uncertainty Source	Setup Impact	Hold Impact	Reason
Jitter	Reduces time	Increases req. time	Edge movement
Skew Variation	Reduces time	Increases req. time	Clock arrival diff.
OCV	Reduces time	Increases req. time	Delay variation
Other	Reduces time	Increases req. time	Modeling margin

PRACTICAL OBSERVATIONS

- Uncertainty ensures real silicon reliability.
- Setup uncertainty always subtracts from available time.
- Hold uncertainty always adds to minimum required time.
- Too much uncertainty → pessimism.
- Too little uncertainty → silicon risk.

VIVA QUESTIONS

Q1. What is clock uncertainty?
Q2. Why is uncertainty required in STA?
Q3. What is setup uncertainty?
Q4. What is hold uncertainty?
Q5. What are the sources of clock uncertainty?
Q6. How jitter affects setup and hold timing?
Q7. How skew variation contributes to uncertainty?
Q8. What is the effect of OCV on uncertainty?

QUICK REVISION

- ✓ Uncertainty accounts for real clock variations.
- ✓ Setup uncertainty reduces required time.
- ✓ Hold uncertainty increases required time.
- ✓ Sources: Jitter, Skew variation, OCV, Other.
- ✓ U_{setup} = Σ (or RMS) of all setup contributions.
- ✓ U_{hold} = Σ (or RMS) of all hold contributions.
- ✓ Essential for robust timing closure.

KEY FORMULAS

Skew = CCA − LCA

U_{setup} = Σ / RMS

U_{hold} = Σ / RMS

RT_{setup} = CCA − T_{setup} − U_{setup}

RT_{hold} = LCA + $T_{clk\to q(min)}$ + T_{hold} + U_{hold}

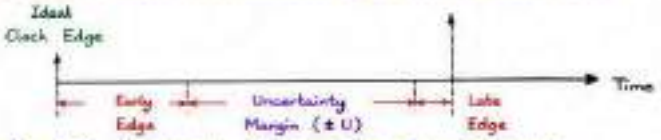
STATIC TIME ANALYSIS

Clock Uncertainty – Margin , Pre-CTS Uncertainty , Post-CTS Uncertainty , Setup Equation , Hold Equation , SDC Clock Uncertainty

1. Clock Uncertainty – Margin

- Clock uncertainty (or margin) accounts for variations in clock edge arrival due to jitter, OCV, PVT, on-chip variation etc.
- It is added/subtracted to required time and hold time to avoid optimistic timing.
- Provides guard band to ensure reliable operation.

> Uncertainty Components :
> Jitter + OCV + PVT variation + Quantization + DUT margin + PLL variation + Measurement margin + Other factors



2. Pre-CTS Uncertainty (Before Clock Tree Synthesis)

- Before CTS, clock network is unknown.
- Only source related uncertainty is known (PLL, jitter, OCV etc.).
- Usually larger uncertainty value is used for safety.
- Same uncertainty is applied to all endpoints (uniform).

> Pre-CTS Uncertainty = Source Uncertainty (large value)
> Applied uniformly to all endpoints.

Example Values : 50 ps – 150 ps (depends on design/technology)

3. Post-CTS Uncertainty (After Clock Tree Synthesis)

- After CTS, real clock tree delays and variation are known.
- Uncertainty can be reduced because actual data is available.
- May be different for setup and hold.
- Can be path specific (OCV based) or corner based.

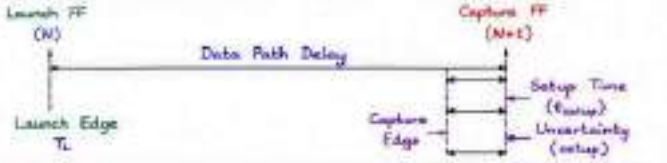
> Post-CTS Uncertainty = Reduced Value (from real data)
> May be setup_uncertainty and hold_uncertainty

Example Values : 10 ps – 50 ps (typical)

4. Setup Equation

- Checks if data arrives early enough before next active capture edge.

> Setup Slack = Required Time (setup) – Arrival Time
> Required Time (setup) = Capture Edge Time – Setup Time – Clock Uncertainty (setup)

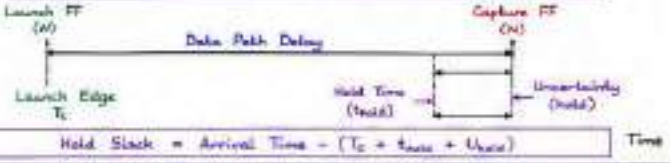


> Setup Slack = (T_c – t_{setup} – U_{setup}) – Arrival Time

5. Hold Equation

- Checks if data does not arrive too early after capture edge.

> Hold Slack = Arrival Time – Required Time (hold)
> Required Time (hold) = Capture Edge Time + Hold Time + Clock Uncertainty (hold)



> Hold Slack = Arrival Time – (T_c + t_{hold} + U_{hold})

6. SDC Clock Uncertainty

- Applied using set_clock_uncertainty command.
- Can be specified for setup, hold or both.
- Can be applied before CTS (pre-CTS) and after CTS (post-CTS).

```
set_clock_uncertainty <value> [-setup] [-hold]
    [-rise_from <clk>] [-fall_from <clk>]
```

SDC Options	Meaning
-setup	Applies uncertainty for setup checks (reduces required time)
-hold	Applies uncertainty for hold checks (increases required time)
-rise_from / -fall_from	Applies uncertainty only for specific transition
Default (no option)	Applies for both setup and hold

Example Commands

- set_clock_uncertainty 0.10 [get_clocks clk]
- set_clock_uncertainty 0.08 -setup [get_clocks clk]
- set_clock_uncertainty -hold 0.05 [get_clocks clk]
- set_clock_uncertainty -rise_from clk -setup 0.07

7. Physical vs Virtual Timing with Uncertainty

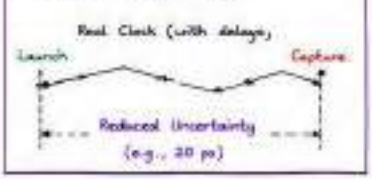
Pre-CTS (Virtual Timing)

- No real clock tree.
- Ideal 0 latency clocks.
- Large uniform uncertainty.



Post-CTS (Physical Timing)

- Real clock tree delays known.
- Actual insertion delays.
- Reduced uncertainty.



8. Relation with Clock Latency

- Clock Insertion Delay (latency) = time from source to endpoint.
- Clock Uncertainty = variation in that arrival time.

For any endpoint :

> Clock Arrival Time = Source Latency + Insertion Delay (Latency)
> Actual Arrival = Latency ± Uncertainty

Hence ,

- Latency shifts the clock edge.
- Uncertainty gives margin around that shifted edge.

9. Impact on Setup / Hold

Aspect	Setup Check	Hold Check
Uncertainty Effect	Reduces Required Time (subtract)	Increases Required Time (add)
If Uncertainty ↑	Setup Slack ↓ (worse)	Hold Slack ↓ (worse)
If Uncertainty ↓	Setup Slack ↑ (better)	Hold Slack ↑ (better)

> Key Point :
> Too large uncertainty → pessimistic (may fail unnecessarily)
> Too small uncertainty → optimistic (risk of silicon failure)

10. Impact Summary

Action / Change	Skew Effect	Setup Impact	Hold Impact	Purpose
Capture delayed	Positive Skew (+)	Improves	Worsens	Fix setup
Capture advanced	Negative Skew (-)	Worsens	Improves	Fix hold
Launch delayed	Negative Skew (-)	Worsens	Improves	Fix hold
Launch advanced	Positive Skew (+)	Improves	Worsens	Fix setup
Increase Uncertainty	–	Worse	Worse	More safe, more pessimistic
Decrease Uncertainty	–	Better	Better	Less safe, less margin

11. Practical Observation

- Choose uncertainty based on process, voltage, temperature and design variation.
- Pre-CTS uses larger value to avoid optimism.
- Post-CTS can use tighter value based on real clock data.
- Separate setup and hold uncertainties help better optimization.
- Always validate with sign-off corners and silicon correlation.
- Uncertainty is crucial for timing sign-off reliability.

12. Numerical Example

Setup Check Example

Capture Edge (T_c) = 2.000 ns
Data Arrival Time = 1.600 ns
Setup Time = 0.100 ns
Setup Uncertainty = 0.080 ns
Required Time = 2.000 – 0.100 – 0.080 = 1.820 ns
Setup Slack = 1.820 – 1.600 = +0.220 ns (Pass)

Hold Check Example

Capture Edge (T_c) = 2.000 ns
Data Arrival Time = 2.120 ns
Hold Time = 0.050 ns
Hold Uncertainty = 0.030 ns
Required Time = 2.000 + 0.050 + 0.030 = 2.080 ns
Hold Slack = 2.120 – 2.080 = +0.040 ns (Pass)

13. Viva Questions

Q1. What is clock uncertainty?
Q2. Why is uncertainty needed in STA?
Q3. What is pre-CTS uncertainty?
Q4. What is post-CTS uncertainty?
Q5. How does uncertainty affect setup?
Q6. How does uncertainty affect hold?
Q7. Give setup and hold timing equations with uncertainty.
Q8. What is set_clock_uncertainty command?

14. QUICK REVISION

- Uncertainty → margin for clock variation
- Pre-CTS → large uniform value
- Post-CTS → reduced / path/corner specific
- Setup Eqn → RT = T_c – t_{setup} – U_{setup}
- Hold Eqn → RT = T_c + t_{hold} + U_{hold}
- SDC → set_clock_uncertainty
- Uncertainty ↑ → both setup & hold worse
- Balance uncertainty for safe & optimized timing

STATIC TIMING ANALYSIS (STA)

CLOCK JITTER – OVERVIEW

1. DEFINITION

- Clock jitter is the short-term variation in the clock edge from its ideal position.
- It is a random deviation in the timing of clock edges caused by noise, supply variation, PLL, crosstalk, etc.
- Jitter reduces timing margin and can cause setup or hold violations.

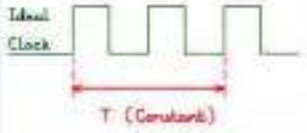
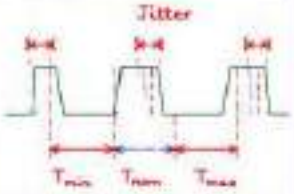
Total Clock Jitter (J)

J = Peak-to-Peak Jitter (J_{pp})

or

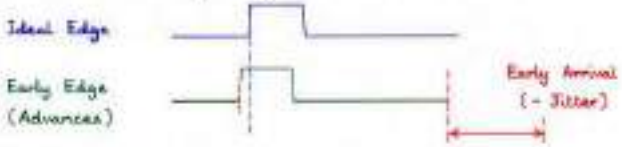
J_{rms} = Root Mean Square Jitter

2. IDEAL CLOCK VS ACTUAL CLOCK

Ideal Clock	Actual Clock
• Perfect periodic waveform. • Constant period. • No variation in edge positions. • No jitter.	• Edge positions vary around ideal edges. • Period varies from cycle to cycle. • Contains jitter.
	

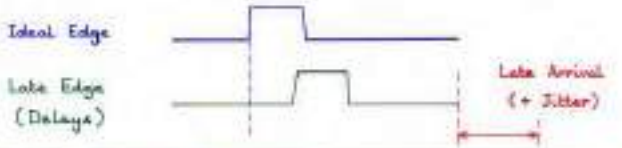
3. EARLY CLOCK ARRIVAL

- Actual clock edge arrives earlier than the ideal edge.
- Clock edge is shifted to the left.
- Can cause hold timing violation (less time for data to stay stable).
- Considered as negative jitter.



4. LATE CLOCK ARRIVAL

- Actual clock edge arrives later than the ideal edge.
- Clock edge is shifted to the right.
- Can cause setup timing violation (less time for data to arrive).
- Considered as positive jitter.

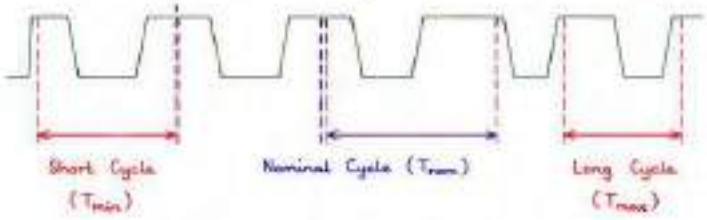


5. CYCLE VARIATION

- Due to jitter, the clock period is not constant.
- Each cycle can be shorter or longer than the nominal period.
- Defined by minimum period (T_{min}) and maximum period (T_{max}).

$$T_{min} = T_{nom} - \frac{J_{pp}}{2}$$

$$T_{max} = T_{nom} + \frac{J_{pp}}{2}$$



SUMMARY TABLE

Condition	Edge Behavior	Type of Jitter	Effect on Timing
Early Clock Arrival	Edge comes earlier	Negative Jitter	Hold slack reduces (Hold violation risk)
Late Clock Arrival	Edge comes later	Positive Jitter	Setup slack reduces (Setup violation risk)
No Deviation	Edge at ideal time	Zero Jitter	No timing impact
Cycle Variation	Period varies	Peak-to-Peak Jitter (J_{pp})	Affects both setup and hold checks

JITTER IMPACT ON TIMING CHECKS

- Positive jitter (late arrival) reduces setup margin.
- Negative jitter (early arrival) reduces hold margin.
- Both must be considered in timing analysis.

Setup Required Time with Jitter (worst case)

$$RT_{setup}(jitter) = RT_{setup}(ideal) - U_{jitter}(setup)$$

Hold Required Time with Jitter (worst case)

$$RT_{hold}(jitter) = RT_{hold}(ideal) + U_{jitter}(hold)$$

Where, $U_{jitter}(setup) = J_{pp}/2$ (worst-case positive jitter)

$U_{jitter}(hold) = J_{pp}/2$ (worst-case negative jitter)

EXAMPLE

- Nominal Clock Period (T_{nom}) = 1.00 ns
- Peak-to-Peak Jitter (J_{pp}) = 80 ps

$$T_{min} = 1.00 - \frac{80}{2} = 1.00 - 40 = 0.96 \text{ ns}$$

$$T_{max} = 1.00 + \frac{80}{2} = 1.00 + 40 = 1.04 \text{ ns}$$

$U_{jitter}(setup)$ = +40 ps

$U_{jitter}(hold)$ = +40 ps

Interpretation

- Worst-case cycle can be as short as 0.96 ns or as long as 1.04 ns.
- Setup check uses +40 ps uncertainty.
- Hold check uses +40 ps uncertainty.

PRACTICAL OBSERVATIONS

- Jitter is unavoidable in real designs.
- Sources: PLL, ring oscillator, supply noise, crosstalk, temperature variation.
- Large jitter increases probability of timing failures.
- Always include jitter uncertainty in both setup and hold timing analysis.

VIVA QUESTIONS

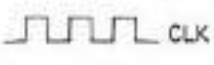
Q1. What is clock jitter ?
Q2. How is peak-to-peak jitter defined ?
Q3. What is the effect of early clock arrival ?
Q4. What is the effect of late clock arrival ?
Q5. How does jitter affect setup timing ?
Q6. How does jitter affect hold timing ?
Q7. What is cycle variation due to jitter ?
Q8. How is jitter considered in STA ?

QUICK REVISION

✓ Jitter = variation in clock edge positions
✓ Early arrival → Negative jitter → Hold risk
✓ Late arrival → Positive jitter → Setup risk
✓ J_{pp} = peak-to-peak jitter
✓ $T_{min} = T_{nom} - J_{pp}/2$; $T_{max} = T_{nom} + J_{pp}/2$
✓ $U_{jitter}(setup) = J_{pp}/2$; $U_{jitter}(hold) = J_{pp}/2$
✓ Jitter reduces timing margin



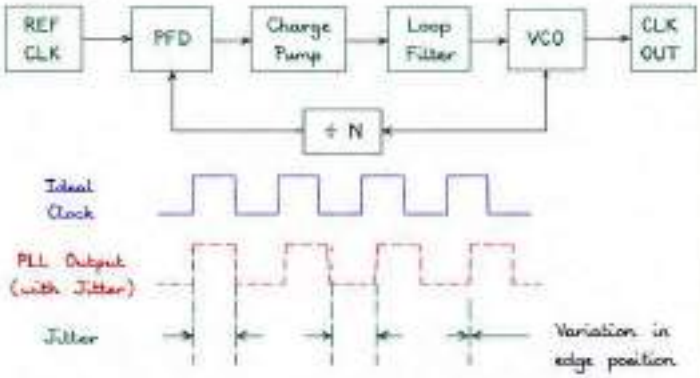
STATIC TIMING ANALYSIS (STA)



CLOCK JITTER – DETAILED SOURCES

1. PLL-RELATED JITTER

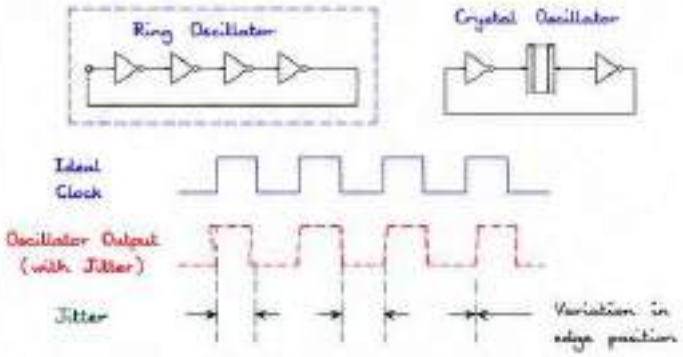
- Jitter generated inside Phase Locked Loop (PLL)
- Caused by noise in charge pump, loop filter, VCO, PFD / clk divider, etc
- Includes phase noise of VCO which gets transferred to output clock



PLL jitter is usually specified as J_{pp} (peak-to-peak) or J_{rms} (root mean square)

2. OSCILLATOR-RELATED JITTER

- Jitter generated by free-running oscillator (ring oscillator, crystal oscillator, RC oscillator, etc.)
- Due to phase noise, flicker noise, supply variations, and device mismatch
- Exists even without PLL



Oscillator jitter increases as frequency increases

3. THERMAL NOISE (WHITE NOISE)

- Random noise due to thermal agitation of charge carriers in resistors, MOS devices, and interconnects
- Present in all circuits and always exists
- Contributes to random jitter

Concept

Thermal noise → random voltage fluctuations
→ random delay variations
→ clock edge variations



Effect

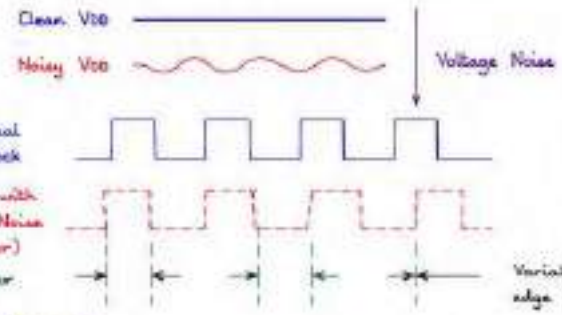
Increases jitter and reduces timing margin

4. POWER NOISE (SUPPLY NOISE)

- Noise on power supply (V_{DD}/GND) due to switching activity of digital circuits
- Causes variation in delay of clock buffers and clock distribution network
- Results in correlated or random jitter depending on activity

Mechanism

Switching activity → voltage droop / bounce
→ delay variation in clock path
→ clock edge variation (jitter)



COMPARISON OF JITTER SOURCES

Jitter Source	Origin	Nature	Typical Dependency	Impact on STA
PLL-Related Jitter	PLL blocks (PFD, CP, VCO, Loop Filter, Divider)	Random (mostly)	Depends on loop BW, VCO phase noise	Affects both setup and hold
Oscillator-Related Jitter	Oscillator (Ring, Crystal, RC, etc.)	Random	Increases with freq, supply variation	Affects both setup and hold
Thermal Noise	Resistors, MOS devices, interconnects	Random (White noise)	Depends on temperature, BW, resistance	Reduces timing margin
Power Noise	Supply network (V_{DD}/GND)	Correlated / Random	Depends on switching activity, IR drop	Causes jitter, can degrade timing closure

PRACTICAL OBSERVATIONS

- Jitter is unavoidable in real silicon
- All jitter sources must be modeled and included in STA as clock uncertainty
- Excessive jitter can lead to setup/hold violations even if nominal timing is met
- Good clock design aims to minimize jitter sources and their impact

VIVA QUESTIONS

Q1. What is clock jitter ?
Q2. What are the sources of clock jitter ?
Q3. Differentiate PLL jitter and oscillator jitter
Q4. What is thermal noise and how does it affect jitter ?
Q5. What is power noise and why does it cause jitter ?
Q6. How is jitter considered in STA ?

QUICK REVISION

- ✓ Jitter = variation in clock edge position
- ✓ Main sources: PLL jitter, Oscillator jitter, Thermal noise, Power noise
- ✓ Jitter reduces setup margin and increases hold risk
- ✓ Modeled as clock uncertainty in STA

STATIC TIMING ANALYSIS (STA)

Clock Jitter Types – Overview

1. Introduction To Clock Jitter

- Clock jitter is the short-term variation in the clock edge from its ideal position.
- It is caused by noise, supply variation, PLL, crosstalk, temperature variation, etc.
- Jitter reduces the timing margin and can cause setup or hold violations.

Peak-to-Peak Jitter (J_{PP})

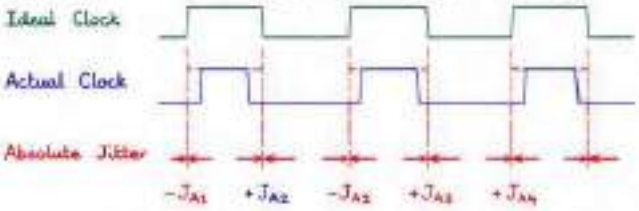
J_{PP} = Maximum deviation between the earliest and latest edge within a given time window.

RMS Jitter (J_{RMS})

J_{RMS} = Root Mean Square value of jitter over time.

2. Absolute Jitter

- Absolute jitter is the deviation of any clock edge from its ideal (reference) edge.
- Measured with respect to a fixed reference edge.
- Represents the total timing uncertainty of that edge.

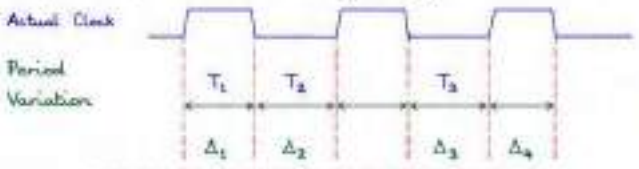


(Absolute jitter of each edge w.r.t ideal edge)

Where, J_A = Actual Edge Time − Ideal Edge Time

3. Cycle-To-Cycle Jitter (Short-Term Jitter)

- Variation in the period between two consecutive clock cycles.
- It shows how the clock period changes from one cycle to the next.
- Important for short-term timing analysis.

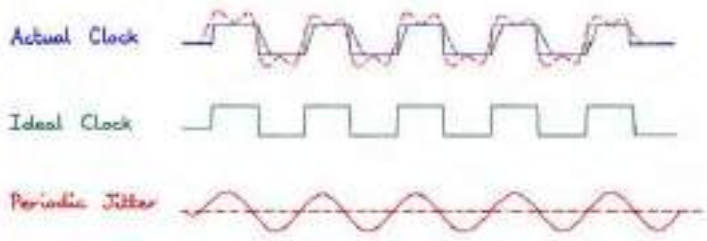


Cycle-to-Cycle Jitter for T_1 = $T_1 - T_{nom}$

Where, T_{nom} = Nominal Clock Period

4. Periodic Jitter

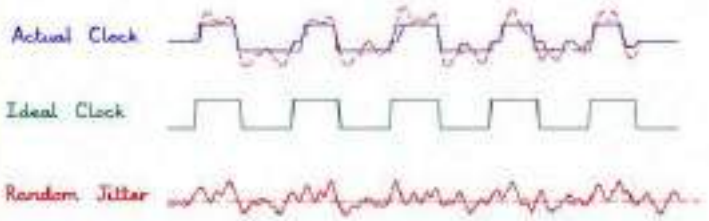
- Jitter that has a repeating or periodic pattern.
- Usually caused by power supply, ripple, crosstalk, EMI, etc.
- Has a specific frequency and amplitude.



(Repeating pattern with fixed frequency)

5. Random Jitter

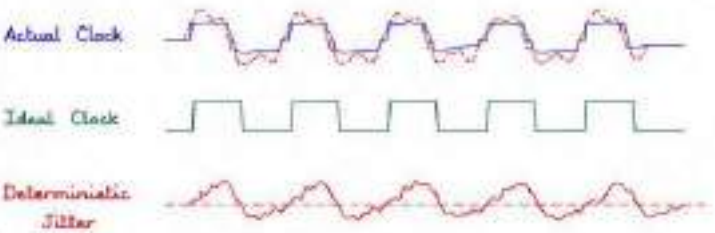
- Jitter that is non-periodic and unpredictable.
- Caused by random noise sources such as thermal noise, flicker noise, shot noise, etc.
- Has Gaussian (normal) distribution.



(No repeating pattern, random variations)

6. Deterministic Jitter

- Jitter that is predictable and repeatable.
- Caused by known sources like PLL, data-dependent crosstalk, duty-cycle distortion, etc.
- Can be periodic or non-periodic but is deterministic.



(Predictable and repeatable behavior)

Comparison of Clock Jitter Types

Jitter Type	Reference	What it Represents	Time Scale				
Absolute Jitter	Ideal (reference) edge	Deviation of a clock edge from its ideal position	Per edge				
Cycle-to-Cycle Jitter	Previous clock edge	Variation in clock period between consecutive cycles	Short-term	Power supply, ripple, xtalk, duty-cycle, EMI	Any	Rapid fluctuations	PLL, RC-tuning
Periodic Jitter	Ideal edge	Deviation with periodic behavior	Any	Thermal noise, shot noise	Any	Random noise jitter	R, shot noise
Random Jitter	Ideal edge	Irregular, non-repeating variation	Any	PLL, crosstalk, data dependent	Any	Thermal supply and logic noise	PLL, impedance

Practical Observations

- High jitter reduces timing margin.
- Both setup and hold checks are affected.
- Jitter must be modeled in STA as clock uncertainty.
- Jitter increases with poor power integrity and noisy environment.
- Good clock design, clean power, and shielding reduce jitter.

Viva Questions

Q1. What is clock jitter ?
Q2. What is absolute jitter ?
Q3. What is cycle-to-cycle jitter ?
Q4. Differentiate periodic jitter and random jitter.
Q5. What is deterministic jitter ?
Q6. How does jitter affect setup and hold checks ?
Q7. What are the sources of clock jitter ?
Q8. How is jitter modeled in STA ?

Quick Revision

- ✓ Absolute jitter → deviation from ideal edge.
- ✓ Cycle-to-cycle jitter → variation in period.
- ✓ Periodic jitter → repeating pattern.
- ✓ Random jitter → non-repeating, due to noise.
- ✓ Deterministic jitter → predictable and repeatable.
- ✓ Jitter reduces timing margin and can cause setup/hold violations.

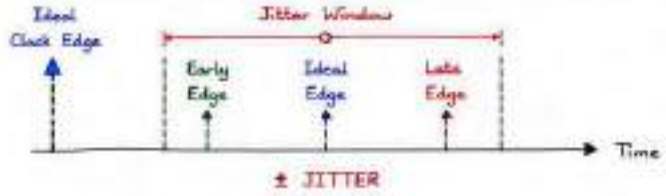
STATIC TIME ANALYSIS

Jitter Reduction – Techniques and Best Practices

1. Jitter – Quick Recall

- Jitter is the short term variation in clock edge arrival time.
- Causes uncertainty in the sampling instant of flip-flops.
- Affects both setup and hold timing checks.
- Reducing jitter improves timing margin and reliability.

> Effective clock edge = Ideal clock edge ± Jitter
> (Early or Late by Jitter amount)



2. Low-Jitter Clock Source

- Use high quality oscillators / PLLs with low phase noise.
- Lower phase noise → Lower period jitter.
- Prefer crystal oscillators over RC oscillators.
- Choose PLL with narrow loop bandwidth for better jitter.



Key Points
- Low phase noise → Low integrated jitter.
- Stable power supply → Stable oscillation.

3. Jitter Buffers

- Jitter buffer (or jitter attenuator) reduces input clock jitter.
- Re-clocks the signal and filters out high frequency variations.
- Provides clean edges to the clock distribution network.

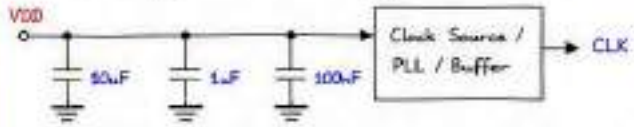


Types
- Analog jitter attenuator
- Digital re-clocking buffer
- PLL based cleaner

Effect
- Reduces random jitter
- Removes high frequency noise
- Improves deterministic jitter
- Sharper clock edges

4. Power Filtering

- Noise on power rail modulates delay of clock circuitry.
- Proper filtering reduces supply induced jitter.
- Use decoupling capacitors close to clock source and buffers.
- Use LDO / regulator for clean supply.



Best Practices
- Multi stage decoupling (bulk + mid + high frequency).
- Short connections, low inductance.
- Separate analog and digital supplies where possible.

5. Shielding

- Clock is a high-speed, noise sensitive signal.
- Shielding reduces electromagnetic interference (EMI).
- Prevents crosstalk from aggressor signals.
- Use grounded shields to surround clock traces.



6. PCB / Interconnect / Layout Improvements

- Short, wide and straight clock routes reduce jitter.
- Minimize vias, stubs and sharp bends.
- Maintain controlled impedance for clock traces.
- Separate clock lines from noisy / switching signals.
- Use solid reference plane (continuous ground).
- Follow differential routing for long / external clock lines.

Do's
- Short routes, minimal vias.
- Matched length for balanced clock paths.
- Avoid parallel run with aggressors.
- Use differential clocking where possible.

Don'ts
- Long stubs, unnecessary vias.
- Routing close to high switching nets.
- Cross splitting reference planes.

7. Techniques Summary Table

Technique	How it Reduces Jitter	Key Benefits
Low-Jitter Clock Source	Low phase noise, stable oscillation	Lower period jitter, stable clock
Jitter Buffers	Filters high frequency variations	Cleaner edges, lower random jitter
Power Filtering	Removes supply noise and ripple	Reduces delay modulation
Shielding	Blocks EMI and crosstalk	Less noise coupling, less jitter
PCB / Interconnect Improvements	Better routing & Impedance control	Stable propagation, low deterministic jitter

8. Impact of Jitter on Timing Checks

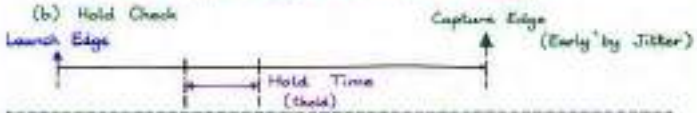
(a) Setup Check



> Required Time (Setup) = Capture Edge Time − t_{setup} − Jitter
> Setup Slack = Required Time − Arrival Time

More jitter (capture late) → Less setup margin

(b) Hold Check



> Required Time (Hold) = Capture Edge Time + t_{hold} + Jitter
> Hold Slack = Arrival Time − Required Time

More jitter (capture early) → Hold violation risk ↑

9. Practical Observation

- Jitter budget is shared by source, network, crosstalk and supply noise.
- Always validate jitter at silicon level.
- Small improvements at each stage give large overall reduction.
- Lower jitter → Higher timing margin → Better yield and reliability.

10. Numerical Example

Given:
- Clock period (Tclk) = 2.000 ns
- Data path delay = 1.450 ns
- Setup time (t_{setup}) = 0.100 ns
- Hold time (t_{hold}) = 0.050 ns

Case	Jitter (ns)	Setup Slack (ns)	Hold Slack (ns)
Low Jitter	0.050	0.420	0.390
High Jitter	0.100	0.380	0.370

↑ Jitter → Slack reduces (Setup & Hold)

11. Viva Questions

Q1. What is clock jitter?
Q2. How does jitter affect setup and hold?
Q3. Why do we use jitter buffers?
Q4. What is the role of power filtering in jitter reduction?
Q5. How does shielding help in jitter reduction?
Q6. List best PCB practices to reduce jitter.
Q7. What is the impact of jitter on STA?
Q8. How do you specify jitter in SDC?

12. QUICK REVISION

- Low-Jitter clock source → stable edges.
- Jitter buffers → clean the clock.
- Power filtering → removes supply noise.
- Shielding → blocks EMI / crosstalk.
- PCB improvements → stable routing.
- Less jitter → more timing margin.
- Always validate jitter in sign-off.
- Jitter affects both Setup & Hold.
- Specify using set_clock_uncertainty.

STATIC TIME ANALYSIS

Slew / Transition Analysis - Concepts, Definitions, Impact and Violations

1. What is Transition ?

- Transition is the change of a signal from one logic level to another.
- It is the waveform shape of a signal edge (rising or falling).
- In STA we analyze how fast or slow the signal changes between voltage levels.

> Transition quality (slew) directly affects cell delays and timing correctness.

2. Slew Rate

- Slew rate is the rate of change of voltage with respect to time.
- Indicates how steep or gradual an edge is.

$$\text{Slew Rate (SR)} = \frac{\Delta V}{\Delta t} \quad (\text{V/time})$$

ΔV : Voltage Change, Δt : Time Taken

- High SR → Fast edge (steep). Low SR → Slew edge (gradual).
- In STA tools, slew is generally reported in terms of time (transition time) rather than V/ns.

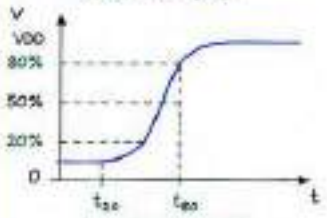
3. Transition Time

- Transition time (Slew) is the time taken for a signal to transition between two specified voltage levels.
- Commonly measured between 20% and 80% (rise) or between 80% and 20% (fall).

> In STA reports, "Slew" usually refers to this transition time.

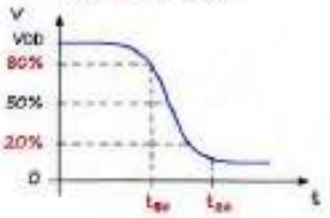
4. Rise Time and Fall Time

Rise Time (Tr)



$Tr = t_{80} - t_{20}$

Fall Time (TF)

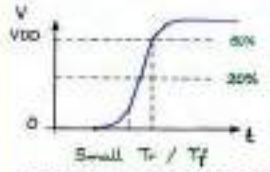


$Tf = t_{20} - t_{80}$

5. Fast Transition and Slow Transition

Fast Transition (Good)

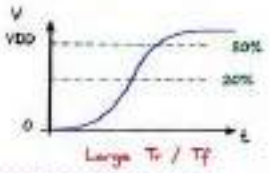
- Steep edge
- Small transition time
- High slew rate



Small Tr / Tf

Slow Transition (Bad)

- Gradual edge
- Large transition time
- Low slew rate

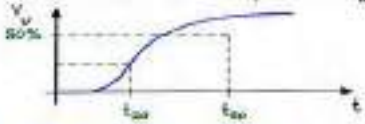


Large Tr / Tf

> Balance is required: too fast → noise / ringing, too slow → delay and power increase.

6. 20 - 80% Rise - Time Definition

- Rise time is measured between 20% of VDD to 80% of VDD on a rising edge.
- It excludes initial noise and final settling.

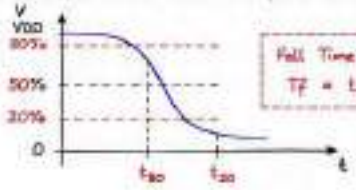


Rise Time (20-80%)

$Tr = t_{80} - t_{20}$

7. 80-20% Fall-Time Definition

- Fall time is measured between 80% of VDD to 20% of VDD on a falling edge.
- It excludes initial noise and final settling.



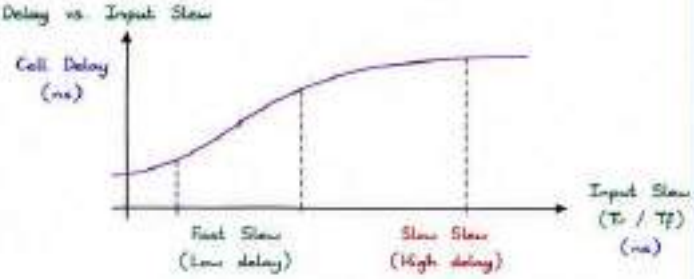
Fall Time (80-20%)

$Tf = t_{80} - t_{20}$

8. Slew Impact on Cell Delay

- Cell delay depends on input slew.
- Slow input (large transition time) → longer delay.
- Fast input (small transition time) → shorter delay.
- Each cell has a delay vs. input slew characteristic.

Delay vs. Input Slew



> Slow transition increases delay and can cause timing violations.

9. Slew Violation Concept

- Every cell / pin has a specified max and min transition (slew) limit.
- If actual slew is outside allowed range → Slew violation.
- Too slow → may cause delay increase.
- Too fast → may cause noise, ringing, power spikes and reliability issues.

Slew Limits



10. Summary Table - Slew / Transition Analysis

Term	Definition / Description
Transition	Change of signal from one logic level to another
Slew Rate	Rate of voltage change ($\Delta V/\Delta t$). Indirectly related to edge steepness
Transition Time (Slew)	Time taken for signal to move between two voltage levels (usually 20-80% or 80-20%)
Rise Time (Tr)	Time from 20% to 80% of VDD on rising edge
Fall Time (Tf)	Time from 80% to 20% of VDD on falling edge
Fast Transition	Small Tr / Tf, high slew rate, good for delay but may increase noise
Slow Transition	Large Tr / Tf, low slew rate, increases delay and power
Slew Impact	Cell delay increases with slower input slew
Slew Violation	Actual slew outside [Min Slew, Max Slew] limits

11. Practical Observation

- Proper slew is critical for accurate delay modeling.
- During CTS and buffer insertion, tools try to keep slew within limits.
- Overly aggressive buffering may create too-fast edges → ringing.
- Under-buffering leads to slow edges → delay / power increase.
- Always fix slew violations along with timing violations.

13. Viva Questions

Q1. What is transition in STA?
Q2. What is slew rate?
Q3. How is rise time defined?
Q4. How is fall time defined?
Q5. What is difference between fast and slow transition?
Q6. How does input slew affect cell delay?
Q7. What causes slew violations?
Q8. What are the effects of too fast and too slow slew?

14. QUICK REVISION

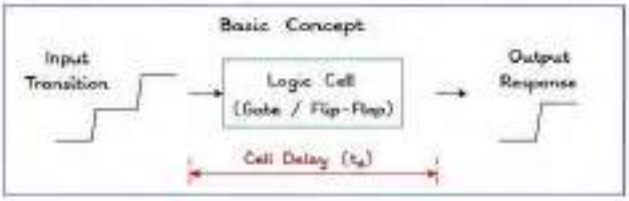
- Slew = Transition time between two voltage levels.
- Rise time = 20-80% of VDD (Tr).
- Fall time = 80-20% of VDD (Tf).
- Fast edge → small Tr/Tf → low delay.
- Slow edge → large Tr/Tf → high delay.
- Each cell has min & max slew limits.
- Slew outside limits → Slew violation.
- Keep slew within limits for accurate timing and reliability.

STATIC TIMING ANALYSIS (STA)

CELL DELAY – OVERVIEW

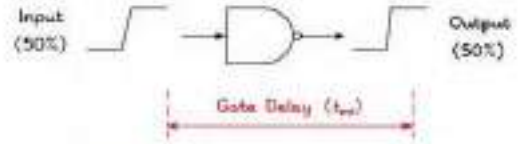
1. DEFINITION

- Cell delay is the time taken by a logic cell to produce a valid output after a valid input transition.
- It is a fundamental parameter used by STA tools to calculate timing of paths.
- Cell delay depends on input slew, output load, PVT conditions, etc.



2. GATE DELAY

- Delay of a basic logic gate (e.g., AND, OR, NAND, NOR, INV).
- Measured between input 50% point and output 50% point.



3. LOGIC-CELL DELAY

- General term for delay of any combinational or sequential cell.
- Includes internal propagation delay from input to output.

4. INPUT SLEW DEPENDENCY

- Cell delay increases as input transition becomes slower.
- Slower input slew takes longer to charge/discharge internal nodes, increasing delay.

Input Slew	Cell Delay	Reason
Fast Slew	Lower Delay	Faster internal transitions
Slow Slew	Higher Delay	Slower internal charging / discharging

5. OUTPUT CAPACITANCE DEPENDENCY

- Cell delay increases as load capacitance at output increases.
- More load → more time to charge/discharge.

Output Capacitance (C_L)	Cell Delay	Reason
Small C_L	Lower Delay	Less time to charge/discharge
Large C_L	Higher Delay	More charge/discharge time

6. PVT DEPENDENCY

- Cell delay varies with Process, Voltage, and Temperature.
- Worst delay usually occurs at High Temp, Low Voltage, Slow process.

Condition	Delay Trend
Low Voltage	Increases
High Temperature	Increases
Slow Process (SS)	Increases
Fast Process (FF)	Decreases

7. LIBRARY CHARACTERIZATION

- Library characterization is the process of measuring cell delays for different operating conditions.
- Performed using SPICE simulation or silicon measurement.
- Results are stored in Standard Cell Library (.lib) files.

8. DELAY TABLES

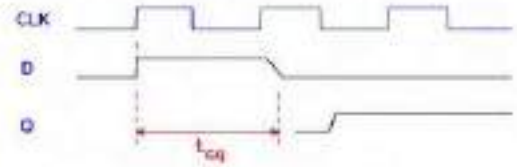
- Delays are stored in multi-dimensional tables inside the .lib file.
- Typically indexed by:
  - Input slew (rise/fall)
  - Output load capacitance
  - PVT corner
- Example: 2D table (rise delay)

Output Load C_L (fF)	Input Slew (ps)			
	20	50	100	200
10	18	21	24	30
20	19	23	27	34
50	22	27	33	42
100	25	31	38	48

(Values are in picoseconds)

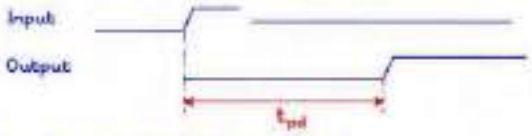
9. SEQUENTIAL CELL DELAY

- Sequential cells have two main delays:
  - Clock-to-Q Delay (t_{cq})
  - Setup/Hold related delays (not covered here)
- Example cells: Flip-Flop, Latch, Register.



10. COMBINATIONAL CELL DELAY

- Delay between input transition and output response.
- Example: Gate, MUX, Buffer, Level Shifter.
- Measured as propagation delay (t_{pd}).



11. CLOCK-TO-Q DELAY (t_{cq})

- Time from the active clock edge to the point when output Q starts changing.
- Depends on cell type, clock edge (rise/fall), PVT, load.
- Critical for launch timing in sequential paths.

Cell Type	Delay Name	Definition
Flip-Flop (DFF)	t_{cq}	Clock active edge → Q output 50%
Latch (Transparent)	t_q	Enable active → Q output 50%

COMPARISON: COMBINATIONAL VS SEQUENTIAL DELAYS

Aspect	Combinational Cell Delay	Sequential Cell Delay
Cells	Logic gates, MUX, Buffer, etc.	Flip-Flops, Latches, Registers
Main Delay	Propagation delay (t_{pd})	Clock-to-Q delay (t_{cq})
Measured Between	Input transition → Output response	Active clock edge → Output response
Depends On	Input slew, Output load, PVT	Clock, Output load, PVT
Used For	Internal data path timing	Launch timing in sequential paths

PRACTICAL OBSERVATIONS

- Slow input slew and high output load increase cell delay.
- Always apply worst-case PVT corner in STA.
- Accurate library characterization is key for reliable timing sign-off.
- Cell delay is the building block for path delay calculation.

VIVA QUESTIONS

Q1. What is cell delay?
Q2. What factors affect cell delay?
Q3. Why does delay increase with slow input slew?
Q4. How does output capacitance affect delay?
Q5. What is library characterization?
Q6. What is difference between combinational and sequential cell delay?
Q7. What is clock-to-Q delay?
Q8. Why is clock-to-Q delay important in STA?

QUICK REVISION

- ✓ Cell delay = time for cell to produce valid output.
- ✓ Depends on input slew, output load, and PVT.
- ✓ Delays are stored in library tables.
- ✓ Combinational delay = propagation delay (t_{pd}).
- ✓ Sequential delay main component = t_{cq}.
- ✓ Clock-to-Q delay = clock active edge → Q output 50%.
- ✓ Library files (.lib) contain all delay information.

KEY FORMULAS

Propagation Delay (t_{pd})
= $T_{output}(50\%) - T_{input}(50\%)$

Clock-to-Q Delay (t_{cq})
= $T_Q(50\%) - T_{clock}$(active edge)

Path Delay
= Σ Cell Delays along the path

STATIC TIME ANALYSIS

Net Delay / Interconnect Delay – Concepts, Modeling and Analysis

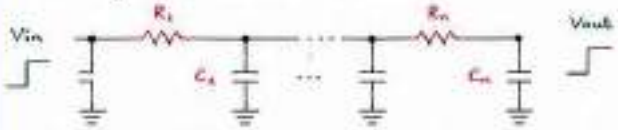
1. Net Delay / Interconnect Delay – Overview

- Interconnect (wire) between two pins introduces delay.
- This delay is called Net Delay or Interconnect Delay.
- It depends on wire length, resistance, capacitance and routing conditions.

> Net Delay = Delay of the signal through the interconnect between driving pin and receiving pin.

2. Wire Delay

- Delay occurs as signal charges/discharges the capacitance through the resistance of the wire.
- Modeled as Distributed RC network.
- For long wires, RC must be considered.



3. Resistance (R)

- Resistance is offered by the metal wire.
- Proportional to wire length.
- Depends on metal resistivity (ρ), length (L) and cross sectional area (A).

$$R = \rho \frac{L}{A} \quad (\Omega)$$

4. Capacitance (C)

- Capacitance exists between wire and ground (C to GND) and between adjacent wires (C coupling).
- Proportional to wire length.

$$C = C' \times L \qquad (C' = \text{capacitance per unit length})$$

5. RC Delay

- First-order Elmore delay approximation for a net:

$$t_{delay} \approx 0.69 \times R_{eq} \times C_{eq}$$

- R_{eq} : Effective resistance seen by the capacitance.
- C_{eq} : Total capacitance seen at the receiving node.

6. Wire Length

- Longer the wire → Higher R and C.
- Therefore, delay increases with length.
- Roughly, delay $\propto L^2$ for long on-chip interconnects.



> For long wires Delay increases super-linearly ($\approx L^{1.5}$ to L^2)

7. Metal Layer

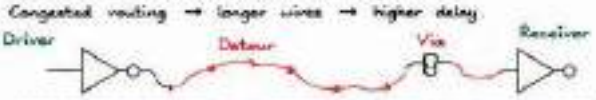
- Higher metal layers (e.g., M5, M6, ...) are wider and thicker.
- They have lower resistance than lower layers.
- But they have higher capacitance to substrate.

Metal Layer	Width	Thickness	Resistance	Capacitance
M1 (Low)	Small	Thin	High	Low
M3 (Mid)	Medium	Medium	Lower	Medium
M6 (High)	Large	Thick	Low	High

> Higher layer → Lower R → Lower delay (but higher C)

8. Routing Impact

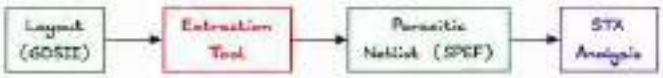
- Detours, jogs and long routes increase length.
- More vias → extra resistance and capacitance.
- Congested routing → longer wires → higher delay.



> Short, direct routes with fewer vias → Lower delay

9. Parasitic Extraction

- Actual R and C values are obtained by extracting parasitics from the layout.
- Extraction includes all wires, vias, couplings and substrate caps.
- Ensures accurate timing analysis.



10. SPEF (Standard Parasitic Exchange Format)

- SPEF is the standard file format to store extracted parasitics.
- Contains resistances (R), capacitances (C) and couplings.
- Used by STA tools for accurate delay computation.

> SPEF → Standard format for exchanging parasitic data between extraction and STA tools.

11. Pre-Route Estimated Delay

- Before routing, accurate parasitics are not known.
- STA uses estimated RC models.
- Based on wire load models, placement and technology data.
- Less accurate but useful for early timing estimation.

Pre-Route Delay Estimation Inputs
• Wire load models (fan-out based)
• Estimated wire length
• Technology RC per unit length
• Average metal layer assumptions

12. Post-Route Actual Delay

- After routing, real parasitics are extracted.
- STA uses SPEF for accurate delay calculation.
- More accurate than pre-route estimation.
- Used for sign-off timing.

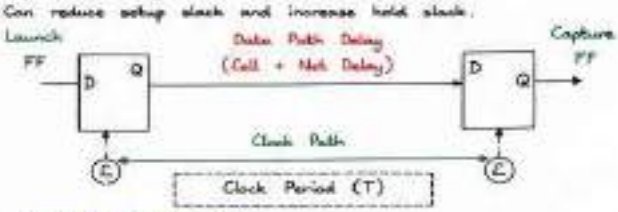
Post-Route Delay Inputs
• Extracted R and C from real layout
• Exact wire length, metal layer and width
• Actual vias and coupling caps

13. Pre-Route vs Post-Route Delay Comparison

Aspect	Pre-Route Estimated Delay	Post-Route Actual Delay
Parasitics	Estimated (wire load model)	Extracted (SPEF)
Accuracy	Lower (approximate)	High (accurate)
Wire Length	Estimated	Exact
Metal Layer	Assumed	Actual
Use	Early timing analysis	Sign-off timing
Variation	More optimistic	More pessimistic/accurate

14. Effect of Net Delay on Timing

- Higher net delay → Arrival time increases.
- Can reduce setup slack and increase hold slack.



15. Practical Observation

- Long interconnects dominate delay in modern chips.
- Accurate parasitic extraction is critical for sign-off.
- Post-route delay is always ≥ pre-route delay.
- Proper routing, layer selection and congestion management reduce interconnect delay.

16. Quick Formula Summary

- Resistance: $R = \rho \frac{L}{A}$
- Capacitance: $C = C' \times L$
- RC Delay (Elmore): $t \approx 0.69 R_{eq} C_{eq}$
- Delay vs Length: $t \propto L^2$ (approx.)

17. Viva Questions

Q1. What is net delay?
Q2. How does wire length affect delay?
Q3. Write formula for wire resistance.
Q4. Write formula for wire capacitance.
Q5. How is RC delay calculated?
Q6. Why do higher metal layers give less delay?
Q7. What is SPEF?
Q8. Difference between pre-route and post-route delay?

18. Common Terms

- R : Resistance (Ω)
- C : Capacitance (F)
- C' : Capacitance per unit length (F/m)
- SPEF : Standard Parasitic Exchange Format
- RC Delay : Delay due to R and C of wire

19. QUICK REVISION

- Net delay comes from R & C of wires.
- R ↑ with length, C ↑ with length.
- Delay ≈ $0.69 R_{eq} C_{eq}$.
- Higher metal → Lower R, higher C.
- Extract parasitics → SPEF.
- Pre-route → estimated, less accurate.
- Post-route → actual, accurate.
- Longer nets → higher arrival time.

STATIC TIMING ANALYSIS (STA)

PVT VARIATION – OVERVIEW

1. PROCESS VARIATION

- Process variation occurs due to manufacturing of transistors.
- It causes variation in threshold voltage (V_{th}), mobility (μ), gate length (L), oxide thickness (t_{ox}), etc.
- These variations change transistor drive current → affects delay.

1.1 Slow Process

- Transistors are weaker than nominal.
- Higher resistance, lower drive current.
- Results in larger delays.

1.2 Fast Process

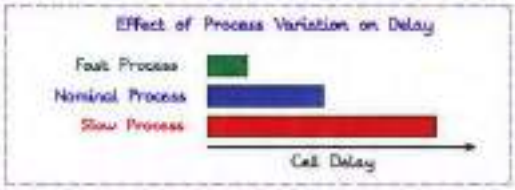
- Transistors are stronger than nominal.
- Lower resistance, higher drive current.
- Results in smaller delays.

1.3 Process Variation

- Random variation across wafer, lot, die, and device.
- Even transistors on same die may vary.

1.4 Manufacturing Variation

- Caused by lithography, etch, deposition, doping, CMP, and other fabrication steps.
- Leads to both global (die-to-die) and local (within-die) variation.



Process Corner Definition		
Corner	Description	Delay Impact
Slow (SS)	Worst transistor drive	Maximum delay
Nominal (TT)	Typical transistor drive	Typical delay
Fast (FF)	Best transistor drive	Minimum delay

2. VOLTAGE VARIATION

- Supply voltage variations directly affect transistor speed.

2.1 Vmin and Vmax

- Vmin : Minimum allowed supply voltage.
- Vmax : Maximum allowed supply voltage.
- Delays are worst at Vmin and best at Vmax.



2.2 IR Drop

- Voltage drop in power distribution network due to current flow.
- Causes local supply to be lower than ideal.
- Increases delay and degrades timing.

2.3 Supply Noise

- High frequency noise on supply due to switching activity, crosstalk, ripple, etc.
- Causes momentary voltage fluctuation.
- Adds uncertainty in delay.



2.4 Voltage Impact on Delay

- Delay ∝ $1/(V_{DD} - V_{th})^{\alpha}$ ($\alpha \approx 1.5$ to 2)
- Lower voltage → smaller overdrive ($V_{DD} - V_{th}$) → lower drive → higher delay.

Impact of Voltage on Delay		
Condition	Voltage Level	Delay
Low Voltage	VDD close to Vmin	High delay
Nominal Voltage	Typical VDD	Typical delay
High Voltage	VDD close to Vmax	Low delay

3. TEMPERATURE VARIATION

- Temperature change affects carrier mobility and leakage.
- It has a strong impact on cell delay.

3.1 High Temperature

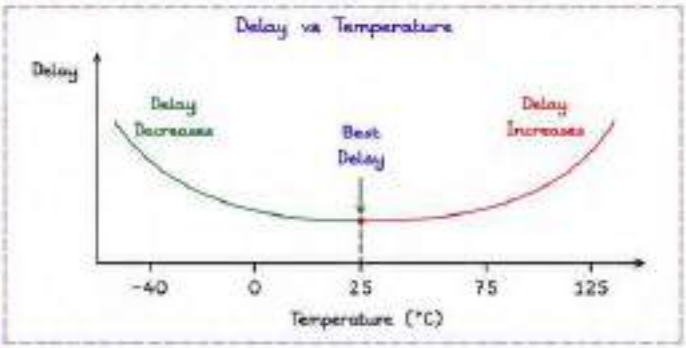
- Carrier mobility decreases.
- Leakage current increases.
- Drive current reduces → delay increases.

3.2 Low Temperature

- Carrier mobility increases.
- Leakage current reduces.
- Drive current improves → delay decreases.

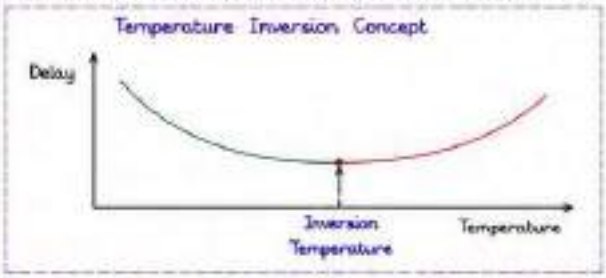
3.3 Temperature Impact on Delay

- Delay generally increases with temperature.
- High temperature → worst delays.
- Low temperature → best delays (until inversion point).



3.4 Temperature Inversion

- For some cells (especially complex or with series PMOS/NMOS), delay stops improving after a point at very low temperature.
- Due to increased resistance and threshold voltage variation.
- Hence delay may increase again → called temperature inversion.



PVT CORNERS SUMMARY

Corner	Process	Voltage	Temperature	Use Case
SS, Vmin, Tmax	Slow	Min	High	Worst setup
SS, Vmin, Tmin	Slow	Min	Low	Worst hold
TT, Nom, Nom	Typical	Nominal	Nominal	Typical case
FF, Vmax, Tmin	Fast	Max	Low	Best setup
FF, Vmax, Tmax	Fast	Max	High	Best hold

PRACTICAL OBSERVATIONS

- Always analyze timing at multiple PVT corners.
- Worst setup usually at: Slow process, Low voltage, High temperature.
- Worst hold usually at: Fast process, High voltage, Low temperature.
- IR drop and supply noise can make local conditions worse than global.
- PVT variation is the largest source of timing uncertainty.

VIVA QUESTIONS

Q1. What is process variation?
Q2. What is the difference between slow and fast process?
Q3. What is IR drop?
Q4. How does voltage affect cell delay?
Q5. What is supply noise?
Q6. Why does delay increase at high temperature?
Q7. What is temperature inversion?
Q8. Which PVT corner is worst for setup?
Q9. Which PVT corner is worst for hold?
Q10. Why PVT analysis is important in STA?

QUICK REVISION

- ✓ Process: Slow ↑ delay, Fast ↓ delay.
- ✓ Variation comes from manufacturing.
- ✓ Voltage: Vmin, ↑ delay; Vmax ↓ delay.
- ✓ IR drop lowers local V_{DD} → ↑ delay.
- ✓ Supply noise adds uncertainty.
- ✓ Temperature: High ↑ delay, Low ↓ delay.
- ✓ Temperature inversion at very low T.
- ✓ Worst setup: SS, Vmin, Tmax.
- ✓ Worst hold: FF, Vmax, Tmin.
- ✓ Analyze all PVT corners in STA.

KEY FORMULAS

- Cell Delay = $1/(V_{DD} - V_{th})^{\alpha}$ ($\alpha \approx 1.5$ to 2)
- Delay Change = $k_1 \cdot \Delta V / (V_{DD} - V_{th})^2$
- Temperature impact on mobility: $\mu(T) \propto T^{-m}$ ($m \approx 1.5$)
- IR Drop = $I \times R_{pd}$

DATA TO REMEMBER

- Typical Temp (Nominal) = 25°C
- Operating Temp Range: −40°C to +125°C
- Vmin < Nominal < Vmax
- Process Corners: SS (Slow), TT (Typical), FF (Fast)
- PVT drives worst case timing.

Author : Konda Tharun Vivekananda

STATIC TIME ANALYSIS

RC Corners – Concepts, Impact and Timing Interpretation

1. RC Corners – Introduction

- RC corner represents different combinations of Resistance (R) and Capacitance (C) values.
- Variation in R and C causes variation in wire (net) delay and hence timing.
- STA analyses both worst and best RC conditions.

> Net Delay (Interconnect) = f (R, C, Length, Metal Layer)

2. RC Variation

- Manufacturing, process, temperature and routing cause variation in R and C.
- These variations are modeled in STA as RC corners.

> RC Variation → Change in R and/or C from nominal values.

3. Resistance Variation

- Resistance (R) of a wire depends on:
  - Metal type, width, thickness, length, temperature.
- Higher R → larger voltage drop and more delay.
- Lower R → less delay.

$$R = \rho \frac{L}{A}$$ (ρ → resistivity, L → length, A → cross area)

4. Capacitance Variation

- Capacitance (C) depends on:
  - Metal layer, spacing, width, length, coupling to neighbours.
- Higher C → slower charging/discharging → more delay.
- Lower C → faster transitions → less delay.

$$C = C' \times L$$ (C' → capacitance per unit length)

5. Worst RC Corner

- Occurs when both R and C are high.
- Produces maximum delay on interconnect.
- Critical for Setup timing (data arrives late).

> Worst RC = High R + High C → Max Delay

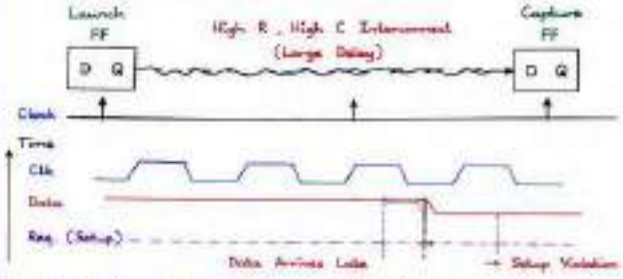
6. Best RC Corner

- Occurs when both R and C are low.
- Produces minimum delay on interconnect.
- Critical for Hold timing (data arrives early).

> Best RC = Low R + Low C → Min Delay

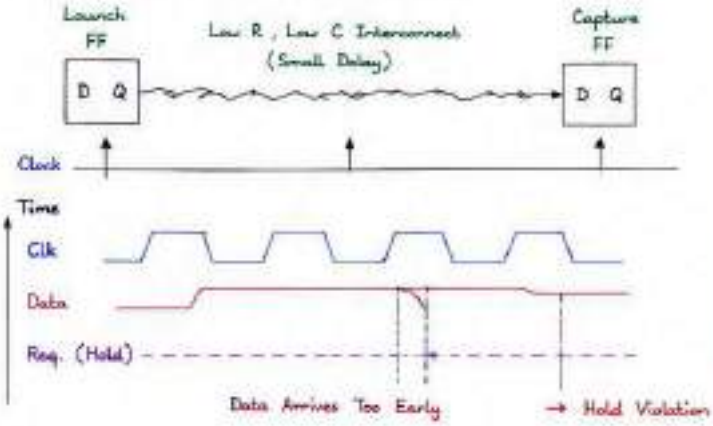
7. Setup with High R / High C (Worst RC)

- High R + High C → large wire delay.
- Data arrival time increases.
- May cause Setup violation (data too late).



8. Hold with Low R / Low C (Best RC)

- Low R + Low C → small wire delay.
- Data arrival time reduces.
- May cause Hold violation (data too early).



9. Interconnect Impact

- Interconnect delay is a major part of total path delay.
- Impacted by R, C, length, coupling, metal layer and routing.
- Dominant for long nets and higher layers.

> Interconnect Delay ↑ → Data Arrival Time ↑ (Setup risk)
> Interconnect Delay ↓ → Data Arrival Time ↓ (Hold risk)

10. Routing Dependency

- Routing topology directly controls R and C.
- Detours, vias, narrow metals → increases R and C.
- Well-planned routing → balanced R/C → better timing.



11. RC Corners Summary Table

Corner	R	C	Interconnect Delay	Critical Check	Effect
Worst RC (HRHC)	High	High	Maximum	Setup	Data arrives late (Setup violation risk)
Best RC (LRLC)	Low	Low	Minimum	Hold	Data arrives early (Hold violation risk)
Typical RC	Nominal	Nominal	Nominal	General timing	Balanced condition

12. Pre-Route Estimated Delay vs Post-Route Actual Delay

Aspect	Pre-Route Estimated Delay	Post-Route Actual Delay
RC Values	Estimated (wire load model)	Extracted (SPEF)
Accuracy	Lower (approximate)	Higher (accurate)
Data	Based on fan-out, placement, wire length estimate	Based on real layout parasitics (R, C, vias, coupling)
Use	Early STA (Planning stage)	Sign-off STA (Final stage)

13. RC Delay (Elmore Approximation)

For a single π or distributed RC network, first-order Elmore delay:

$$t_{elmore} = R_{eq} \times C_{eq}$$

R_{eq} → Effective resistance seen by the capacitance.
C_{eq} → Total capacitance seen at the receiving node.

14. Practical Observation

- Always analyse both worst and best RC corners.
- High R or High C → setup problems.
- Low R or Low C → hold problems.
- Routing quality is key to control RC.
- Final timing sign-off must use extracted parasitics (SPEF).

15. Example

Setup (Worst RC)

Given:
R = 200 Ω
C = 300 fF
R_{eq} = 200 Ω, C_{eq} = 300 fF
$t_{delay} = R_{eq} \times C_{eq}$
= $200 \times 300 \times 10^{-15}$
= 60 ps (Max Delay)

Hold (Best RC)

Given:
R = 50 Ω
C = 60 fF
R_{eq} = 50 Ω, C_{eq} = 60 fF
$t_{delay} = R_{eq} \times C_{eq}$
= $50 \times 600 \times 10^{-15}$
= 3 ps (Min Delay)

16. Viva Questions

Q1. What is an RC corner?
Q2. Why High R & High C is worst for setup?
Q3. Why Low R & Low C is worst for hold?
Q4. How does interconnect delay affect timing?
Q5. What factors affect resistance of a wire?
Q6. What factors affect capacitance of a wire?
Q7. What is SPEF?
Q8. Difference between pre-route and post-route delay estimation?

17. QUICK REVISION

- RC Corners → Combinations of R and C.
- Worst RC → High R + High C → Setup.
- Best RC → Low R + Low C → Hold.
- High R/C → Delay ↑ → Data late.
- Low R/C → Delay ↓ → Data early.
- Interconnect + Routing → Major timing factor.
- Use SPEF for accurate timing sign-off.

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

MCMM – Multi-Corner Multi-Mode

1. Introduction

- Real silicon works under different Process, Voltage, Temperature (PVT) conditions and in different operating modes.
- To get timing sign-off across all conditions, STA uses MCMM (Multi-Corner Multi-Mode) analysis.
- Ensures that design meets timing in all possible scenarios.

MCMM Concept

Multi-Corner (PVT and RC combinations) **X** Multi-Mode (Different operating modes) → MCMM Analysis

2. Multi-Corner

Multi-Corner considers all significant variations in PVT and RC parasitics.

2.1 Process Corners

- Due to fabrication variations, transistor parameters change.
- Corners used in STA:
  - Slow (SS) : Worst drive, higher delay
  - Typical (TT) : Nominal behavior
  - Fast (FF) : Best drive, lower delay

2.2 Voltage Corners

- Supply voltage variation affects delay.
- Common corners:
  - Vmin : Minimum voltage (worst delay)
  - Vnom : Nominal voltage
  - Vmax : Maximum voltage (best delay)

2.3 Temperature Corners

- Silicon behavior changes with temperature.
- Common corners:
  - Low Temp (e.g., -40°C)
  - Nominal Temp (e.g., 25°C)
  - High Temp (e.g., 125°C)

2.4 RC Corners

- Interconnect parasitics (R and C) also vary.
- Common RC corners:
  - Best RC (min R, min C) → lower delay
  - Typical RC (nominal R, C)
  - Worst RC (max R, max C) → higher delay

Example : PVT × RC Corner Combinations

Corner ID	Process	Voltage	Temperature	RC
SS_Vmin_Tmax_WC	Slow (SS)	Vmin	125°C	Worst RC
SS_Vmin_Tmin_BC	Slow (SS)	Vmin	-40°C	Best RC
FF_Vmax_Tmin_BC	Fast (FF)	Vmax	-40°C	Best RC
FF_Vmax_Tmax_WC	Fast (FF)	Vmax	125°C	Worst RC
TT_Vnom_Tnom_TC	Typical (TT)	Vnom	25°C	Typical RC

More corners → More accuracy → More runtime

3. Multi-Mode

Design operates in different modes, each with different timing requirements.

3.1 Functional Mode

- Normal operating mode of the chip.
- All logic active according to specification.

3.2 Scan / Test Mode

- Scan chains are enabled, functional logic is bypassed.
- High switching activity in scan chains.
- Different timing paths exist.

3.3 Low-Power Mode

- Some blocks are power-gated or clock-gated.
- Reduced voltage / frequency.
- Timing characteristics change.

3.4 Different Operating Scenarios

- Multiple frequency points.
- Different clock sources.
- Different data arrival scenarios.

Example Modes

Mode	Description	Typical Use
Functional Mode	Normal operation	Performance validation
Scan / Test Mode	Scan enabled, functional logic inactive	ATPG and test timing
Low-Power Mode	Power-gating / clock-gating configured	Power intent validation
Alternate Frequency	Different clock frequencies	DVFS / frequency scaling

4. MCMM Flow

Define PVT Corners → Define Modes → Create MCMM Scenarios → Run STA for all scenarios → Analyze Worst-case Results

Scenario = 1 PVT Corner + 1 Mode + 1 RC Model

5. Importance of MCMM

- Captures real-world variations.
- Prevents timing failures in any condition.
- Ensures robust and silicon-proven design.

Sample MCMM Scenario Set

Scenario	Process	Voltage	Temp	RC	Mode
S1	SS	Vmin	125°C	WC	Functional
S2	SS	Vmin	125°C	WC	Scan
S3	TT	Vnom	25°C	TC	Functional
S4	FF	Vmax	-40°C	BC	Low-Power
S5	SS	Vmin	25°C	WC	Low-Power

TC = Typical RC, WC = Worst RC, BC = Best RC

Practical Observations

- Worst setup usually at: SS, Vmin, Tmax, Worst RC, Functional mode.
- Worst hold usually at: FF, Vmax, Tmin, Best RC, Scan mode.
- Corner explosion is controlled by selecting only critical combinations.
- Always ensure coverage of power intent related modes.

Viva Questions

Q1. What is MCMM in STA ?
Q2. What are the components of multi-corner ?
Q3. Which corner gives worst setup delay ?
Q4. Which corner gives worst hold delay ?
Q5. What are common voltage corners ?
Q6. What is scan/test mode in MCMM ?
Q7. Why is RC variation considered ?
Q8. What is the benefit of MCMM analysis ?

Quick Revision

- ✓ MCMM = Multi-Corner + Multi-Mode
- ✓ Multi-corner → PVT and RC variations
- ✓ Multi-mode → different operating modes
- ✓ Worst setup → SS, Vmin, Tmax, WC
- ✓ Worst hold → FF, Vmax, Tmin, BC
- ✓ Ensures timing closure in all conditions

Key Formulas

- Setup Delay $\propto 1/(V_{DD} - V_{th})^{\alpha}$ ($\alpha \approx 1.5$ to 2)
- Delay = R × C
- Temperature impact on mobility: $\mu(T) \propto T^{-m}$ ($m \approx 1.5$)

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

MCMM Importance – Why MCMM is Critical

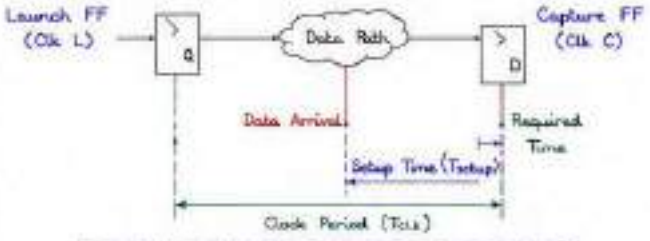
1. Timing Closure

- MCMM ensures that all timing paths meet timing requirements for all PVT corners, RC conditions and operating modes.
- Helps to achieve zero (or positive) slack across all scenarios.
- Avoids late-stage surprises and re-spins.



2. Setup Closure

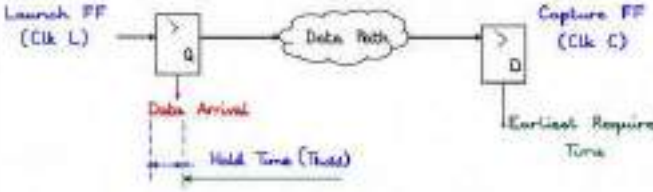
- Setup timing checks that data arrives before the required time at the capture flop.
- Worst-case for setup occurs at:
  - Slow process, Low voltage, High temperature, Worst RC, Functional mode.
  - MCMM checks all these combinations.



Setup Slack = Required Time − Data Arrival

3. Hold Closure

- Hold timing ensures data does not change too early after the capture edge.
- Worst-case for hold occurs at:
  - Fast process, High voltage, Low temperature, Best RC, Functional or Scan mode.
  - MCMM ensures hold slack ≥ 0 in all scenarios.



Hold Slack = Data Arrival − Earliest Required Time

4. Yield

- MCMM reduces silicon failure by covering all variations.
- Considering multiple corners/modes increases the probability of chips meeting timing in real-world.
- Higher yield → More good dies → Lower cost per chip.

	Without MCMM	With MCMM
Variation Coverage	Limited	Wide (All PVT + Modes)
Timing Violations	May exist in some cases	Fixed in all cases
Silicon Yield	Low	High
Risk of Respins	High	Low

5. Reliability

- MCMM accounts for worst operating conditions → robust design.
- Prevents timing failures due to environmental and operational stresses.
- Improves long-term reliability of the chip.

Reliability Factors Improved by MCMM
- ✓ Variation robust design
- ✓ Lower chance of field failures
- ✓ Stable operation across environments
- ✓ Handles aging and variation

6. Performance

- MCMM helps to optimise design for best performance.
- Identifies performance-limiting scenarios and allows targeted optimization.
- Ensures chip meets required speed across all modes.

Aspect	How MCMM Helps Performance
Speed Optimization	Finds slowest scenarios and optimizes paths
Balanced Design	Avoids over-designing for only one scenario
Mode-Specific Tuning	Optimize speed for each operating mode
Clock Frequency	Accurately predict max frequency in all scenarios

7. Signoff

- Signoff requires proof that design meets timing in all specified PVT corners, RC conditions and modes.
- MCMM is industry-standard requirements for signoff.
- Without MCMM, timing signoff is incomplete.

Signoff Checklist (MCMM)
- ✓ All Setup checks pass in all scenarios
- ✓ All Hold checks pass in all scenarios
- ✓ All operating modes verified
- ✓ All corners (PVT + RC) verified
- ✓ Timing report archived for each scenario

Summary Table – MCMM Importance

Importance Area	Why it is Important	Worst-Case for Setup	Worst-Case for Hold	Benefit
Timing Closure	Ensure zero slack in all scenarios	Slow, Low V, High T, Worst RC	Fast, High V, Low T, Best RC	No violations anywhere
Setup Closure	Data arrives before required time	Slow, Low V, High T, Worst RC	–	Meets setup in all cases
Hold Closure	Data does not change too early	–	Fast, High V, Low T, Best RC	Meets hold in all cases
Yield	More good chips in production	Full variation coverage	Full variation coverage	Higher yield, lower cost
Reliability	Robust operation across conditions	All corners	All corners	Fewer field failures
Performance	Achieve target speed in all modes	Identify slow scenarios	Avoid hold failures	Better speed, balance
Signoff	Industry requirement for closure	All PVT + RC + Modes	All PVT + RC + Modes	Complete and reliable

Practical Observations
- Without MCMM, timing may pass in typical case but fail in silicon.
- Always analyze worst-case for setup and best-case for hold.
- More scenarios → more runtime, but better confidence.
- MCMM reduces late-stage surprises and silicon respins.

Viva Questions
- Q1. What is MCMM?
- Q2. Why is MCMM important?
- Q3. What is the worst-case corner for setup?
- Q4. What is the worst-case corner for hold?
- Q5. How does MCMM improve yield?
- Q6. What is mode in MCMM?
- Q7. What are the benefits of MCMM?
- Q8. What is required for MCMM signoff?

Quick Revision
- ✓ MCMM = Multi-Corner Multi-Mode analysis
- ✓ Covers PVT, RC variations and operating modes
- ✓ Ensures timing closure in all scenarios
- ✓ Setup worst-case: Slow, Low V, High T, Worst RC
- ✓ Hold worst-case: Fast, High V, Low T, Best RC
- ✓ Improves yield, reliability and performance
- ✓ Essential for timing signoff

Key Formulas
- Setup Slack = Required Time − Arrival Time
- Hold Slack = Arrival Time − Earliest Required Time
- Delay $= 1 / (V_{DD} - V_{th})^{\alpha}$ (α = 1.5 to 2)
- $\mu(T) = T^{1-m}$ ($m \approx 1.5$)

STATIC TIME ANALYSIS

OCV - On-Chip Variation - Concepts, Sources, Types and Impact

1. What is OCV?

- OCV stands for On-Chip Variation.
- It refers to the variations in device and interconnect parameters across a chip.
- These variations cause timing to differ from nominal values.

> OCV → Variation of delay / timing within a chip due to R, C, process, voltage and temperature.

2. Why Variation Occurs Within a Chip?

- Manufacturing process creates non-uniformities.
- Local changes in doping, oxide thickness, line width etc.
- IR drop and local heating cause voltage and temperature gradients.
- Interconnect geometry and density variations.
- These lead to different delays in different locations.

3. Process Variation

- Random variation in transistor parameters.
- Affects mobility, threshold voltage (V_{th}), drive strength.
- Fast process → smaller delay.
- Slow process → larger delay.
- Typically modeled as ± variation around nominal.

4. Voltage Variation

- Caused by IR drop, power grid variations.
- Lower Vdd → slower cells, higher delay.
- Higher Vdd → faster cells, lower delay.

5. Temperature Variation

- Caused by uneven power dissipation across chip.
- Higher Temperature → lower mobility → more delay.
- Lower Temperature → higher mobility → less delay.

6. Types of Variation

(A) Based on Scope

(i) Global Variation

- Variation that affects the whole chip in the same way.
- Example : All cells slower / faster together.
- Causes : Global process shift, global voltage shift.

(ii) Local Variation

- Variation that differs from one region to another.
- Example : Certain regions slower, others faster.
- Causes : Local process, IR drop, temperature gradients.

(B) Based on Chip-to-Chip

(iii) Inter-die Variation

- Variation from one die (chip) to another.
- Due to process lot, wafer position, die-to-die.

(iv) Intra-die Variation

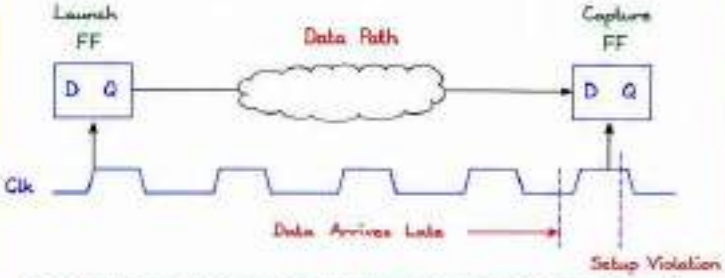
- Variation within the same die.
- Due to local pattern density, IR drop, temperature, local lithography effects.

7. Variation Summary Table

Variation Type	Scope	Example Cause	Effect on Delay
Global	Whole Chip	Global process / Vdd	All paths increase / decrease together
Local	Within Chip	Local IR drop / Temp gradient	Some paths ↑ delay, others ↓ delay
Inter-die	Chip to Chip	Wafer lot / process shift	Different chips have different delays
Intra-die	Within Same Chip	Local RC, density, heating	Different paths have different delays

8. Impact on Setup (Worst Case)

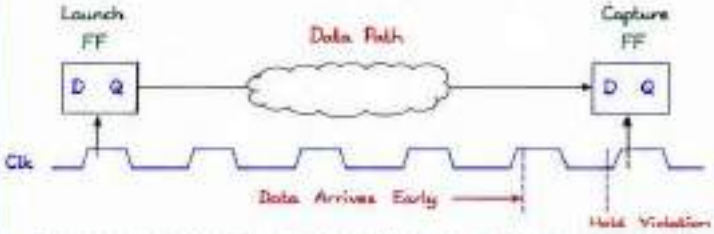
- For Setup timing, we consider conditions that make data path delay maximum.
- Slow process, Low Vdd, High Temperature, High R, High C → increase delay.



> High R / High C / Slow Process / Low Vdd / High Temp → Longer Delay → Data Arrives Late → Setup Slack Decreases → Setup Violation

9. Impact on Hold (Worst Case)

- For Hold timing, we consider conditions that make data arrive as early as possible.
- Fast process, High Vdd, Low Temperature,
- Low R, Low C → reduce delay.

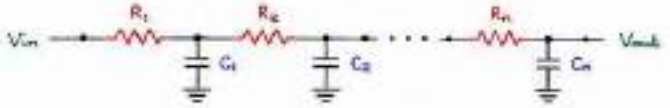


> Low R / Low C / Fast Process / High Vdd / Low Temp → Shorter Delay → Data Arrives Early → Hold Slack Decreases → Hold Violation

10. Interconnect Impact

- Interconnect (wires) have R and C.
- Variation in R and C strongly affects net delay.
- Delay of RC interconnect (Elmore approximation) :

$$t_{delay} = 0.69 \times R_{eq} \times C_{eq}$$



- Increase in R or C → increases net delay (setup risk).
- Decrease in R or C → decreases net delay (hold risk).

11. Routing Dependency

- Routing decides wire length, width, spacing and metal layer.
- Longer routes, narrow width, lower metal layers → higher R and C → higher delay.
- Shorter routes, wider width, higher metal layers → lower R and C → lower delay.
- Hence routing quality directly impacts OCV and timing.

12. Quick Comparison

Corner	R	C	Process	Vdd	Temp	Used for
Setup (Worst RC)	High	High	Slow	Low	High	Setup Check
Hold (Worst RC)	Low	Low	Fast	High	Low	Hold Check

13. Practical Observation

- Not all paths see same variation.
- Critical paths are most sensitive.
- OCV ensures guardband for real silicon behavior.
- Sign-off must include OCV models to avoid silicon failures.

14. Viva Questions

Q1. What is OCV?
Q2. Why does variation occur within a chip?
Q3. What are global and local variations?
Q4. What is inter-die variation?
Q5. What is intra-die variation?
Q6. Which RC corner is worst for setup?
Q7. Which RC corner is worst for hole?
Q8. How does routing affect OCV and timing?

15. QUICK REVISION

- OCV → On-Chip Variation.
- Causes → Process, Voltage, Temp, R, C variations.
- Types → Global, Local, Inter-die, Intra-die.
- Setup worst → High R, High C, Slow, Low Vdd, High Temp.
- Hold worst → Low R, Low C, Fast, High Vdd, Low Temp.
- Routing impacts R, C → Net Delay.

STATIC TIME ANALYSIS

OCV Derating - Concepts , Types , Application and Impact

1. What is Derating ?

- Derating is the technique used in OCV (On-Chip Variation) based STA to account for variations in cell and interconnect delays.
- We scale (derate) the nominal delays using multipliers so that analysis becomes more realistic and conservative.

> Derating = Adjusting delays by a multiplier (>1 or <1) to cover variations.

2. Delay Multiplier (Derate Factor)

- A numeric factor applied to nominal delay.
- > 1.0 → increases delay (pessimistic).
- < 1.0 → decreases delay (optimistic).
- Denoted as : M (multiplier) or D (derate).

> Derated Delay = Nominal Delay × Derate Multiplier

3. Cell - Delay Derate

- Applied on intrinsic cell delays (gate delay).
- Accounts for variation in transistor parameters of cells.
- Separate derates for early (fast) and late (slow) conditions.

> Cell Delay (Early) = Cell Delay (Nom) × M_{early}
> Cell Delay (Late) = Cell Delay (Nom) × M_{late}

4. Net - Delay Derate

- Applied on interconnect delays (RC delay).
- Accounts for variation in wire R and C.
- Separate derates for early and late as well.

> Net Delay (Early) = Net Delay (Nom) × M_{early_net}
> Net Delay (Late) = Net Delay (Nom) × M_{late_net}

5. Early Derate

- Multiplier < 1.0 used to model fast (best-case) condition.
- Reduces delay → data arrives earlier.
- Used for HOLD timing (worst case).

> Early Derate Factor (M_{early}) < 1.0

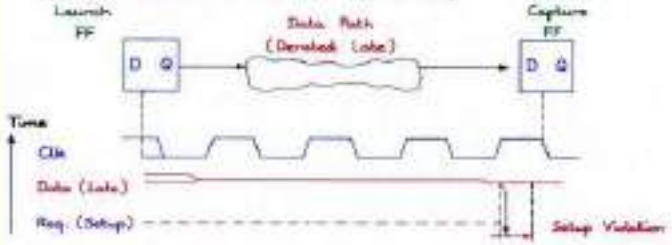
6. Late Derate

- Multiplier > 1.0 used to model slow (worst-case) condition.
- Increases delay → data arrives later.
- Used for SETUP timing (worst case).

> Late Derate Factor (M_{late}) > 1.0

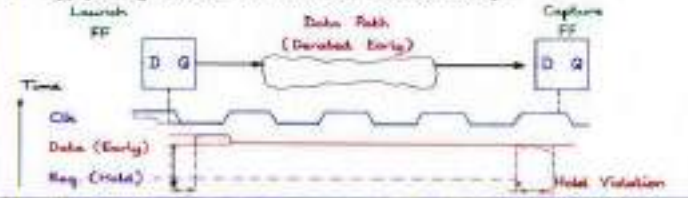
7. Setup Derating

- We want to test worst case for setup → data should arrive late.
- Apply late derates to both cell and net delays.



8. Hold Derating

- We want to test worst case for hold → data should arrive early.
- Apply early derates to both cell and net delays.



9. Conservative Timing Analysis

- Derates introduce margin to cover variations.
- Setup uses worst (late) → pessimistic delay.
- Hold uses worst (early) → pessimistic arrival.
- Ensures manufactured chips meet timing with high probability.

> Derating makes STA conservative but results in reliable silicon.

10. Why Derating is Needed ?

- To model local and global variations within the chip.
- Variation cannot be captured by corners alone.
- Derating provides a continuous margin across paths.
- Improves silicon success rate.

11. Types of Derates Used in STA

Derate Type	Applied On	Multiplier	Used For	Effect
Early (Fast)	Cell / Net	< 1.0	Hold Check	Reduces delay (Data early)
Late (Slow)	Cell / Net	> 1.0	Setup Check	Increases delay (Data late)
Setup Derate	Cell + Net	> 1.0	Setup Timing	Conservative for setup
Hold Derate	Cell + Net	< 1.0	Hold Timing	Conservative for hold

12. Common Derate Usage

Timing Check	Cell Delay	Net Delay	Purpose
Setup (Worst Case)	Late (>1)	Late (>1)	Data should arrive late
Hold (Worst Case)	Early (<1)	Early (<1)	Data should arrive early

13. Impact of Derating

Check	Derate Applied	Delay Effect	Timing Effect	Risk if Ignored
Setup	Late Derate (>1)	Delay ↑	Data arrives late	Setup violation (Data too late)
Hold	Early Derate (<1)	Delay ↓	Data arrives early	Hold violation (Data too early)

14. Example - Numerical Illustration

(A) Setup (Late Derate)

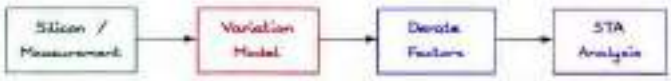
- Nominal cell delay = 120 ps , net delay = 80 ps
- Late cell derate = 1.15 , Late net derate = 1.10
- Total path delay (derated) = (120 × 1.15) + (80 × 1.10)
  = 138 + 88 = 226 ps

(B) Hold (Early Derate)

- Nominal cell delay = 50 ps , net delay = 40 ps
- Early cell derate = 0.85 , Early net derate = 0.90
- Total path delay (derated) = (50 × 0.85) + (40 × 0.90)
  = 42.5 + 36 = 78.5 ps

15. Relation with OCV

- OCV → Variation modelling (global + local).
- Derating → Applying numeric multipliers to include these variations in STA.
- Derates are derived from characterization / silicon data.



16. Practical Observation

- Too small derate → risk of silicon failure.
- Too large derate → over-conservation, may reduce performance.
- Derates are technology, design and path dependent.
- Always sign-off with proper derate values as per foundry recommendation.

17. Viva Questions

Q1. What is OCV derating?
Q2. What is a derate multiplier?
Q3. What is cell-delay derate?
Q4. What is net-delay derate?
Q5. What is early derate?
Q6. What is late derate?
Q7. How is derating used for setup?
Q8. How is derating used for hold?
Q9. Why is STA made conservative?
Q10. Difference between early and late derate?

18. Common Terms

OCV	: On-Chip Variation
Derating	: Scaling delays by multiplier
Early Derate	: < 1.0 , fast case
Late Derate	: > 1.0 , slow case
Cell Derate	: Applied to cell (gate) delays
Net Derate	: Applied to interconnect delays
Setup Check	: Data must arrive early enough
Hold Check	: Data must not arrive too early

19. QUICK REVISION

- Derating accounts for OCV variations.
- Early derate < 1 → reduces delay.
- Late derate > 1 → increases delay.
- Setup → use late derate (worst case).
- Hold → use early derate (worst case).
- Cell and Net derates applied separately.
- Derating ensures conservative timing.
- Proper derating → silicon reliability.

Author : Konda Tharun Vivekananda

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

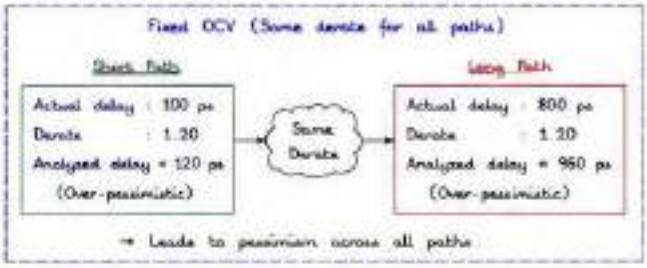
AOCV – Advanced OCV – Overview

1. Need for AOCV

- OCV (On-Chip Variation) models guardband for PVT variations.
- Fixed OCV uses same derate for all paths → too pessimistic.
- AOCV (Advanced OCV) changes derate based on path depth (logic effort) and provides more accurate timing.
- Improves timing closure and reduces over-design.

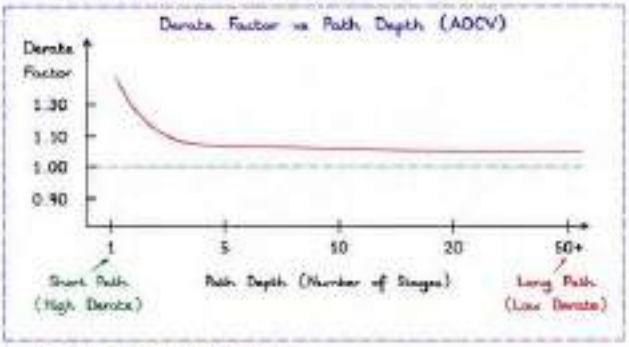
2. Fixed OCV Limitations

- Same derating factor applied to all paths irrespective of depth.
- Over-pessimism on short paths.
- Under-pessimism on long paths in some cases.
- Results in worse slack and unnecessary area/power.



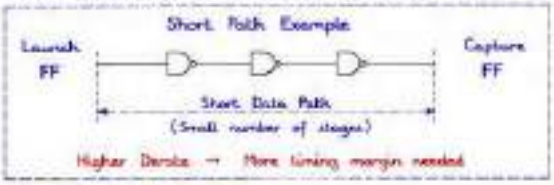
3. Path-Depth Dependency in AOCV

- AOCV scales derate factor based on path depth (number of logic stages).
- Short paths → higher derate (more uncertainty).
- Long paths → lower derate (variation average out).



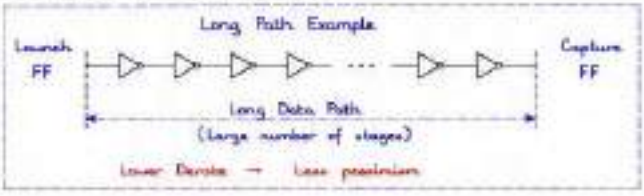
4. Short Path Variation

- Short paths have less logic → variations do not average out.
- More sensitive to local variations.
- AOCV applies higher derate to cover worst-case.



5. Long Path Variation

- Long paths have more logic → variations tend to average out.
- AOCV applies lower derate.
- Reduces unnecessary pessimism.



6. Table-Based Derating

- AOCV uses tables vs path depth to give derate factors.
- Separate tables for setup and hold.
- Tool looks up derate based on effective path depth.

AOCV Setup Derate Table (Example)

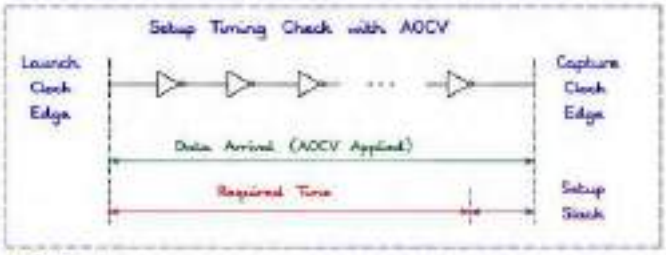
Path Depth (Stages)	Setup Derate Factor	
	Slow	Fast
1	1.28	0.92
2	1.18	0.82
5	1.08	0.92
10	1.03	0.99
20	1.01	0.99
50+	1.00	1.00

AOCV Hold Derate Table (Example)

Path Depth (Stages)	Hold Derate Factor	
	Slow	Fast
1	0.92	1.28
2	0.82	1.18
5	0.92	1.08
10	0.99	1.03
20	0.99	1.01
50+	1.00	1.00

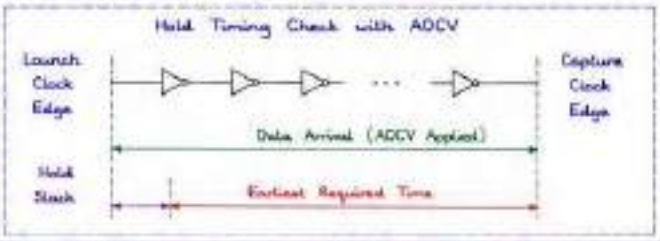
7. AOCV Setup

- For setup checks:
  - Slow corner (max delay) → derate > 1
  - Fast corner (min delay) → derate < 1
- Derate depends on path depth from AOCV setup table.



8. AOCV Hold

- For hold checks:
  - Slow corner (min delay) → derate < 1
  - Fast corner (max delay) → derate > 1
- Derate depends on path depth from AOCV hold table.



9. Advantages

- ✓ Reduces pessimism compared to fixed OCV.
- ✓ Improves achievable timing and slack.
- ✓ Better correlation with silicon.
- ✓ Helps in area, power and performance optimization.
- ✓ Industry standard for signoff STA.

10. Limitations

- × Requires accurate path depth calculation.
- × Slightly complex to set up and debug.
- × Depends on quality of library characterization.
- × More memory and runtime than fixed OCV.
- × Not effective if path depth estimation is wrong.

11. Reduced Pessimism – Impact

- AOCV reduces unnecessary guardbands on long paths.
- Helps in timing closure with smaller area and lower power.
- Improves performance by allowing faster designs.

Fixed OCV vs AOCV (Example)

Metric	Fixed OCV	AOCV
Short Path Slack	Very pessimistic	Reasonable
Long Path Slack	Very pessimistic	Less pessimistic
Area / Power	Higher (due to over-design)	Lower
Silicon Correlation	Poor	Better
Overall	Less efficient	More efficient

Practical Observations

- Use AOCV with accurate libraries and SDF / RC data.
- Check derate tables for both setup and hold.
- Monitor slack improvement after enabling AOCV.
- Ensure proper path depth extraction and mapping.
- AOCV is mandatory for signoff in advanced nodes.

Viva Questions

Q1. What is AOCV?
Q2. Why is AOCV needed?
Q3. What is the limitation of Fixed OCV?
Q4. How does AOCV handle path depth?
Q5. How are short and long paths treated in AOCV?
Q6. What are AOCV setup and hold tables?
Q7. What are the advantages of AOCV?
Q8. What are the limitations of AOCV?

Quick Revision

- ✓ AOCV → derate depends on path depth.
- ✓ Short path → higher derate.
- ✓ Long path → lower derate.
- ✓ Separate tables for setup and hold.
- ✓ Setup: Slow>1, Fast<1 ; Hold: Slow<1, Fast>1.
- ✓ Reduces pessimism → better area, power, performance.
- ✓ Essential for signoff STA.

Key Formulas

Setup Slack = Required Time − Data Arrival
Hold Slack = Data Arrival − Earliest Required Time
Derate Setup (Slow) > 1
Derate Setup (Fast) < 1
Derate Hold (Slow) < 1
Derate Hold (Fast) > 1

STATIC TIME ANALYSIS

POCV - Parametric OCV - Concepts, Modeling, Benefits and Usage

1. Need for POCV

- OCV models variations using fixed derate factors (pessimistic).
- AOCV uses depth based derates but still empirical.
- POCV models delay as a statistical (random) variable.
- Gives more accurate and less pessimistic timing.

> POCV → Parametric statistical modeling of delay variations.

2. Statistical Variation

- Every delay has natural variation due to random factors.
- Delay is treated as a random variable (X).
- Represented by statistical parameters (mean, σ, distribution).

> Variation → Delay is not single value, it is a range with probability.

3. Mean Delay (μ)

- Mean is the expected (average) delay of a cell/net.
- Represents nominal or typical delay.

$$\mu = E[X] = \text{Average expected delay}$$

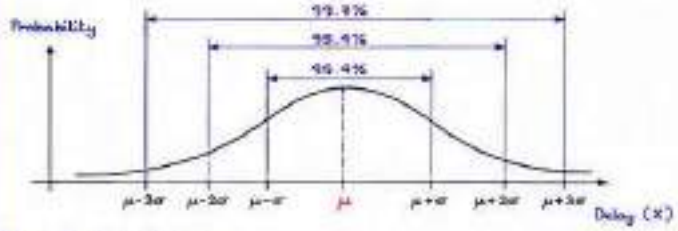
4. Standard Deviation (σ)

- Measures how much delay deviates from mean.
- Larger σ → Larger variation.
- Smaller σ → More stable delay.

$$\sigma = \sqrt{E[(X - \mu)^2]}$$

5. Probability Distribution

- Delay variations follow a probability distribution.
- Commonly modeled as Gaussian (Normal) distribution.
- Defined by mean (μ) and standard deviation (σ).



6. Cell Correlation

- Delays of cells are not completely independent.
- Nearby cells share common process, temperature, voltage.
- Correlation coefficient (ρ) captures this relationship.
- ρ = 1 → Fully correlated, ρ = 0 → Uncorrelated.



7. Statistical Delay Calculation

- Path delay = Sum of individual cell/net delays.
- Using statistical formula considering mean, σ and correlation.

$$\mu_{path} = \sum \mu_i$$

$$\sigma^2_{path} = \sum \sigma_i^2 + 2 \sum_{i<j} \rho_{ij}\, \sigma_i\, \sigma_j$$

Where, μ_i, σ_i → mean and std dev of element i
ρ_{ij} → correlation between i and j

8. Early Derate

- Used to model best-case (early) delays.
- Delay is reduced compared to nominal.
- Applied on cell and net delays for HOLD analysis.

> Early Derate Factor (M_{early}) < 1.0

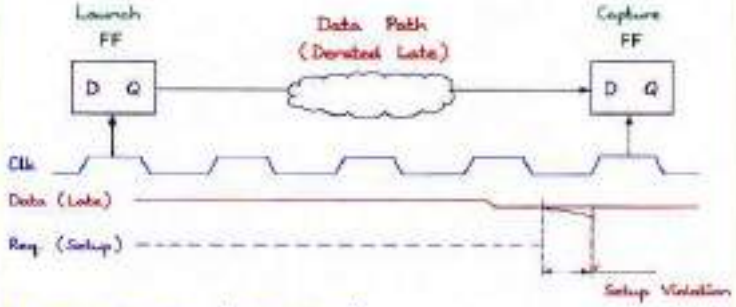
9. Late Derate

- Used to model worst-case (late) delays.
- Delay is increased compared to nominal.
- Applied on cell and net delays for SETUP analysis.

> Late Derate Factor (M_{late}) > 1.0

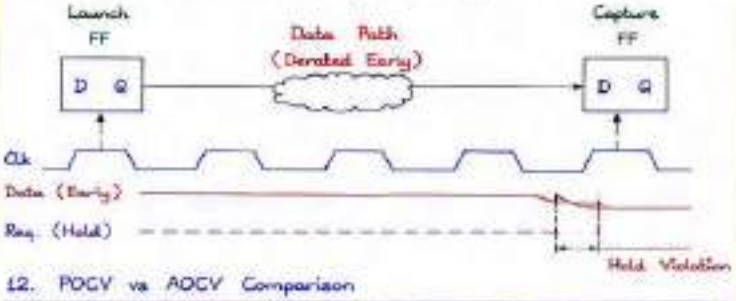
10. Setup Derating (Worst Case)

- Use late derate on both cell and net delays.
- Models slow process, low Vdd, high temperature.
- Ensures data arrives before required time (with margin).



11. Hold Derating (Worst Case)

- Use early derate on both cell and net delays.
- Models fast process, high Vdd, low temperature.
- Data should not arrive too early (with margin).



12. POCV vs AOCV Comparison

Aspect	AOCV	POCV (Parametric OCV)
Model Type	Depth based empirical	Statistical (μ, σ, ρ based)
Variation	Fixed derate per depth	Parametric with mean, σ
Correlation	Not explicitly modeled	Modeled using correlation (ρ)
Accuracy	Better than OCV	Better than AOCV (more accurate)
Pessimism	Moderate	Lower
Data Needed	Characterization tables	Statistical characterization
Complexity	Medium	High
Best For	General designs	Advanced nodes, high performance

13. POCV Advantages

- Uses statistical model → more realistic.
- Models correlation between cells.
- Reduces pessimism → less area and power.
- Improves timing accuracy and silicon correlation.

14. POCV Limitations

- Requires statistical characterization (μ, σ, ρ).
- More complex setup and computation.
- Needs good data and tools support.
- Not essential for older / less aggressive nodes.

15. Advanced-Node Usage

- Variations are higher in advanced nodes.
- POCV widely used in 28nm and below.
- Helps in sign-off with accurate margins.
- Required for high-performance, low-power designs.

> Typical POCV Data
> - μ → Mean delay
> - σ → Std. deviation
> - ρ → Correlation
> - %3σ → Percentile

16. Example - Statistical Delay

Given:
- Cell1: μ_1 = 100 ps, σ_1 = 10 ps
- Cell2: μ_2 = 80 ps, σ_2 = 8 ps
- Correlation ρ_{12} = 0.5

Path Delay

μ_{path} = 100 + 80 = 180 ps

σ^2_{path} = $10^2 + 8^2 + 2(0.5)(10)(8)$
= 100 + 64 + 80 = 244

σ_{path} = $\sqrt{244}$ = 15.62 ps

3σ (99.7% case) = 46.86 ps

17. Viva Questions

Q1. What is POCV?
Q2. What are mean and standard deviation?
Q3. Why is correlation important in POCV?
Q4. What is early derate used for?
Q5. What is late derate used for?
Q6. How is path mean and sigma calculated?
Q7. What is difference between POCV and AOCV?
Q8. Why is POCV used in advanced nodes?

18. Common Terms

OCV	: On-Chip Variation
POCV	: Parametric OCV
μ	: Mean delay
σ	: Standard deviation
ρ	: Correlation coefficient
Early Derate	: Fast / Best case (< 1)
Late Derate	: Slow / Worst case (> 1)
%3σ	: Percentile (e.g. 3σ = 99.7%)

19. QUICK REVISION

- POCV → statistical model of delay variation
- Delay modeled with μ, σ and ρ
- Early derate < 1 → Hold check
- Late derate > 1 → Setup check
- Less pessimism than AOCV
- High accuracy → better silicon correlation
- Used in advanced nodes

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

OCV / AOCV / POCV Comparison

1. Introduction

- Variation in PVT causes uncertainty in delays.
- STA applies derating to model worst-case (setup) and best-case (hold) delays.
- OCV, AOCV and POCV are three generations of derating methods.

2. Method Overview

OCV (On-Chip Variation)	AOCV (Advanced OCV)	POCV (Parametric OCV)
• Fixed derate applied to all cells • Does not consider path depth • More pessimism	• Derate depends on path depth • Uses table-based derating • Less pessimism than OCV	• Derate based on statistical (μ, σ) of delay • Models probability of violation • Least pessimism

3. Need for AOCV

- Fixed OCV over-penalizes short paths and under-penalizes long paths.
- AOCV uses path-depth information to apply appropriate derates.
- Achieves better correlation with silicon results.

4. Fixed OCV Limitations

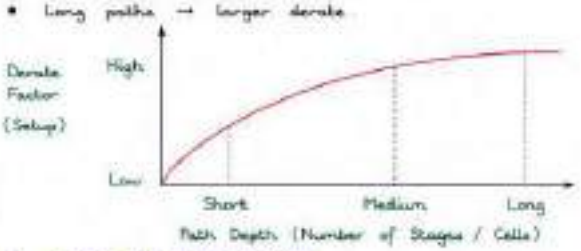
- Same derate for all paths regardless of length.
- Short paths → unnecessary pessimism.
- Long paths → insufficient derating, risk of violation.
- Leads to either over-design (area/power) or silicon failure.

Effect of Fixed OCV

Short Path (e.g. 2 cells)	Long Path (e.g. 30 cells)
Actually less variation	Actually more variation
but fixed OCV applies	but fixed OCV applies
large derate →	same derate →
more pessimism	less protection

5. Path-Depth Dependency (AOCV Core Idea)

- Derate factor increases with path depth.
- Short paths → smaller derate.
- Long paths → larger derate.



6. Short Path Variation

- Fewer stages → variations tend to average out.
- AOCV applies smaller derate.

7. Long Path Variation

- More stages → variations accumulate.
- AOCV applies larger derate for safety.

8. Table-Based Derating (AOCV)

- Library contains derate tables for setup and hold.
- Rows → path depth (number of stages).
- Columns → transition (slew) / load or simply depth.
- Tool picks derate based on path depth and applies to each cell.

Example AOCV Setup Table (Derate Factor)

Path Depth (Stages)	Slew / Load		
	Small	Medium	Large
1 – 5	1.02	1.03	1.04
6 – 10	1.04	1.06	1.08
11 – 20	1.07	1.10	1.13
21 – 40	1.10	1.15	1.18
> 40	1.12	1.17	1.20

(Values shown are example only)

9. AOCV Setup

- For setup checks → derate > 1.0 (increase delay).
- Longer path depth → larger derate.
- Protects against late arrival of data.

10. AOCV Hold

- For hold checks → derate < 1.0 (decrease delay).
- Shorter path depth → smaller reduction.
- Ensures data does not arrive too early.

11. POCV Overview (for comparison)

- Uses statistical delay (Mean μ and Std Dev σ).
- Derate based on required probability (e.g., 3σ, 6σ).
- Better accuracy than AOCV.
- Requires more library data and tool support.

12. Comparison Summary Table

Feature	OCV	AOCV	POCV
Derate Type	Fixed	Path-depth based	Statistical (μ, σ)
Depends on	None	Path depth	Path depth + Statistics
Library Data	Single derate value	Derate tables	μ and σ tables
Pessimism	High	Medium	Low
Accuracy	Low	Medium	High
Silicon Correlation	Poor	Good	Very Good
Complexity	Low	Medium	High
Runtime	Low	Medium	High
Best Use	Legacy designs	Most designs today	High performance critical designs

13. Advantages (AOCV)

- ✓ Reduces pessimism compared to OCV.
- ✓ Considers path depth (variation accumulation).
- ✓ Improves performance without compromising reliability.
- ✓ Good silicon correlation.
- ✓ Widely supported by EDA tools.

14. Limitations (AOCV)

- ✗ Still not statistical like POCV.
- ✗ Table size increases with depth and conditions.
- ✗ Needs careful library characterization.
- ✗ May not capture all correlation effects.

15. Practical Observations

- OCV is simple but over-pessimistic.
- AOCV is the industry standard for most designs.
- POCV gives best accuracy but needs advanced flows.
- Choose method based on design criticality and tool support.

16. Viva Questions

Q1. What is OCV?
Q2. Why is fixed OCV pessimistic?
Q3. How does AOCV reduce pessimism?
Q4. What does path depth mean?
Q5. How is AOCV setup different from hold?
Q6. What is POCV?
Q7. Compare OCV, AOCV and POCV.
Q8. When would you prefer POCV?

17. Quick Revision

- ✓ OCV → Fixed derate for all paths.
- ✓ AOCV → Derate depends on path depth.
- ✓ POCV → Derate based on statistics (μ, σ).
- ✓ Setup derate > 1.0, Hold derate < 1.0.
- ✓ Short path → smaller variation.
- ✓ Long path → larger variation.
- ✓ AOCV gives good balance of accuracy and runtime.

18. Key Formulas

- Setup Derated Delay = Cell Delay × Setup Derate (> 1)
- Hold Derated Delay = Cell Delay × Hold Derate (< 1)
- Path Depth = Number of logic stages / cells in path
- POCV Delay = $\mu + (K \times \sigma)$ ($K = 3$ for 3σ, else..)

STATIC TIME ANALYSIS

CPPR / CRPR - Concepts, Purpose, Impact and Benefits

1. What is CPPR / CRPR ?

- CPPR - Common Path Pessimism Removal
- CRPR - Clock Reconvergence Pessimism Removal
- STA tools initially analyze launch and capture clocks independently (pessimistic)
- CPPR / CRPR removes the part of clock path that is common or reconverged, reducing pessimism

> CPPR → Removes pessimism due to common path between launch and capture clocks.
> CRPR → Removes pessimism due to reconverged clock paths at the capture clock

2. Purpose

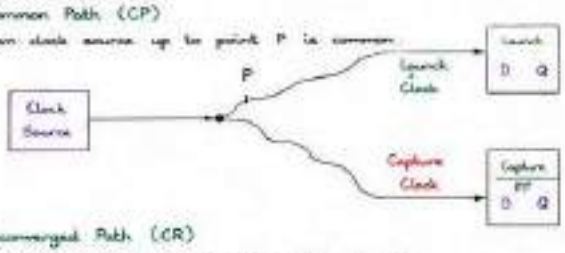
- Clock paths are often shared or reconverge
- Without removal, delays on common / reconverged paths are counted twice
- Leads to overly pessimistic slack (false violations)
- CPPR / CRPR gives more realistic timing

> Goal → Eliminate double-counted clock delays
> Result → More accurate slack and fewer false violations

3. Clock Path Scenarios

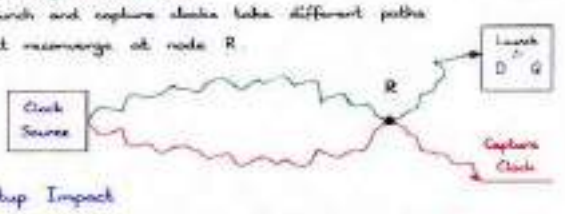
(A) Common Path (CP)

- From clock source up to point P is common



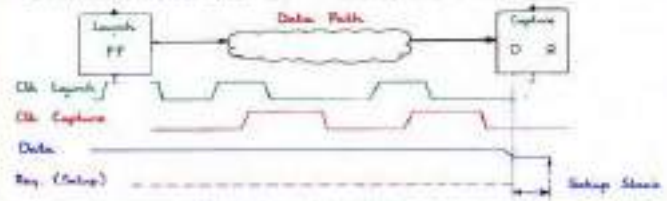
(B) Reconverged Path (CR)

- Launch and capture clocks take different paths but reconverge at node R



4. Setup Impact

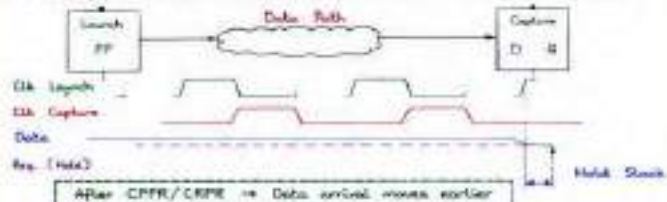
- Setup slack = Required Time - Arrival Time - Setup Time
- Pessimism makes data arrival look later
- CPPR / CRPR reduces pessimism → more positive setup slack



> After CPPR / CRPR → Required time moves later
> → Setup slack increases

5. Hold Impact

- Hold slack = Data Arrival Time - Required Hold Time
- Pessimism makes data arrival look later than actual
- CPPR / CRPR reduces arrival time pessimism → hold slack reduces



> After CPPR / CRPR → Data arrival moves earlier
> → Hold slack increases

6. Removing Excessive Pessimism

- Without removal → common or reconverged delays counted twice
- With CPPR / CRPR → remove portion of common path from capture clock
- Only the differential (unique) part is considered

> Effective Capture Clock Path = Total Capture Path - Common / Reconverged Portion

7. More Realistic Timing

- Gives slack closer to silicon behavior
- Reduces false violations and saves unnecessary ECOs
- Helps in closing design with correct priorities

> Result → Accurate timing sign-off with less pessimism

8. Statistical View (Intuition)

- Common paths affect launch and capture in same direction
- Their effect cancels out in differential timing
- Removing them reduces variation pessimism

Scenario	Without CPPR / CRPR	With CPPR / CRPR
Common Path	Delay counted twice	Counted once
Reconverged Path	Delay counted twice	Counted once
Setup Slack	More pessimistic	Less pessimistic
Hold Slack	More optimistic	Less optimistic

9. CPPR vs CRPR

Aspect	CPPR (Common Path)	CRPR (Clock Reconvergence)
Definition	Removes common path	Removes reconverged path
Where occurs	Shared path from source	Paths meet at reconvergence
Node	Common point P	Reconvergence point R
Impact	Reduces double counting	Reduces reconvergence pessimism
Used for	Setup and Hold	Setup and Hold

10. CPPR / CRPR Advantages

- Removes double-counted clock delays
- Improves accuracy of setup and hold slacks
- Reduces false violations → saves area and power
- Helps closure with real design priorities
- Required for sign-off in advanced flows

11. CPPR / CRPR Limitations

- Depends on accurate clock tree modeling
- Requires correct identification of common / reconverged portions
- Not effective if correlation is weak or unknown
- Tool and methodology dependent
- Misconfiguration can hide real violations

12. Advanced-Node Usage

- Essential in modern nodes due to large clock trees
- High variation and deep sub-micron processes → more clock sharing and reconvergence
- Used with OCV / POCV to achieve realistic sign-off

Key Formulas

Setup Slack = Required Time - Arrival Time - Setup Time

Hold Slack = Arrival Time - Required Hold Time

Effective Cap Clock Delay = Cap Clock Delay - Common / Reconverged Delay

Slack after Removal = Slack before Removal + Removed Pessimism (Setup)

Slack after Removal = Slack before Removal - Removed Pessimism (Hold)

13. Numerical Example (Setup)

Given:

- Launch clock delay : 1.20 ns
- Capture clock delay : 1.50 ns
- Common path delay : 1.00 ns
- Data path delay : 2.30 ns
- Setup time : 0.10 ns
- Clock period : 4.00 ns

(A) Without CPPR

Required Time = 4.00 - 1.50 = 2.50 ns

Arrival Time = 1.20 + 2.30 = 3.50 ns

Setup Slack = 2.50 - 3.50 = -0.10 ns (VIOLATION)

(B) With CPPR

Effective Cap. Delay = 1.50 - 1.00 = 0.50 ns

Required Time = 4.00 - 0.50 = 3.50 ns

Setup Slack = 3.50 - 3.50 = 0.00 ns (IMPROVED)

14. Viva Questions

Q1. What is CPPR ?
Q2. What is CRPR ?
Q3. Why is pessimism present in STA?
Q4. How does CPPR remove pessimism?
Q5. How does CRPR remove pessimism?
Q6. What is the effect on setup slack?
Q7. What is the effect on hold slack?
Q8. Difference between CPPR and CRPR?
Q9. What are advantages of CPPR/CRPR?
Q10. Are there any limitations?

15. QUICK REVISION

- CPPR → Common Path Pessimism Removal
- CRPR → Clock Reconvergence Pessimism Removal
- Removes double-counted clock delays
- Setup → Slack increases
- Hold → Slack decreases
- More realistic & accurate timing
- Reduces false violations
- Widely used in advanced nodes

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

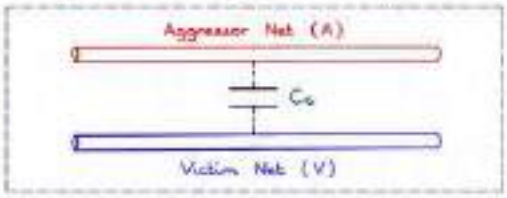
CROSSTALK FUNDAMENTALS

1. WHAT IS CROSSTALK?

- Crosstalk is an unwanted electrical interference between adjacent interconnects due to capacitive coupling.
- Switching on one net (aggressor) induces noise or delay on the neighboring net (victim).
- It can affect both the logic value (noise) and the timing (delay) of the victim net.

2. COUPLING CAPACITANCE

- When two conductors run in parallel, a capacitance exists between them called coupling capacitance (C_c).
- The aggressor and victim wires form a small capacitor.
- Noise depends on C_c, aggressor activity and victim load.



3. ADJACENT NETS

- Nets routed close to each other are considered adjacent.
- Larger length of parallel run → higher total coupling (C_c).
- Inter-layer coupling can also exist between metal layers.

4. AGGRESSOR

- Net that switches and creates a time-varying voltage.
- Its transition ($0 \to 1$ or $1 \to 0$) causes current to flow through C_c.
- Example: A switching clock line next to a data line.

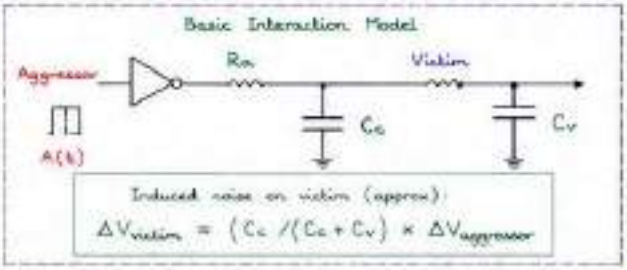
5. VICTIM

- Net that is affected by the switching of the aggressor.
- May experience additional delay or unwanted noise.
- Example: A data net adjacent to a switching clock.

Aggressor Type	Effect on Victim
$0 \to 1$ (Rising)	Positive noise (may cause false 1)
$1 \to 0$ (Falling)	Negative noise (may cause false 0)

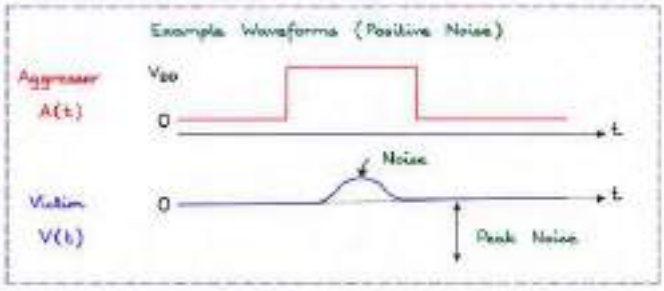
6. SWITCHING INTERACTION

- Current flows via C_c only when aggressor is switching.
- No switching on aggressor → No crosstalk.
- Noise magnitude depends on:
  - Victim load capacitance (C_v)
  - Coupling capacitance (C_c)
  - Transition time of aggressor
  - Relative timing between aggressor and victim.



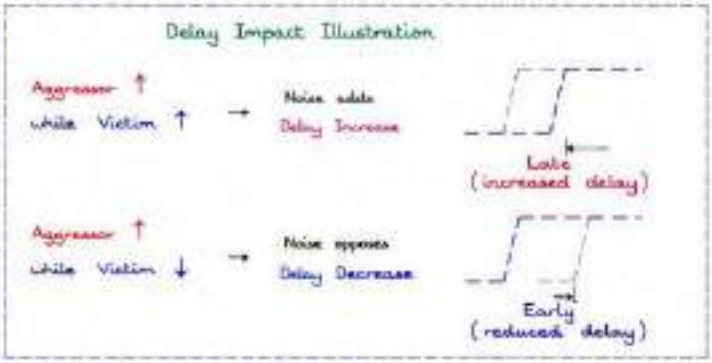
7. CROSSTALK NOISE

- Unwanted voltage glitch induced on the victim net.
- Can reduce noise margin and cause functional errors.
- Peak noise depends on C_c, C_v and aggressor transition.
- Types:
  - Positive noise (victim pulled up)
  - Negative noise (victim pulled down)



8. CROSSTALK DELAY

- Victim net delay changes due to crosstalk.
- If noise adds to transition → Delay increase.
- If noise opposes transition → Delay decrease (rare but possible).
- Two main delay types:
  - Early arrival (negative delay impact)
  - Late arrival (positive delay impact)



9. SUMMARY COMPARISON TABLE

Aspect	Definition / Description	Key Points
What is Crosstalk?	Interference between adjacent nets	Caused by capacitive coupling
Coupling Capacitance	Capacitance between two adjacent nets (C_c)	Larger C_c → More noise
Adjacent Nets	Nets routed close to each other	Longer parallel run → Higher coupling
Aggressor	Switching net that induces noise	$0 \to 1$ or $1 \to 0$ transition creates effect
Victim	Net affected by aggressor switching	Receives noise / delay from aggressor
Switching Interaction	Interaction occurs only during switching	Depends on C_c, C_v, timing and transition
Crosstalk Noise	Unwanted voltage glitch on victim	Can be positive or negative
Crosstalk Delay	Change in arrival time of victim	May increase or decrease delay

10. PRACTICAL OBSERVATIONS

- High switching activity on clock lines can inject noise into nearby data lines.
- Victim nets with small load capacitance are more sensitive.
- Proper routing, shielding and spacing help reduce crosstalk.
- Crosstalk must be analysed for accurate timing signoff.

11. VIVA QUESTIONS

Q1. What is crosstalk?
Q2. What is coupling capacitance?
Q3. What is an aggressor?
Q4. What is a victim?
Q5. How does crosstalk noise occur?
Q6. What are positive and negative crosstalk noise?
Q7. How does crosstalk affect delay?
Q8. How can crosstalk be reduced?

12. QUICK REVISION

- ✓ Crosstalk is due to capacitive coupling.
- ✓ Aggressor switching → Noise on victim.
- ✓ Noise can be positive or negative.
- ✓ Noise depends on C_c, C_v and timing.
- ✓ Crosstalk can increase or decrease delay.
- ✓ Proper routing and shielding reduce crosstalk.

13. KEY FORMULAS

- Induced Noise on Victim:

$$\Delta V_{victim} \approx \left(\frac{C_c}{C_c + C_v}\right) \times \Delta V_{aggressor}$$

- Total Coupling Capacitance:

$$C_{c_{total}} = C_{side} + C_{above} + C_{below}$$

- Noise Margin Reduction:

$$NM_{reduced} = NM_{original} - |Noise|$$

STATIC TIME ANALYSIS

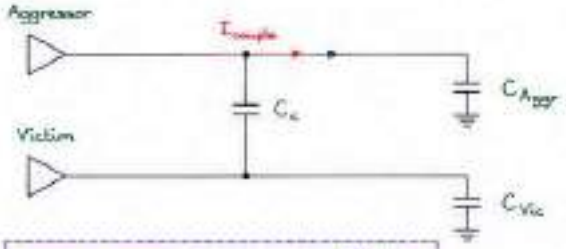
Crosstalk Noise / Glitch - Concepts, Types, Amplitude and Impact

1. Crosstalk Noise / Glitch - Introduction

- Occurs due to capacitive coupling between adjacent nets.
- When one net (aggressor) switches, it induces unwanted voltage on another net (victim).
- This unwanted voltage is called noise or glitch.

> Victim remains constant (no switching)
> Aggressor switches → induces glitch on victim.

2. Basic Coupling Model

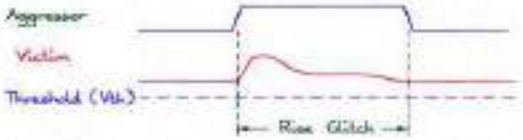


Coupling current $I_{couple} = C_c \frac{dV_{aggr}}{dt}$

Induced noise on victim $\propto \frac{C_c}{C_{Vic}}$

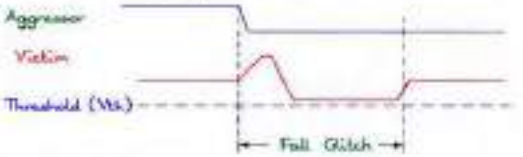
3. Rise Glitch

- Aggressor makes a low → high transition.
- Victim sees a positive glitch (voltage rises temporarily).
- May cross logic threshold and cause wrong capture.



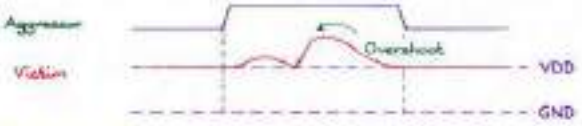
4. Fall Glitch

- Aggressor makes a high → low transition.
- Victim sees a negative glitch (voltage dips temporarily).
- May cause wrong capture or hold violation.



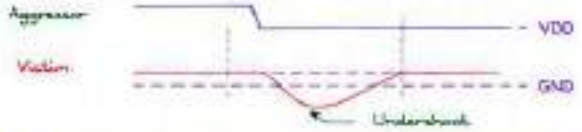
5. Overshoot

- Glitch rises above the steady high level.
- Voltage goes above VDD (temporary).



6. Undershoot

- Glitch goes below the steady low level.
- Voltage goes below GND (temporary).



7. Noise Amplitude

- Maximum deviation of victim voltage from its steady state.
- Depends on coupling capacitance, aggressor slew, victim load.

$$\text{Noise Amplitude } (\Delta V) = \left(\frac{C_c}{C_{Vic}}\right) \times \Delta V_{aggr}$$

8. Functional Impact

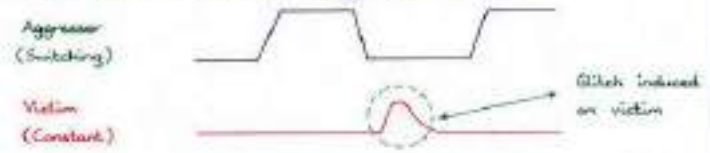
- If glitch crosses logic threshold → functional error.
- Can cause:
  - Wrong data captured (setup violation)
  - False transition (hold violation)
  - Extra switching → power and noise increase
  - Reduced design margin

> Risk increases when:
> - Large C_c (adjacent long parallel nets)
> - Small victim load (low C_{Vic})
> - Fast aggressor edge (high dV/dt)

9. Types Summary Table

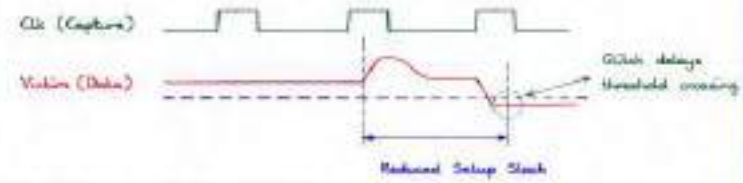
Type	Aggressor Transition	Victim Response	Effect
Rise Glitch	Low → High	Voltage rises temporarily	May cause setup violation
Fall Glitch	High → Low	Voltage falls temporarily	May cause hold violation
Overshoot	Low → High	Voltage > VDD temporarily	Stress, leakage, false switching
Undershoot	High → Low	Voltage < GND temporarily	Leakage, reliability degradation

10. Victim Remains Constant, Aggressor Switches



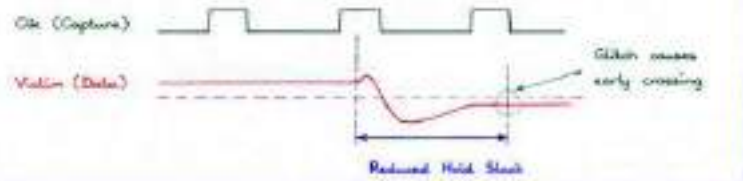
11. Effect on Timing (Setup)

- Rise glitch → delays the rising edge reaching threshold.
- Data arrives later → reduces setup slack.



12. Effect on Timing (Hold)

- Fall glitch → pulls voltage down below threshold early.
- Data appears to change early → hold violation.



13. Practical Observation

- Crosstalk is worst on long parallel routes and high metal layers.
- Shielding, spacing and layer assignment help in reducing noise.
- STA with crosstalk (SI-STA) gives accurate timing sign-off.

> Good physical design reduces glitch amplitude and improves timing reliability.

14. Viva Questions

Q1. What is crosstalk noise/glitch?
Q2. What is victim and aggressor?
Q3. What is rise glitch? When does it occur?
Q4. What is fall glitch? When does it occur?
Q5. What are overshoot and undershoot?
Q6. How is noise amplitude calculated?
Q7. How does crosstalk affect setup timing?
Q8. How does crosstalk affect hold timing?
Q9. How can crosstalk be reduced in design?
Q10. Difference between overshoot and undershoot?

15. QUICK REVISION

- Victim constant, Aggressor switches
- Rise glitch → victim voltage rises
- Fall glitch → victim voltage falls
- Overshoot → voltage > VDD
- Undershoot → voltage < GND
- Noise amplitude $\propto (C_c / C_{Vic}) \times \Delta V_{aggr}$
- Glitches can cause setup/hold violations
- Crosstalk increases power and reduces margin
- Good routing and shielding reduce glitches

IN → OUT

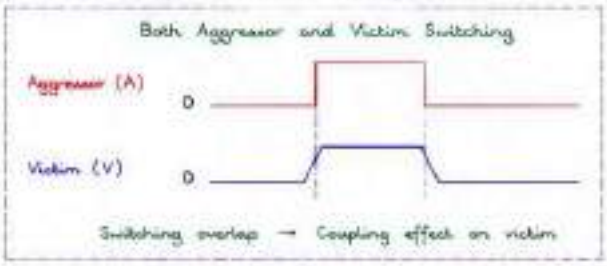
STATIC TIMING ANALYSIS (STA)

CLK

CROSSTALK DELAY

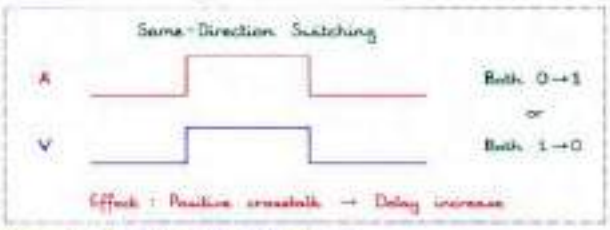
1. BOTH AGGRESSOR AND VICTIM SWITCHING

- Crosstalk occurs only when the aggressor net is switching
- If victim is also switching, the interaction changes its delay
- Both nets switching → maximum coupling effect



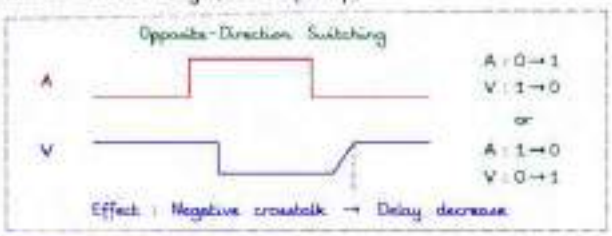
2. SAME-DIRECTION SWITCHING

- Aggressor and victim switch in the same direction ($0\to1$ with $0\to1$ or $1\to0$ with $1\to0$)
- Induces voltage of same polarity on victim
- Increases victim's delay (victim slowdown)



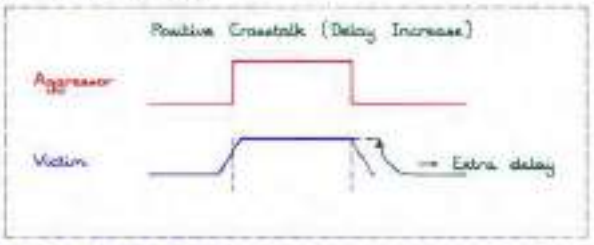
3. OPPOSITE-DIRECTION SWITCHING

- Aggressor and victim switch in opposite directions ($0\to1$ with $1\to0$ or $1\to0$ with $0\to1$)
- Induces voltage of opposite polarity on victim
- Reduces victim's delay (victim speedup)



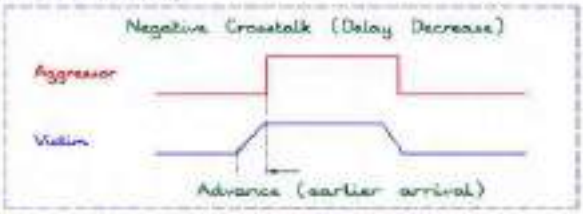
4. POSITIVE CROSSTALK DELAY

- Occurs in same-direction switching
- Victim experiences additional propagation delay
- Bad for setup timing



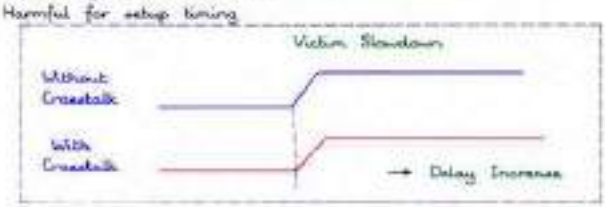
5. NEGATIVE CROSSTALK DELAY

- Occurs in opposite-direction switching
- Victim arrives earlier due to coupling
- Bad for hold timing



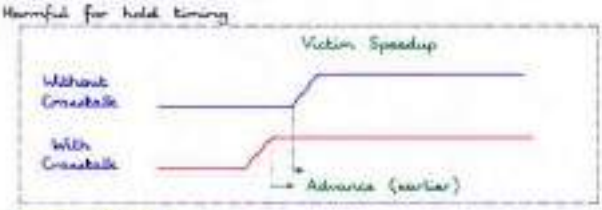
6. VICTIM SLOWDOWN

- Delay of victim increases (arrival becomes later)
- Caused by positive crosstalk
- Harmful for setup timing



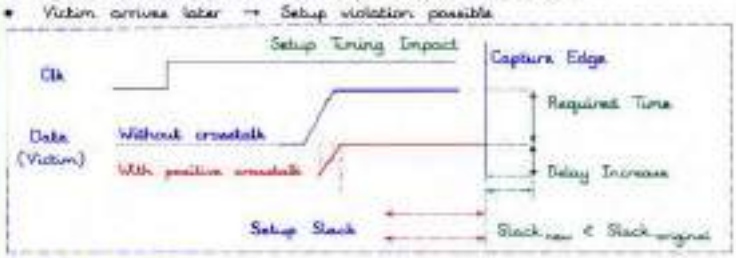
7. VICTIM SPEEDUP

- Delay of victim decreases (arrival becomes earlier)
- Caused by negative crosstalk
- Harmful for hold timing



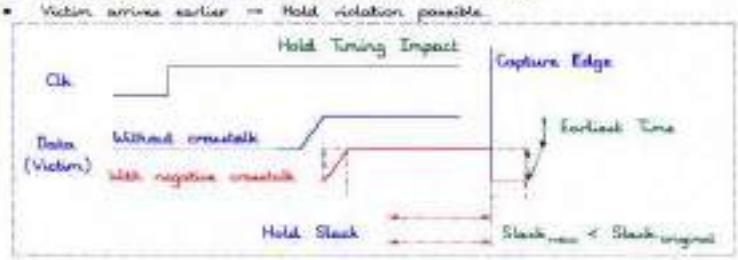
8. SETUP IMPACT

- Positive crosstalk (victim slowdown) reduces setup slack
- Victim arrives later → Setup violation possible



9. HOLD IMPACT

- Negative crosstalk (victim speedup) reduces hold slack
- Victim arrives earlier → Hold violation possible



10. SUMMARY TABLE

Case	Aggressor	Victim	Crosstalk Type	Victim Effect	Delay Change	Setup Impact	Hold Impact
Both Switching	Switching	Switching	Either positive or negative	Coupling effect seen	May increase or decrease	May degrade	May degrade
Same-Direction	0→1 / 1→0	0→1 / 1→0	Positive	Victim slows down	Increases	Worse (Slack ↓)	Better
Opposite-Direction	0→1 1→0	1→0 0→1	Negative	Victim speeds up	Decrease	Better	Worse (Slack ↓)

11. PRACTICAL OBSERVATIONS

- Crosstalk effect increases with
  - Higher switching activity
  - Larger coupling capacitance (C_c)
  - Smaller spacing between nets
  - Longer parallel run length
  - Aggressor transition time
- Proper shielding and spacing reduce crosstalk

12. VIVA QUESTIONS

Q1. What is crosstalk delay?
Q2. When does crosstalk occur?
Q3. What is positive crosstalk?
Q4. What is negative crosstalk?
Q5. What happens in same-direction switching?
Q6. What happens in opposite-direction switching?
Q7. How does crosstalk affect setup timing?
Q8. How does crosstalk affect hold timing?

13. QUICK REVISION

- ✓ Crosstalk occurs due to coupling between adjacent nets
- ✓ Same-direction → positive CT → delay ↑
- ✓ Opposite-direction → negative CT → delay ↓
- ✓ Positive CT → victim slowdown
- ✓ Negative CT → victim speedup
- ✓ Positive CT hurts setup, helps hold
- ✓ Negative CT hurts hold, helps setup

14. KEY FORMULAS

- Induced Noise on Victim

$$\Delta V_{victim} \approx \left(\frac{C_c}{C_c + C_v}\right) \times \Delta V_{aggressor}$$

- Total Delay Change (approx)

$$\Delta t \approx \Delta V_{victim}$$

- Slack Impact

$$Slack_{new} = Slack_{original} - |\Delta t|$$

STATIC TIME ANALYSIS

Crosstalk Mitigation - Techniques, Methods, Impact and Benefits

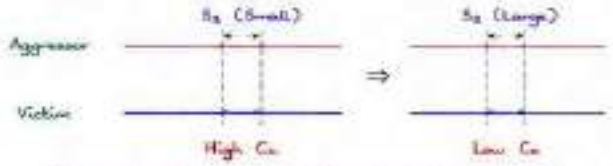
1. Crosstalk Mitigation - Introduction

- Crosstalk occurs due to capacitive coupling between adjacent nets.
- Victim net experiences glitches / noise when aggressor switches.
- Mitigation techniques reduce coupling, noise amplitude and delay.
- Goal: Improve timing accuracy, reduce violations and improve signal integrity.

> Mitigation → Reduce coupling (C_c) and its impact on victim → Better timing and reliability.

2. Increase Wire Spacing

- Increase distance (S) between victim and aggressor.
- Coupling capacitance (C_c) decreases as spacing increases.
- Simple and effective technique.



> $C_c \propto \frac{1}{S}$ (More spacing → Low C_c)

3. NDR (Non-Default Routing)

- Use wider wires and larger spacing rules than default.
- Reduces resistance, coupling capacitance and delay.
- Used for critical nets / buses.

Parameter	Default Routing	NDR Routing	Effect
Wire Width	Normal	Wider	↓ R, ↓ C_c
Wire Spacing	Normal	Larger	↓ C_c
Delay	Higher	Lower	↓ Delay
Crosstalk Noise	Higher	Lower	↓ Noise

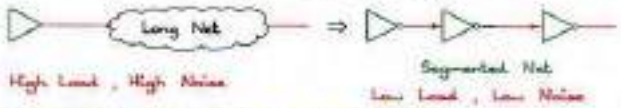
4. Higher Metal Layers

- Upper metal layers have larger spacing and thicker metals.
- Less coupling to other nets → lower C_c.
- Lower resistance → lower RC delay.



5. Buffer Insertion

- Insert buffers along long victim net.
- Breaks long wire into small segments.
- Reduces capacitive loading and glitch propagation.



6. Shielding (Ground Shield or Shield Net)

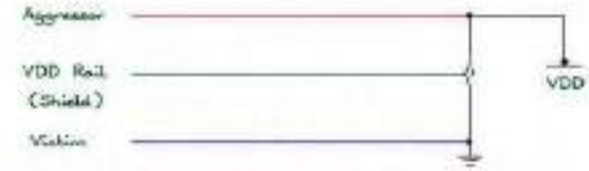
- Place a grounded shield wire between victim and aggressor.
- Shield intercepts coupling and reduces noise on victim.



> Shield absorbs coupling → → Noise on victim ↓

7. VDD / VSS Shielding

- Use power (VDD) or ground (VSS) rails as shields.
- Power rails are low impedance → strong shielding effect.
- Commonly used in standard cell designs.



> Power rail shielding → Strong protection, Low noise.

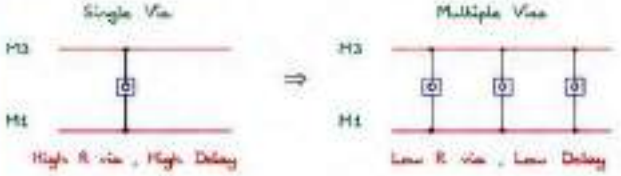
8. Drive-Strength Adjustment

- Use proper driver strength for aggressor net.
- Too strong driver → large dV/dt → more noise.
- Too weak driver → slow transition → more switching window.
- Choose optimal driver for balanced noise and delay.

Driver Strength	Transition (dV/dt)	Noise	Delay	Comment
Very Strong	Very High	High	Low	↑ Crosstalk
Medium (Optimal)	Moderate	Low	Balanced	Best Choice
Very Weak	Very Low	High	High	↑ Delay, Noise

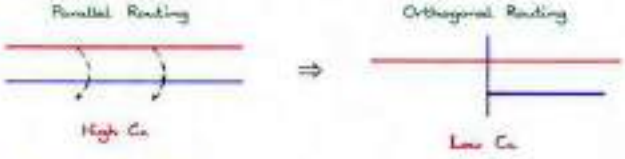
9. Multiple Vias

- Use multiple vias in parallel for vertical connections.
- Reduces via resistance and inductance.
- Helps reduce RC delay and improves signal quality.



10. Reduce Coupling Capacitance (Overall Concept)

- Use larger spacing, shielding, higher layers, NDR.
- Reduce parallel adjacency length.
- Orthogonal routing (perpendicular) reduces C_c.



11. Reduce RC Delay

- Reduce wire resistance (R) → wider wire, higher metal.
- Reduce capacitance (C) → spacing, shielding, less load.
- RC delay = R × C → both R and C must be minimized.

> Low R + Low C → Low RC delay → Better Timing

12. Mitigation vs Effect Summary

Technique	C_c	Noise	RC Delay	Timing Impact
Increase Spacing	↓	↓	↓	Positive
NDR	↓	↓	↓	Positive
Higher Metal	↓	↓	↓	Positive
Buffer Insertion	↓	↓	↓	Positive
Shielding	↓↓	↓↓	↓	Strong Positive
VDD/VSS Shield	↓↓	↓↓	↓	Strong Positive
Drive Adjust.	↓	↓	→	Balanced
Multiple Vias	–	–	↓	Positive

13. Practical Observation

- Crosstalk is critical in advanced nodes.
- Long parallel runs are most risky.
- Shielding and higher metal layers are very effective.
- Buffering improves both timing and signal integrity.
- Proper physical design reduces timing violations and noise-related functional failures.

14. Viva Questions

Q1. What is crosstalk?
Q2. What happens when aggressor switches and victim remains constant?
Q3. What is glitch on victim net?
Q4. What is rise glitch?
Q5. What is fall glitch?
Q6. What is overshoot and undershoot?
Q7. How does shielding reduce crosstalk?
Q8. What is NDR and why is it used?
Q9. How does buffer insertion help?
Q10. How do we reduce RC delay?

15. QUICK REVISION

- Victim constant, Aggressor switches.
- Rise glitch → temporary upward spike.
- Fall glitch → temporary downward dip.
- Overshoot → above VDD.
- Undershoot → below GND.
- Use spacing, shielding, higher layers, NDR to reduce C_c.
- Buffer insertion → reduce long load.
- VDD/VSS rails act as excellent shields.
- Multiple vias → reduce RC delay.
- Mitigation → Low noise, Low delay, Reliable timing.

IN → OUT

STATIC TIMING ANALYSIS (STA)

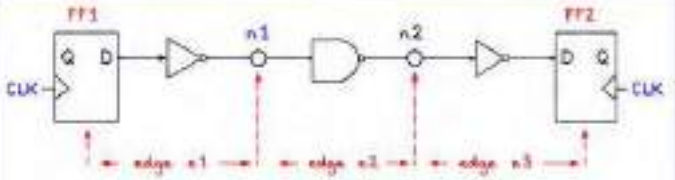
CLK

TIMING ANALYSIS METHODS - GBA (GRAPH-BASED ANALYSIS)

1. GBA OVERVIEW

- Graph-Based Analysis (GBA) is the most commonly used STA method.
- The entire design is modeled as a timing graph.
- Arrival times are propagated from all sources to all destinations using graph traversal.
- It uses pre-characterized delays from libraries.
- Fast and scalable but can be pessimistic.

2. TIMING GRAPH



FF : Flip-Flop (sequential node)
n1, n2 : Combinational nodes
e1, e2, e3 : Timing edges (combinational arcs)

3. NODES

- Nodes represent points in the timing graph.
- Types:
  - Sequential Nodes (registers, latches, memories, IO pins)
  - Combinational Nodes (gate outputs, net points)
- Each node has arrival time and required time values.

4. EDGES

- Edges represent connections between nodes.
- Carry timing information (delays).
- Edge delay comes from library characterization.
- Types:
  - Combinational edges (gate delays)
  - Sequential edges (clock-to-Q, setup, hold)

5. ARRIVAL TIME (AT)

- Time at which data arrives at a node.
- Propagated from all possible start points.
- Arrival time at node = max of all incoming arrival times.

$$AT(node_j) = \max_{i \in Fanin(j)} [AT(node_i) + Delay(edge_{i \to j})]$$

6. REQUIRED TIME (RT)

- Latest time by which data must arrive at a node to meet timing constraints.
- Propagated backward from all endpoints.
- Required time at node = min of all outgoing required times.

$$RT(node_i) = \min_{j \in Fanout(i)} [RT(node_j) - Delay(edge_{i \to j})]$$

7. SLACK

- Slack indicates how much timing margin is available.
- Positive slack → timing is met.
- Negative slack → timing violation.

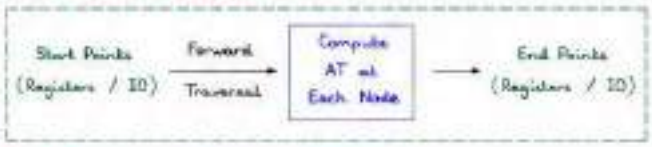
$$Slack(node) = RT(node) - AT(node)$$

Slack Value	Interpretation
> 0	Positive slack (safe)
= 0	Critical
< 0	Negative slack (violation)

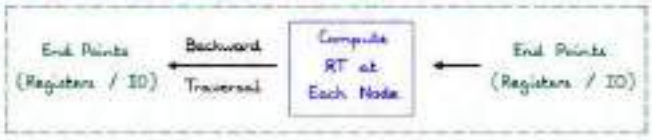
8. GRAPH PROPAGATION

- GBA performs two passes:

(1) Forward Propagation (Arrival Time)



(2) Backward Propagation (Required Time)

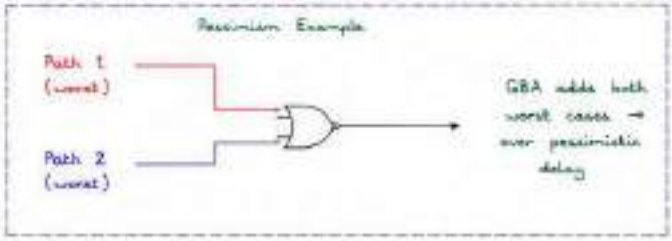


9. SPEED

- GBA is very fast and scalable.
- Complexity ~ O(N + E)
  - N : number of nodes
  - E : number of edges
- Suitable for very large designs.

10. PESSIMISM

- GBA assumes worst-case for all paths independently.
- Does not consider correlation between paths.
- Multiple worst-cases can stack up → more pessimism.
- May report violation even if real silicon is safe.



11. EARLY TIMING CLOSURE

- GBA provides quick and conservative results early in the flow.
- Helps identify bad paths and guide optimization.
- Allows early decisions on buffering, sizing and floorplanning.
- Later detailed methods (AOCV/POCV, PBA) can refine the results.

12. GBA CHARACTERISTICS SUMMARY

Aspect	GBA (Graph-Based Analysis)
Model	Design modeled as timing graph
Nodes	Sequential nodes + Combinational nodes
Edges	Carry delays between nodes
AT / RT	Forward: compute AT, Backward: compute RT
Slack	Slack = RT - AT
Speed	Very Fast, Scalable
Pessimism	Higher (independent worst-case stacking)
Use	Early timing analysis, signoff with POCV/AOCV

13. PRACTICAL EXAMPLES

Example Timing Path



Node	AT (ns)	RT (ns)	Slack (ns)
FF1 (Q)	0.00	2.50	-
n1	0.30	2.20	1.90
n2	0.85	1.70	0.85
FF2 (D)	1.20	1.50	0.30

14. VIVA QUESTIONS

Q1. What is Graph-Based Analysis?
Q2. What is a timing graph?
Q3. What are nodes in timing graph?
Q4. What are edges in timing graph?
Q5. How is arrival time computed?
Q6. How is required time computed?
Q7. How is slack calculated?
Q8. What are the advantages and limitations of GBA?

15. QUICK REVISION

- ✓ GBA models design as a graph.
- ✓ Nodes: sequential + combinational.
- ✓ Edges carry delays.
- ✓ Forward pass → Arrival Time.
- ✓ Backward pass → Required Time.
- ✓ Slack = RT - AT.
- ✓ Fast but pessimistic.
- ✓ Used for early timing closure.

16. KEY FORMULAS

- Arrival Time Propagation:

$$AT(j) = \max_{i \in Fanin(j)} (AT(i) + d_{i \to j})$$

- Required Time Propagation:

$$RT(i) = \min_{j \in Fanout(i)} (RT(j) - d_{i \to j})$$

- Slack: Slack = RT - AT

STATIC TIME ANALYSIS

TEMING ANALYSIS METHODS - PBA (Path-Based Analysis) - Concepts and Comparison

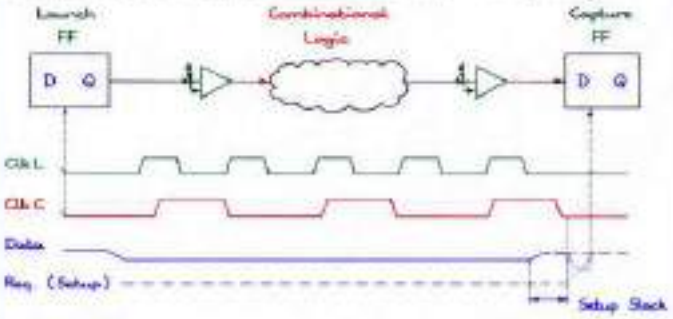
1. What is PBA (Path-Based Analysis)?

- PBA analyses each timing path individually from launch to capture.
- Uses exact clock and data arrival times along the path.
- Considers actual delays, derates, CPPR/CRPR and crosstalk.
- Gives more accurate slack with reduced pessimism.

> PBA → Path-by-path computation for accurate timing.
> (As opposed to GBA which uses graph propagation)

2. Individual Path Analysis

- Each path is traced from launch flip-flop to capture flip-flop.
- All elements on the path are counted once.
- Clock launch and capture latencies are computed separately.



3. Path Delay

- Sum of cell delays + net (interconnect) delays on the path.
- Includes wire RC delay, parasitics, derates and crosstalk.

> Path Delay = $\sum$ Cell Delay (i) + $\sum$ Net Delay (j)

4. Accurate Timing

- Counts only the real delays that exist on the path.
- Applies launch and capture clock latencies specifically for that path.
- Removes unnecessary pessimism from reconvergent graphs.

> Result → Slack closer to silicon behaviour.

5. Reduced Pessimism

- GBA may count common logic multiple times.
- PBA counts each element only once.
- PBA accounts for actual path correlation and common path removal.

> GBA → Higher pessimism
> PBA → Lower pessimism → More realistic slack

6. Critical-Path Analysis

- Finds the path with minimum (most negative) slack.
- That path decides the timing limit of the design.
- PBA identifies true critical paths more accurately.

> Critical Path → Worst slack path (after accurate analysis)

7. Signoff Usage

- Used in pre-signoff and signoff timing verification.
- Essential for high-performance and advanced-node designs.
- Ensures timing closure with reliable sign-off results.

> PBA is widely used in signoff STA sign-off.

8. Key Formulas in PBA

- Setup Slack (PBA)

> Setup Slack = (Tclk + Tlaunch + Tunc) - (Tdata_arr + Tsetup)

- Hold Slack (PBA)

> Hold Slack = (Tdata_arr + Thold) - (Tlaunch + Tunc)

Where,
- Tclk : Clock period
- Tlaunch : Launch clock latency
- Tunc : Uncertainty
- Tdata_arr : Data arrival time (on the path)
- Tsetup : Setup time of capture FF
- Thold : Hold time of capture FF

9. PBA vs GBA Comparison

Aspect	GBA (Graph-Based Analysis)	PBA (Path-Based Analysis)
Analysis Type	Graph propagation	Path-by-path calculation
Delay Counting	Common logic counted multiple times	Each element counted only once
Pessimism	High (more pessimism)	Low (reduced pessimism)
Accuracy	Less accurate	More accurate
CPPR / CRPR	Approximate or partial	Exact and effective
Runtime	Faster	Slower (more computation)
Use Case	Early analysis, exploration	Signoff, final verification
Slack Result	More negative	Closer to silicon
Best For	Large designs, quick estimation	High performance, critical paths

10. Pessimism Example (Conceptual)

(A) GBA (Pessimistic)



→ Delay of common logic counted twice → More delay → Less slack

(B) PBA (Accurate)



→ Delay of common logic counted once → Less delay → More slack

11. How PBA Works?

1) Select a path from launch FF to capture FF.
2) Compute launch clock arrival time to launch pin.
3) Compute capture clock arrival time to capture pin.
4) Traverse the path and sum actual cell + net delays.
5) Apply derates, CPPR/CRPR and crosstalk on this path.
6) Calculate arrival time and required time.
7) Compute slack for setup and hold.
8) Repeat for all required paths.

12. Practical Observation

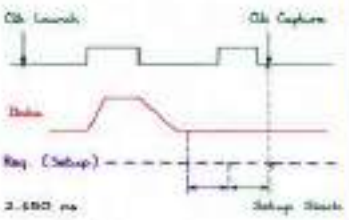
- PBA removes artificial delay from reconvergent clock/data paths.
- Slack reported by PBA is closer to real silicon timing.
- Helps to avoid over-design and unnecessary optimizations.
- Essential for advanced nodes where noise & variations are high.

> Accurate path analysis → Reliable timing closure.

13. Example - Setup Slack Calculation (PBA)

Given:

Clock period (Tclk)	=	2.000 ns
Launch latency (Tlaunch)	=	0.150 ns
Capture latency (Tcapture)	=	0.200 ns
Data path delay (Tdata_arr)	=	1.350 ns
Setup time (Tsetup)	=	0.120 ns
Uncertainty (Tunc)	=	0.050 ns



Required time = Tclk + Tcapture - Tunc
= 2.000 + 0.200 - 0.050 = 2.150 ns

Setup Slack = Required time - (Tdata_arr + Tsetup)
= 2.150 - (1.350 + 0.120)
= 2.150 - 1.470 = 0.680 ns (Positive Slack ✓)

14. Viva Questions

Q1. What is PBA?
Q2. What is the difference between PBA and GBA?
Q3. Why does PBA give less pessimism?
Q4. How does PBA count delays on a path?
Q5. How does PBA handle common path pessimism?
Q6. What is the main advantage of PBA?
Q7. When is PBA used in timing analysis?
Q8. How is slack calculated in PBA (setup)?
Q9. How is slack calculated in PBA (hold)?
Q10. What is the limitation of PBA?

15. Key Points

- Analyses each path individually.
- Considers actual delays and latencies.
- Removes reconvergent pessimism.
- Gives more accurate and realistic slack.
- Identifies true critical paths.
- Used in pre-signoff and signoff.
- More computation time than GBA.

16. QUICK REVISION

- PBA = Path-Based Analysis
- Individual path analysis for accurate timing
- Path Delay = Sum(cell delays + net delays)
- Reduced pessimism → More realistic slack
- Critical path analysis → True worst path
- Used in signoff → Reliable closure
- PBA vs GBA → Accurate but slower vs fast but pessimistic

17. Abbreviations

PBA	: Path-Based Analysis
GBA	: Graph-Based Analysis
CPPR	: Common Path Pessimism Removal
CRPR	: Clock Reconvergence Pessimism Removal
Tclk	: Clock Period
Tdata_arr	: Data Arrival Time
Tunc	: Uncertainty
FF	: Flip-Flop

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

TIMING VIOLATION DEBUGGING - SETUP VIOLATION DEBUGGING

1. IDENTIFY VIOLATING PATH

- First step is to find the path that violates setup check.
- Use STA report → worst negative setup slack.
- Note the path, slack value, and endpoint.
- This path guides the entire debugging process.



Path = Launch FF → Combinational logic → Capture FF

Slack (setup) < 0 → Violation

2. CHECK LAUNCH POINT

- Verify the launching flip-flop (source).
- Check if clock to launch FF is correct and not gated.
- Ensure launch edge is the expected active edge.
- Confirm there is no unintentional delay in launch clock path.

3. CHECK CAPTURE POINT

- Verify the capturing flip-flop (destination).
- Check clock to capture FF is correct.
- Ensure capture edge is the expected active edge.
- Confirm destination constraints (set_clock_groups, exceptions, multicycle) are as intended.

4. CHECK CLOCK PATH

- Analyze both launch and capture clock paths.
- Look for long / short wire delays, buffers, or congestion.
- Inspect clock tree quality, balance and insertion delays.
- Ensure no unintended clock gating or mux on clock.



5. CHECK CLOCK SKEW

- Clock skew = Capture clock arrival − Launch clock arrival.
- Positive skew (capture late) → reduces setup slack.
- Negative skew (capture early) → improves setup slack.
- Large positive skew often causes setup violations.

$$\text{Skew} = T_{clk,cap} - T_{clk,launch}$$

Setup Slack = Required Time − Arrival Time

$= (T_{clk,cap} - \text{Setup Time}) - \text{Arrival Time}$

6. CHECK UNCERTAINTY

- Total uncertainty reduces the available setup time.
- Includes : Clock uncertainty + Jitter + OCV + Margin.
- Larger uncertainty → less required time → violation.
- Review and optimize uncertainty components if possible.

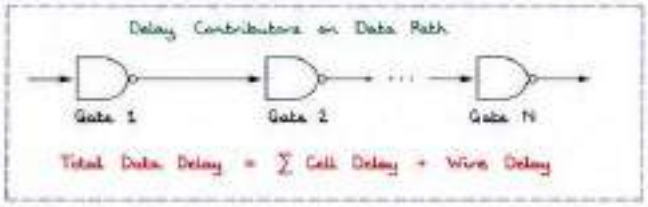
Setup Required Time

$RT = T_{clk,cap} - \text{Setup Time} - \text{Uncertainty}$

(Higher Uncertainty → Lower RT)

7. CHECK CELL DELAY

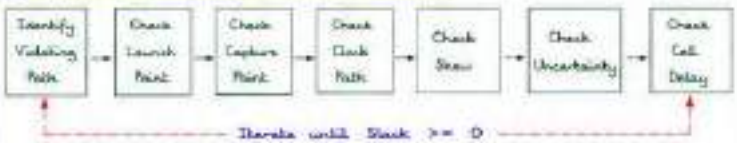
- Analyze cell delays on the path.
- Identify gates with high intrinsic delay or large load.
- Check fanout, output capacitance, and drive strength.
- Replace high delay cells, upsizing, or buffering if needed.
- Ensure cell library corner matches analysis corner.



8. SUMMARY OF CHECKS

Step	What to Check	Why Important	Impact on Setup Slack
1. Identify Violating Path	Worst setup slack path	Focus debugging efforts	Directly identifies critical issue
2. Check Launch Point	Source FF, launch clock	Incorrect launch can mislead	Wrong launch reduces slack
3. Check Capture Point	Destination FF, capture clock	Incorrect capture causes error	Wrong capture reduces slack
4. Check Clock Path	Clock tree to launch & capture	Clock delays affect available time	Longer path → less slack
5. Check Clock Skew	Skew between clocks	Skew shifts required time	Positive skew → less slack
6. Check Uncertainty	Clock unc., jitter, OCV, margins	Reduces required time	Higher unc. → less slack
7. Check Cell Delay	Cell delays, load, fanout	High delay increases arrival time	Higher delay → less slack

9. DEBUGGING FLOW (SETUP VIOLATION)



10. PRACTICAL OBSERVATIONS

- Setup violations are usually caused by late data arrival or early required time.
- Positive clock skew and high uncertainty are major contributors.
- Start fixing from clock related issues first, then data path.
- Always re-run STA after each fix to confirm improvement.

QUICK REVISION

- ✓ Find worst setup violating path
- ✓ Verify launch and capture flip-flops and their clocks
- ✓ Check clock paths, skew and uncertainty
- ✓ Analyze cell delays and optimize data path
- ✓ Iterate until slack is non-negative

11. VIVA QUESTIONS

Q1. What is setup violation?
Q2. How do you identify a violating path?
Q3. What is clock skew?
Q4. How does positive skew affect setup?
Q5. What is uncertainty in STA?
Q6. What factors contribute to uncertainty?
Q7. How does cell delay affect setup slack?
Q8. Order the steps to debug setup violation.

12. PRACTICAL EXAMPLE

Given :
Clock Period = 2.0 ns
Setup Time = 0.10 ns
Uncertainty = 0.30 ns
Arrival Time = 2.25 ns
Required Time = 2.0 − 0.10 − 0.30
= 1.90 ns
Slack = RT − AT = 1.90 − 2.25
= −0.35 ns (Violation)

13. VIOLATION FIX IDEAS

- Reduce positive skew
- Balance clock tree
- Reduce uncertainty
- Upsize cells / reduce load
- Add buffers to shorten route
- Use better drive cells
- Optimize logic and routing

14. KEY FORMULAS

- Setup Slack = RT − AT
- RT = $T_{clk,cap}$ − Setup Time − Uncertainty
- AT = Launch AT + Data Path Delay
- Skew = $T_{clk,cap} - T_{clk,launch}$

(Positive Skew → Bad for Setup)

STATIC TIME ANALYSIS

TIMING VIOLATION DEBUGGING - Setup Violation Debugging - Key Checks and Methods

Setup Violation: Data arrives after the required time, point before the capture clock edge.

Goal of Debugging → Identify the root cause of negative setup slack and fix it.

1. Check Net Delay

- Interconnect (wire) delay is the major contributor to path delay.
- Longer wires, high resistance, high capacitance increase net delay.
- Check if any specific net has abnormally high delay.



Check Points	What to Look For	Impact
Net Length	Very long routes	↑ Delay
Fanout Load	High capacitive load	↑ Delay
Metal Layer	Lower metals (M1, M2)	↑ R, ↑ C, ↑ Delay
Congestion	Detours, rip-up routes	↑ Delay

2. Check Slew

- Bad (slow) slew increases cell delay and net delay.
- Caused by high fanout, long net, weak drivers.
- Check input slew at each cell on the path.

Slow slew → Higher delay through the path
→ More negative setup slack

3. Check PVT Corner

- Verify the violation across all PVT corners.
- Worst case usually: Slow Process, Low Voltage, High Temperature (SS - 0.72V - 125°C).
- A corner may give worst delay.

Corner Type	Process	Voltage	Temperature	Effect on Delay
Best Case	FF	High (1.08V)	Low (-40°C)	Min Delay
Typical	TT	Nominal (0.90V)	25°C	Nominal Delay
Worst Case	SS	Low (0.72V)	High (125°C)	Max Delay

4. Check RC Corner

- Interconnect parasitics vary with RC corners.
- Worst RC = High R / High C.
- Check if violation is worse in any RC corner.

RC Corner	Resistance	Capacitance	Effect on Delay
Best RC	Low	Low	Min Delay
Typical RC	Nominal	Nominal	Nominal Delay
Worst RC	High	High	Max Delay

5. Check OCV Derating

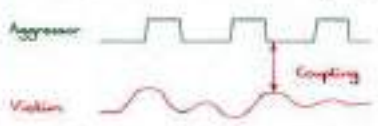
- OCV derating increases delays (late derate) in setup.
- Check if the derating factors are high.
- Excessive derating can increase pessimism.

Setup → Late derate (> 1.0)
Hold → Early derate (< 1.0)

Higher late derate → Higher delay → More violation

6. Check Crosstalk

- Switching on adjacent nets induces noise on victim net.
- Rise glitch → increases delay.
- Fall glitch → decreases delay (may cause hold issue, but rise glitch hurts setup).
- Check for max crosstalk delay impact.

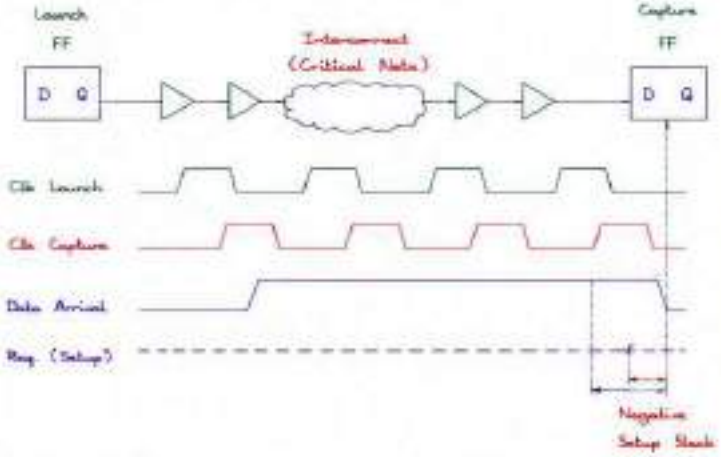


Rise glitch slows victim transition → Higher delay → Setup violation

7. Critical-Path Analysis

- Identify the worst negative slack path.
- Analyze each component: cell delay, net delay, load, slew, derates, crosstalk.
- Determine the dominant cause.

Critical Path = Path with most negative setup slack



8. Signoff Usage

- All the above checks are performed in signoff timing.
- Ensure worst-case violations are removed.
- Final signoff must show positive setup slack in all scenarios.

Signoff = Zero violations across all PVT, RC, OCV, and crosstalk

9. PBA vs GBA (Quick View)

Aspect	GBA (Graph-Based)	PBA (Path-Based)
Method	Graph propagation	Analyze each path separately
Delay Calc	Common logic counted multiple times	Each element counted only once
Pessimism	High (more pessimism)	Low (reduced pessimism)
Accuracy	Less accurate	More accurate
CPPR	Approx / Partial	Exact and effective
Runtime	Fast	Slower
Use	Early analysis Exploration	Signoff Final verification
Best For	Large designs, quick estimation	High performance critical paths

10. Practical Observation

- Always start debug by looking at path report summary.
- Examine Top 10 violating paths.
- Focus on nets with abnormal delay and cells with high delay.
- Fix root cause → re-run timing → verify slack improvement.
- Iterative debugging ensures clean signoff.

12. Debug Checklist (Summary)

- ☑ Check net delay
- ☑ Check slew (input & arrival)
- ☑ Check PVT corner (SS - Low V - High T)
- ☑ Check RC corner (High R / High C)
- ☑ Check OCV derating (late derate)
- ☑ Check crosstalk (rise glitch impact)
- ☑ Identify critical path
- ☑ Fix root cause and re-verify

13. QUICK REVISION

- Setup violation → data arrives late.
- Debug steps: Net delay, Slew, PVT, RC, OCV derating, Crosstalk.
- Long nets, slow slew, worst corner, derates, and noise → increase delay.
- Critical path gives most negative slack.
- PBA gives accurate results with less pessimism.
- Signoff requires zero violations in all scenarios.

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

SETUP FIXES

Setup Violation

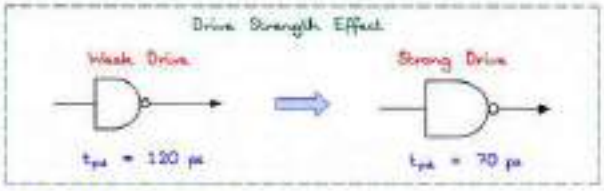
- Data arrives after the required time at the capture FF
- Slack (setup) < 0
- We need to either reduce arrival time or increase required time



Setup Slack = Required Time − Arrival Time

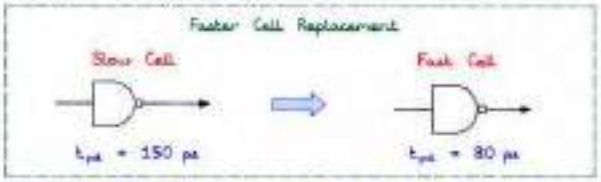
1. INCREASE DRIVE STRENGTH

- Use a stronger version of the driving cell
- Reduces output resistance and improves slew
- Lowers delay of the cell and helps reduce arrival time



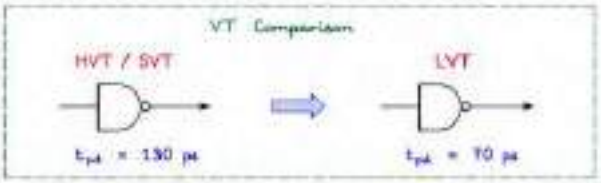
2. USE FASTER CELLS

- Replace cells with lower intrinsic delay
- Example: Use high-performance library cells
- Directly reduces logic delay of the path



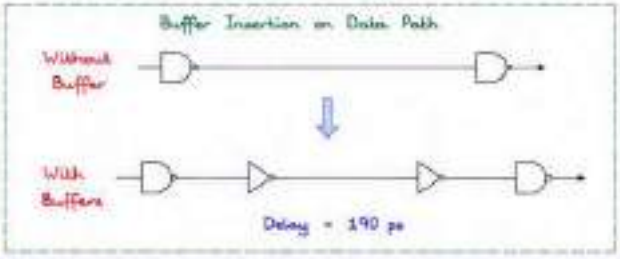
3. LVT USAGE (LOW THRESHOLD VOLTAGE)

- Use Low-Vt cells for critical path
- Lower threshold voltage → higher drive current
- Results in lower delay at the cost of higher leakage



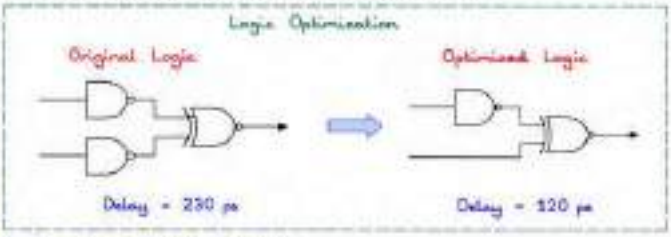
4. BUFFER INSERTION

- Insert buffers on long nets or high fanout nodes
- Breaks long wire into smaller segments
- Improves slew and reduces overall delay



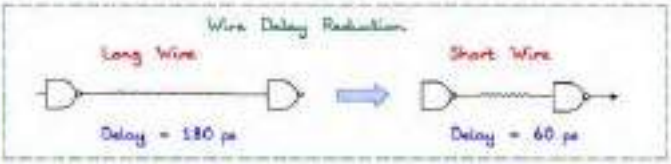
5. REDUCE LOGIC DELAY

- Simplify logic to reduce number of gates
- Remove unnecessary levels
- Apply logic restructuring / boolean optimization



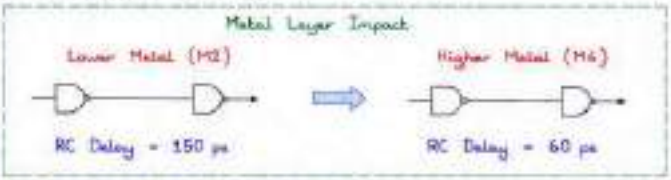
6. REDUCE WIRE DELAY

- Shorten the wire length
- Reduce fanout on long nets
- Use repeaters / buffers where required



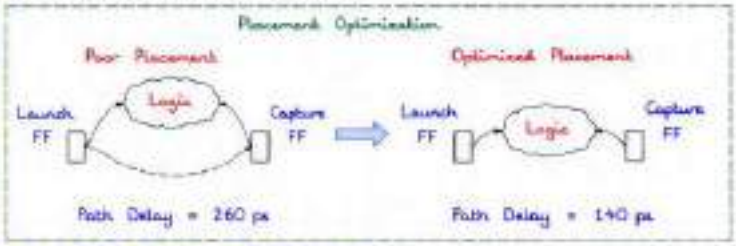
7. HIGHER METAL LAYER

- Route critical nets on higher metal layers
- Higher layers have lower resistance and capacitance
- Leads to lower RC delay



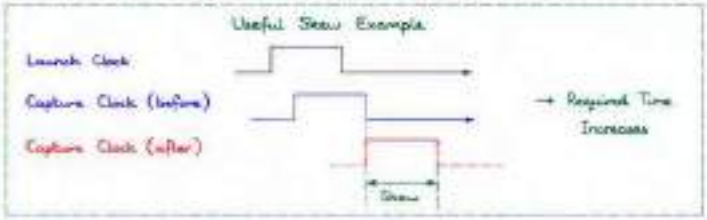
8. PLACEMENT OPTIMIZATION

- Place the cells closer to reduce interconnect length
- Keep logic levels compact
- Minimize detours and congestion



9. CLOCK SKEW OPTIMIZATION

- Apply beneficial skew (capture clock later / launch earlier)
- Positive skew increases required time
- Control skew using clock tree synthesis / useful skew



10. PRACTICAL OBSERVATIONS

- Start with quick fixes (buffers, drive)
- Check all paths, not just critical
- Fix root cause, not only symptoms
- Always re-run STA after each fix
- Evaluate area, power and timing trade-off

11. VIVA QUESTIONS

01. What is setup violation?
02. How do you fix setup violation?
03. Why do we use stronger cells?
04. What is benefit of using LVT?
05. How does buffer insertion help?
06. What is useful skew?
07. Which layer reduces RC delay?
08. What is placement optimization?

12. QUICK REVISION

- ✓ Reduce arrival time or increase required time
- ✓ Faster cells, stronger drive → lower delay
- ✓ LVT → lower delay (higher leakage)
- ✓ Buffers, shorter wires → less delay
- ✓ Higher metal → less RC delay
- ✓ Better placement → short interconnect
- ✓ Useful skew → more setup margin

13. KEY FORMULAS

- Setup Slack:

$$Slack_{setup} = RT - AT$$

- Skew:

$$Skew = T_{cap} - T_{launch}$$

- Delay:

$$Total\ Delay = Cell + Wire$$

STATIC TIME ANALYSIS

TIMING VIOLATION DEBUGGING - Hold Violation Debugging - Key Checks and Methods

Hold Violation : Data changes too early and does not meet the required hold time at capture FF.

> Goal of Debugging → Identify root cause of negative hold slack and fix it.

1. Identify Minimum-Delay Path

- Find the path with most negative hold slack.
- Minimum data delay path usually causes hold violation.



2. Check Launch Clock

- Verify launch clock latency (incl. source + network).
- Check for short path in launch clock tree.
- Early launch clock arrival increases hold violation.

> Earlier Launch Clock → More chance of hold violation

3. Check Capture Clock

- Verify capture clock latency.
- Late capture clock arrival → reduces hold violation.
- Early capture clock → increases hold violation.

> Earlier Capture Clock → Higher hold violation

4. Check Clock Skew

- Clock Skew = Capture Clock Arrival − Launch Clock Arrival.
- Negative skew (capture earlier) → increases hold violation.
- Positive skew (launch earlier) → reduces hold violation.

> Skew < 0 → Worse for Hold
> Skew > 0 → Better for Hold

5. Check Uncertainty

- Includes clock uncertainty, jitter, variation.
- Hold uncertainty reduces the effective required time.
- (makes it harder to meet hold).

> Effective Hold Requirement = Hold Time − Uncertainty
> Lower requirement → More hold violation

6. Check Minimum Cell Delay

- Use cells with adequate minimum (contamination) delay.
- Very small cell delay (fast cells) → causes early data arrival.
- Review fast cells on the min-delay path.

Cell Type	Min Delay	Impact on Hold
Fast Cells (LVT)	Very Low	↑ Hold Violation
Typical Cells	Normal	Balanced
Slow Cells (HVT)	Higher	↓ Hold Violation

7. Check Minimum Net Delay

- Interconnect parasitics affect min delay.
- Short routes, thin wires, low R/C → small net delay.
- Increase wire length / use higher layer → increases delay.

Net Condition	Min Net Delay	Impact on Hold
Short, Thin, Lower Metal	Very Small	↑ Hold Violation
Typical Routing	Normal	Balanced
Long, Wide, Higher Metal	Large	↓ Hold Violation

8. Check PVT Corner

- Hold violation is worst in FAST PVT.
- Fast Process → lower transistor delay.
- High Voltage → faster switching.
- Low Temperature → faster devices.

PVT Corner	Process	Voltage	Temp	Hold Impact
Worst Case	Fast (FF)	High (1.08V)	Low (−40°C)	Max Violation
Typical Case	Typical (TT)	Nom (0.90V)	25°C	Moderate
Best Case	Slow (SS)	Low (0.72V)	High (125°C)	Min Violation

9. Check RC Corner

- Affects interconnect minimum delay.
- Worst RC for hold → Low R / Low C (minimum delay).

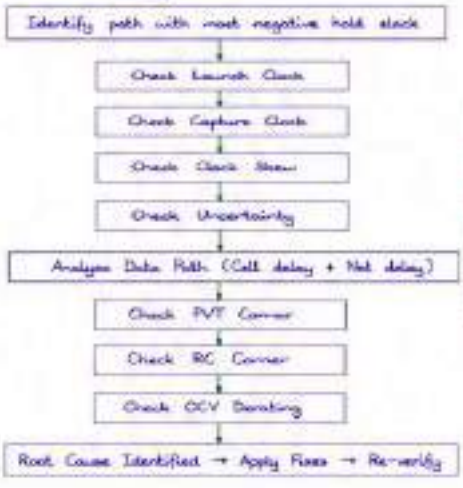
RC Corner	Resistance (R)	Capacitance (C)	Hold Impact
Worst for Hold	Low	Low	Max Violation
Typical RC	Nominal	Nominal	Moderate
Best for Hold	High	High	Min Violation

10. Check OCV (On-Chip Variation)

- OCV derating affects min delay.
- Early derate (< 1.0) reduces delay further → worse hold.
- Check early derate factors used for cells and nets.

> Early Derate < 1.0 → Delay decreases → Hold worse
> Late Derate > 1.0 → Delay increases → Hold better

11. Hold Debug Flow



Common Fixes

- Add buffers on data path
- Use slower cells
- Increase wire length / use higher layer
- Insert delay cell
- Adjust clock skew

12. Practical Observation

- Hold violations are difficult to see in logic simulation.
- They appear due to silicon variations and min delays.
- Min delay paths are usually short and use fast cells.
- Good clock tree balance helps to reduce hold issues.
- Fix early in flow → avoids rework.

13. Viva Questions

Q1. What is a hold violation?
Q2. Why does hold violation occur?
Q3. How does clock skew affect hold?
Q4. Which PVT corner is worst for hold?
Q5. How does uncertainty impact hold?
Q6. Which RC corner is worst for hold?
Q7. How does OCV affect hold timing?
Q8. How do you fix hold violation?
Q9. What is minimum-delay path?
Q10. Difference between setup and hold debugging?

14. QUICK REVISION

- Hold violation → Data arrives too early.
- Debug by checking clock path, data path, PVT, RC, OCV and uncertainty.
- Negative clock skew → worse for hold.
- Fast PVT and Low RC → worst for hold.
- Increase delay or skew to fix hold violation.
- Re-verify after applying fixes.

STATIC TIMING ANALYSIS (STA)

HOLD FIXES

Hold Violation :

- Data arrives too early at the capture FF
- Hold Slack < 0
- We need to delay the data arrival or delay the capture edge

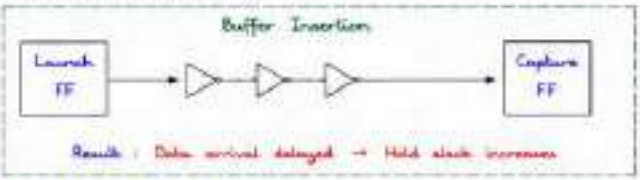


Hold Slack = Data Arrival Time − Required Time

(Negative Slack → Violation)

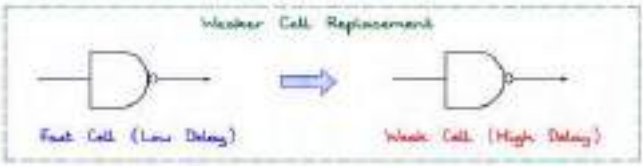
1. DELAY BUFFERS

- Insert buffers on the data path to add delay
- Increases data path delay



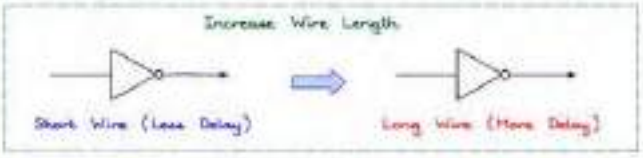
2. WEAKER CELLS

- Replace fast cells with weaker (higher delay) cells
- Slower cells add more delay



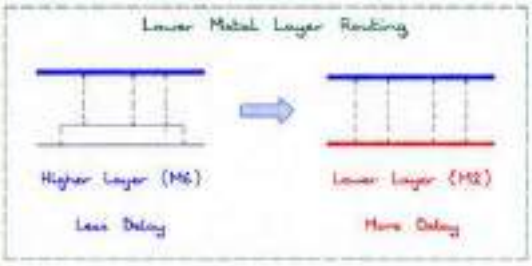
3. INCREASE WIRE DELAY

- Add extra wire length
- Increases RC of the net → more delay
- Use longer routes or add repeaters



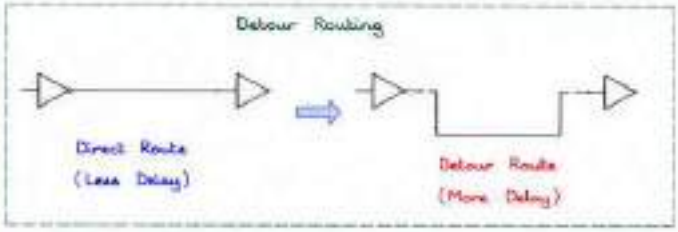
4. LOWER METAL LAYER

- Route critical nets on lower metal layers
- Lower layers have higher resistance and capacitance
- Results in higher RC delay



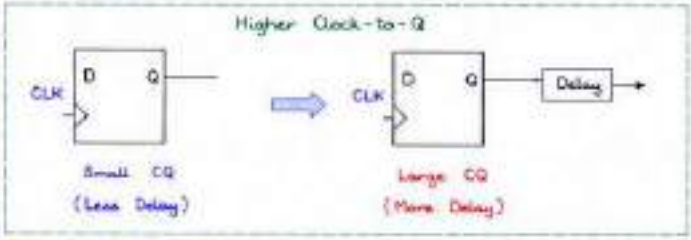
5. DETOUR ROUTING

- Take a longer path for the net intentionally
- Adds extra length without affecting functionality
- Simple but effective for small violations



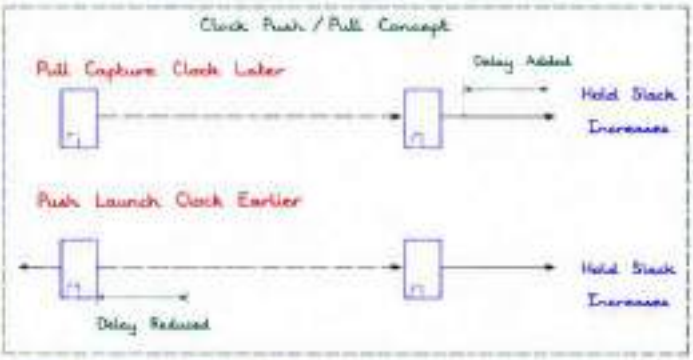
6. INCREASE CLOCK-TO-Q (CQ)

- Use flip-flops with larger clock-to-Q delay
- Data launches later → more delay on path



7. CLOCK PUSH / PULL

- Adjust clock arrival times to fix hold
- Pull Capture Clock later (add delay) → increases hold slack
- Push Launch Clock earlier (reduce delay) → increases hold slack



EFFECT OF HOLD FIXES ON DATA PATH

Fix Method	What it Changes	Data Path Delay	Hold Slack	Impact on Setup	Notes
Delay Buffers	Adds buffer delay	Increases	Increases	May worsen setup	Common and controllable
Weaker Cells	Use slower cells	Increases	Increases	May worsen setup	Check power/area
Increase Wire Delay	Add wire length	Increases	Increases	May worsen setup	Simple method
Lower Metal Layer	Higher RC of wire	Increases	Increases	May worsen setup	Check congestion
Detour Routing	Longer route	Increases	Increases	May worsen setup	Use carefully
Increase Clock-to-Q	Larger CQ delay	Increases	Increases	Slight impact	Check library options
Clock Push/Pull	Clock timing adjustment	(Effective Delay)	Increases	May degrade/improve	Use with balance

8. PRACTICAL OBSERVATIONS

- Fix only the violating paths
- Over-fixing hold can create setup violations
- Always run STA after each fix
- Prefer logical fixes (buffers, cells) over routing detours when possible

9. VIVA QUESTIONS

Q1. What is a hold violation?
Q2. How do buffers help in hold fixing?
Q3. What is the effect of using weaker cells?
Q4. Why does lower metal layer increase delay?
Q5. What is detour routing?
Q6. How can increasing clock-to-Q fix hold?
Q7. What is clock push/pull?
Q8. Which fix may degrade setup timing?

10. QUICK REVISION

✓ Increase data path delay or delay capture clock
✓ Buffers, weaker cells, wire delay → add delay
✓ Lower metal, detour routing → increase delay
✓ Higher CQ, clock pull, launch push → improve hold
✓ Always verify setup after hold fixes

11. KEY FORMULAS

- Hold Slack

$$Slack_{hold} = AT - RT$$

- Hold Condition :

$$AT \geq RT$$

- RT (Hold) = Capture Clock Arrival + Hold Time
- Slack > 0 → Safe
  Slack = 0 → Critical
  Slack < 0 → Violation

STATIC TIME ANALYSIS

SPECIAL TIMING CHECKS - Recovery & Removal → Concepts, Checks, Relations and Reset Timing

Special Timing Checks: Some signals do not follow data path setup/hold rules. They are asynchronous control signals (e.g., Reset, Set, Clear, Enable, Preset).

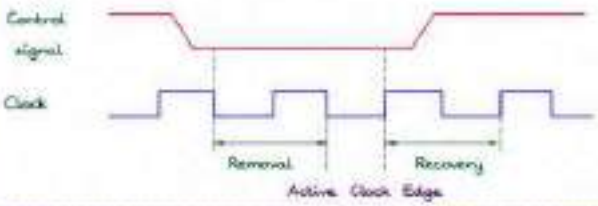
Recovery & Removal checks ensure these control signals are released safely w.r.t. clock edges.

1. Synchronous vs Asynchronous Timing

Synchronous Timing	Asynchronous Timing
• Data changes are controlled by the clock.	• Control signal can change independent of clock.
• Setup & Hold checks apply.	• Recovery & Removal checks apply.
• Example: D input of FF.	• Example: Reset, Set, Clear

2. Concept of Recovery & Removal

- Control signal must be de-asserted safely relative to the active clock edge.
- If released too late → may cause metastability / illegal state.



Removal → Control must be inactive for some time BEFORE the active clock edge.

Recovery → Control must remain inactive for some time AFTER the active clock edge.

3. Recovery Check

- Minimum time that control signal must be stable (inactive) AFTER the clock edge.
- Ensures FF/logic returns to normal mode safely.
- If recovery time is violated → incorrect capture.

Recovery Slack = (Control release time after clock) − Recovery requirement

Violation if Recovery Slack < 0

4. Removal Check

- Minimum time that control signal must be stable (inactive) BEFORE the clock edge.
- Ensures FF/logic exits asynchronous state safely before capture.

Removal Slack = (Control inactive time before clock) − Removal requirement

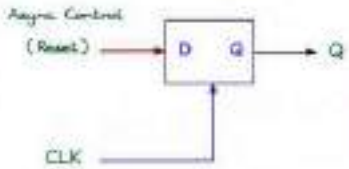
Violation if Removal Slack < 0

5. Relation to Setup / Hold

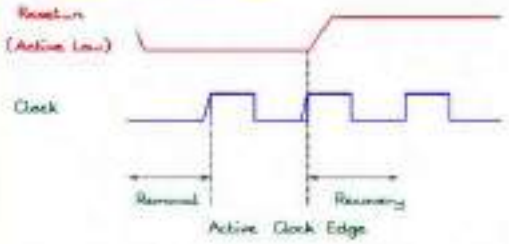
Check Type	Applies To	Relative to Clock Edge	Goal
Setup	Data Inputs	Before next edge	Data stable before capture
Hold	Data Inputs	After same/prev edge	Data stable after capture
Removal	Async Control	Before active edge	Exit async state before capture
Recovery	Async Control	After active edge	Remain stable after capture

6. Asynchronous Control Signals

+ Common control signals that use recovery/removal checks:
  - Reset (active High/Low)
  - Set / Preset
  - Clear
  - Enable / Gate
  - Scan enable, Test mode
+ These signals do not follow data setup/hold rules.
+ Must meet recovery/removal to avoid metastability.



7. Reset Timing Example



De-assert Reset_n must satisfy both Removal and Recovery.

8. Debug Checklist for Hold Violation (Recovery Removal)

Step	Check Item	What to Verify	Impact If Wrong
1	Identify minimum delay path	Path with most negative hold slack	Root cause path identification
2	Check launch clock	Launch clock latency (source + network)	Early arrival → hold violation
3	Check capture clock	Capture clock latency (source + network)	Late arrival → hold violation
4	Check clock skew	Skew = Capture − Launch	Negative skew hurts hold
5	Check uncertainty	Clock uncertainty value	Higher uncertainty hurts hold
6	Check minimum cell delay	Min delay of cells on data path	Too small delay → hold violation
7	Check minimum net delay	Interconnect min delay (RC effect)	Too small net delay hurts hold
8	Check PVT corner	Worst case usually FAST PVT (FF, High V, Low T)	Fast corner → min delay small
9	Check RC corner	Lowest R / C (min RC)	Min RC → min delay small
10	Check OCV derating	Early derate for min delay (< 1.0)	More early derate → time becomes smaller

9. Effect Summary on Hold

Factor Increases Hold Slack	Factor Decreases Hold Slack
• Larger data path min delay	• Smaller data path min delay
• Larger net min delay (higher R/C)	• Smaller net min delay (low R/C)
• Positive clock skew	• Negative clock skew
• Lower uncertainty	• Higher uncertainty
• Early PVT (SS, Low V, High T)	• Fast PVT (FF, High V, Low T)
• Later OCV (< 1.0 less early derate)	• High early derate for min delay

10. Practical Observation

+ Hold violations are subtle and hard to observe in functional simulation.
+ Usually appear in min delay paths with fast conditions.
+ Early fixing in floorplan (skew, buffering, RC control) saves late iterations.
+ Always re-verify after applying fixes.

11. Viva Questions

Q1. What is recovery time?
Q2. What is removal time?
Q3. Difference between setup/hold and recovery/removal?
Q4. Why hold violations occur in fast PVT?
Q5. How does clock skew affect hold?
Q6. Which factor increases hold slack?
Q7. Which factor decreases hold slack?
Q8. What are asynchronous control signals?
Q9. Why reset needs recovery/removal checks?
Q10. How do we debug hold violation?

12. QUICK REVISION

+ Recovery → after clock edge
+ Removal → before clock edge
+ Check launch/capture clock latency
+ Check clock skew and uncertainty
+ Use min cell delay and min net delay
+ Worst case → Fast PVT, Low RC
+ Early derate reduces hold slack
+ Fix by increasing delay or skew

13. Abbreviations

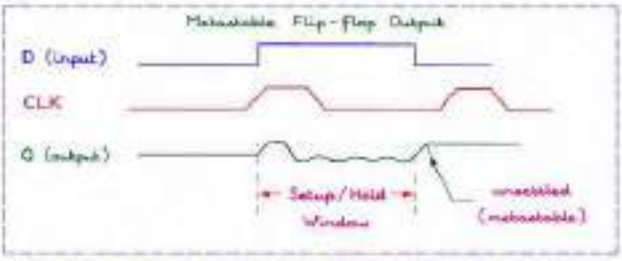
FF	:	Flip-Flop
PVT	:	Process, Voltage, Temperature
RC	:	Resistance, Capacitance
OCV	:	On-Chip Variation
SS	:	Slow-Slow
FF	:	Fast-Fast
CLK	:	Clock
Skew	:	Capture − Launch clock arrival difference

STATIC TIMING ANALYSIS (STA)

METASTABILITY

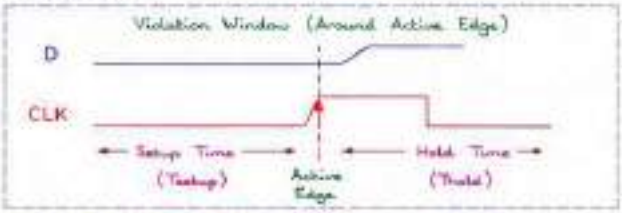
1. WHAT IS METASTABILITY?

- Metastability occurs when a flip-flop input changes within the setup and hold window around the active clock edge.
- The flip-flop output does not resolve to a valid '0' or '1' immediately.
- It enters an unstable state and may take a long (unpredictable) time to settle.



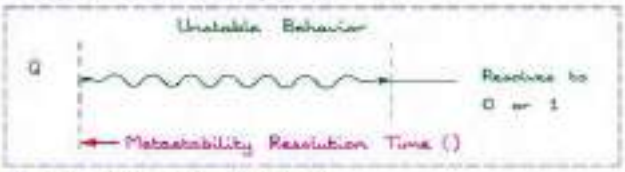
2. SETUP / HOLD VIOLATION RELAIONSHIP

- If data changes inside the setup/hold window, violation occurs.
- This may push the flip-flop into metastability.
- The closer the change to the clock edge, the longer the resolution time.



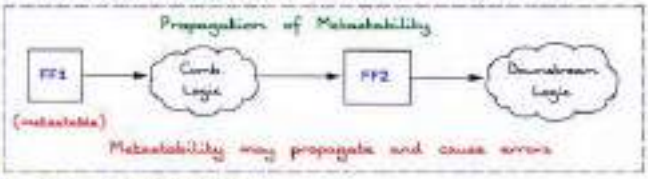
3. UNSTABLE FLIP-FLOP STATE

- When violation happens, internal regenerative feedback cannot decide quickly.
- Output may hover between 0 and 1 for some time.
- Finally, it resolves randomly to 0 or 1.



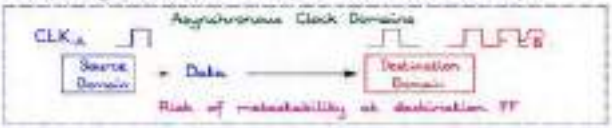
4. SYNCHRONIZATION PROBLEM

- Metastability in one flip-flop can propagate to downstream logic.
- Can cause intermediate glitches, wrong data capture, functional failure.
- Hence signals crossing clock domains must be synchronized.



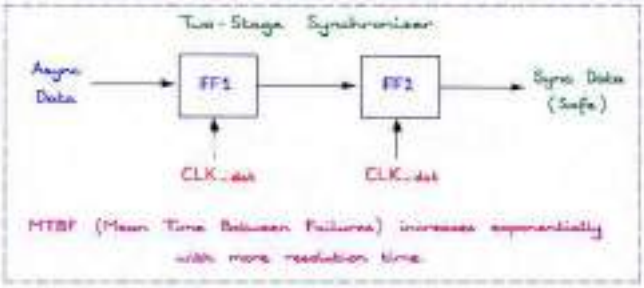
5. CDC RELAIONSHIP

- Metastability is a major issue in Clock Domain Crossing (CDC).
- Occurs when data from one clock domain is sampled by another asynchronous clock domain.
- Proper synchronization is required to make data safe.



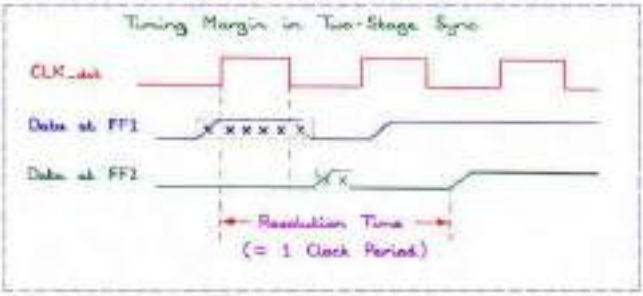
6. TWO-STAGE SYNCHRONIZER

- Most common and effective way to handle metastability.
- First flip-flop may go metastable.
- Second flip-flop gives more time (one full clock cycle) to resolve.
- Significantly reduces probability of error.



7. TIMING MARGIN

- Extra time given for metastability to resolve.
- More margin → less chance of failure.
- In two-stage synchronizer, margin ≈ one clock period.



8. SAFE DATA TRANSFER

- Synchronizer ensures stable and correct data in destination.
- Data is sampled only after it has settled.
- Guarantees reliable and safe communication across domains.



9. SUMMARY TABLE

Fix / Concept	What it Does	Effect on Hold	Notes
Delay Buffers	Adds delay in data path	Increases	Simple and common fix
Weaker Cells	Slower cells add delay	Increases	May hurt setup
Increase Wire Delay	Longer wires / repeaters	Increases	Use carefully
Lower Metal Layer	Higher RC → more delay	Increases	May worsen setup
Detour Routing	Longer route	Increases	Useful for minor fixes
Increase Clock-to-Q	Larger CK→Q delay	Increases	Slight setup impact
Clock Push / Pull	Skew adjustment	Increases	Use with balance

10. PRACTICAL OBSERVATIONS

- Metastability is unpredictable.
- Cannot be completely eliminated, only minimized.
- Always use synchronizers for CDC.
- Provide sufficient resolution time.
- Verify CDC paths using CDC tools.

11. VIVA QUESTIONS

Q1. What is metastability?
Q2. When does metastability occur?
Q3. What happens in a metastable state?
Q4. How is metastability related to CDC?
Q5. What is a two-stage synchronizer?
Q6. Why do we use two flip-flops?
Q7. What is timing margin?
Q8. How do we ensure safe data transfer?

12. QUICK REVISION

- ✓ Violation in setup/hold window → metastability
- ✓ Output becomes unstable before resolving
- ✓ Major issue in CDC
- ✓ Use two-stage synchronizer
- ✓ More resolution time → safer design
- ✓ Ensure timing margin
- ✓ Leads to safe data transfer

13. KEY POINTS

- Metastability → Unstable FF state
- Occurs due to setup/hold violation
- Propagates → Synchronization problem
- CDC needs synchronization
- Two-stage synchronizer → best practice
- Extra time → safe and reliable transfer

STATIC TIME ANALYSIS

SPECIAL TIMING CHECKS – CLOCK DOMAIN CROSSING (CDC) – Concepts and Techniques

CDC (Clock Domain Crossing) : Transfer of data/control from one clock domain to another clock domain

Improper handling → Data corruption, metastability, functional failure

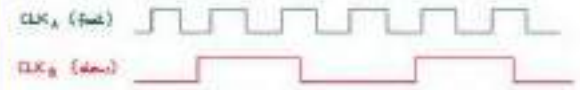
1. What is CDC ?

- When signals cross between two different asynchronous clock domains
- Data may change at any time w.r.t. the destination clock
- Requires special synchronization techniques



2. Different Clock Frequencies

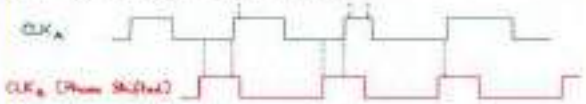
- Source and destination clocks have different periods
- Faster clock captures data more frequently
- Slower clock may miss short pulses



Case : Use level based or protocol based transfer.

3. Different Clock Phases

- Clocks have same frequency but different phase
- Timing relationship keeps shifting



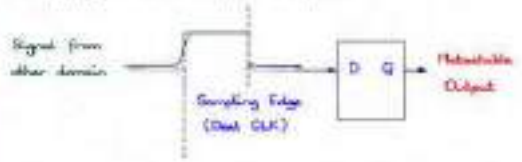
4. Asynchronous Domains

- No fixed phase or frequency relationship
- Data arrival in destination domain is uncertain
- Requires synchronization or protocol based transfer



5. Metastability

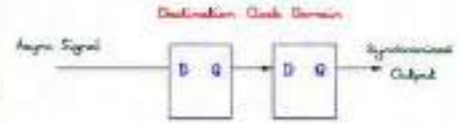
- Occurs when a signal changes very close to the sampling edge of destination flip-flop
- Output may remain in unstable state for some time
- Can propagate and cause functional failure



Solution : Allow signal to settle using synchronizers

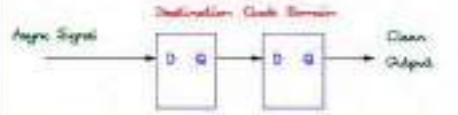
6. Single-Bit Synchronization

- Used for single bit control or status signals
- Destination clock safely samples the signal



7. Two-Flop Synchronizer

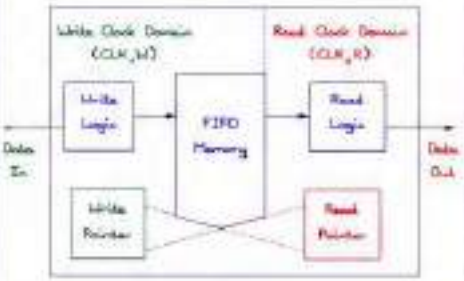
- Most common and recommended technique
- First FF may go metastable, second FF allows more time to settle
- Greatly reduces probability of metastability propagation



MTBF increases exponentially with more settling time

8. FIFO (First-In First-Out)

- Used for high speed data transfer between domains
- Separate write clock domain and read clock domain
- Pointer synchronization ensures safe read/write



9. Handshake (Request-Acknowledge)

- Used for control or low throughput data transfer
- Sender requests, receiver acknowledges
- Ensures data is captured safely



10. Gray-Code Counter

- Only one bit changes between consecutive counts
- Used for pointer synchronization in FIFOs
- Reduces risk of sampling multiple changing bits



Only one bit change → safer across domains

11. CDC Timing Concerns

- No direct timing path between asynchronous domains
- STA tools do not analyze CDC paths
- Must rely on proper CDC design techniques
- Avoid combinational logic between synchronizer stages
- Ensure enough settling time

Main Concerns :

- Metastability
- Data loss / duplication
- Glitch on control signals
- Reset release across domains
- CDC verification (Lint / formal)

12. CDC Technique Selection

Scenario	Recommended Technique
Single bit control	Two-flop synchronizer
Single pulse	Pulse synchronizer / Handshake
Multi-bit data (low speed)	Handshake
Multi-bit data (high speed)	FIFO with pointer sync.
Counters / pointers	Gray-code counter
Reset / Clear	Reset synchronizer

13. Comparison of CDC Methods

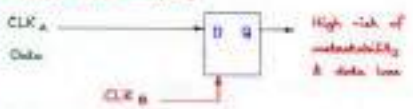
Method	Pros	Cons
Two-Flop Synchronizer	Simple, low area, high reliability	Only for single bit, not for data buses
FIFO	High throughput, no data loss	More area, complex control
Handshake	Reliable, works for multi-bit	Lower throughput, more latency
Gray-Code	Safe pointer transfer	Only for counters / pointers

14. Practical Observation

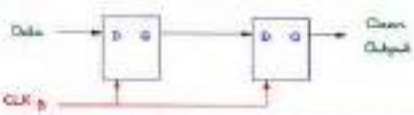
- Never use single flip-flop to cross asynchronous domains
- Always use two-flop synchronizer for single-bit CDC
- For multi-bit data, use FIFO or handshake protocol
- Proper reset synchronization is critical for functional bring-up
- CDC violations are not timing violations but functional risks

15. Example – Unsafe vs Safe CDC

(A) Unsafe Direct Sampling



(B) Safe Two-Flop Synchronizer



16. Viva Questions

Q1. What is Clock Domain Crossing (CDC) ?
Q2. Why is CDC required in digital design ?
Q3. What problems occur due to CDC ?
Q4. What is metastability ?
Q5. How does a two-flop synchronizer work ?
Q6. Why is two-flop synchronizer preferred ?
Q7. When do we use FIFO for CDC ?
Q8. Explain handshake based CDC
Q9. Why Gray-code is used in CDC pointers ?
Q10. What are CDC timing concerns ?

17. QUICK REVISION

- CDC = Clock Domain Crossing
- Data crosses between asynchronous clocks
- Metastability is the main risk
- Two-flop synchronizer for single-bit
- FIFO for high throughput multi-bit
- Handshake for low speed reliable transfer
- Gray-code for safe pointer/counter transfer
- STA does not time CDC paths
- Use proper CDC techniques for safety

18. Abbreviations

CDC	:	Clock Domain Crossing
CLK	:	Clock
FF	:	Flip-Flop
FIFO	:	First-In First-Out
REQ/ACK	:	Request / Acknowledge
MTBF	:	Mean Time Between Failure
STA	:	Static Timing Analysis
PVT	:	Process, Voltage, Temperature
RC	:	Resistance, Capacitance

STATIC TIME ANALYSIS

PHYSICAL EFFECTS ON TIMING - Concepts and Impact on Path Delay

Physical Effects : Physical implementation significantly affects timing. Poor placement or routing increases delay and reduces slack.

1. Placement Impact

- Placement dictates the Manhattan distance between cells.
- Longer distance → longer wires → more delay.
- Good placement → shorter data paths → better timing.



Closer placement → shorter interconnect → lower delay

2. Cell Proximity

- Cells placed very close cause coupling capacitance.
- Switching in one net induces noise in nearby nets.
- Increases delay and can cause functional issues.



C_c : Coupling Capacitance

More proximity → Higher C_c → More delay & noise

3. Wire Length

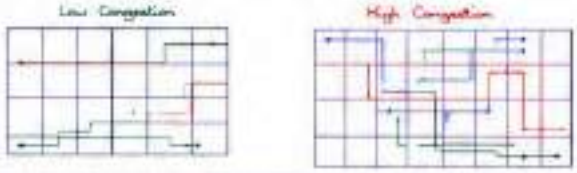
- Longer wire → more resistance and capacitance.
- Delay increases approximately with RC.
- Longer wires also increase signal transition time (slew).



Delay ∝ (R · C) , Longer L → Higher R & C → Higher Delay

4. Congestion

- Congested area has many nets competing for space.
- Leads to detours, layer changes and longer wires.
- Increases RC and makes timing closure difficult.



More congestion → Detours & Longer paths → Higher delay

5. Routing Layers

- Higher metal layers have lower resistance and capacitance.
- Higher layers → lower delay and better signal quality.
- Lower layers → more RC → more delay.

Metal Layer	R (ohm / unit)	C (fF / unit)	Impact on Delay
M1 (Lower)	High	High	High Delay
M2 - M3	Medium	Medium	Medium Delay
M4 - M5	Low	Low	Low Delay
Top Metal	Very Low	Very Low	Lowest Delay

Use higher layers for timing critical nets

6. Resistance (R)

- Resistance comes from wire material and length.
- R increases with length and decreases with wider wires / higher layers.
- Higher R → higher voltage drop → slower transitions.

$$R = \rho \cdot \frac{L}{W \cdot t}$$

ρ : Resistivity , L : Length , W : Width , t : Thickness



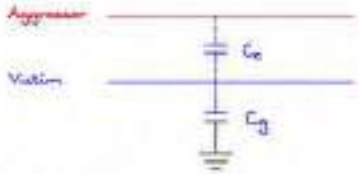
Higher R → More delay

7. Capacitance (C)

- Capacitance includes ground capacitance (C_g) and coupling capacitance (C_c).
- C increases with wire length, width and proximity to other nets.
- Higher C → more charging time → more delay.

$$C_{total} = C_g + C_c$$

Higher C → More delay



8. Overall Delay Due to R and C

- Elmore delay of an RC interconnect:

$$t_{delay} = 0.5 \cdot R_{total} \cdot C_{total}$$

- Increase in either R or C directly increases delay.

Reduce R and C → Reduce Delay → Improve Timing

9. Impact Summary Table

Physical Effect	What Increases	What Decreases	Net Impact on Delay
Placement (distance)	Distance	Proximity	↑ Delay Increases
Cell Proximity	Coupling Capacitance	Spacing	↑ Delay Increases
Wire Length	Length	Shorter Length	↑ Delay Increases
Congestion	Detours, Layer Jumps	Clean Routing	↑ Delay Increases
Routing Layers	Lower Layers	Higher Layers	↓ Delay Decreases
Resistance (R)	Length, Narrow Width, Lower Layer	Wider, Shorter, Higher Layer	↑ Delay Increases
Capacitance (C)	Length, Width, Proximity	Shorter, Narrow, More Spacing	↑ Delay Increases

10. Practical Observation

- Physical design quality directly decides timing closure.
- Good placement and congestion-free routing reduce delay.
- Always check RC values of critical nets during timing analysis.
- Use higher layers and wider wires for timing critical paths.
- Maintain proper cell spacing to reduce coupling.

11. Viva Questions

Q1. What is CDC and why is it required?
Q2. What are different clock domain issues?
Q3. What is metastability?
Q4. What is a two-flop synchronizer?
Q5. When do we use FIFO?
Q6. What is Gray-code counter?
Q7. What are CDC timing constraints?
Q8. Why recovery/removal checks are used?
Q9. How does placement affect timing?
Q10. Why higher metal layers reduce delay?

12. QUICK REVISION

- Placement for → long wire → more RC → more delay
- Close cells → more coupling C_c → more delay & noise
- Longer wire → more R & C → more delay & slew
- Congestion → detours → more length → more delay
- Higher metal layer → lower R & C → less delay
- $R \propto L / (W \cdot t)$, $C_{total} = C_g + C_c$, $t_{delay} \approx 0.5RC$
- Keep critical path short, clean and on higher layers
- Manage proximity and congestion for timing success

13. Abbreviations

R	:	Resistance
C	:	Capacitance
RC	:	Resistance - Capacitance
M1/M2..	:	Metal layers
C_g	:	Ground capacitance
C_c	:	Coupling capacitance
STA	:	Static Timing Analysis
L	:	Length W : Width t : Thickness

IN → OUT

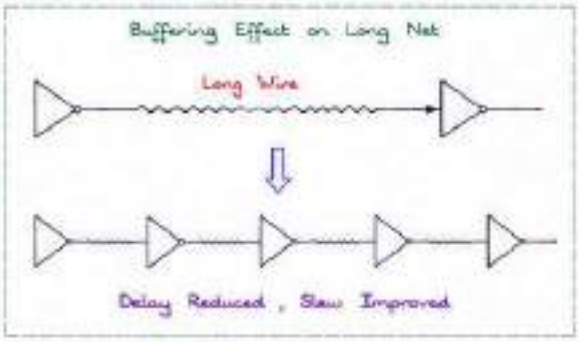
STATIC TIMING ANALYSIS (STA)

CLK

PHYSICAL EFFECTS ON TIMING

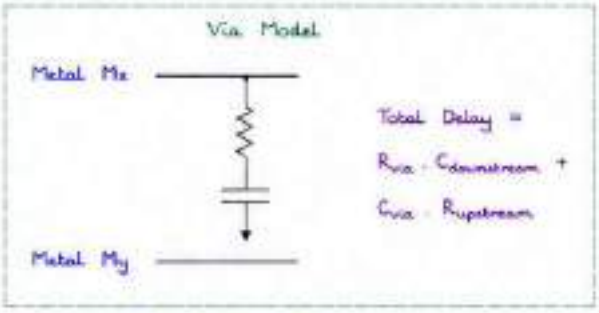
1. BUFFERING

- Inserting buffers on long nets or critical paths
- Breaks large RC into smaller segments
- Reduces effective capacitance seen by each stage
- Improves slew and reduces delay



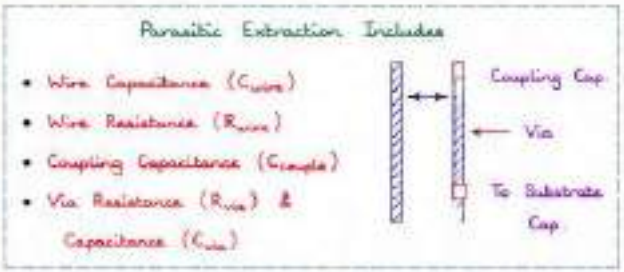
2. VIA RESISTANCE

- Vias connect two metal layers
- Each via has resistance (R_{via}) and capacitance (C_{via})
- Adds extra delay and degrades signal integrity
- More vias → more delay



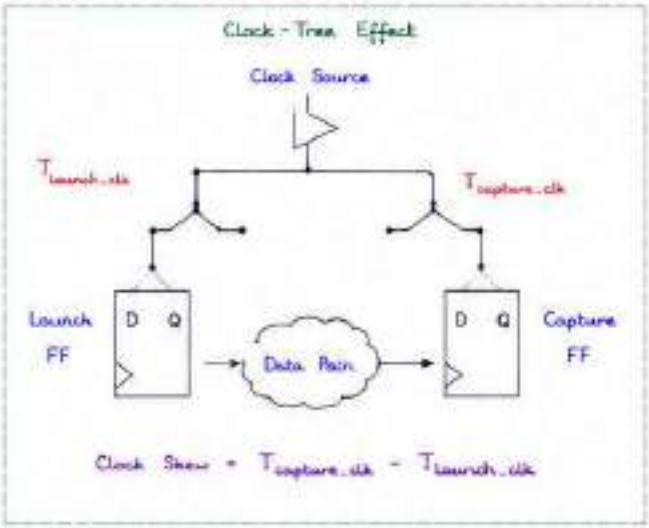
3. PARASITIC EXTRACTION

- Extracts actual parasitic R and C from final layout
- Includes wire resistance, capacitance to substrate, coupling capacitance, via R/C
- More accurate delays compared to pre-route estimates
- Essential for sign-off timing



4. CLOCK-TREE EFFECTS

- Clock travels through clock tree before reaching sequential elements
- Causes clock latency, skew and uncertainty
- Skew = $T_{capture_clk} - T_{launch_clk}$
- Affects both setup and hold timing



5. POST-ROUTE TIMING

- Performed after routing when actual parasitics are known
- Uses extracted RC and actual clock tree
- Most accurate timing numbers
- Used for final sign-off (setup & hold)

Pre-Route vs Post-Route Timing

Aspect	Pre-Route Timing	Post-Route Timing
Parasitics	Estimated / Generic	Extracted from Layout
Clock	Ideal / Approx	Actual Clock Tree
Accuracy	Less Accurate	Very Accurate (Sign-off)
Use	Early Optimization	Final Verification
Risk	May be Overly Optimistic	Realistic, No Surprises

Effect of Physical Factors on Delay

Buffering	Decreases Delay
Via Resistance	Increases Delay
Parasitic Loads	Increases Delay
Clock-Tree Skew	May Increase/Decrease Slack
Post-Route Parasitics	More Accurate Delay

6. PRACTICAL OBSERVATIONS

- Physical implementation greatly affects timing
- Pre-route timing is only a guide
- Post-route analysis may reveal new violations
- Good buffering and placement reduce risk
- Clock-tree quality is critical for both setup and hold

7. VIVA QUESTIONS

Q1. What is buffering and how does it help timing?
Q2. Why does via resistance increase delay?
Q3. What parasitics are included in extraction?
Q4. How does the clock tree affect timing?
Q5. What is post-route timing?
Q6. Why is post-route STA important?
Q7. What is clock skew?
Q8. How do physical factors impact slack?

8. QUICK REVISION

- ✓ Buffering breaks long nets → reduces delay
- ✓ Via resistance adds extra delay
- ✓ Parasitic extraction gives real delays
- ✓ Clock-tree adds latency, skew and uncertainty
- ✓ Post-route timing = accurate, used for sign-off

9. KEY FORMULAS

- Clock Skew :
  $Skew = T_{capture_clk} - T_{launch_clk}$
- Setup Slack :
  $Slack_{setup} = RT - AT$
- Hold Slack :
  $Slack_{hold} = AT - RT$

AT = Data Arrival Time
RT = Required Time

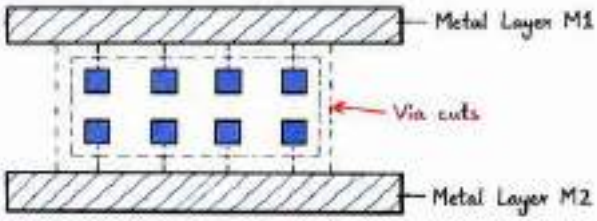
STATIC TIME ANALYSIS

(a) MULTI-CUT VIAS

- Multi-cut vias use multiple via cuts in parallel between two metal layers to improve electrical and reliability performance.

1) MULTIPLE VIA CUTS

- Instead of single via, multiple cuts are placed in an array.
- Cuts are connected in parallel.



Example: 3×2 via array (6 cuts)

2) RESISTANCE REDUCTION

- Via resistance is inversely proportional to number of cuts.
- More cuts ⇒ Low resistance ⇒ Better timing.

$$R_{eq} = \frac{R_{single}}{N}$$

R_{eq} = Equivalent resistance
R_{single} = Resistance of one cut
N = Number of cuts

3) RELIABILITY

- Parallel cuts reduce current density per cut.
- Less chance of electromigration (EM) failure.
- Improves long-term reliability.

4) CURRENT HANDLING

- Higher current can flow through multiple cuts.
- Prevents via over-stress and EM damage.
- Suitable for high-current power/ground nets.

5) CRITICAL NETS

- Used on timing-critical and high-current nets.
- Examples: Clock nets, Reset nets, Power nets.

Multi-cut Via Benefits Summary

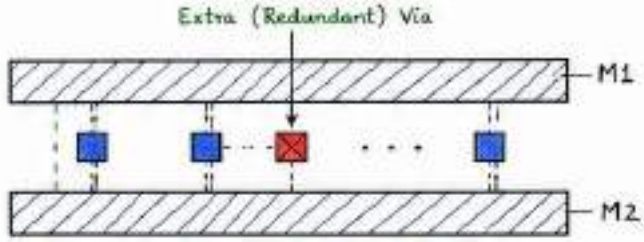
Aspect	Benefit
Resistance	Reduced via resistance
Reliability	Lower EM risk, higher life
Current Handling	Handles high current safely
Timing	Better signal integrity & timing
Usage	Used for critical & high-current nets

(b) REDUNDANT VIAS

- Redundant vias provide backup connections to protect against via failures.
- Ensures reliability and manufacturability.

1) BACKUP CONNECTION

- Extra vias are added along the path.
- If one via fails, others maintain the connection.



Blue = Active Via , Red = Failed Via

2) VIA FAILURE PROTECTION

- Protects against open via defect.
- Prevents signal open or intermittent connection.
- Part of design-for-manufacturability (DFM).

3) YIELD

- Reduces impact of manufacturing defects.
- Improves chip yield by avoiding open circuits.
- Especially useful in advanced technology nodes.

4) RELIABILITY

- Improves robustness of critical connections.
- Survives stress, aging, and random via failures.

5) CRITICAL SIGNAL PATHS

- Used on important nets where failure is not tolerable.
- Examples: Clock, Reset, Scan enable, Power critical nets.

Multi-cut Via vs Redundant Via

Feature	Multi-cut Via	Redundant Via
Purpose	Reduce resistance	Provide backup
Connection	Cuts in parallel	Extra vias as backup
Focus	Electrical performance	Reliability & yield
Failure Handling	Not for failure (all cuts work)	Survives via failure
Typical Use	High current / critical nets	Critical signal paths

Example - Resistance Improvement

Single via resistance = 20 mΩ
Number of cuts (N) = 4
Equivalent resistance - $R_{eq} = \frac{20}{4} = \boxed{5\ m\Omega}$
⇒ 75% reduction in resistance.

Practical Observation

- Use multi-cut vias on high-current and timing-critical nets to reduce delay.
- Use redundant vias on critical signals to avoid open failures.
- Both improve timing robustness and overall chip reliability.

Where to Use ?

Multi-cut Via → When performance (low resistance) is priority.

Redundant Via → When reliability and failure protection is priority.

VIVA QUESTIONS

Q1. What is a multi-cut via ?
Q2. Why does multi-cut via reduce resistance ?
Q3. Give the formula for equivalent resistance of multi-cut via.
Q4. What is the benefit of multi-cut via in timing ?
Q5. What is a redundant via ?
Q6. How does redundant via improve yield ?
Q7. Where are redundant vias typically used ?
Q8. Difference between multi-cut via and redundant via ?

QUICK REVISION

★ Multi-cut vias use multiple via cuts in parallel to reduce resistance and handle high current.
★ More cuts ⇒ lower resistance ⇒ better timing & reliability.
★ Redundant vias are extra vias used as backup connections to protect against via failures.
★ Redundancy improves yield, reliability and protects critical signal paths.
★ Use multi-cut for performance, redundant via for reliability.

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

TIMING CONSTRAINTS – SDC FUNDAMENTALS

1. WHAT IS SDC ?

- SDC (Synopsys Design Constraints) is a text based format
- to specify timing constraints to the STA tool.
- It tells the tool how the design should behave in silicon.
- SDC defines clocks, I/O timing, exceptions and uncertainties.

Example SDC Commands

```
create_clock -name clk -period 2.0 [get_ports clk]      # clock
set_input_delay 0.5 -clock clk [get_ports in]           # input constraint
set_output_delay 0.8 -clock clk [get_ports out]         # output constraint
set_false_path -from [get_ports test_in]                # exception
set_clock_uncertainty 0.1 [get_clocks clk]              # uncertainty
```

2. WHY CONSTRAINTS ARE REQUIRED ?

- Without constraints STA tool cannot determine required times.
- Constraints define environment, clocking and data interfaces.
- They ensure analysis matches real chip behavior.
- Help tool identify real violations and ignore false paths.

Without Constraints	With Constraints
Tool assumes default clocks	Realistic clock and I/O info
Unrealistic required times	Accurate required times
Many false violations	Only real violations reported
No clear interface behavior	Correct chip behavior modeled

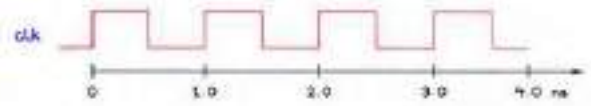
3. CLOCK DEFINITION (create_clock)

- Defines primary clocks in the design.
- Specifies name, period, waveform and source.
- All sequential timing is referenced to these clocks.

Clock Definition Example

```
create_clock -name clk -period 2.0 -waveform {0.0 1.0} [get_ports clk]
```

↑ Rising edge at 0.0ns , Falling edge at 1.0ns , Period = 2.0ns



Useful Options : -name , -period , -waveform , -rise , -fall , -add

Clock Object Attributes

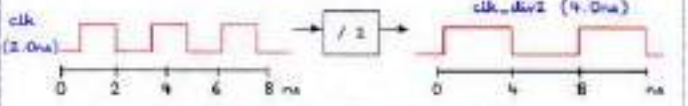
Name	:	Unique clock name
Period	:	Clock period
Waveform	:	List of {rise fall} times
Source	:	Port or pin where clock is defined
Type	:	Primary / Generated / Virtual

4. GENERATED CLOCK (create_generated_clock)

- Clock created from an existing clock after passing through a divider, multiplier or phase shift element.
- Used to model clocks at outputs of PLL, dividers, muxes, etc.
- Keeps correct timing relationship with the master clock.

Generated Clock Example

```
create_generated_clock -name clk_div2 -source [get_clocks clk] \
-divide_by 2 [get_pins u_div/clk_out]
```



clk_div2 period = clk period × 2

Edges are aligned according to source relationship.

5. VIRTUAL CLOCK

- A clock that does not physically exist in the design.
- Used to define timing of input/output ports w.r.t an external reference.
- Ensures correct interface timing checks.

Virtual Clock Example

```
create_clock -name vclk -period 2.0
```



Used with set_input_delay / set_output_delay

6. CLOCK UNCERTAINTY

- Accounts for variations in clock arrival time.
- Includes jitter, OCV, PVT variations, clock distribution variations.
- Reduces available (required) time for data arrival.

Clock Uncertainty Example

```
set_clock_uncertainty 0.1 [get_clocks clk]
```

Total uncertainty = 0.5 ns



Required Time = Ideal Capture Time − Uncertainty

COMPARISON OF CLOCK TYPES

Clock Type	Defined By	Physically Present ?	Purpose	Example Command
Primary Clock	create_clock	Yes (at port/pin)	Reference for sequential timing	create_clock -name clk -period 2.0 [get_ports clk]
Generated Clock	create_generated_clock	Yes (derived)	Clocks after dividers, PLL, muxes	create_generated_clock -divide_by 2 -source [get_clocks clk] [get_pins u_div/clk_out]
Virtual Clock	create_clock (no object)	No (external ref)	Define I/O constraints	create_clock -name vclk -period 2.0
Uncertainty	set_clock_uncertainty	N/A	Account for clock variations	set_clock_uncertainty 0.5 [get_clocks clk]

7. PRACTICAL OBSERVATIONS

- Correct constraints are more important than tool settings.
- Bad or missing constraints lead to wrong reports.
- Always review constraints before sign-off.
- Use reports (check_timing, report_clocks) to verify constraints.

8. VIVA QUESTIONS

Q1. What is SDC?
Q2. Why are timing constraints required?
Q3. What is a primary clock?
Q4. What is a generated clock?
Q5. What is a virtual clock?
Q6. When do we use a virtual clock?
Q7. What is clock uncertainty?
Q8. How does uncertainty affect timing?

9. QUICK REVISION

- ✓ SDC specifies clocks, I/O, exceptions, uncertainties.
- ✓ Constraints define environment → accurate analysis
- ✓ create_clock → primary clocks
- ✓ create_generated_clock → derived clocks
- ✓ Virtual clocks → I/O timing reference
- ✓ Uncertainty reduces available required time

10. KEY FORMULAS

- Clock Skew = $T_{capture} - T_{launch}$
- Setup Slack = Required Time − Arrival Time
- Hold Slack = Arrival Time − Required Time
- Required Time with Uncertainty :

RT = Capture Time − Uncertainty

IN → OUT

STATIC TIMING ANALYSIS (STA)

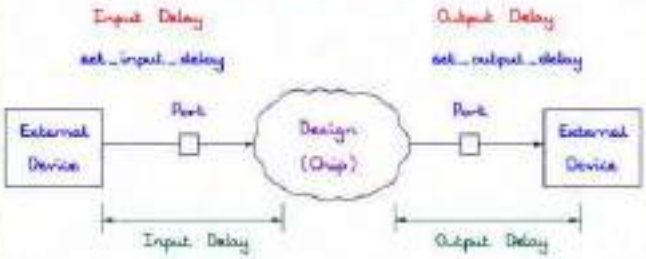
CLK

TIMING CONSTRAINTS – SDC FUNDAMENTALS

1. I/O DELAYS

- Used to model delay between the design pins and external environment.
- Tells the tool when data is available at inputs and when outputs are required.
- Essential for correct path analysis at the chip boundary.

I/O Delay Concept



2. TIMING EXCEPTIONS

- Used to tell STA tool to ignore or modify certain paths.
- Applied for false paths, multi-cycle paths or path-based exceptions.
- Prevents reporting of unwanted or unrealistic violations.

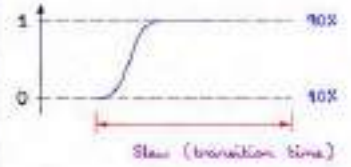
Common Timing Exceptions

Exception	Purpose	SDC Command
False Path	Ignore a path completely	set_false_path
Multi-Cycle Path	Allow more than one clock cycle	set_multicycle_path
Max Delay Path	Constrain max delay between points	set_max_delay
Min Delay Path	Constrain min delay (for hold)	set_min_delay

3. MAXIMUM TRANSITION

- Limits the maximum allowed signal transition (slew).
- High transition increases dynamic power and noise.
- Ensures signals do not become too slow on the nodes.

Maximum Transition Concept



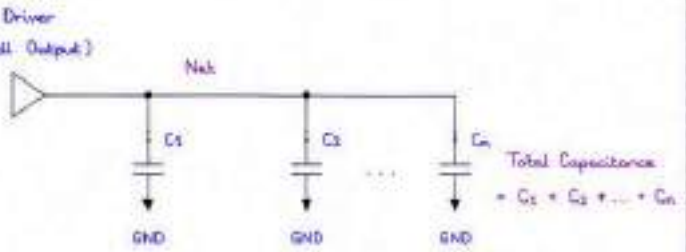
Constraint:
set_max_transition
< value (e.g., 0.2ns)

Applies on ports, nets or clock.

4. MAXIMUM CAPACITANCE

- Limits the total capacitance seen at a net or pin.
- Excessive capacitance increases delay and power.
- Helps in controlling fanout and loading.

Maximum Capacitance Concept

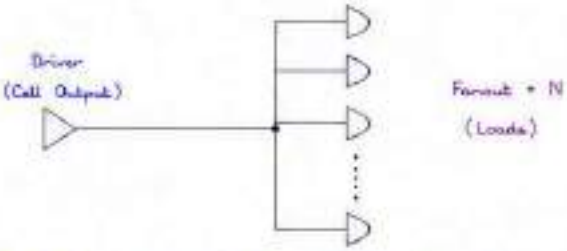


Constraint : set_max_capacitance < value (e.g., 0.1pF)

5. MAXIMUM FANOUT

- Limits the number of loads driven by a single cell output.
- Too many loads increases delay, transition and power.
- Helps maintain signal integrity and timing quality.

Maximum Fanout Concept



Constraint : set_max_fanout < value (e.g., 8)

SUMMARY TABLE

Constraint	What it Controls	Why it is Needed	Example Command
I/O Delays	External interface timing	Model external devices	set_input_delay, set_output_delay
Timing Exceptions	Ignore/relax or modify paths	Avoid false violations	set_false_path, set_multicycle_path
Max Transition	Signal slew (time for rise/fall)	Control power, noise & delay	set_max_transition
Max Capacitance	Total capacitance on a net/pin	Limit delay & loading	set_max_capacitance
Max Fanout	Number of loads driven	Maintain timing & signal integrity	set_max_fanout

6. PRACTICAL OBSERVATIONS

- Proper constraints are as important as good design.
- Wrong or missing constraints lead to false violations or missed issues.
- I/O delays must match PCB or system timing.
- Exceptions should be used carefully with clear justification.
- Transition, capacitance and fanout control power and timing.

7. VIVA QUESTIONS

Q1. What is SDC?
Q2. Why are timing constraints required?
Q3. What command is used for input delay?
Q4. What is the purpose of set_false_path?
Q5. What is a multi-cycle path?
Q6. What does set_max_transition control?
Q7. How does maximum capacitance affect timing?
Q8. What is fanout and how is it limited?

8. QUICK REVISION

- ✓ I/O delays → model external timing
- ✓ Exceptions → ignore/modify paths
- ✓ Max transition → limit signal slew
- ✓ Max capacitance → limit loading
- ✓ Max fanout → limit number of loads
- ✓ Proper constraints → accurate & reliable STA results

9. KEY FORMULAS / NOTES

- Slew (Transition time)

$$t_{slew} = t_{90\%} - t_{10\%}$$

- Total Capacitance

$$C_{total} = \Sigma C_{loads}$$

- Fanout

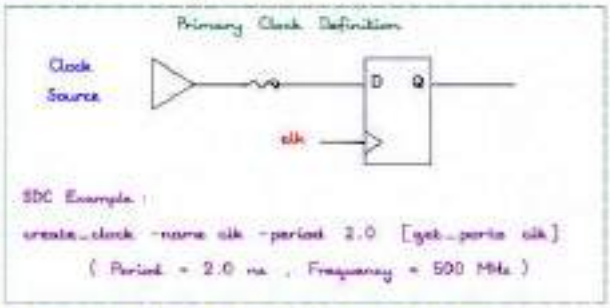
Fanout = Number of Loads

STATIC TIMING ANALYSIS (STA)

CLOCK CONSTRAINTS – SDC FUNDAMENTALS

1. PRIMARY CLOCK (create_clock)

- Primary (root) clock is defined at the clock source.
- It is the base clock for all other clocks.
- All sequential timing is referenced to this clock.



SDC Example :

```
create_clock -name clk -period 2.0 [get_ports clk]
```

(Period = 2.0 ns , Frequency = 500 MHz)

2. GENERATED CLOCK (create_generated_clock)

- Clocks generated by dividers, multipliers, MUX, phase shift, gating etc. are defined as generated clocks.
- Maintains correct clock relationships with the master clock.

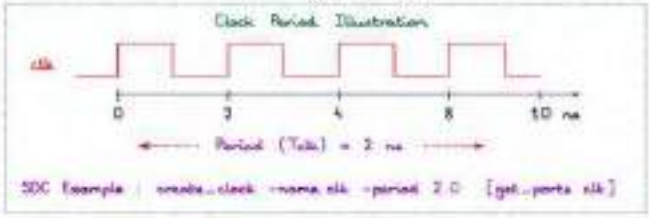


SDC Example (Divider by 2) :

```
create_generated_clock -name clk_div2 -source [get_ports clk] \
                       -divide_by 2 [get_pins u_div/clk_out]
```

3. CLOCK PERIOD

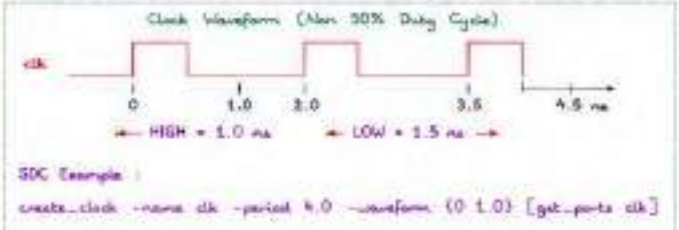
- Defines the time between two consecutive active edges.
- It sets the available time for data to arrive.
- Determines maximum operating frequency.



SDC Example : `create_clock -name clk -period 2.0 [get_ports clk]`

4. CLOCK WAVEFORM

- Defines shape of the clock (rise and fall edges).
- Specifies widths of HIGH and LOW portions.
- Important for clocks with non-50% duty cycle.

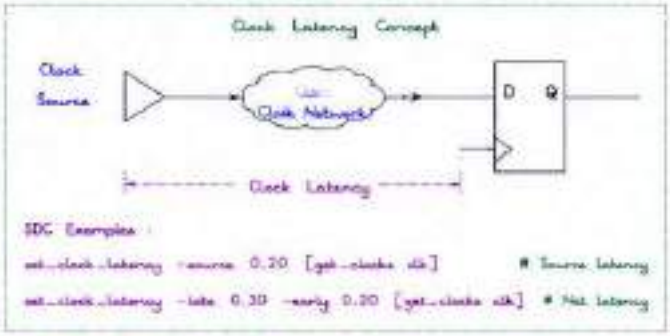


SDC Example :

```
create_clock -name clk -period 4.0 -waveform {0 1.0} [get_ports clk]
```

5. CLOCK LATENCY

- Delay from clock source to the sequential element clock pin.
- Includes clock network delay and insertion delay.
- Affects arrival time of clock at each sequential element.

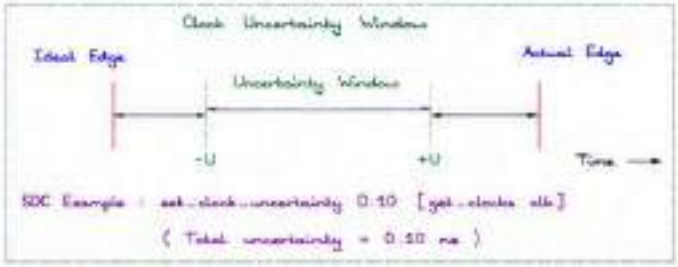


SDC Examples :

```
set_clock_latency -source 0.20 [get_clocks clk]      # Source latency
set_clock_latency -late 0.30 -early 0.20 [get_clocks clk]   # Net latency
```

6. CLOCK UNCERTAINTY

- Accounts for variations and unknowns in clock network.
- Includes jitter, PLL variation, OCV, PVT, duty cycle distortion.
- Reduces available timing margin to avoid false failures.



SDC Example : `set_clock_uncertainty 0.10 [get_clocks clk]`

(Total uncertainty = 0.20 ns)

7. CLOCK RELATIONSHIP

- Defines relationship between two clocks.
- Used for multi-clock/multi-domain designs.
- Types : -related , -asynchronous , -logically_exclusive



SDC Examples :

```
set_clock_groups -physically_exclusive -group {clk_a} -group {clk_b}
set_clock_groups -asynchronous -group {clk_a} -group {clk_b}
```

8. CLOCK SOURCE

- Specifies the origin of the clock (port or pin).
- Required for create_clock and create_generated_clock.
- Ensures STA builds correct clock tree path.



SDC Example : `create_clock -name clk -period 2.0 [get_ports clk]`

9. PRACTICAL OBSERVATIONS

- Accurate clock constraints are foundation of reliable STA.
- Wrong period or waveform leads to false timing results.
- Clock uncertainty is essential to avoid optimistic slack.
- Always define relationships between independent clocks.
- Check clock latency reports during sign-off.

10. VIVA QUESTIONS

Q1. What is a primary clock?
Q2. How is a generated clock defined?
Q3. What does clock period represent?
Q4. Why do we define clock waveform?
Q5. What is clock latency?
Q6. What does clock uncertainty account for?
Q7. What are types of clock relationships?
Q8. What is clock source in SDC?

11. QUICK REVISION

- ✓ Primary clock → base reference clock
- Generated clock → derived from master clock
- Period → time between active edges
- ✓ Waveform → shape (rise/fall times)
- Latency → delay to reach sequential element
- Uncertainty → variation margin
- ✓ Relationship → defines clock interactions
- ✓ Source → origin (port or pin) of clock

12. KEY FORMULAS / NOTES

Period (Tclk) = Time between two consecutive active edges

Clock Skew = $T_{capture_clk} - T_{launch_clk}$

Required Time (setup) = Capture Edge − Setup Time − Uncertainty

Required Time (hold) = Capture Edge + Hold Time + Uncertainty

STATIC TIME ANALYSIS

TIMING EXCEPTIONS

1) FALSE PATH :

- A path that is functionally impossible or never sensitized.
- No timing requirement should be checked.
- Useful for paths crossing domains or logic that cannot propagate.

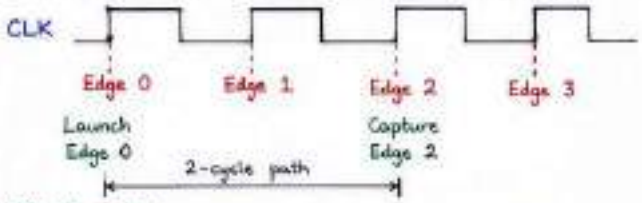
SDC Example : set_false_path -from A -to B



Example :
Reset signal driving data input of a FF.
Reset is never asserted in functional mode.
→ Mark path from reset source to FF D as false path.

2) MULTI-CYCLE PATH :

- Data requires more than one clock cycle to propagate.
- Relax timing by allowing data to arrive after multiple cycles.
- Setup and hold must be set accordingly.



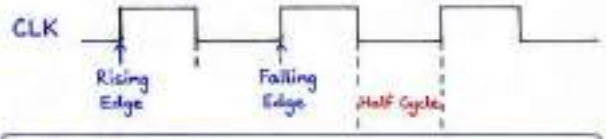
SDC Example :

Setup (N cycles)	set_multicycle_path N -setup -from A -to B
Hold (N-1 cycles)	set_multicycle_path (N-1) -hold -from A -to B

Example :
A path updates output once every 3 cycles.
Use → -setup 3
-hold 2

3) HALF-CYCLE BEHAVIOR :

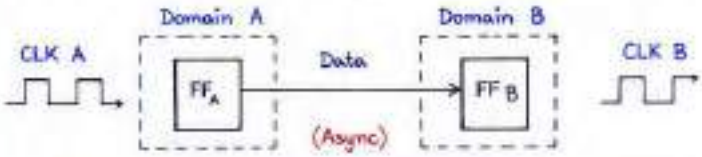
- Data must be captured within half of the clock period.
- Occurs in paths between opposite clock phases or dual-edge elements.



Example :
Data launched on rising edge and captured on falling edge of same clock = 0.5 T requirement.
→ Critical and needs careful optimization.

4) ASYNCHRONOUS PATH CONSIDERATIONS :

- Paths between clock domains (no common clock).
- No valid timing relationship exists.
- Either mark false or use synchronizers.



Example :
Data from slow clock domain to fast clock domain through combinational logic.
→ Use synchronizer or mark as false path.

5) PATH EXCLUSION :

- Exclude a specific timing path or endpoint from analysis.
- Similar to false path but more granular.
- Can exclude setup, hold or both.

SDC Examples :

Purpose	SDC Command
Exclude setup check	set_disable_timing -setup -from A -to B
Exclude hold check	set_disable_timing -hold -from A -to B
Exclude both	set_disable_timing -from A -to B

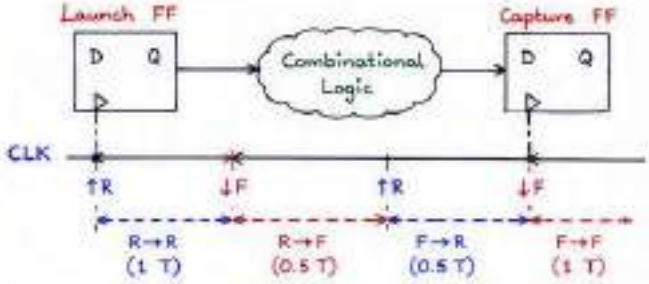
Example :
A test logic path not used in silicon mode.
→ Exclude setup and hold from reports.

6) LAUNCH / CAPTURE EDGE SELECTION :

- Choose appropriate launch and capture edges for timing analysis.
- Determines which clock edges form the timing relationship.
- Affects required time and slack.

Edge Selection Types :

Type	Description
Rising → Rising	Default full-cycle path
Rising → Falling	Half-cycle path (0.5 T)
Falling → Rising	Half-cycle path (0.5 T)
Falling → Falling	Full-cycle path



TOPIC - SPECIFIC VIVA QUESTIONS

Q1. What is a false path ? Give one example.
Q2. How do you declare a multi-cycle path for setup and hold ?
Q3. What is half-cycle behavior ? When does it occur ?
Q4. How do you handle asynchronous paths in STA ?
Q5. What is path exclusion ? How is it different from false path ?
Q6. How does launch/capture edge selection affect timing analysis ?
Q7. What is the effect of multi-cycle on hold requirement ?
Q8. When should you use -hold (N-1) with multi-cycle path ?
Q9. Can a path be both false path and multi-cycle ? Explain.
Q10. Why are timing exceptions necessary ?

QUICK REVISION

* False Path → No timing check (functionally impossible).
* Multi-Cycle Path → Allow data N cycles; use -setup N and -hold (N-1).
* Half-Cycle → Data captured within 0.5 clock period.
* Asynchronous Path → No timing relation; use synchronizer or mark false.
* Path Exclusion → Disable setup/hold/both checks on specific path.
* Edge Selection → Choose correct launch & capture edges (R-R, R-F, F-R, F-F).

IN —▷— OUT

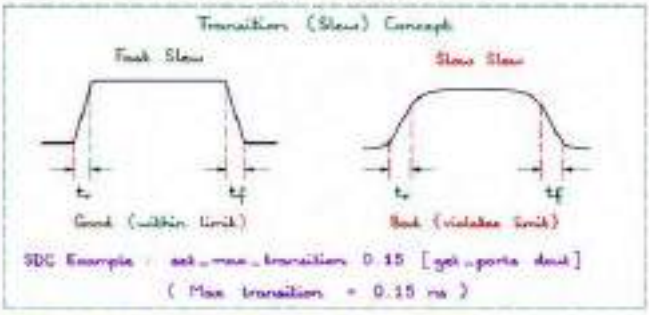
STATIC TIMING ANALYSIS (STA)

CLK

DESIGN RULE TIMING CONSTRAINTS

1. MAXIMUM TRANSITION (SLEW LIMIT)

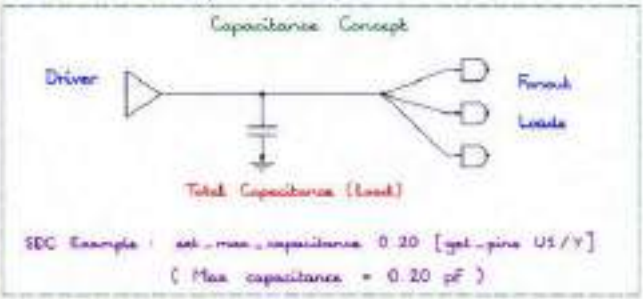
- Limits the output transition time (slew) of a pin.
- Slow transitions increase dynamic power and reduce noise margin.
- Ensures reliable operation of downstream cells.



SDC Example : set_max_transition 0.15 [get_ports dout]
(Max transition = 0.15 ns)

2. MAXIMUM CAPACITANCE (LOAD LIMIT)

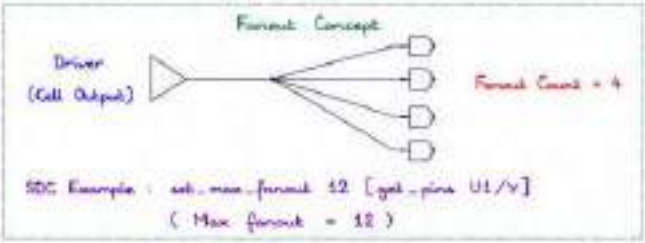
- Limits the capacitive load seen at a pin.
- High load increases delay, power and noise.
- Prevents overloading the driving cell.



SDC Example : set_max_capacitance 0.20 [get_pins U1/Y]
(Max capacitance = 0.20 pF)

3. MAXIMUM FANOUT (DRIVING LIMIT)

- Limits number of loads driven by a pin.
- High fanout causes high delay and transition degradation.
- Helps to maintain balanced and robust design.



SDC Example : set_max_fanout 12 [get_pins U1/Y]
(Max fanout = 12)

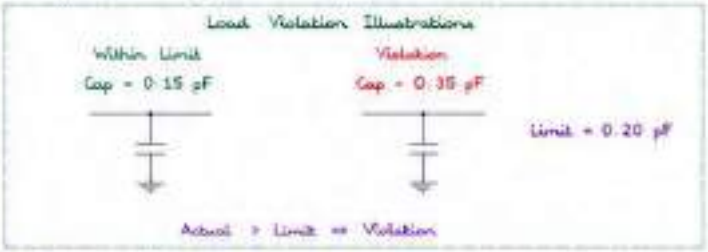
4. SLEW VIOLATIONS (TRANSITION VIOLATIONS)

- Occurs when actual slew > allowed maximum transition.
- Causes extra dynamic power and signal integrity issues.
- May lead to functional failures in worst cases.



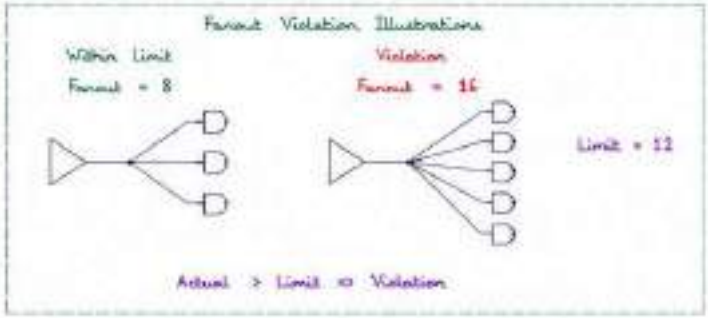
5. LOAD VIOLATIONS (CAPACITANCE VIOLATIONS)

- Occurs when actual capacitance > allowed maximum.
- Increases delay and power consumption.
- May cause timing and reliability problems.



6. FANOUT VIOLATIONS

- Occurs when number of loads > allowed maximum fanout.
- Leads to increased delay, poor transition and reliability issues.
- Can cause functional failures under variation.



7. THEIR EFFECT ON TIMING

Constraint	If Violated	Effect on Timing	Other Effects
Max Transition	Slew > Limit	Increases delay	More dynamic power, noise, unreliable
Max Capacitance	Cap > Limit	Increases delay	More power, IR drop, reliability risk
Max Fanout	Fanout > Limit	Increases delay	Poor slew, congestion, unreliable operation
Slew Violations	Slow edges	Higher delay in downstream paths	High power, noise, setup/hold risk
Load Violations	High cap load	Higher cell delay, lower slack	Power increase, voltage droop
Fanout Violations	Too many loads	Higher delay, negative slack	Slew degradation, functionality risk

8. FIXING CONSTRAINT VIOLATIONS (GENERAL APPROACH)

Violation Type	Fixing Techniques	Typical SDC / Actions
Max Transition	• Insert buffer • Use stronger driver • Reduce load • Shorter wires	set_max_transition <value> Buffer insertion in ECO Use higher drive cells
Max Capacitance	• Reduce load • Insert buffer • Split net • Use higher metal layer	set_max_capacitance <value> Buffering / Net restructuring Route on higher layer
Max Fanout	• Insert buffer • Tree buffering • Increase driving strength	set_max_fanout <value> Buffer tree insertion
Slew Violations	• Increase drive • Buffer insertion • Reduce wire RC	Fix using transition methods set_max_transition <value>
Load Violations	• Reduce load • Buffer / Split net • Change placement	Fix using capacitance methods set_max_capacitance <value>
Fanout Violations	• Buffer insertion • Balance load	Fix using fanout methods set_max_fanout <value>

9. PRACTICAL OBSERVATIONS

- All DRC-like timing constraints must be satisfied for robust silicon.
- Violations increase power, delay and risk of functional failure.
- Fix violations early in the flow (pre-CTS / pre-route).
- Always review reports: slew, cap, fanout, transition.
- Proper constraints improve QoR.

10. VIVA QUESTIONS

Q1. What is maximum transition constraint?
Q2. What happens if slew limit is violated?
Q3. What is maximum capacitance constraint?
Q4. How do you fix capacitance violations?
Q5. What is maximum fanout constraint?
Q6. What are slew violations?
Q7. What are load violations?
Q8. What are fanout violations?
Q9. How do you fix fanout violations?
Q10. What is the effect of constraint violations?

11. QUICK REVISION

✓ Transition limits output slew.
✓ Capacitance limits pin load.
✓ Fanout limits number of loads.
✓ Slew, load and fanout violations are DRC-like timing errors.
✓ Violations increase delay, power and risk.
✓ Use buffering, stronger cells, net splitting and routing improvements to fix.

12. KEY FORMULAS / NOTES

- Slew (Transition) = time for fall or rise between 10%–90%
- Load (Capacitance) = total C at pin
- Fanout = No. of loads driven

Effect on Delay (Qualitative)

Delay ∝ (Cload + Rdriver)
& Delay ∝ Fanout

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

STA REPORT ANALYSIS - TIMING REPORT READING

1. TIMING REPORT READING

- STA report lists the timing of paths analyzed in the design
- Helps to identify worst (critical) paths and violations
- Each path entry shows launch details, data path delays, capture details and timing summary
- Report is analyzed for slack, arrival, required time and path contributions

Typical Timing Report Summary (Example)

Path Group : REG2REG Path Type : Setup (max)

WNS = -0.020 ns TNS = -0.060 ns WHS = 0.045 ns THS = 0.000 ns

Point	Incr Delay (ns)	Path Delay (ns)
Startpoint (FF1/Q)	-	0.000
U1/IN	0.030	0.030
U1/Z (NAND2)	0.080	0.110
U2/Z (INV)	0.060	0.170
Net (wire)	0.045	0.215
Endpoint (FF2/D)	-	-

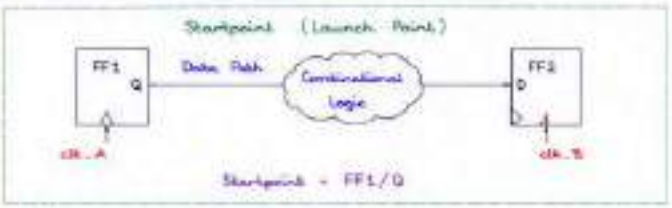
Data Arrival Time (A) = 0.215 ns

Data Required Time (R) = 0.195 ns

Setup Slack = R - A = -0.020 ns → VIOLATION

2. STARTPOINT

- The launch point of the path from where data is launched
- Usually a flip-flop (Q pin) or a primary input port
- Determines the launch clock domain



3. ENDPOINT

- The capture point where data is sampled
- Usually a flip-flop (D pin) or a primary output port
- Determines the capture clock domain



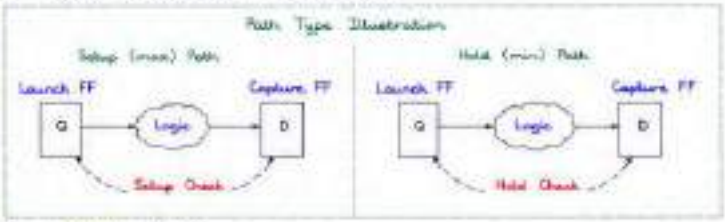
4. PATH GROUP

- Collection of timing paths grouped based on clock relationship
- Used to apply constraints and analyze paths together
- Examples : REG2REG, REG2OUT, IN2REG, IN2OUT

Path Group	Description
REG2REG	From flip-flop to flip-flop (main or min)
REG2OUT	From flip-flop to primary output
IN2REG	From primary input to flip-flop
IN2OUT	From primary input to primary output

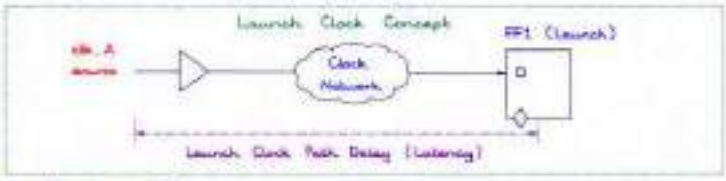
5. PATH TYPE

- Indicates the type of timing check the path is analyzed for
- Affects how required time is calculated
- Types : Setup (max), Hold (min)



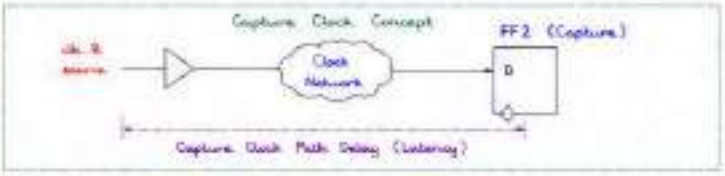
6. LAUNCH CLOCK

- Clock that launches data from the startpoint
- Used as reference for calculating data arrival time
- May include clock latency and uncertainty



7. CAPTURE CLOCK

- Clock that captures data at the endpoint
- Used as reference for calculating required time
- Arrival of capture clock determines timing window



8. CLOCK LATENCY

- Delay between clock source and the sequential element clock pin
- Includes clock network delay and clock insertion delay
- Affects both arrival (launch) and required (capture) times



SDC Example

```
set_clock_latency -source 0.25 [get_clocks clk]
set_clock_latency -late   0.35 [get_clocks clk]
# Early / Late latency
```

9. CLOCK UNCERTAINTY

- Uncertainty in clock arrival time due to jitter, variation, OCV, etc.
- Reduces available timing margin to avoid false violations



SDC Example :

```
set_clock_uncertainty 0.10
[get_clocks clk]
```

(Total uncertainty = 0.20 ns)

10. DATA ARRIVAL TIME (A)

- Time when data actually arrives at the endpoint (D pin)
- Calculated from startpoint through combinational path
- Includes cell delays + net delays

11. DATA REQUIRED TIME (R)

- Time by which data must arrive at the endpoint to meet timing
- Calculated from capture clock edge
- For hold check, it is based on same/previous edge

Timing Relationship

A < R → Positive Slack (OK)

A = R → Zero Slack (Critical)

A > R → Negative Slack (Violation)

Key Formulas (Setup)

Setup Slack = R - A

Where,

R = Capture Edge Time - Setup Time

A = Data Arrival Time

12. PRACTICAL OBSERVATIONS

- Always check WNS/TNS first. Negative WNS indicates violation
- Slack negative paths are priority
- Check data arrival vs required time to understand root cause
- Look at clock latency & uncertainty impact on margin
- Identify path group and path type before debugging

13. VIVA QUESTIONS

Q1: What is a startpoint in STA report ?

Q2: What is an endpoint in STA report ?

Q3: What are path groups ? Give examples

Q4: What are path types in STA ?

Q5: What is launch clock ?

Q6: What is capture clock ?

Q7: What is clock latency ?

Q8: What is clock uncertainty ?

Q9: What is data arrival time ?

Q10: What is data required time ?

14. QUICK REVISION

- ✓ Startpoint = Launch point (FF Q or PI)
- ✓ Endpoint = Capture point (FF D or PO)
- ✓ Path Groups: REG2REG, REG2OUT, IN2REG, IN2OUT
- ✓ Path Types: Setup (max), Hold (min)
- ✓ Launch clock → reference for arrival
- ✓ Capture clock → reference for required
- ✓ Latency → clock source to sequential pin
- ✓ Uncertainty → variation margin
- ✓ A < R → Good, A > R → Violation

15. KEY FORMULAS / NOTES

- Setup Slack : R - A
  - R = Capture Edge - Setup Time
  - A = Data Arrival Time
- Hold Slack : A - R_hold
  - R_hold = Capture Edge - Hold Time
- Clock Latency = Source Latency + Network Latency
- Uncertainty reduces available margin

STATIC TIME ANALYSIS

STA REPORT ANALYSIS

1) SLACK :

- Slack is the time difference between the required time and the data arrival time.
- It indicates how much timing margin exists.
- Positive slack ⇒ timing is met.
- Negative slack ⇒ timing violation.

> Setup Slack = Required Time − Arrival Time
> Hold Slack = Arrival Time − Required Time

Example :

Clock Period = 2.0 ns, Data Arrival = 1.7 ns
Required Time = 2.0 ns
Setup Slack = 2.0 − 1.7 = +0.3 ns ✓
Hold Slack = 1.7 − 0.0 = +1.7 ns ✓

2) CELL DELAY :

- Delay introduced by a cell from input pin to output pin.
- Includes intrinsic delay + internal wire delay.
- Reported as (cell_delay) in timing reports.
- Depends on cell type, input slew, output load.

Example Report Line :

U12/INVX1 (cell_delay) 0.085 r (→)

3) NET DELAY :

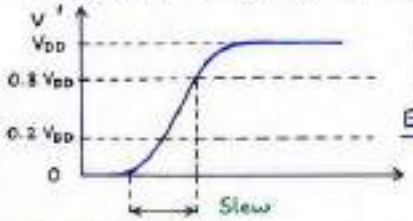
- Delay due to interconnect (wires, vias, RC).
- Depends on wire length, resistance & capacitance.
- Reported as (net_delay) in timing report.

Example Report Line :

net_45 (net_delay) 0.172 f (→)

4) SLEW :

- Slew is the signal transition time (rise or fall time) at a pin.
- Measured between 20% to 80% of V_{DD}.
- Affects cell delay and power.
- Reported as (slew) in timing report.



> Slew = $t_{80\%} - t_{20\%}$

Example : Slew = 45 ps

5) CAPACITANCE :

- Capacitance is the total load seen by the driving pin.
- Includes pin capacitance + net capacitance.
- Higher capacitance ⇒ higher delay.
- Reported as (capacitance) in timing report.

Example Report Line :

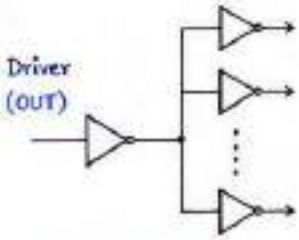
net_45 (capacitance) 0.056 pF

6) FANOUT :

- Number of inputs driven by an output pin.
- Higher fanout ⇒ higher capacitance ⇒ higher delay.
- Fanout = Total Load Capacitance / Input Capacitance

Effect of Fanout

Fanout	Effect on Delay
Low	Lower delay
Medium	Moderate delay
High	Higher delay

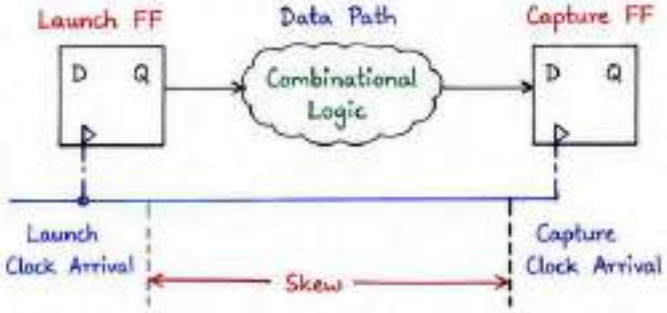


Fanout = 4

7) CLOCK SKEW :

- Clock skew is the difference between capture clock arrival and launch clock arrival.
- Skew = Capture Clock Arrival − Launch Clock Arrival

> Clock Skew = T_{clk}(capture) − T_{clk}(launch)



Positive Skew : Capture clock arrives later (helps setup)
Negative Skew : Capture clock arrives earlier (hurts setup)

8) UNCERTAINTY :

- Margin added to account for variations and unknowns in the system.
- Includes variation in clock, delay, environment.
- Reduces available timing margin.

> Total Uncertainty = Clock Uncertainty + Delay Uncertainty

9) CRITICAL PATH :

- The path with the worst (most negative) slack.
- Determines the maximum operating frequency.
- Any improvement on critical path improves overall timing.

> Negative Slack on critical path ⇒ Violation
> Zero Slack on critical path ⇒ Just meets timing
> Positive Slack on critical path ⇒ Timing has margin

TOPIC – SPECIFIC VIVA QUESTIONS

Q1. What is slack and how is it calculated ?
Q2. What is the difference between setup slack and hold slack ?
Q3. What is cell delay and what factors affect it ?
Q4. What is net delay and what affects it ?
Q5. What is slew and how is it measured ?
Q6. What is capacitance in STA ?
Q7. What is fanout and how does it impact timing ?
Q8. How is clock skew calculated ?
Q9. What is uncertainty and why is it added in STA ?
Q10. What is a critical path ?

QUICK REVISION

* Slack = Required Time − Arrival Time (Setup)
* Cell Delay = Delay inside the cell
* Net Delay = Delay due to interconnect
* Slew = Signal transition time (20% to 80%)
* Capacitance = Total load at a pin
* Fanout = Number of loads driven
* Clock Skew = Capture Clock − Launch Clock
* Uncertainty = Margin for variations
* Critical Path = Path with worst slack

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

REPORT_TIMING & STA DEBUGGING

1. TIMING-PATH REPORTING

- report_timing command is used to report timing paths.
- Shows detailed path information, delays, and slack values.
- Helps in identifying critical paths and violations.
- Can be customized using options (from, to, group, max_paths, etc.)

Basic Command

```
report_timing -delay_type max|min \
              -nworst <N> -max_paths <N> \
              -path_type full_clock_expanded
```

Typical Usage

- Worst setup path → report_timing -delay_type max
- Worst hold path → report_timing -delay_type min
- Specific path group → report_timing -group <group_name>
- From specific pin → report_timing -from <pin/cell>
- To specific pin → report_timing -to <pin/cell>

2. WORST SETUP PATH

- Path with most negative setup slack.
- Determines maximum delay and sets the performance limit.
- Analyzed with -delay_type max.



Setup Slack = Required Time - Arrival Time

Negative slack → Setup Violation

3. WORST HOLD PATH

- Path with most negative hold slack.
- Short paths (minimum delay) are critical.
- Analyzed with -delay_type min.



Hold Slack = Data Arrival Time - Required Min Time

Negative slack → Hold Violation

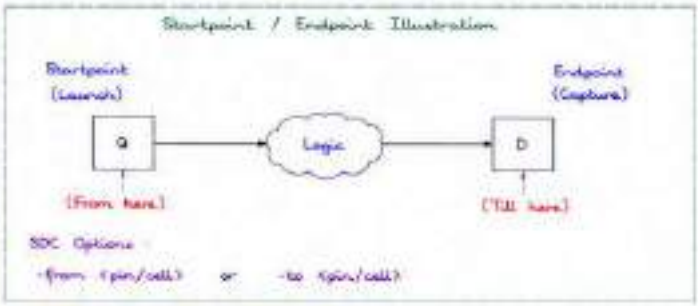
4. PATH GROUPS

- Collection of timing paths grouped based on clock relationships.
- Used to apply constraints and analyze path categories.
- Defined in SDC using set_path_group

Path Group	Description	Typical Use
REG2REG	Flip-flop to Flip-flop (same clock domain)	Core synchronous paths
REG2OUT	Flip-flop to primary output	Output timing checks
IN2REG	Primary input to Flip-flop	Input timing checks
IN2OUT	Primary input to primary output	Combinational paths
ASYNC	Asynchronous paths	CDC / Exceptions

5. STARTPOINT / ENDPOINT SELECTION

- Startpoint → where data is launched (Q pin or input port)
- Endpoint → where data is captured (D pin or output port)
- Selection helps focus on specific logic and paths



6. TIMING REPORT INTERPRETATION

- Each timing report entry shows delay components and slack info.
- Key fields in a timing path report :

Field	Meaning
Point	Cell/Pin/Net name in the path
Incr Delay	Incremental delay from previous point
Path Delay	Cumulative delay from startpoint
Arrival	Data arrival time at this point
Required	Required time for data at this point
Slack	Required - Arrival (for setup) or Arrival - Required(min) (for hold)
Edge	Clock edge used for calculation

7. VIOLATION IDENTIFICATION

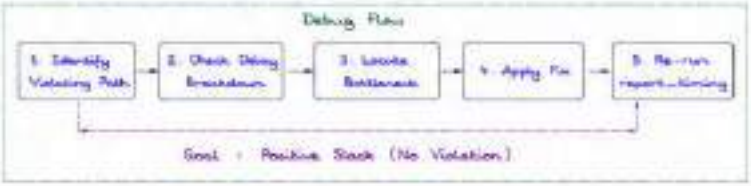
- Violation exists when slack is negative.
- Setup violation → data arrives late.
- Hold violation → data arrives too early.
- Identify violation type, path, and amount of slack.

Slack Interpretation

Slack Value	Interpretation
> 0	Positive slack → No violation
= 0	Zero slack → Critical path
< 0	Negative slack → Timing violation

8. DEBUGGING FROM REPORT

- Use timing report to locate cause of violation.
- Analyze path delay breakdown and fix root cause.
- Iterate until slack is positive.



9. KEY TIMING INFORMATIONS SUMMARY

Term	Definition	Used For	Equation / Note
Launch Clock	Clock associated with startpoint	Calculates data arrival	Includes clock latency and uncertainty
Capture Clock	Clock associated with endpoint	Calculates required time	Includes clock latency and uncertainty
Clock Latency	Delay from clock source to sequential pin	Affects arrival & required times	Source latency + Network latency
Data Arrival Time (A)	Time when data arrives at endpoint	Setup & Hold analysis	A = Launch Edge + Launch Latency + Path Delay
Data Required Time (R)	Time by which data must arrive	Setup & Hold analysis	R = Capture Edge - Capture Latency - Setup Time (for setup)
Slack (Setup)	Extra time available (or shortage)	Setup check	Slack = R - A
Slack (Hold)	Extra time available (or shortage)	Hold check	Slack = A - R_hold (min)

10. PRACTICAL OBSERVATIONS

- Always check from worst paths (most negative slack).
- Small negative slack can grow after ECO or route changes.
- Check both setup and hold violations.
- Path groups help focus analysis on specific categories.
- Use path details to find real bottleneck (cell, net, cap, fanout).

11. VIVA QUESTIONS

Q1. What is report_timing command?
Q2. How do you find worst setup path?
Q3. How do you find worst hold path?
Q4. What are path groups? Give examples.
Q5. What is startpoint and endpoint?
Q6. How do you interpret timing report?
Q7. How do you identify violations?
Q8. What is clock latency?
Q9. What is data arrival time?
Q10. How do you debug timing violations?

12. QUICK REVISION

- ✓ report_timing → shows path details
- ✓ Worst setup → most negative setup slack
- ✓ Worst hold → most negative hold slack
- ✓ Path groups → REG2REG, REG2OUT, IN2REG, IN2OUT, ASYNC
- ✓ Startpoint → launch point
- ✓ Endpoint → capture point
- ✓ Negative slack → violation
- ✓ Use report to debug and fix root cause

13. KEY FORMULAS / NOTES

- Setup Slack :
  Slack_setup = Data Required Time (R) - Data Arrival Time (A)
- Hold Slack :
  Slack_hold = Data Arrival Time (A) - Required Min Time (R_hold)
- Data Arrival Time :
  A = Launch Edge + Lat_launch + Path Delay
- Data Required Time (Setup) :
  R = Capture Edge + Lat_capture - Setup Time

Author : Kanda Tharun Vivekananda

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

TIMING CLOSURE - TIMING CLOSURE FUNDAMENTALS

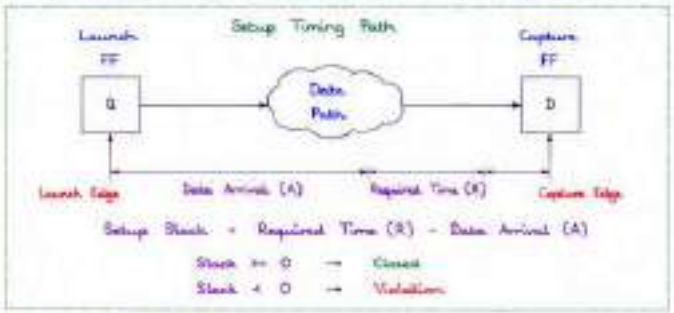
1. WHAT IS TIMING CLOSURE?

- Timing closure is the process of ensuring that all timing requirements are met across the entire design.
- No timing violations (setup, hold, constraints, etc.)
- Achieved under all operating conditions and scenarios.
- Goal → Zero violations with sufficient timing margin.



2. SETUP CLOSURE

- Ensure data arrives at the capture element before the required time.
- Setup slack must be >= 0 for all paths.
- Improve by reducing data path delay or increasing required time.



3. HOLD CLOSURE

- Ensure data does not arrive too early at the capture element.
- Hold slack must be >= 0.
- Improve by increasing data path delay or reducing early arrival.



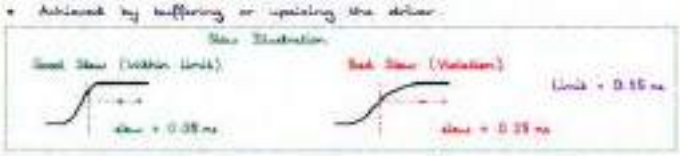
4. CONSTRAINT CLOSURE

- All design intent constraints must be properly defined and met.
- Includes clocks, I/O delays, exceptions, uncertainties.
- No unconstrained or conflicting constraints.

Item	Goal	Unclosed Issue
Clocks	All clocks defined	Unspecified clock
I/O Delays	All I/O have delays	Missing I/O delay
Exceptions	Valid and complete	Over/Under constrained
Skew	Proper margins added	Too less / Too much

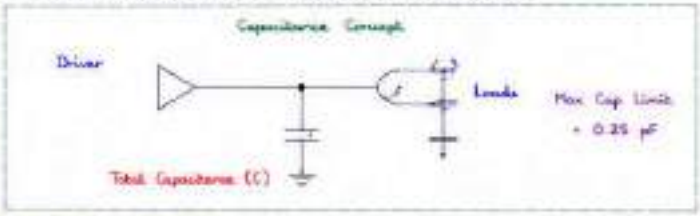
5. SLEW CLOSURE

- Ensure signal transition (slew) is within the allowed limit.
- High slew increases delay, power and noise.
- Achieved by buffering or upsizing the driver.



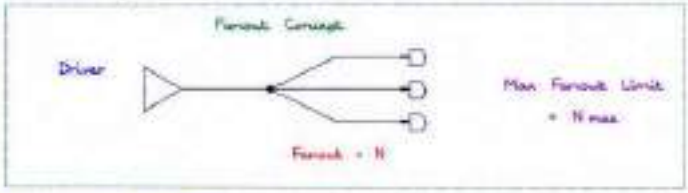
6. CAPACITANCE CLOSURE

- Ensure total capacitance seen at a pin is within the allowed limit.
- High capacitance increases delay and power.
- Improve by reducing load, buffering or upsizing driver.



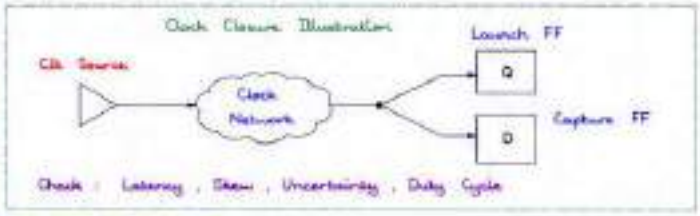
7. FANOUT CLOSURE

- Ensure number of loads driven by a cell is within limit.
- High fanout increases delay, slew and power.
- Improve by buffering, using higher drive cell or reducing loads.



8. CLOCK CLOSURE

- Ensure clock paths meet all timing requirements.
- Includes clock latency, skew, uncertainty and duty cycle.
- Aim for balanced clock tree and minimal skew.



9. MCMM CLOSURE

- Timing must be closed across all modes, corners, and scenarios.
- Includes PVT, voltage, mode, and workload variations.
- No violation in any scenario.

MCMM Dimensions	
Dimension	Examples
Mode	Functional mode, Scan mode, Low power mode
Corner (PVT)	SS / 0.72V / 125°C, TT / 0.80V / 25°C, FF / 0.88V / -40°C
Voltage	0.72V, 0.80V, 0.88V, 0.95V
Workload	Best case, Typical, Worst case

10. SIGNOFF CLOSURE

- Final timing signoff before tape-out.
- All checks (setup, hold, constraints, slew, cap, fanout, clock, MCMM) must be clean.
- Reports are reviewed and archived for signoff.

Signoff Checklist

- ✓ Setup slack >= 0 (all paths)
- ✓ Hold slack >= 0 (all paths)
- ✓ All constraints satisfied
- ✓ Slew within limit
- ✓ Capacitance within limit
- ✓ Fanout within limit
- ✓ Clock skew and latency OK
- ✓ MCMM scenarios clean
- ✓ Signoff reports archived

11. EFFECT ON TIMING (IF NOT CLOSED)

Closure Type	If Not Closed	Impact
Setup	Negative setup slack	Functional failure (data miss)
Hold	Negative hold slack	Data changes early
Constraint	Missing/incorrect constraints	Wrong analysis / false closure
Slew	High slew	More delay, noise, power
Capacitance	High cap	More delay, power
Fanout	High fanout	More delay, high slew
Clock	High skew/latency	Setup/Hold degraded
MCMM	Fails in some scenario	Failure in field condition
Signoff	Not all checks clean	Risk at tape-out

12. VIVA QUESTIONS

01. What is timing closure?
02. What is setup closure?
03. What is hold closure?
04. What is constraint closure?
05. What is slew closure?
06. What is capacitance closure?
07. What is fanout closure?
08. What is clock closure?
09. What is MCMM closure?
010. What is signoff closure?
011. What happens if slew is not closed?
012. Why MCMM closure is important?

13. QUICK REVISION

- ✓ Timing closure → Zero violations everywhere.
- ✓ Setup → Data must arrive before required time.
- ✓ Hold → Data must not arrive too early.
- ✓ Constraint → All constraints correct & complete.
- ✓ Slew → Transition within limit.
- ✓ Capacitance → Load within limit.
- ✓ Fanout → Number of loads within limit.
- ✓ Clock → Latency, skew, uncertainty closed.
- ✓ MCMM → All modes/corners closed.
- ✓ Signoff → Final clean timing data.

14. KEY FORMULAS / NOTES

- Setup Slack = Required Time - Arrival Time
- Hold Slack = Earliest Arrival - Hold Time
- Slew (Transition) = t_r / t_f (10% - 90%)
- Capacitance = Total C at pin
- Fanout = Number of loads

Goal of Timing Closure → Zero Violations with Sufficient Margin

STATIC TIME ANALYSIS

73. SETUP vs HOLD OPTIMIZATION STRATEGY

a) SETUP (Make data faster / give data more time)

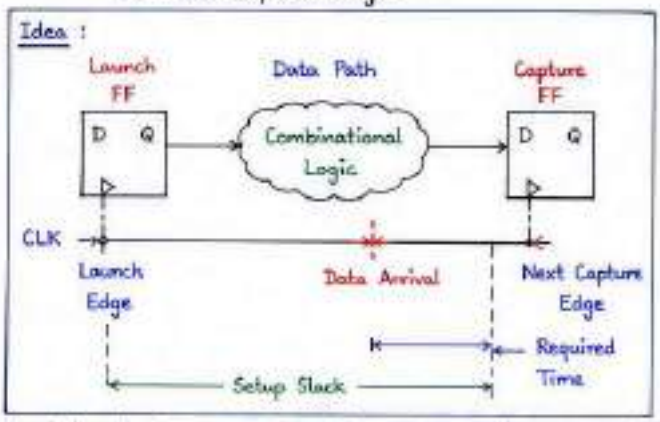
Goal : Meet setup requirement by

- Increasing data arrival speed

OR

- Giving data more time to arrive before the next capture edge.

Idea :



Strategies to Improve Setup (Data faster / More time)

1. **Stronger cells**
   - Use high drive cells to reduce cell delay.
2. **Faster cells**
   - Choose faster libraries / higher drive variants.
3. **Buffer / repeater insertion**
   - Insert buffers / repeaters on long nets to reduce wire delay and load.
4. **Higher metal layers**
   - Route on higher metal to reduce RC delay.
5. **Placement optimization**
   - Place cells closer to reduce wire length and congestion.
6. **Useful skew (positive skew)**
   - Intentionally delay launch clock or advance capture clock so that data gets more time.

Example – Setup Improvement

Action	Before	After
Data Path Delay	1.30 ns	0.95 ns
Required Time	2.00 ns	2.00 ns
Setup Slack	+0.70 ns	+1.05 ns ✓

Practical Observation (Setup)

- Optimize setup without creating hold violation.
- Useful skew must be within clock tree limits.

b) HOLD (Make data slower / reduce available early-arrival window)

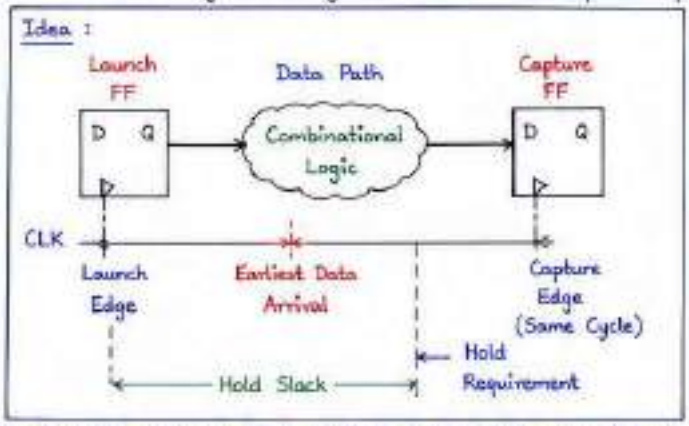
Goal : Meet hold requirement by

- Slowing down the data path

OR

- Reducing the early arrival window at capture flop.

Idea :



Strategies to Improve Hold (Data slower / Reduce window)

1. **Delay buffers**
   - Insert buffers in data path to add delay.
2. **Weaker cells**
   - Use low drive cells to increase cell delay.
3. **Longer routing**
   - Route with longer paths or more vias to increase RC delay.
4. **Lower metal layers**
   - Route on lower metal layers (higher R, C) to add delay.
5. **Clock adjustment (negative skew)**
   - Intentionally advance launch clock or delay capture clock to reduce early window.

Example – Hold Improvement

Action	Before	After
Earliest Data Arrival	0.15 ns	0.32 ns
Hold Requirement	0.20 ns	0.20 ns
Hold Slack	−0.05 ns ✗	+0.12 ns ✓

Practical Observation (Hold)

- Do not over-buffer; it can hurt setup timing.
- Balance hold fix with minimum area / power.

SETUP vs HOLD OPTIMIZATION – SUMMARY

Aspect	Setup Optimization	Hold Optimization
Objective	Make data faster / Give more time	Make data slower / Reduce early window
Effect on Data Path	Decrease delay	Increase delay
Typical Techniques	Stronger cells, Faster cells, Buffer/repeater, Higher metal, Placement optimization, Useful skew	Delay buffers, Weaker cells, Longer routing, Lower metal, Clock adjustment
Slack Effect	Increases setup slack	Increases hold slack
Risk	May create hold violation	May reduce setup slack
Skew Used	Positive (beneficial for setup)	Negative (beneficial for hold)

TOPIC – SPECIFIC VIVA QUESTIONS

Q1. What is the goal of setup optimization ?

Q2. List the common techniques used to improve setup timing.

Q3. What is the effect of buffer insertion on setup timing ?

Q4. What is useful skew and how does it help setup timing ?

Q5. What is the goal of hold optimization ?

Q6. List the common techniques used to improve hold timing.

Q7. How does negative skew help hold timing ?

Q8. Why does buffer insertion fix hold but hurts setup ?

Q9. How do you balance setup and hold during optimization ?

Q10. Give one practical observation for setup and one for hold optimization.

QUICK REVISION

- Setup ⇒ Data faster / More time
  - Stronger cells, Faster cells, Buffer, Higher metal, Placement, Positive skew.
- Hold ⇒ Data slower / Reduce early window
  - Delay buffers, Weaker cells, Longer routing, Lower metal, Negative skew.
- Balance both to close timing without swapping violations.

IN → OUT

STATIC TIMING ANALYSIS (STA)

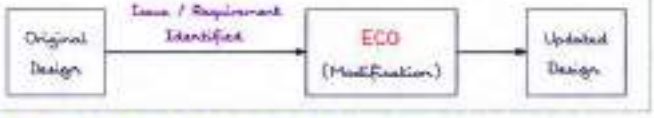
CLK

ECO (ENGINEERING CHANGE ORDER)

1. WHAT IS ECO?

- ECO is a controlled modification to the design after one or more design stages (synthesis / place & route / signoff) or even after tape-out.
- Made to fix timing, functional, power, metal or silicon issues.
- ECO must maintain correctness and design intent.

ECO Concept



2. WHY ECO IS REQUIRED?

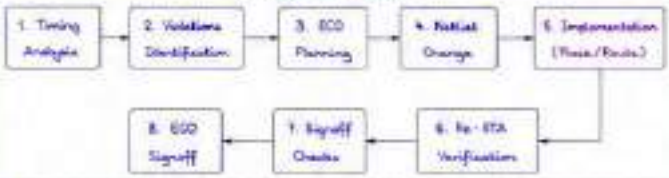
- Timing violations (setup/hold) after signoff.
- Functional bugs or incorrect logic.
- Power / leakage too high.
- Metal / routing DRC or antenna issues.
- Post-silicon failures or parametric escapes.
- Boundary scan, test, or connectivity changes.

3. TYPES OF ECO

ECO Type	Purpose
Timing ECO	Fix setup/hold violations and timing issues.
Functional ECO	Correct functional bugs / logic errors.
Power ECO	Reduce dynamic power, leakage or toggling.
Metal ECO	Fix DRC, antenna, short / open, wire edits.
Post-silicon ECO	Changes after fabrication using fuses, OTP, or re-programmable elements.

4. TIMING ECO FLOW

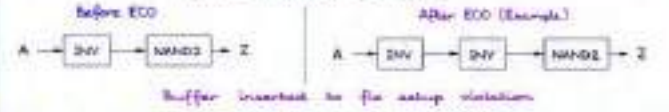
Timing ECO Flow



5. NETLIST MODIFICATION

- Modify the gate-level netlist to fix the issue.
- Add / remove / replace cells or change connections.
- Maintain functional equivalence. (except functional ECO)
- Ensure design rules and timing constraints are followed.

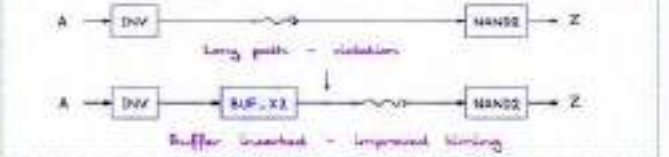
Netlist Modification Example



6. BUFFER INSERTION

- Add buffer(s) in data path to improve timing.
- Reduces delay by splitting long path and driving strength.
- Improves slew and reduces load seen by previous cell.

Buffer Insertion Illustration



7. CELL REPLACEMENT

- Replace a cell with a faster / stronger / lower power cell.
- Useful when buffering alone is not sufficient.
- Example: Replace NAND2_X1 with NAND2_X2 (faster)

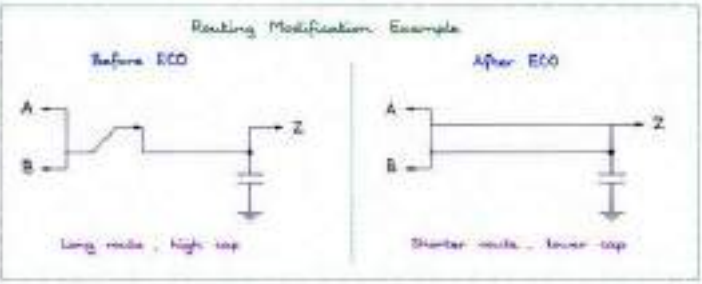
Cell Replacement Example



8. ROUTING MODIFICATION

- Modify routing to reduce capacitance / resistance.
- Change wire layer, use wider wire or straighter route.
- Fix DRC, antenna or crosstalk issues.

Routing Modification Example



9. VERIFICATION AFTER ECO

- Run STA to ensure violations are fixed without new issues.
- Check setup, hold, constraints, DRC, LVS.
- Verify functionality (formal / simulation).
- Ensure power and other signoff metrics are within targets.

ECO Verification Checklist

- ✓ STA (Setup / Hold)
- ✓ DRC (Design Rule Check)
- ✓ LVS (Layout vs Schematic)
- ✓ Functional Verification
- ✓ Power Check
- ✓ Antenna / IR / EM (if needed)
- ✓ Signoff Reports Updated

10. SUMMARY OF CLOSURE OBJECTIVES

Closure Type	Objective
Setup Closure	All setup slacks ≥ 0 (with margin)
Hold Closure	All hold slacks ≥ 0
Constraint Closure	All paths have correct constraints
Slew Closure	All pin slews within limit
Capacitance Closure	All net capacitance within limit
Fanout Closure	All fanouts within limit
Clock Closure	Clock latency, skew, uncertainty OK
MCMM Closure	No violations in any scenario / mode
Signoff Closure	All signoff checks clean and reports archived

11. PRACTICAL OBSERVATIONS

- Small ECOs done early are easy and safe.
- Large ECOs late in flow risk DRC and schedule.
- Always analyze impact of ECO on power, area and reliability.
- ECO should not create new violations or DRC.
- Good naming and documentation of ECO changes is essential.

12. VIVA QUESTIONS

Q1. What is ECO?
Q2. Why ECO is required?
Q3. What are the types of ECO?
Q4. What is timing ECO?
Q5. How does buffer insertion help?
Q6. When do we do cell replacement?
Q7. What is metal ECO?
Q8. What is post-silicon ECO?
Q9. What checks are done after ECO?
Q10. What is the goal of ECO flow?

13. QUICK REVISION

- ✓ ECO = Engineering Change Order
- ✓ Types: Timing, Functional, Power, Metal, Post-silicon
- ✓ Timing ECO flow: Analysis → Identify → Plan → Netlist change → Implement → Re-STA → Signoff
- ✓ Techniques: Netlist mod, Buffer insert, Cell replace, Routing mod
- ✓ Verify: STA, DRC, LVS, Function, Power
- ✓ Goal: Zero violations with all signoff clean

14. KEY FORMULAS / NOTES

Setup Slack = Required Time − Arrival Time

Hold Slack = Arrival Time − Required Min Time

Total Capacitance (C) ≤ Cap Limit

Fanout = Number of Loads ≤ Fanout Limit

Slew ≤ Slew Limit

After ECO → Re-run STA and signoff

IN → OUT

STATIC TIMING ANALYSIS (STA)

CLK

SIGNOFF STA - FINAL TIMING VERIFICATION

1. WHY SIGNOFF IS REQUIRED?

- To ensure the design meets all timing requirements before tape-out.
- To guarantee correct operation across all PVT conditions, corners, modes and scenarios.
- To avoid timing failures in silicon and improve reliability.
- To signoff with sufficient margin considering on-chip variations.

> **Signoff Objective**
>
> Zero timing violations in all signoff scenarios with required margins and applied derates.

2. FINAL TIMING VERIFICATION

- Done after post-route with signoff parasitics.
- All constraints, exceptions and derates are applied.
- Reports are analysed for setup, hold, skew, cap, fanout, etc.
- All paths must have slack ≥ 0 (with margin).

> **Final Checks**
>
> ✓ No setup / hold violations
> ✓ All DRC / LVS clean
> ✓ All timing constraints met
> ✓ All signoff scenarios clean
> ✓ power / IR / EM within limit
> ✓ Signoff reports reviewed

3. POST-ROUTE PARASITICS

- Parasitics are extracted from routed layout.
- Include accurate wire RC, coupling caps and vias.
- More accurate delays compared to pre-route estimation.

Types of Parasitics

a) Lumped Parasitics : single RC per net
b) Coupled Parasitics : RC network with coupling caps



SPEF / DSPF / RSPF files are used in signoff STA.

4. MCMM ANALYSIS

- Multi-Corner Multi-Mode analysis verifies timing across different PVT, modes and workloads.
- Ensures robust operation under all possible conditions.

Dimension	Examples
Mode	Functional, Scan, Low-power, Test
Corner (PVT)	SS/0.72V/125°C, TT/0.80V/25°C, FF/0.88V/-40°C
Voltage	0.72V, 0.80V, 0.88V, 0.95V
Temperature	-40°C, 25°C, 125°C
Workload	Typical, Best-case, Worst-case

5. SETUP SIGNOFF

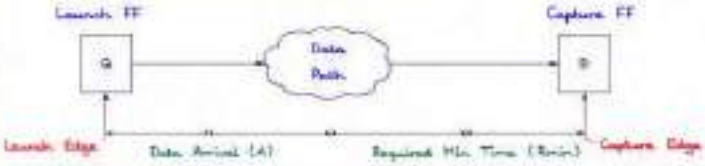
- Verify all setup timing paths across all signoff scenarios.
- Slack = Required Time − Arrival Time ≥ 0
- Ensure margin after OCV/AOCV/POCV.



Setup Slack = R − A ≥ 0 → OK
< 0 → Setup Violation

6. HOLD SIGNOFF

- Verify all hold timing paths.
- Slack = Arrival Time − Required Min Time ≥ 0
- Critical for short paths and clock skew.



Hold Slack = A − Rmin ≥ 0 → OK
< 0 → Hold Violation

7. CLOCK SIGNOFF

- Verify clock network for latency, skew and uncertainty.
- Ensure balanced clock tree and minimal skew.
- Check duty cycle, transition (slew) at clock pins.
- No violations due to clock related issues.

> **Clock Signoff Checks**
>
> ✓ Clock latency
> ✓ Clock skew
> ✓ Uncertainty
> ✓ Duty cycle
> ✓ Clock slew
> ✓ Gating / CTS sanity

8. OCV / AOCV / POCV

- Account for on-chip variations in delay and uncertainty.
- Reduces timing margins and prevents silicon failures.

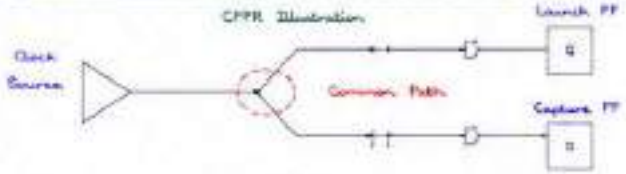
Model	Description	Application
OCV	Fixed % derate on cells and nets	Older nodes / conservative
AOCV	Derate varies with path depth	More accurate, depth based
POCV	Statistical model (σ based)	Advanced signoff accuracy

> **Effect on Timing (Setup Example)**
>
> Derated Delay = Nominal Delay × (1 + Derate %)
> Derated Slack = Required Time − Derated Arrival Time
> (Similar for Hold with early derate)

9. CPPR / CRPR

- Common Path Pessimism Removal (CPPR) removes pessimism from common clock paths.
- Common Reference Point Removal (CRPR) is an extended form of CPPR.

CPPR Illustration



Pessimism removed from common clock path.

10. ECO

ECO Type	Purpose	Typical Changes
Timing ECO	Fix setup/hold violations	Buffer insertion
Functional ECO	Fix functional bugs / logic errors	Gate fix, logic modification
Metal ECO	Only metal layers changed	Reuse existing cells

11. PRACTICAL OBSERVATIONS

- Signoff is the final gate to tape-out.
- Post-route parasitics accuracy is critical.
- MCMM signoff takes most of the runtime.
- Without OCV/AOCV/POCV, silicon may fail.
- CPPR can recover useful slack in many paths.
- Always keep reports and constraints versioned.
- ECOs must be minimal and well verified.

12. VIVA QUESTIONS

Q1. What is signoff STA ?
Q2. Why is signoff necessary ?
Q3. What are post-route parasitics ?
Q4. What is MCMM analysis ?
Q5. What is setup signoff ?
Q6. What is hold signoff ?
Q7. What is clock signoff ?
Q8. What is the need of OCV/AOCV/POCV ?
Q9. What is CPPR ?
Q10. List types of ECO

13. QUICK REVISION

✓ Signoff = Zero violation across all scenarios
✓ Post-route parasitics = accurate delays
✓ MCMM = all PVT / modes / workloads
✓ Setup slack = R − A ≥ 0
✓ Hold slack = A − Rmin ≥ 0
✓ Clock signoff = latency, skew, uncertainty
✓ OCV/AOCV/POCV = variation aware
✓ CPPR removes common path pessimism
✓ ECOs fix timing, functional, power, metal issues

14. KEY FORMULAS / NOTES

Setup Slack = Required Time (R) − Data Arrival Time (A)
Hold Slack = Data Arrival Time (A) − Required Min Time (Rmin)

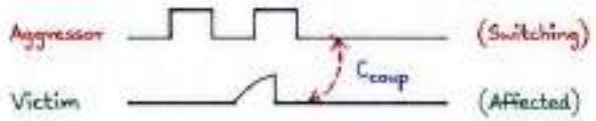
Total Capacitance (C) ≤ Cap Limit
Fanout ≤ Fanout Limit
Slew ≤ Slew Limit

STATIC TIME ANALYSIS

SIGNOFF STA - KEY CONSIDERATIONS

1) CROSSTALK :

- Switching on an aggressor net couples noise on victim net through parasitic capacitance.
- Can increase delay (setup failure) or reduce delay (hold failure).
- Signoff STA uses crosstalk aware analysis.



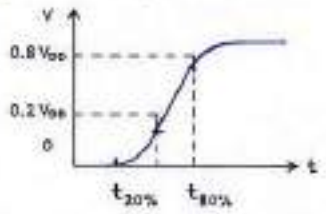
Impact :
- ↑ Delay on victim ⇒ worse setup.
- ↓ Delay on victim ⇒ worse hold.

2) SLEW :

- Slew is the transition time of a signal (rise or fall).
- High slew (slow edge) increases cell delay.
- Low slew reduces delay but may increase power/EM.
- Reported in timing as input and output slew.

Slew Calculation :

$Slew = t_{80\%} - t_{20\%}$

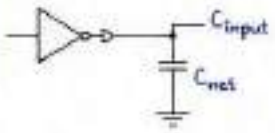


3) CAPACITANCE :

- Total load seen by a pin = input cap of next cell + net cap (wires, vias, pins).
- Higher capacitance ⇒ higher delay & power.
- Reduced by buffering, upsizing, or wire optimization.

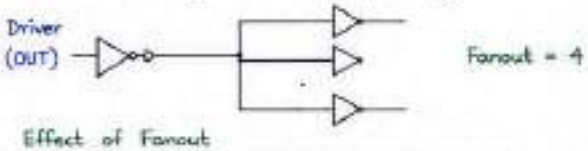
Total Capacitance :

$C_{total} = C_{input(next)} + C_{net}$



4) FANOUT :

- Number of inputs driven by a single output.
- Higher fanout ⇒ higher load capacitance ⇒ higher delay.
- Controlled using buffers or upsizing.



Effect of Fanout

Fanout	Effect on Delay
Low (1-2)	Lower delay
Medium (3-5)	Moderate delay
High (>5)	Higher delay

5) TIMING EXCEPTIONS :

- Specify paths/conditions that are not required to meet default timing.
- Avoid false violations and focus on real paths.

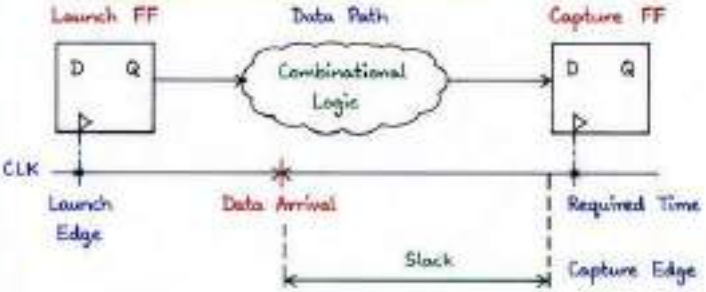
Exception Type	Purpose
False Path	Path never sensitizes
Multi-Cycle Path	Requires more than one cycle
Min Delay (Hold) Exception	Relax hold requirement
Max Delay Exception	Relax setup requirement

6) CRITICAL PATHS :

- Paths with the worst (most negative) slack.
- Define the maximum achievable frequency.
- Any improvement on critical path improves overall timing.

Slack :

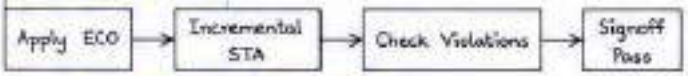
Slack = Required Time - Arrival Time



7) ECO VERIFICATION :

- ECO = Engineering Change Order.
- Any post-route ECO (cell change, buffer add/remove, re-route) must be re-verified.
- Ensure no new violations are introduced.

ECO Check Flow :



8) FINAL TIMING CLOSURE :

- All setup and hold slacks must be ≥ 0 across the chip.
- No unconstrained endpoints.
- Reasonable margins on critical paths.
- Timing met in all signoff corners.

9) TAPEOUT READINESS :

- Timing signoff passed with all checks clean.
- Zero timing violations.
- Exceptions reviewed and justified.
- ECOs verified and frozen.
- Final timing reports, QoR and signoff checklist complete.

Tapeout Checklist :

- ✓ No Setup Violations
- ✓ No Hold Violations
- ✓ No Unconstrained Paths
- ✓ Exceptions Reviewed
- ✓ Crosstalk/Slew/Cap/Fanout OK
- ✓ ECOs Verified
- ✓ Signoff Corners Clean
- ✓ Signoff Reports Archived

TOPIC - SPECIFIC VIVA QUESTIONS (?)

Q1. What is crosstalk and how does it affect timing ?

Q2. Why is slew important in STA ?

Q3. How does capacitance impact timing ?

Q4. What is fanout and how does it affect delay ?

Q5. What are timing exceptions and why are they used ?

Q6. What is a multi-cycle path ? Give an example.

Q7. What is a false path ?

Q8. How are critical paths identified ?

Q9. What is ECO verification and why is it needed ?

Q10. What are the checks for tapeout readiness ?

QUICK REVISION

- Crosstalk ⇒ Alters delay (setup ↑, hold ↓)
- Slew ⇒ Transition time; high slew increases delay
- Capacitance ⇒ Higher cap ⇒ higher delay
- Fanout ⇒ More loads ⇒ higher cap ⇒ higher delay
- Exceptions ⇒ Ignore non-timing / relaxed paths
- Critical Paths ⇒ Worst slack paths
- ECO ⇒ Re-run STA after every physical change
- Closure ⇒ All slacks ≥ 0 in all signoff conditions
- Tapeout ⇒ Clean timing, verified ECOs, closed checklist



THANK YOU!

Thank you for your time, interest and support in exploring Static Timing Analysis (STA). Hope these notes help you understand concepts clearly and apply them confidently in real world VLSI designs.

WHAT YOU EXPLORED

- Timing Concepts & Fundamentals
- Clocking, Skew & Uncertainty
- Setup / Hold Analysis
- Timing Exceptions & Constraints
- PVT, OCV, AOCV / POCV, CPPR
- Crosstalk, Slew, Capacitance, Fanout
- Physical Effects & RC Extraction
- MCMM Analysis & Corner Analysis
- STA Reports, Debug & Timing Closure

KEY TAKEAWAYS

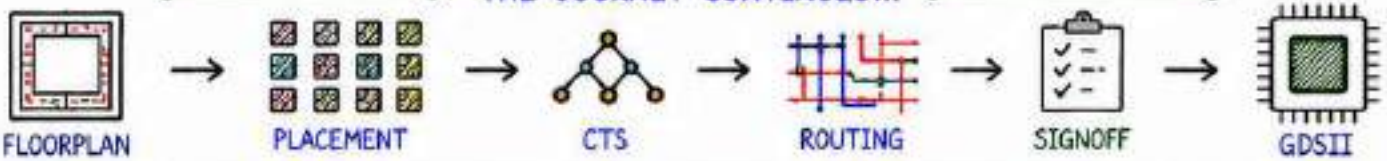
- STA ensures timing correctness without test vectors.
- All possible timing paths are analyzed.
- Accurate constraints and clean design = Reliable signoff.
- Good timing = Good silicon.
- Verification today, Silicon success tomorrow.

WHAT GOOD STA DELIVERS

- Accurate Results
- Better Performance
- Low Power
- Reliable Designs
- Faster Time to Market
- Higher Yield
- Confident Tape-out

> "Good Constraints + Clean Design + Accurate Analysis = Successful Silicon."

THE JOURNEY CONTINUES...



Keep Learning, Keep Building, Keep Innovating!
Wishing you the best for your journey in VLSI Physical Design!

Author :
Konda Tharun Vivekananda